

Kubernetes

для разработчиков

Уильям Деннис



Kubernetes for Developers

WILLIAM DENNISS



MANNING

SHELTER ISLAND

Kubernetes *для разработчиков*

Уильям Деннис

Выпущено
при поддержке
КРОК



Санкт-Петербург • Москва • Минск

2025

ББК 32.973.2-018+ 32.988.02-0
УДК 004.41+004.75
Д33

Деннис Уильям

Д33 Kubernetes для разработчиков. — СПб.: Питер, 2025. — 384 с.: ил. — (Серия «Для профессионалов»).

ISBN 978-5-4461-4129-6

Современное программное обеспечение должно хорошо поддаваться масштабированию, эффективно справляясь при этом с балансировкой нагрузки, отслеживанием состояния и обеспечением безопасности. Узнайте, как можно решать эти и другие важные задачи. В издании рассматриваются все составляющие процесса контейнеризации и развертывания приложения на платформе Kubernetes с точки зрения разработчика. Сначала вы создадите небольшое приложение, способное работать в облачном кластере Kubernetes, а затем поэтапно изучите рекомендуемые практики, позволяющие сделать развертывание стабильным в долгосрочной перспективе, включая методы масштабирования, планирования емкости и оптимизации ресурсов.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018+ 32.988.02-0
УДК 004.41+004.75

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1617297175 англ.

Authorized translation of the English edition © 2024 Manning Publications.
This translation is published and sold by permission of Manning Publications, the owner of all rights to publish and sell the same.

ISBN 978-5-4461-4129-6

© Перевод на русский язык ООО «Прогресс книга», 2025
© Издание на русском языке, оформление ООО «Прогресс книга», 2025
© Серия «Для профессионалов», 2025

Краткое содержание

Часть I

Начало работы с Kubernetes

Глава 1. Использование Kubernetes для развертывания приложений	27
Глава 2. Контейнеризация приложений	48
Глава 3. Развертывание в Kubernetes	76
Глава 4. Автоматизированные операции	123
Глава 5. Управление ресурсами	147

Часть II

Переходим в продуктивную среду

Глава 6. Масштабирование	171
Глава 7. Внутренние сервисы и балансировка нагрузки	201
Глава 8. Выбор параметров узлов	224
Глава 9. Stateful-приложения	251
Глава 10. Фоновая обработка	289
Глава 11. Методология GitOps: конфигурация как код	325
Глава 12. Безопасность в Kubernetes	344

Оглавление

Предисловие	14
Благодарности	16
О книге.	18
Для кого предназначена эта книга.	18
Структура издания	19
О примерах кода.	20
Дополнительные онлайн-ресурсы.	21
От издательства	22
О научном редакторе русского издания	22
Об авторе	23
Иллюстрация на обложке	24

Часть I

Начало работы с Kubernetes

Глава 1. Использование Kubernetes для развертывания приложений	27
1.1. Зачем использовать контейнеры	29
1.1.1. Преимущества контейнеров	30
1.2. Зачем использовать Kubernetes.	35
1.2.1. Составные структурные элементы	36
1.2.2. Особенности и преимущества Kubernetes.	38

1.2.3. Отличия Kubernetes от PaaS-платформы	43
1.2.4. Когда стоит использовать Kubernetes, а когда — нет	45
Резюме	47
Глава 2. Контейнеризация приложений	48
2.1. Создание Docker-контейнеров	50
2.1.1. Настройка среды разработчика	50
2.1.2. Выполнение команд в Docker	51
2.1.3. Создание собственного образа	53
2.1.4. Использование базовых образов	55
2.1.5. Добавление команды по умолчанию	57
2.1.6. Добавление зависимостей	58
2.1.7. Компиляция кода в Docker	59
2.1.8. Компиляция кода с помощью многоэтапной сборки	60
2.2. Контейнеризация серверного приложения	62
2.2.1. Контейнеризация сервера приложений	62
2.2.2. Отладка	65
2.3. Локальное тестирование с помощью утилиты Docker Compose	66
2.3.1. Отображение локальных каталогов на каталоги контейнера	67
2.3.2. Добавление сервисных зависимостей	70
2.3.3. Имитация внешних зависимостей	72
Резюме	74
Глава 3. Развертывание в Kubernetes	76
3.1. Архитектура системы Kubernetes	77
3.1.1. Кластер Kubernetes	78
3.1.2. Объекты Kubernetes	80
3.2. Процесс развертывания приложения	83
3.2.1. Создание кластера	83
3.2.2. Загрузка в сеть собственного контейнера	88
3.2.3. Развертывание в Kubernetes	93
3.2.4. Спецификация пода	99

8 Оглавление

3.2.5. Публикация сервиса	102
3.2.6. Взаимодействие с объектом Deployment	107
3.2.7. Обновление приложения	108
3.2.8. Освобождение ресурсов	111
3.3. Императивные команды	112
3.4. Локальные среды Kubernetes	113
3.4.1. Кластер Kubernetes приложения Docker Desktop	115
3.4.2. Minikube	116
3.4.3. Использование собственного локального кластера Kubernetes	117
Резюме	121
Глава 4. Автоматизированные операции	123
4.1. Автоматическое обеспечение безотказной работы с помощью проверок работоспособности	124
4.1.1. Проверки жизнеспособности и готовности	124
4.1.2. Добавление проверки готовности	126
4.1.3. Добавление проверки жизнеспособности	128
4.1.4. Как создать эффективные проверки работоспособности	129
4.1.5. Перераспределение неготовых контейнеров	133
4.1.6. Типы проверок	136
4.2. Обновление работающих приложений	136
4.2.1. Скользящая стратегия обновления	137
4.2.2. Стратегия повторного создания	140
4.2.3. Сине-зеленая стратегия	141
4.2.4. Выбор стратегии развертывания	145
Резюме	145
Глава 5. Управление ресурсами	147
5.1. Распределение подов	148
5.1.1. Определение ресурсов подов	150
5.1.2. Качество обслуживания	153
5.1.3. Вытеснение, приоритет и приоритизация	155

5.2. Расчет ресурсов для подов	160
5.2.1. Определение запросов и лимитов на объем памяти	162
5.2.2. Определение запросов и лимитов на ресурсы процессора	164
5.2.3. Снижение затрат путем избыточного выделения ресурсов процессора	165
5.2.4. Нахождение баланса между количеством реплик и степенью внутреннего параллелизма пода.	166
Резюме	168

Часть II

Переходим в продуктивную среду

Глава 6. Масштабирование	171
6.1. Масштабирование подов и узлов	172
6.2. Горизонтальное автомасштабирование подов	177
6.2.1. Внешние показатели	181
6.3. Автомасштабирование узлов и планирование мощностей	187
6.3.1. Автомасштабирование кластера	187
6.3.2. Свободные мощности при автомасштабировании кластера	189
6.4. Как сделать свое приложение масштабируемым	192
6.4.1. Отказ от сохранения состояния	193
6.4.2. Микросервисная архитектура	197
6.4.3. Фоновая обработка	199
Резюме	199
 Глава 7. Внутренние сервисы и балансировка нагрузки	 201
7.1. Внутренние сервисы	202
7.1.1. Сетевое взаимодействие в кластере Kubernetes	203
7.1.2. Создание внутреннего сервиса	205
7.1.3. Обнаружение сервиса	207
7.2. Объект Ingress: балансировка HTTP(S)-запросов	212
7.2.1. Обеспечение безопасных соединений с помощью протокола TLS	219
Резюме	222

Глава 8. Выбор параметров узлов	224
8.1. Как выбрать параметры узлов.	225
8.1.1. Селекторы узлов	226
8.1.2. Предпочтение и антипредпочтение узлов.	228
8.1.3. Наложение на узлы ограничений, запрещающих распределение по умолчанию.	231
8.1.4. Разделение рабочих нагрузок	237
8.2. Размещение подов	240
8.2.1. Создание высокодоступных развертываний	241
8.2.2. Совместное размещение взаимозависимых подов	245
8.2.3. Избегание определенных подов	247
8.3. Отладка проблем с размещением.	248
8.3.1. Отказываются работать правила размещения	248
8.3.2. Поды остаются в состоянии ожидания	249
Резюме.	250
Глава 9. Stateful-приложения.	251
9.1. Тома, постоянные тома, заявки и классы хранения	252
9.1.1. Тома	252
9.1.2. Постоянные тома и заявки	256
9.1.3. Классы хранения	262
9.1.4. Одноподовые развертывания для stateful-нагрузок	265
9.2. Набор с сохранением состояния (StatefulSet).	267
9.2.1. Процесс развертывания StatefulSet.	267
9.2.2. Процесс развертывания StatefulSet с несколькими ролями	274
9.3. Перемещение/восстановление дисков	279
9.4. Использование универсальных эфемерных томов в качестве временного рабочего пространства	285
Резюме.	288
Глава 10. Фоновая обработка.	289
10.1. Очереди фоновых задач	290
10.1.1. Создание собственной очереди задач.	292
10.1.2. Обработка сигналов в подах с обработчиками	300

10.1.3. Масштабирование подов с обработчиками	302
10.1.4. Свободно распространяемые реализации очереди задач.	303
10.2. Объекты Job.	303
10.2.1. Выполнение разовых задач с помощью объектов Job.	304
10.2.2. Выполнение запланированных задач с помощью объектов CronJob.	308
10.3. Пакетное выполнение задач с использованием объектов Job	312
10.3.1. Обработка динамической очереди с помощью объекта Job	312
10.3.2. Обработка статической очереди с помощью объекта Job.	317
10.4. Проверка жизнеспособности для фоновых задач.	320
Резюме	324
Глава 11. Методология GitOps: конфигурация как код.	325
11.1. Работа в продуктивной и предпродуктивной средах с использованием пространств имен	326
11.1.1. Процесс развертывания в созданном нами пространстве имен.	328
11.1.2. Синхронизация изменений в пределах кластера.	328
11.2. Как реализуется методология «конфигурация как код» в Kubernetes	329
11.3. Безопасное развертывание.	332
11.3.1. Конвейеры развертывания	334
11.3.2. Непрерывное развертывание с помощью сервиса Cloud Build	335
11.4. Секреты	336
11.4.1. Строковые секреты (пароли).	336
11.4.2. Секреты в формате Base64	338
11.4.3. Файловые секреты	339
11.4.4. Секреты и методология GitOps	342
Резюме	343
Глава 12. Безопасность в Kubernetes	344
12.1. Выполнение обновлений.	345
12.1.1. Обновление кластера и узлов.	345
12.1.2. Обновление контейнеров	346
12.1.3. Управление прерываниями	347

12 Оглавление

12.2. Развертывание узловых агентов с помощью объектов DaemonSet	352
12.3. Контекст безопасности пода.	354
12.4. Запуск контейнеров не от имени суперпользователя	356
12.4.1. Изменение контейнеров в целях их запуска от имени обычного пользователя	359
12.5. Контроллеры допуска.	362
12.5.1. Допуск безопасности пода.	363
12.5.2. Соблюдение баланса между безопасностью и совместимостью . . .	368
12.6. Ролевое управление доступом	369
12.6.1. Роль на уровне пространства имен	369
12.6.2. Роль на уровне кластера	373
12.6.3. Применение профиля безопасности подов	376
12.6.4. Использование механизма RBAC для аккаунтов сервисов	376
12.7. Что еще можно сделать	377
12.7.1. Сетевые политики.	377
12.7.2. Изоляция контейнеров.	378
12.7.3. Защита кластера	378
Резюме	379

Посвящается Питеру.

Предисловие

Рождественские каникулы 2016 года я решил провести за изучением платформы Kubernetes. Я собирался присоединиться к команде разработчиков сервиса Google Kubernetes Engine (GKE), и мне нужно было изучить, как работает эта платформа, причем как можно скорее. Ранее я уже использовал PaaS-платформы (Platform as a Service, платформа как услуга) для развертывания stateless-приложений с возможностью масштабирования, поэтому был очень воодушевлен функционалом Kubernetes: платформа одновременно предлагала и высокую степень гибкости, позволяя запускать практически любой код, который можно упаковать в контейнер, и широкий набор конструкций для представления самых разных рабочих нагрузок, от приложений без сохранения состояния (stateless) до приложений с сохранением состояния (stateful) и пакетных заданий. Однако, как и многие другие, приступающие к работе с этой платформой, я обнаружил, что сначала придется немного поучиться.

К началу 2017 года я уже обрел некоторые знания и начал работать менеджером по продуктам в команде разработчиков сервиса GKE, при этом часто взаимодействуя с ними. Одним из этих разработчиков был мой брат, который решил перевести на Kubernetes свой стартап, развернутый на PaaS-платформе. Я поделился с братом всем, чему научился на тот момент, и предостерег от тех ошибок новичка, которые поначалу совершал и я. В то же время я понял, что было бы хорошо передавать эти знания более простым способом, например, с помощью обучающего руководства, написанного специально для разработчика приложений, который хочет произвести развертывание в системе эксплуатационного уровня, обойдясь без глубокого изучения всех тонкостей платформы Kubernetes.

Эта книга, по сути, и есть то обучающее руководство, которое я хотел иметь в таких случаях. Чтобы ее читать, вам необязательно иметь предварительные знания о Kubernetes или использовании контейнеров. В ней пошагово показан весь процесс разработки и развертывания простого контейнеризированного приложения. Сначала мы разберем, как запустить и сделать доступным в Интернете простейший контейнер, а затем будем постепенно погружаться в такие дополнительные концепции Kubernetes, как stateful-приложения и очереди фоновых задач, пополняя ваш багаж знаний.

Надеюсь, что, прочитав эту книгу, вы сможете легко выполнять развертывание своих приложений в Kubernetes и уже не будете считать эту платформу трудной

в освоении. Это действительно большая и сложная система, но вскоре вы поймете, что вам совсем не обязательно изучать сразу все многочисленные возможности, которые могут пригодиться при развертывании сложных рабочих нагрузок. На самом деле в развертывании stateless-приложения нет ничего сложного, поэтому начните с него и добавляйте остальные возможности по мере необходимости.

К концу книги вы, надеюсь, по достоинству оцените гибкость платформы Kubernetes, которая может быть очень полезной в том случае, когда требования к системе сложно удовлетворить при использовании более специализированной платформы, например, какой-либо PaaS-среды.

Приступая к написанию этой книги, я немного боялся, что к моменту ее выхода описываемые в ней технологии сильно изменятся и мне придется снова и снова переписывать те или иные главы, чтобы их содержимое не устаревало. К счастью, на данный момент можно сказать следующее: в том, что касается общедоступных компонентов платформы Kubernetes, в частности API версии 1, содержимое книги по-прежнему актуально. Это позволяет мне надеяться, что платформа будет достаточно стабильно работать в ваших проектах, а книга еще долго будет полезной. Первоначально я рассматривал и несколько необщедоступных API, которые со временем подверглись значительным изменениям. К счастью, к концу работы над книгой все такие компоненты стали общедоступными, что позволило обновить соответствующие фрагменты текста так, чтобы везде использовались стабильные версии компонентов (исходя из этого опыта, я рекомендую использовать бета-версии API только в экспериментальных целях).

Когда я принимал решение о том, чтобы присоединиться к команде разработчиков сервиса GKE, платформа Kubernetes быстро набирала популярность, но еще не использовалась повсеместно. Я помню, насколько был взбудоражен идеей участия в разработке свободно распространяемого продукта, способного произвести кардинальный сдвиг в компьютерной отрасли; тем не менее я был несколько обеспокоен тем, как сложится судьба нашего детища (и надеялся положительно повлиять на нее). В то время практически каждый облачный провайдер старался предложить собственный управляемый сервис (к счастью, эти сервисы были в значительной мере совместимы друг с другом, а их разработчики стремились к использованию единообразных программных решений), и система Kubernetes постепенно становилась основным инструментом оркестрации контейнеров в продуктивной среде. Я очень рад, что мне довелось присоединиться к сообществу разработчиков Kubernetes и принять участие в ее создании. Надеюсь, что данная книга поможет вам начать свое путешествие с Kubernetes!

Благодарности

Прежде всего я хотел бы поблагодарить редактора-консультанта Элешу Хайд (Elesha Hyde), которая научила меня выходить за рамки исключительно технических моментов и тем самым значительно повлияла на конечный результат. Хочу поблагодарить и технического редактора Шона Смита (Shawn Smith) за его подробные рецензии первых черновиков книги и предложения по доработке содержимого. Большое спасибо Майклу Стивенсу (Michael Stephens) за то, что поверил в меня, когда я озвучил идею книги четыре года назад, и всем сотрудникам издательства Manning, которые помогали претворить эту идею в жизнь: редактору проекта Дирдре Хиама (Deirdre Hiam), выпускающему редактору Алисе Ларсон (Alisa Larson) и корректору Майку Биди (Mike Beady). До начала работы над этим проектом я и не подозревал, насколько весомый вклад в качество книги будет вносить сотрудничество со всеми вами!

Спасибо и вам, мои рецензенты: Амар Манигандан (Amar Manigandan), Андрес Сакко (Andres Sacco), Ариэль Гаминьо (Ariel Gamiño), Атул С. Кхот (Atul S. Khot), Бекки Хьюетт (Becky Huett), Бонни Малек (Bonnie Malec), Чейз Силлевис (Chase Sillevs), Крис Хенеган (Chris Heneghan), Крис Винер (Chris Viner), Конор Редмонд (Conor Redmond), Дэвид Паккуд (David Paccoud), Давиде Фиорентино (Davide Fiorentino), Дипика (Deepika), Джордж Томас (George Thomas), Джулиано Латини (Giuliano Latini), Грегорио Пикколи (Gregorio Piccoli), Гай Ндженг (Guy Ndjeng), Ханс Доннер (Hans Donner), Харинат Маллепалли (Harinath Mallepally), Иоаннис Полизос (Ioannis Polyzos), Джавид Асгаров (Javid Asgarov), Хуан Хименес (Juan Jimenez), Жюльен Похи (Julien Pohie), Камеш Ганесан (Kamesh Ganesan), Картике-яраджан Раджендран (Karthikeyarajan Rajendran), Келум Прабат Сенанаяке (Kelum Prabath Senanayake), Кельвин Джонсон (Kelvin Johnson), Космас Чазимикалис (Kosmas Chatzimichalis), Кшиштоф Камычек (Krzysztof Kamyczek), Марк Дехампс (Mark Dechamps), Майкл Брайт (Michael Bright), Майк Райт (Mike Wright), Митчелл Фокс (Mitchell Fox), Наджиб Ариф (Najeeb Arif), Пьер-Мишель Ансель (Pierre-Michel Ansel), Рахул Модпур (Rahul Modpur), Раджасилан Ганесваран (Rajaseelan Ganeswaran), Рамбабу Поса (Rambabu Posa), Ричард Воган (Richard Vaughan), Роберт Килти (Robert Kielty), Сатадру Рой (Satadru Roy), Себастьян Чех (Sebastian Czech), Серджиу Попа (Sergiu Popa), Симеон Лейзерзон (Simeon Leyzerzon), Сухас Кришнаия (Suhas Krishnayya), Башируддин Ахмед (Basheeruddin Ahmed), Венкатеш Сундарамурти (Venkatesh Sundaramoorthy), Вальдемар Модзелевски (Waldemar Modzelewski) и Залан Сомогивари (Zalán Somogyváry). Ваши предложения помогли улучшить эту книгу.

Высокой оценки заслуживают и технические специалисты издательства Manning, которые создали такой невероятный инструмент, как Программа раннего доступа к книгам (Manning Early Access Program, MEAP), что позволило мне публиковать материал этой книги по ходу работы над ней и получать обратную связь от первых читателей. Спасибо и всем, кто купил предварительную версию книги в рамках программы раннего доступа.

Хочу поблагодарить и своих коллег по работе. Спасибо вам, Дрю Брэдсток (Drew Bradstock) и Йохай Кириати (Yochay Kiriati), за то, что верили в меня и поддерживали в работе над этим проектом, создав в нашем коллективе атмосферу, превращающую работу в праздник. Спасибо вам, Тим Хокин (Tim Hockin), Чен Голдберг (Chen Goldberg), Келси Хайтауэр (Kelsey Hightower), Эрик Брюер (Eric Brewer), Джереми Олмстед-Томпсон (Jeremy Olmsted-Thompson), Ежи Форыцяж (Jerzy Foryciarz) и Брайан Грант (Brian Grant), за то, что оказывали на меня положительное влияние, и тебе, Апарна Синха (Aparna Sinha), за то, что одной из первых поверила в мой успех.

Фотограф Том Эдвардс (Tom Edwards) (работы которого можно увидеть на сайте tomedwardsphotography.com) фотографировал меня для профиля на сайте перед конференцией KubeCon. Спасибо, дружище!

Наконец, хочу сказать спасибо членам моей семьи за поддержку во время работы над книгой. Аарон и Эшли, вы были слишком маленькими, чтобы понимать, над чем работает ваш папа, но я всегда буду признателен вам за то, что в течение многих месяцев вы каждый день ходили со мной в Imua, где я мог спокойно поработать над книгой, пока вы играли рядом в свои игры. Спасибо моей жене Фионе за то, что поддерживала все мои начинания и всегда могла вернуть меня с небес на землю, когда мне это требовалось. Сяньцон, без вашей помощи по хозяйству у меня вряд ли было бы время на реализацию своих проектов. Спасибо Джули, Грэму и Лорел за то, что с самых ранних лет поощряли мой интерес к технологиям, и моему брату Питеру и сестре Джессике за их готовность прийти на помощь и за наши совместные авантюры.

Вы хотите разместить свое приложение в продуктивной облачной среде, которая благодаря своей мощности и гибкости сможет легко удовлетворять быстро изменяющиеся требования к приложению и при необходимости позволит выполнить широкое масштабирование? Из книги вы узнаете, как приложение, написанное на Python, Java, Ruby или любом другом языке, можно запросто развернуть на профессиональной облачной платформе, которая может удовлетворить ваши требования и сейчас, и в дальнейшем, — с использованием контейнеров и Kubernetes.

Если у вас еще нет готового к развертыванию собственного приложения, то в качестве примера можете использовать мое демонстрационное приложение. Кроме того, для работы с книгой необязательно иметь предварительные знания о платформах Docker или Kubernetes. Прочитав ее, вы сможете развертывать в Kubernetes различные виды рабочих нагрузок: от stateless-приложений до пакетных заданий и баз данных.

ДЛЯ КОГО ПРЕДНАЗНАЧЕНА ЭТА КНИГА

Существует много книг по Kubernetes, каждая из которых написана для определенной целевой аудитории. Данная книга предназначена для тех разработчиков, которые приступают к изучению Kubernetes, исходя из примерно таких соображений: «Я написал на своем ноутбуке небольшой код и хочу сделать его доступным для пользователей по всему миру с помощью платформы, которая позволит выполнить масштабирование, если мой продукт станет успешным». Когда я только начинал использовать Kubernetes, мне очень не хватало наглядных примеров того, как можно развернуть код на этой платформе, выполнить его обновление, обеспечить его бесперебойную работу и при необходимости подготовить к масштабированию. Я надеюсь, читатели смогут найти такие примеры в этой книге.

Если вы еще никогда не использовали контейнеры, то вам не стоит волноваться по этому поводу, поскольку целая глава посвящена контейнеризации приложений перед их развертыванием. Если же вы уже знакомы с Docker и контейнерами и хотите сразу же приступить к развертыванию своего кода в Kubernetes, то можете пропустить первые главы и начать с главы 3.

СТРУКТУРА ИЗДАНИЯ

Содержимое книги разделено на две части. В первой я излагаю основы работы с платформой, начиная с операций сборки контейнера и запуска его в Kubernetes. Вы научитесь выделять оптимальное количество ресурсов и настраивать их так, чтобы автоматизированные операции платформы использовались максимально эффективно. И конечно, вы научитесь выполнять обновление своего приложения. Если вы хотите развернуть в Kubernetes stateless-приложение, то все, что вам нужно знать, изложено в следующих пяти главах.

- Прочитав главу 1, вы получите общее представление о платформе и ее преимуществах.
- Глава 2 представляет собой краткий курс по Docker для разработчиков приложений. Вы можете пропустить эту главу, если уже знакомы с данной платформой.
- В главе 3 вы развернете свое приложение на платформе, сделав его доступным в Интернете, и выполните его обновление. Таким образом, ваше приложение уже будет работать в системе Kubernetes.
- В главе 4 вы добавите в развертывание проверки работоспособности, чтобы сделать его более надежным.
- В главе 5 вы научитесь выделять для рабочей нагрузки оптимальное количество ресурсов, чтобы они не расходовались напрасно.

Во второй части я подробно описываю эксплуатационные аспекты платформы Kubernetes. Речь пойдет о масштабировании приложений, настройке внутренних сервисов и развертывании таких рабочих нагрузок, как stateful-приложения и очереди фоновых задач. Заключительные главы будут посвящены использованию конфигурации в виде кода, непрерывному развертыванию и вопросам безопасности.

- Глава 6 посвящена масштабированию (как расширению, так и сужению), в том числе ручному и автоматическому, масштабированию узлов и подов и т. д.
- В главе 7 мы поговорим о настройке внутренних сервисов и приложений, основанных на микросервисной архитектуре, и в общих чертах рассмотрим процесс балансировки нагрузки HTTP-трафика.
- В главе 8 будет показано, как в определении рабочих нагрузок указывать конкретные требования контейнеров к аппаратному обеспечению и как выполняется группировка и распределение подов по узлам.
- В главе 9 мы поговорим об использовании рабочих нагрузок с сохранением состояния, таких как базы данных.
- Глава 10 посвящена процессам, которые не связаны с обработкой поступающих запросов, в том числе очередям задач и пакетным заданиям.

- В главе 11 вы познакомитесь с методологией GitOps. Вы научитесь использовать пространства имен для определения нескольких отдельных сред и работать с конфигурацией как с кодом.
- В главе 12 мы коснемся вопросов безопасности, которые следует учитывать разработчику приложений.

О ПРИМЕРАХ КОДА

Используемые в книге примеры кода представляют собой открытый исходный код, доступный по лицензии Apache 2.0. Все примеры можно скачать по адресу <https://github.com/WilliamDenniss/kubernetes-for-developers>.

Примеры распределены по папкам, относящимся к отдельным главам и имеющим соответствующее имя: `Chapter02`, `Chapter03` и т. д. Каждая из папок содержит вложенные папки с конфигурациями Kubernetes; при этом имена вложенных папок состоят из номера того раздела или подраздела, в котором используются конфигурации, и соответствующей описательной фразы. Например, конфигурация, используемая в разделе 3.2 главы 3, размещена по адресу `Chapter03/3.2_DeployingToKubernetes`. Используемое в книге демонстрационное контейнеризированное приложение постепенно дорабатывается, а его текущая версия размещается в папке, относящейся к соответствующей главе. Например, первая версия приложения, с которой вы познакомитесь в главе 2, размещена в папке `Chapter02/timeserver`, следующая версия, которая будет использоваться в главе 4, находится в папке `Chapter04/timeserver2` и т. д.

Чтобы вам было проще ориентироваться, в заголовке каждого листинга указывается полный путь к соответствующему файлу. В примерах команд, приводимых по ходу книги, в качестве отправной точки всегда используется базовый каталог, что позволяет изучать содержимое разделов в произвольном порядке. Прежде чем изучать каждый следующий раздел, просто возвращайтесь в командной оболочке в базовый каталог перед изучением каждого нового раздела.

Примеры из книги можно выполнять на любой из трех платформ: Mac, Linux или Windows, инструкции по начальной настройке которых приводятся в главах 2 и 3.

Пользователи облачной платформы Google Cloud могут использовать командную оболочку на базе браузера Cloud Shell (<https://cloud.google.com/shell>). Она позволяет выполнять все приводимые в книге инструкции, в том числе те, которые касаются использования Docker и Minikube, не устанавливая что-либо на локальной машине. Исключение составляют инструкции по работе в локальной среде Kubernetes приложения Docker Desktop, вместо которого пользователи должны использовать Minikube.

Примеры кода в книге могут приводиться и в виде отдельных пронумерованных листингов, и как фрагменты внутри абзацев обычного текста. И в том и в другом

случае они выделяются моноширинным шрифтом. Кроме того, иногда я выделяю часть кода **жирным шрифтом**, чтобы вы видели, что изменилось по сравнению с предыдущими этапами, например, при добавлении новых строк кода к уже имеющимся.

Чтобы код мог поместиться на странице книги, мне часто приходилось менять формат исходного кода, добавляя разрывы строк и уменьшая размер отступов. В некоторых случаях даже этого было недостаточно, и мне приходилось разрывать строки с помощью символа разрыва строки (`\n`). Кроме того, я часто удалял комментарии из тех примеров кода, которые описываются в тексте. Многие листинги имеют пояснения, особое внимание в которых уделяется тем или иным важным элементам.

Все примеры кода из этой книги доступны для скачивания по адресу <https://github.com/WilliamDenniss/kubernetes-for-developers>.

ДОПОЛНИТЕЛЬНЫЕ ОНЛАЙН-РЕСУРСЫ

- Официальная документация по Kubernetes: <https://kubernetes.io/docs/>.
- Базовые образы контейнеров: <https://hub.docker.com/>.
- Раздел, посвященный Kubernetes, на сайте StackOverflow: <https://stackoverflow.com/questions/tagged/kubernetes>.

Дополнительные материалы, посвященные платформе Kubernetes и сервису GKE, представлены здесь: <https://wdenniss.com/kubernetes>.

От издательства

Мы выражаем огромную благодарность компании «КРОК» за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

О НАУЧНОМ РЕДАКТОРЕ РУССКОГО ИЗДАНИЯ

Алексей Патрин — архитектор и senior-разработчик C# компании «КРОК». Занимается проектированием программной и системной архитектуры широкого спектра решений, разрабатываемых компанией для своих заказчиков. Опыт в IT и разработке — более 25 лет. Выступал как на публичных, так и на внутренних конференциях по темам использования Kubernetes и микросервисной архитектуры.

Об авторе

Уильям Деннис (William Denniss) занимает в компании Google должность менеджера группы продуктов (Group Product Manager) и в этом качестве участвует в разработке сервиса Google Kubernetes Engine (GKE). Был одним из тех, кто снабдил сервис GKE режимом «Автопилот» (Autopilot) и тем самым превратил его в полностью управляемую платформу Kubernetes, которая раскрывает все свои возможности, не вынуждая тратить усилия на управление вычислительными узлами, входящими в состав кластера. Убежден в том, что в развитии компьютерной индустрии важную роль играет использование открытых стандартов и программного обеспечения с открытым исходным кодом. Одна из первых задач Уильяма после присоединения к команде разработчиков сервиса GKE сводилась к сотрудничеству с сообществом Kubernetes и Фондом нативных облачных вычислений (Cloud Native Computing Foundation, CNCF) в целях создания Программы сертификации соответствия платформ Kubernetes Conformance Program, которая призвана повысить совместимость платформы среди облачных провайдеров Kubernetes.

Устроившись в 2014 году на работу в Google, Уильям поначалу занимался системами идентификации, а точнее, работал над улучшением способа взаимодействия пользователей с системами идентификации, применявшимися на мобильных устройствах. Он является автором рекомендаций по использованию протокола OAuth в нативных приложениях, которые были опубликованы в RFC 8252, и является сооснователем свободно распространяемой библиотеки AppAuth для iOS, Android и JavaScript, которая призвана стать общедоступной реализацией этих рекомендаций.

Уильям любит повышать уровень своих знаний, поэтому преподает и много времени уделяет написанию программ и дальнейшему развитию своих проектов. Купив какое-либо устройство, он всегда начинал писать программы для него, будь то графический калькулятор (в старших классах школы), компьютер с операционной системой Windows, Linux или Mac (каждый в свое время), приставка PlayStation Portable (в 2000-е) или iPhone (после выхода модели iPhone 3G). В конечном счете пришлось создать для проектов какой-то сервер, и около десяти лет назад Уильям начал разворачивать свой код на PaaS-платформе, что, в свою очередь, побудило его заинтересоваться разработкой платформы Kubernetes, которая была призвана упростить жизнь разработчикам с такими же, как у него, потребностями.

Будучи менеджером по продукту, Уильям обладает уникальным качеством: он сам является активным пользователем тех продуктов, что создает.

Иллюстрация на обложке

В качестве иллюстрации на обложке использована картина Доминика Серреса (Dominic Serres) «Матрос с военной баржи» (*A Seaman with a Man of War's Barge*), написанная им в 1777 году. Это произведение искусства входит в коллекцию изображений, представляющих различные виды униформы Британского военно-морского флота: от формы матроса до формы среднего офицерского состава и адмирала. Матрос на картине одет в короткий синий пиджак с обтянутыми тканью пуговицами, рукава которого на флотский манер имеют разрез и застегиваются на три пуговицы, и белый жилет. Кроме того, мы видим на матросе похожие на юбку брюки, которые спускаются от талии до верхней части голеней, белые чулки и туфли со светлыми металлическими пряжками.

В наше время, когда одна компьютерная книга мало чем отличается от другой, издательство Manning чествует проявление изобретательности и инициативности в компьютерном бизнесе, представляя на обложках своих книг разнообразные исторические фигуры в старинных костюмах.

Часть I

Начало работы с Kubernetes

Kubernetes — популярная платформа оркестрации контейнеров. В первой части книги мы разберем, что она собой представляет и чем полезна. Кроме того, вы узнаете о некоторых базовых моментах, например, о том, как контейнеризировать приложения в целях их использования в этой среде, и попытаете развернуть в Kubernetes свое первое контейнеризированное приложение, сделав его доступным в Интернете. Вы узнаете, как сконфигурировать приложение, чтобы Kubernetes автоматически обеспечивал его бесперебойную работу даже при проведении обновлений, и как определить, какие ресурсы следует выделять каждому контейнеру.

К концу первой части вы запустите в Kubernetes контейнеризированное приложение без сохранения состояния (stateless) и, если захотите, сделаете его доступным по всему миру через Интернет. Вы научитесь настраивать конфигурацию приложения так, чтобы платформа Kubernetes обеспечивала его присутствие в сети даже в случае, если ваше приложение упало или кластер обновляется, и вам не нужно было следить за этим самим.

Многие считают платформу Kubernetes немного сложной в использовании. Отчасти такое мнение объясняется тем, что эта система обладает чрезвычайно широким спектром возможностей. В первой части я постараюсь показать, насколько легко можно развернуть в Kubernetes простое stateless-приложение и сделать его доступным в Интернете. Совсем не обязательно осваивать сразу все расширенные возможности платформы — обстоятельное знакомство с ними можно отложить до того момента, когда они действительно понадобятся. Лучше начать с таких простых вещей, как развертывание stateless-приложения.

Использование Kubernetes для развертывания приложений

В этой главе

- ✓ Преимущества упаковки приложений в контейнеры.
- ✓ Что делает Kubernetes идеальной платформой для развертывания контейнеров.
- ✓ Когда стоит использовать Kubernetes, а когда — нет.

Наступил вечер пятницы, и у пользователей вдруг проснулся необычайный интерес к тому приложению, над которым вы работали в течение последнего года. Вам нужно его масштабировать, и как можно скорее. Позволит ли ваше приложение и та платформа, на которой оно запущено, увеличить масштаб в 100 раз, обеспечив вам успех, или все ваши усилия окажутся напрасными из-за нестабильно работающего кода и негибкой платформы?

Вы создали отличное stateless-приложение на базе популярной платформы для создания приложений, позволяющей быстро проводить масштабирование, и все прекрасно работает... Но вот наступает момент, когда бизнес-требования изменяются, в связи с чем необходимо запустить специализированное приложение с сохранением состояния (stateful-приложение) для некоторых критически важных бизнес-данных или специально настроенный конвейер ночной пакетной обработки. Сможете ли вы без проблем добавить эти новые рабочие нагрузки к уже

существующим? Или потребуется разработать новое приложение или как-то объединить несколько разнородных систем?

Предлагая разработчикам решение этих и многих других проблем, Kubernetes быстро становится отраслевым стандартом, позволяющим выполнять рабочие нагрузки самых разных форм и размеров в любом необходимом масштабе. Эта платформа позволяет запускать контейнеризированные приложения, предоставляя возможность быстрого их масштабирования, используя при этом несколько сложных схем развертывания: от развертывания приложений без сохранения состояния до развертывания баз данных с сохранением состояния и пакетных заданий с эфемерным хранилищем. Это свободно распространяемое детище компании Google (<https://cloud.google.com/learn/what-is-kubernetes>), которым уже пользуются такие уважаемые организации, как Spotify (<https://kubernetes.io/case-studies/spotify/>), CapitalOne (<https://kubernetes.io/case-studies/capital-one/>) и OpenAI (<https://kubernetes.io/case-studies/openai/>), представляет собой открытую, не привязанную к конкретному поставщику надежную платформу, которая играет в мире облачного развертывания столь же важную роль, как, например, Linux в мире операционных систем.

Однако платой за такую гибкость является достаточно длительный срок освоения возможностей платформы. Будучи универсальным инструментом, Kubernetes позволяет использовать так много разных способов развертывания, что изучение платформы может показаться крайне сложной задачей. Здесь я должен сказать, что, во-первых, изучить ее не так сложно, как может показаться, и, во-вторых, ваши старания окупятся с лихвой. Если вы начнете с основ и будете постепенно добавлять новые конструкции (а именно такой подход описывается в книге), то освоение этой системы уже не будет выглядеть пугающе сложной задачей. Вы можете развернуть stateless-приложение, используя лишь несколько строк YAML-кода, и продолжить дальнейшее изучение, отталкиваясь от этой отправной точки.

Исходя из своего профессионального опыта, могу сказать, что при решении такой проблемы, как выбор наилучшего способа развертывания приложения, нужно иметь в виду следующий момент. Часто вместо того чтобы выбрать самый простой вариант, способный удовлетворить текущие требования, стоит потратить некоторое время на изучение системы, что сможет удовлетворить ваши потребности и сейчас, и в дальнейшем, а также позволит вам развивать свои навыки и расти профессионально по мере увеличения требований к приложению. Именно такой системой является Kubernetes. Вы можете за считанные часы реализовать одну из простейших схем развертывания, зная при этом, что данный инструмент таит в себе множество других функциональных возможностей, которые вы всегда можете освоить и применить в случае необходимости.

Если я уже убедил вас в необходимости использования Kubernetes, то вы можете пропустить оставшуюся часть этой главы и сразу перейти к главе 2, в которой рассказывается о создании Docker-образа, или к главе 3, если уже знаете, что такое Docker-контейнер, и хотите приступить к развертыванию приложений в Kubernetes. Далее в этой главе рассказывается о том, почему разработчики приложений все чаще предпочитают использовать Kubernetes и контейнеры.

1.1. ЗАЧЕМ ИСПОЛЬЗОВАТЬ КОНТЕЙНЕРЫ

Kubernetes — платформа развертывания для контейнеров. При развертывании на ней любого кода, например приложения, он сначала упаковывается в контейнер. Но что такое контейнеры и зачем вообще они нужны?

Контейнеры — современное средство упаковки и запуска приложений. Если вы не собираетесь запускать каждое приложение на отдельном хосте (что крайне неэффективно), то вам как-то нужно будет развернуть несколько приложений на одной машине или группе машин. Рассмотрим доступные варианты.

До появления виртуальных машин было принято устанавливать приложения в отдельные каталоги на общем хосте, выделяя для каждого из них отдельный порт. Этот подход чреват проблемами, поскольку приложения могут влиять друг на друга при совместном использовании зависимостей и таких ресурсов машины, как процессор, память и доступные порты. Кроме того, данная схема трудно поддается масштабированию: если одно из приложений вдруг начнет получать больше трафика, чем другие, то вы не сможете масштабировать только это приложение, оставив другие на прежнем уровне.

После появления виртуальных машин разработчики стали упаковывать каждое приложение в отдельную такую машину. При этом каждое приложение имеет собственную операционную среду с изоляцией зависимостей и выделенной для него частью ресурсов. Но каждая виртуальная машина эмулирует все функции отдельного хоста, поэтому вам приходится обеспечивать управление операционной системой и всеми пакетами для каждого приложения, что достаточно сложно и чревато большими накладными расходами.

Теперь перейдем к контейнерам. Подобно виртуальной машине, контейнер позволяет упаковать приложение и зависимости, необходимые ему для размещения в изолированной среде, но при этом не требует, чтобы вместе с приложением устанавливалась отдельная операционная система.

На рис. 1.1 показан процесс эволюции сервисов хостинга: от запуска нескольких рабочих нагрузок на общем хосте до запуска их в отдельных виртуальных машинах

и, наконец, в контейнерах. Благодаря использованию контейнеров можно получить многие из преимуществ виртуальной машины, обойдясь без накладных расходов на запуск дополнительного ядра операционной системы, что делает переход к их применению логичным следующим шагом.

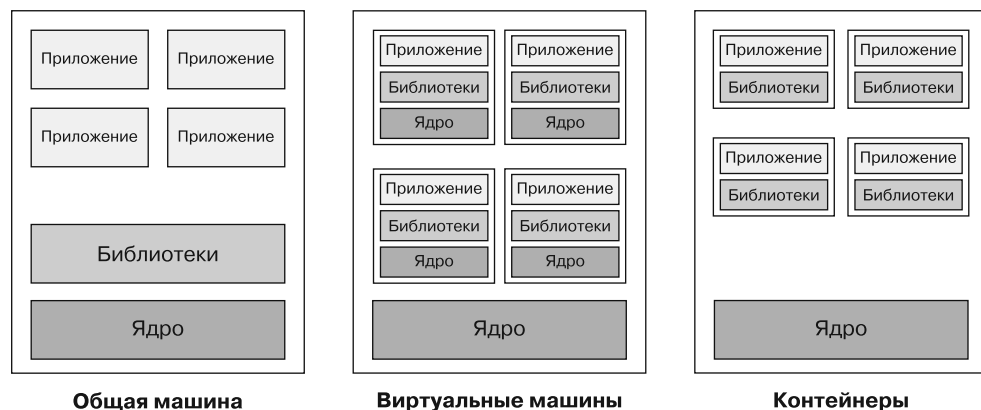


Рис. 1.1. Эволюция архитектур общего хостинга

1.1.1. Преимущества контейнеров

Основные доводы в пользу применения контейнеров:

- *языковая гибкость* — на контейнерной платформе можно использовать любой язык или среду;
- *легковесная изоляция* — можно исключить влияние рабочих нагрузок друг на друга, не используя виртуальные машины;
- *высокая эффективность процесса разработки* — степень различий между продуктивной средой и средой разработки уменьшается, а также упрощается настройка приложения;
- *воспроизводимость* — в сборочном файле контейнера описывается последовательность действий по созданию необходимой среды.

Языковая гибкость

Контейнеры исключают зависимость системы развертывания от конкретного языка или библиотек. Вы можете использовать любой язык и обновлять любой пакет. Вы больше не будете привязаны к конкретному языку или версии языка либо к устаревшей версии критически важной зависимости, установленной вместе с операционной системой много лет назад, как часто происходило при использовании традиционной PaaS-среды (platform as a service, платформа как услуга).

При запуске двух контейнеров на одном хосте они не будут иметь общих библиотек, а значит, конфигурация одного не будет влиять на конфигурацию другого. Вам нужно использовать разные версии языка Java или совершенно разные зависимости? Нет проблем! Изоляция не ограничивается одними библиотеками: разные контейнеры могут применять совершенно разные операционные системы и менеджеры пакетов: например, один пакет может использовать Ubuntu в сочетании с менеджером APT, а другой — CentOS в сочетании с менеджером RPM. Такая гибкость позволяет при необходимости легко объединить в одну систему несколько сервисов (используя так называемую *микросервисную* архитектуру), каждый из которых будет иметь свой набор зависимостей и используемых языков и будет поддерживаться отдельной командой разработчиков. Контейнеры изолируют друг от друга зависимости разных приложений, что упрощает их запуск на одном хосте (рис. 1.2).

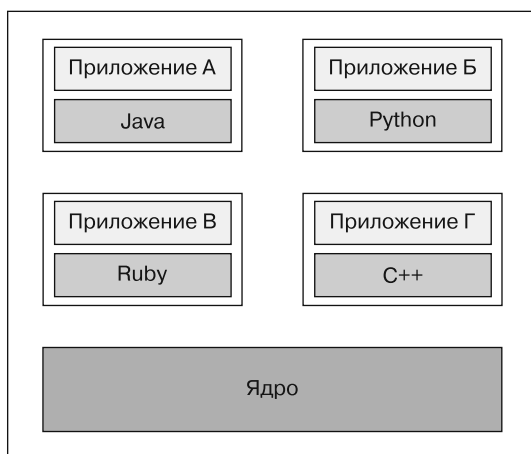


Рис. 1.2. На одном хосте запущены четыре контейнера, в которых используются разные языки

Изоляция без накладных расходов

Раньше для взаимной изоляции приложений, запускаемых на одном и том же хосте, использовались виртуальные машины. Это решение влияет и на размер образа, и на потребление ресурсов процессора и памяти, поскольку в каждой виртуальной машине приходится дублировать ядро и значительную часть операционной системы. Контейнеры являются более подходящим решением, обеспечивая практически те же преимущества изоляции ресурсов. Любой контейнер можно настроить на использование только части ресурсов хоста, и Kubernetes проследит за соблюдением данного ограничения. В конечном счете это позволяет размещать на одном хосте больше приложений, снижая уровень затрат на инфраструктуру (рис. 1.3).

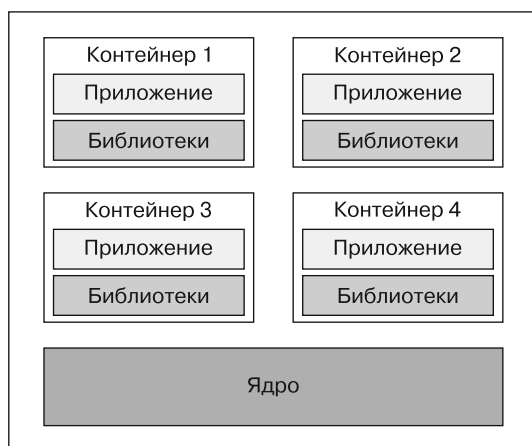


Рис. 1.3. Четыре контейнера, запущенных на одном хосте, полностью изолированы друг от друга, но используют общее ядро

Высокая эффективность процесса разработки

Изоляция зависимостей, возможная благодаря контейнерам, упрощает не только процесс развертывания в продуктивной среде, но и процесс разработки, поскольку вы можете разрабатывать на одной машине множество различных приложений, и при этом вам не придется учитывать в конфигурации хоста зависимости каждого из них (рис. 1.4). Наряду с разработкой Linux-приложений непосредственно в Linux с помощью Docker вы можете создавать контейнеры для Linux на рабочей станции с операционной системой macOS или Windows, не создавая нативную версию приложения для этих платформ, когда на этапе разработки приходится использовать соответствующую платформенно-зависимую конфигурацию.

Разработчикам больше не нужно использовать многостраничные инструкции по начальной настройке, поскольку теперь этот процесс сводится лишь к тому, чтобы установить Docker, проверить код, выполнить сборку и запуск. Упрощается и совместная работа над несколькими проектами в составе одной или нескольких команд разработчиков: каждый проект аккуратно изолируется в своем контейнере, и вам не нужно использовать для него специальную конфигурацию хоста.

Теперь приложение выглядит практически одинаково и в среде разработки, и в продуктивной среде, зачастую представляя собой один тот же контейнер. Благодаря этому можно забыть об особенностях среды разработки, таких как использование конкретной версии библиотеки MySQL в macOS или незначительные различия в способе упаковки кода для развертывания в продуктивной среде. Вам нужно выяснить, чем вызвана определенная проблема в продуктивной среде? Скачайте тот же контейнер и посмотрите, что с ним не так, запустив его в своей среде разработки (рис. 1.5).

Машина разработчика, использующего Docker

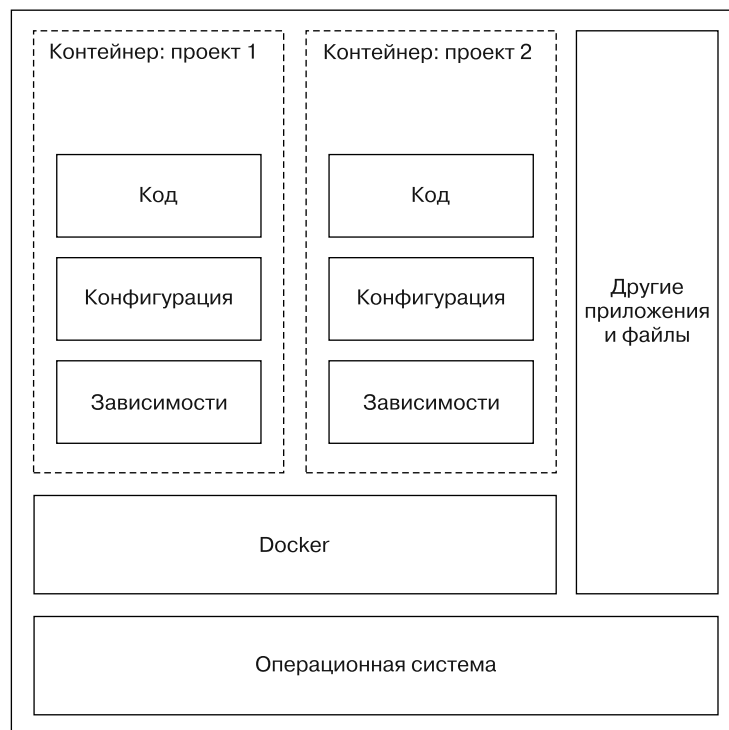


Рис. 1.4. Машина разработчика с двумя контейнеризированными проектами



Рис. 1.5. Развертывание одного и того же контейнера в продуктивной среде и среде разработки

Воспроизводимость

При использовании контейнеров проще воспроизвести необходимую для приложения среду. Допустим, вы развернули приложение на виртуальной машине, и вам нужно настроить протокол TLS (Transport Layer Security, протокол защиты транспортного уровня), чтобы обезопасить HTTP-соединения. Вы устанавливаете SSH-соединение с рабочим хостом и добавляете в одну из папок TLS-сертификат. Это не дает результата, и вы добавляете сертификат в другую папку. Результата снова нет, и вы добавляете сертификат в третью папку. Он срабатывает, и вы больше не вносите никаких изменений. Год спустя наступает момент, когда требуется обновить этот TLS-сертификат. Сможете ли вы теперь вспомнить, каким образом и в какой из трех папок нужно выполнить обновление?

Использование контейнеров позволяет решить эту проблему. Вместо того чтобы устанавливать SSH-соединение и вносить изменения в состояние рабочего хоста, вы можете выполнить добавление TLS-сертификата как один из этапов сборки контейнера. Если данный этап не дает результата, то вы можете скорректировать его так, чтобы он заработал, но, что важно, в конечном счете сохраните только ту его версию, которая действительно дает результат. Кроме того, те файлы, которые вы добавляете на данном этапе сборки, аккуратно изолируются от остальной части системы и поэтому могут представлять собой только приращение или отличия от исходного состояния системы — только те изменения, которые вам необходимо внести. Это означает следующее: когда подойдет время обновления сертификата, вам нужно будет просто заменить старый файл сертификата новым и заново выполнить процесс сборки контейнера, который поместит сертификат в нужное место.

В листинге 1.1 представлен пример псевдокода, описывающего содержимое файла Dockerfile, то есть код настройки контейнера, представленный на обычном языке (примеры реального содержимого этого файла будут рассмотрены в главе 2).

Листинг 1.1. Chapter01/1.1.1_Pseudocode Dockerfile

Использовать операционную систему Ubuntu
Скопировать и настроить TLS-сертификат
Скопировать приложение

ПРИМЕЧАНИЕ

Используя Docker в качестве инструмента для создания контейнеров, вы не сможете обеспечить идеальную воспроизводимость. При установке зависимостей с помощью таких команд, как `apt-get`, Docker взаимодействует с операционной системой в ее текущем виде и поэтому иногда выдает разный результат для одинаковых входных данных, поскольку такие зависимые системы, как репозиторий АРТ, могут изменяться от сборки к сборке. Некоторые инструменты, например, свободно распространяемая система сборки Bazel от компании Google, позволяют решить эту и другие проблемы, но имеют свои сложности и больше подходят для больших корпоративных схем развертывания. Несмотря на это ограничение, система сборки Docker все же обеспечивает намного больший уровень воспроизводимости по сравнению с необходимостью вспоминать, что именно вы делали в SSH-сеансе год назад, когда нужно было устранить проблему на определенной Linux-машине. В этом плане данная система вполне приемлема для большинства проектов.

1.2. ЗАЧЕМ ИСПОЛЬЗОВАТЬ KUBERNETES

Итак, контейнеры — прекрасное средство упаковки приложений. Однако после их создания их нужно как-то запустить и обеспечить управление ими. Конечно, вы могли бы просто запускать один или несколько контейнеров на отдельном хосте, как в случае запуска нескольких разных приложений из отдельных папок или образов виртуальных машин. Но при таком подходе вы в итоге получаете особым образом скомпонованную группу машин, которая трудно поддается масштабированию из-за большого объема неавтоматизируемой ручной работы по настройке и управлению хостами.

Более эффективный подход сводится к тому, чтобы создать общий пул (кластер) ресурсов из некоего множества машин (узлов) и выполнять в нем запуск контейнеров с помощью так называемого «оркестратора контейнеров» (такого как Kubernetes). При этом вы управляете всей группой машин, в которой ни одна из них не играет большой роли. При выходе из строя одной машины ее может заменить другая. Это позволяет не привязываться к отдельным машинам и масштабировать приложение быстрее, чем размер команды разработчиков.

Раньше использование систем для гибкой оркестрации контейнеров в больших масштабах было прерогативой крупных компаний. Появление Kubernetes, а в особенности управляемого (managed) Kubernetes в публичных облачных сервисах, привело к тому, что эта операционная модель может использоваться для схем развертывания любых размеров: от развертывания одного контейнерного приложения на одной машине до развертывания нескольких микросервисов, созданных отдельными командами разработчиков, в громадном пуле из 15 000 машин.

Кроме того, Kubernetes позволяет легко обеспечить *высокий уровень доступности* приложения. Как показано на рис. 1.6, один и тот же сервис можно развернуть в нескольких зонах доступности, что позволит избежать простоя в случае выхода из строя одной из этих зон. Такую схему развертывания обычно сложно реализовать при использовании ручных систем развертывания, однако в случае применения Kubernetes достаточно просто создать определение необходимой конфигурации (в данном случае размещения контейнеров в нескольких зонах). Подробнее об этом будет рассказано в подразделе 8.2.1.

Самое приятное здесь в том, что для обновления сервиса достаточно изменить лишь одну строку в файле конфигурации, и Kubernetes автоматически внесет эти изменения в каждой из зон доступности. В такой же автоматизированной манере выполняется обновление и самой платформы (при условии, что вы используете управляемый сервис): система постепенно заменяет узлы их обновленными версиями и переносит на них рабочую нагрузку, таким образом исключая какие-либо простои. Не стоит волноваться и в том случае, если ваше приложение не настолько большое, чтобы нуждаться в высокодоступной многозональной схеме развертывания: Kubernetes позволяет выполнять развертывание небольших масштабов, дополнительно предоставляя возможность произвести масштабирование в случае необходимости.

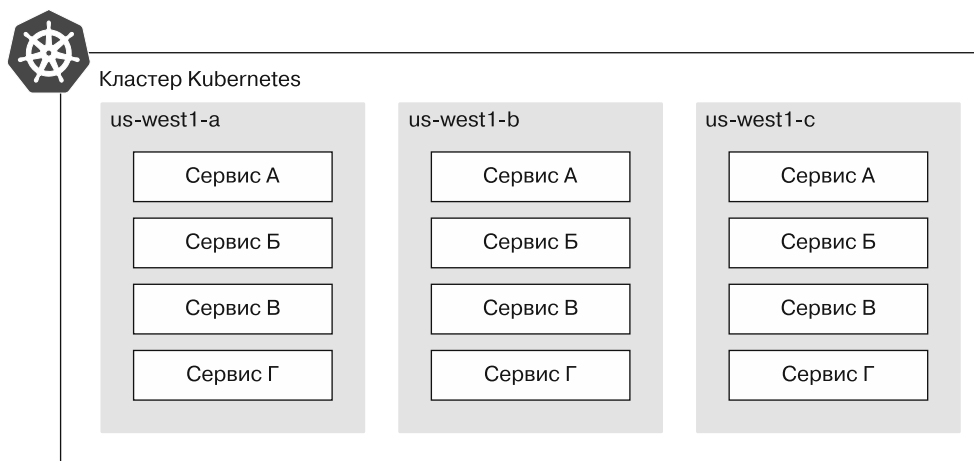


Рис. 1.6. Кластер Kubernetes, управляющий четырьмя сервисами, размещенными в трех зонах доступности

Платформа Kubernetes стала популярной по той причине, что она в значительной мере автоматизирует процесс диспетчеризации и запуска контейнеров в пуле ресурсов и обеспечивает разработчикам хорошо сбалансированный уровень абстракции: не настолько низкий, чтобы нужно было думать об отдельных машинах, и не настолько высокий, чтобы накладывать ограничения на развертываемые рабочие нагрузки.

1.2.1. Составные структурные элементы

В Kubernetes контейнеры группируются в так называемые *поды* (Pods). Под — это просто набор контейнеров, который в ходе диспетчеризации рассматривается как единый блок. Довольно часто под представляет собой лишь один контейнер, но может содержать и несколько контейнеров, если приложение состоит из нескольких взаимосвязанных частей. С концептуальной точки зрения под — это приложение и его зависимости. Используя определенные сервисы, Kubernetes предоставляет возможность как внутреннего (в пределах кластера), так и внешнего подключения к группе подов. Типичный пример того, как могут быть распределены ресурсы приложения, развернутого в кластере Kubernetes, представлен на рис. 1.7.

В Kubernetes есть и несколько инкапсулирующих поды высокоуровневых конструкций для описания рабочих нагрузок, о которых мы еще поговорим. Так, в случае stateless-приложения вам потребуется создать так называемое *Развертывание* (*Deployment*), которое включает в себя описание пода (определяющее используемую

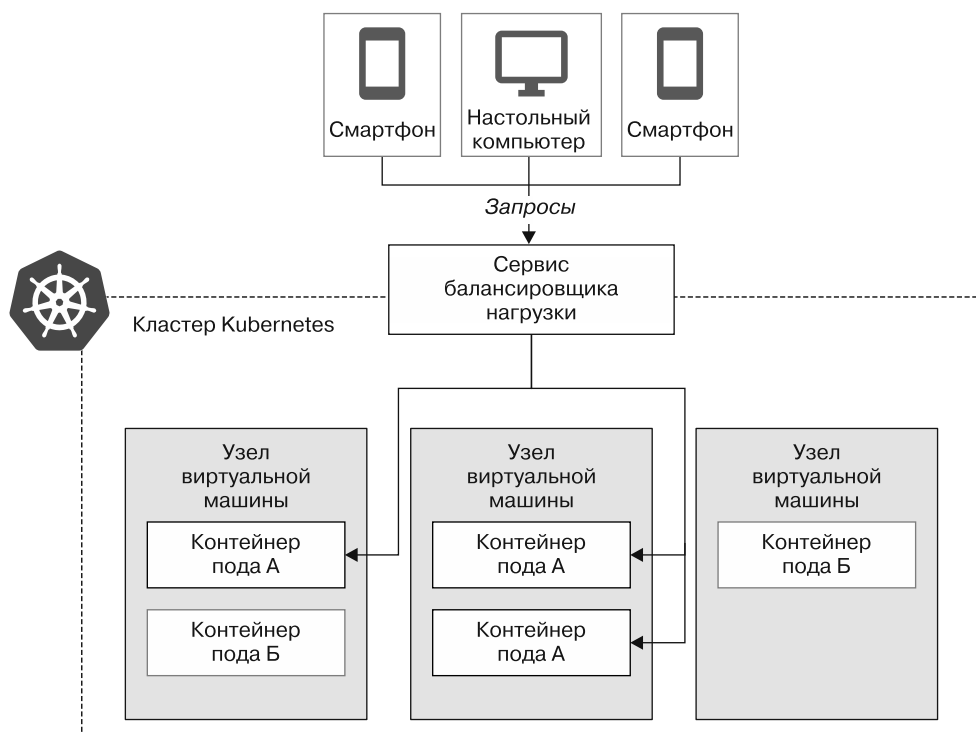


Рис. 1.7. Кластер Kubernetes, в котором два разных контейнера приложения запущены внутри нескольких виртуальных машин и доступны при этом балансировщику нагрузки

версию контейнера), а также нужное вам количество реплик (экземпляров) этого пода. При этом всю работу по поиску в кластере свободного места, где будут размещаться поды в соответствии с вашими требованиями, Kubernetes возьмет на себя.

В файле конфигурации платформы Kubernetes можно определять самые разные виды рабочих нагрузок, в том числе:

- stateless-приложения;
- базы данных и другие приложения, сохраняющие данные на постоянной основе;
- приложения, которые ранее были настроены для работы в виртуальной машине;
- пакетные обработки, которые нужно запускать по определенному графику;
- задание, которое нужно выполнить только один раз, например процедуру обучения модели машинного обучения.

В любом из этих случаев вы должны упаковать приложения в контейнеры и сгруппировать контейнеры в поды, а затем указать в файле конфигурации, как именно платформа Kubernetes должна запускать эту рабочую нагрузку.

1.2.2. Особенности и преимущества Kubernetes

В этом подразделе мы обсудим несколько наиболее веских доводов в пользу применения Kubernetes для развертывания контейнеров.

Автоматизация процессов

При правильной настройке конфигурации развертывания Kubernetes берет на себя выполнение значительной части необходимых операций. Запущенные на узле процессы автоматически перезапускают переставшие работать контейнеры, постоянно контролируя состояние каждого контейнера и его способность к обслуживанию поступающего трафика, используя для этого проверки работоспособности и готовности. При желании вы можете настроить для развертывания механизм автоматического масштабирования подов, который будет автоматически увеличивать количество реплик на основе таких показателей, как уровень использования ресурсов.

Сама платформа Kubernetes не решает проблемы на уровне вычислительных узлов, но если вы выберете использование управляемого сервиса на базе Kubernetes, то вам будет доступна и эта степень автоматизации. Например, при использовании сервиса Google Kubernetes Engine в режиме «Автопилот» (Autopilot) он автоматически выделяет вычислительные мощности для ваших подов, увеличивая или уменьшая их в зависимости от используемого вами количества реплик, а также восстанавливает работоспособность узлов и выполняет их обновление в случае такой необходимости.

Высокая степень масштабируемости

Вне зависимости от размеров вашего приложения вы должны подумать о том, как оно будет масштабироваться в дальнейшем. Какое бы приложение вы ни развертывали: огромное корпоративное или небольшой стартап, — используемое вами решение должно предусматривать возможность масштабирования по мере роста бизнеса. Не стоит думать, что этот вопрос можно отложить до того момента, когда действительно потребуется произвести масштабирование!

Все мы знаем, насколько сложно создать успешный продукт, и вам совсем не нужно, чтобы в момент вашего успеха — когда все захотят опробовать ваш продукт — приложение просто не смогло справиться с возросшей нагрузкой. Если вы начнете думать о масштабировании только в этот момент, то вам не удастся решить данный вопрос сразу, и, возможно, его решение потребует изменения всей архитектуры приложения, что может занять месяцы или годы.

Kubernetes позволяет развертывать приложения любого размера. Вы можете развернуть и небольшой одноузловой кластер с одним процессором и весьма скромным объемом памяти, и громадный кластер, состоящий из многих тысяч узлов с десятками тысяч ядер, как, например, поступила компания Niantic при выпуске на рынок игры Pokemon Go (<https://cloud.google.com/blog/products/gcp/bringing-pokemon-go-to-life-on-google-cloud>). Конечно, при этом придется позаботиться о том, чтобы масштабированию не препятствовали особенности самого приложения и такие его зависимости, как используемые базы данных. Но вы хотя бы будете знать, что по мере роста бизнеса не возникнет никаких проблем с масштабированием вычислительной платформы.

Абстрагирование рабочей нагрузки

Введение новых уровней абстракции полезно лишь до определенного момента, и найти инструмент, который бы абстрагировал именно те детали, которые не представляют для вас важности, не скрывая при этом значимых нюансов, не так просто. Исходя из своего опыта, могу сказать, что Kubernetes представляет собой наилучшее решение в этом плане.

IaaS-платформы (Infrastructure as a Service, инфраструктура как услуга) абстрагируют уровень аппаратного обеспечения. Вместо того чтобы взаимодействовать с реальными машинами с вращающимися дисками и сетевыми картами, вы взаимодействуете с теми API, которые предоставляет вам программное обеспечение, реализующее эти интерфейсы.

В отличие от этого, Kubernetes абстрагирует уровень рабочих нагрузок. Это означает, что вы описываете свое приложение в терминах рабочих нагрузок. Допустим, мне нужно запустить приложение-сервер в распределенной манере, выделить определенный объем дискового пространства для базы данных, запустить на каждом узле утилиту протоколирования или воспроизвести фильм, используя самые дешевые из доступных мне ресурсов. Имеющийся в Kubernetes набор конструкций позволяет легко представить все эти и многие другие варианты развертывания.

Предоставляя вам уровень, расположенный выше физических компьютеров или виртуальных машин, Kubernetes избавляет вас от необходимости управлять отдельными машинами и вообще думать о них. Достаточно указать, какие ресурсы требуются вашему контейнеру: в какой мере ему нужны центральный процессор, память, дисковое пространство, графический процессор и т. д. Кроме того, управляемый сервис на базе Kubernetes обычно предоставляет и вычислительные мощности, которые требуются для функционирования рабочих нагрузок. Избавляя вас от необходимости думать об отдельных машинах, Kubernetes позволяет совершать выполняемые на этом уровне операции, такие как запись данных в персистентное хранилище, что раньше было невозможно на этом уровне абстракции.

Вдобавок предлагаемый уровень абстракции остается достаточно чистым благодаря тому, что платформа отделена от приложения (рис. 1.8). В отличие от многих традиционных PaaS-платформ, Kubernetes оставляет способ запуска приложения прежним, не прибегая к внедрению или изменению кода, и накладывает очень мало ограничений в плане того, что может делать приложение. Если приложение может работать в контейнере, то оно, скорее всего, сможет работать и в Kubernetes.



Рис. 1.8. Разделение ответственности между разными вычислительными слоями

Декларативная конфигурация

Kubernetes использует декларативную модель ресурсов. Вы описываете свою рабочую нагрузку в файлах конфигурации (в большинстве своем это файлы формата YAML), и система стремится реализовать данную конфигурацию на практике. Например, если вы укажете в определении развертывания, что вам нужны три реплики (копии) приложения, подключенные к внешнему миру через балансировщик нагрузки, то платформа найдет в вашем кластере место для запуска этих трех реплик и подключения балансировщика нагрузки. Kubernetes не только обеспечит первоначальное развертывание этих трех реплик, но и проследит за тем, чтобы они оставались в рабочем состоянии, перезапуская их заново в случае сбоя или ошибки.

У такого декларативного подхода к определению конфигурации есть преимущество. Вы можете описать нужное вам состояние (такое как запуск трех копий приложения) и возложить ответственность за обеспечение этого состояния на Kubernetes, вместо того чтобы самому выполнять императивные команды (например, для создания трех копий приложения) и проводить дальнейшее отслеживание и корректировку (например, время от времени выяснять, сколько копий приложения работает в текущий момент, и при необходимости изменять их количество).

Оптимизация затрат

Kubernetes облегчает использование самых низкоуровневых структурных элементов вычислительной системы (виртуальных машин). Раньше вы могли выделять каждому приложению отдельную виртуальную машину просто для того, чтобы упростить техническое обслуживание. Теперь же с помощью Kubernetes можно легко снизить уровень затрат, разместив на одной машине несколько экземпляров приложения или несколько приложений, используя метод упаковки в контейнеры (bin-packing). Применение стандартных структурных элементов (вычислительных узлов) в сочетании с продвинутыми возможностями оркестрации рабочих нагрузок часто делает платформу привлекательной с точки зрения оптимизации затрат.

Снижению затрат в Kubernetes способствует не только упаковка в контейнеры, но и объединение ресурсов. Конфигурацию рабочих нагрузок можно настроить так, чтобы они получали некий гарантированный набор ресурсов и при резком возрастании степени использования задействовали ресурсы, зарезервированные для других контейнеров, но в данный момент не применяемые ими.

Расширяемость

Если Kubernetes не может сделать то, что вам нужно, то вы можете подключить сторонний код или даже самостоятельно написать API в стиле Kubernetes, чтобы реализовать данную возможность. Конечно, этот вариант подходит не всем и совершенно не требуется при развертывании типичных рабочих нагрузок наподобие веб-приложения с сохранением или без сохранения состояния, но может быть очень полезным в том случае, когда нужно добавить определенный вид бизнес-логики или отсутствующую в Kubernetes конструкцию. В таком случае можно создать собственный API в стиле Kubernetes, задействуя пользовательский объект определения ресурсов (Custom Resource Definition) и шаблон оператора (Operator Pattern).

Свободно распространяемая платформа

Kubernetes — свободно распространяемая платформа, которая доступна в виде управляемого сервиса на всех крупных облачных платформах. Существует множество различных платформ, дистрибутивов и установщиков, однако практически все эти продукты сертифицированы в рамках программы сертификации Cloud Native Computing Foundation (<https://www.cncf.io/certification/software-conformance/>), что гарантирует соответствие нескольким требованиям в плане переносимости и совместимости рабочих нагрузок. На самом деле официальную проверку на соответствие этим требованиям проходят все продукты, в названии которых присутствует слово *Kubernetes* (как, например, в случае сервиса Google Kubernetes Engine).

Вы можете запустить Kubernetes с нуля самостоятельно. В таком случае для вас важную роль играет качество исходного кода, поскольку многие решения с открытым

исходным кодом часто далеки от идеала в этом плане. Использование свободно распространяемого продукта позволяет не зависеть от конкретного поставщика, в конечном счете может возникнуть необходимость самостоятельно заниматься технической поддержкой (по принципу «любишь кататься, люби и саночки возить»), если за этим продуктом не стоит мощное сообщество. Исключение составляют крупные открытые проекты с мощной поддержкой — такие как Linux, которыми пользуется так много людей, что вы можете быть уверенными в том, что вам не придется решать все проблемы технической поддержки самостоятельно. К счастью, будучи наиболее продвинутым оркестратором контейнеров, Kubernetes относится к числу как раз таких продуктов.

СОВЕТ

Вы можете развернуть Kubernetes самостоятельно хоть в общедоступном облаке, хоть в кластере из устройств Raspberry Pi, но я не рекомендую делать это для продуктивной среды (разве что в том случае, если вы хотите научиться управлять кластером). Лучше занимайтесь тем, что у вас получается лучше всего: разработкой классных приложений. А такие рутинные задачи, как запуск и обслуживание платформы Kubernetes, пусть выполняет кто-то другой.

Благодаря наличию активного сообщества Kubernetes вы получаете не только свободно распространяемую платформу, но и множество бесплатных сопутствующих инструментов. Они позволяют реализовать практически любые варианты развертывания, поэтому в большинстве случаев у вас будет выбор между использованием управляемого сервиса и самостоятельным развертыванием свободно распространяемого инструмента. Этим Kubernetes выгодно отличается от традиционных PaaS-систем, которые позволяли дополнительно приобретать только платные компоненты, доступные лишь для частных лиц. Вы считаете, что в вашем случае будет полезен управляемый мониторинг? Используйте проприетарный продукт. Хотите выполнять мониторинг самостоятельно? Установите свободно распространяемое приложение Prometheus. Кроме того, количество специалистов, хорошо разбирающихся в особенностях платформы, постоянно растет, поэтому при возникновении каких-либо проблем вы всегда найдете ответы на свои вопросы на сайте Stack Overflow или в книгах, посвященных Kubernetes.

Гибкая организация процесса разработки

Kubernetes отличается гибкостью и в плане организации процесса разработки. Хотите организовать процесс так, чтобы можно было выполнять развертывание с помощью команды `git push`? Существует несколько способов сделать это, и некоторые из них практически не требуют настройки. Обычно вы изначально получаете некий набор структурных элементов непрерывной интеграции и развертывания, из которых можно составить нужный вам рабочий процесс: от простейшего процесса, развертывание в котором выполняется с помощью одного щелчка

кнопкой мыши, до чрезвычайно сложных процессов, в ходе которых выполняется контроль доступа, автоматическое внедрение секретов и сканирование на предмет уязвимостей. Отрицательным моментом является то, что вы не можете сразу приступить к работе, как, например, в случае традиционной PaaS-платформы, но, как будет показано в книге, начальная настройка Kubernetes требует *не слишком* больших усилий.

Степень гибкости, которую обеспечивает Kubernetes в плане организации рабочего процесса, часто является огромным преимуществом в случае большой команды разработчиков: в крупных компаниях разработчики базовой платформы часто задают жестко определенные процессы для разработчиков приложений, чтобы гарантировать применение конкретных методов обеспечения безопасности, эффективное использование ресурсов и т. д.

1.2.3. Отличия Kubernetes от PaaS-платформы

Альтернатива использованию Kubernetes — развертывание приложений с помощью PaaS-платформы. Такая платформа позволяет легко выполнять крупномасштабное развертывание приложений, избавляя вас от необходимости заботиться о различных аспектах процесса упаковки и развертывания. Соблюдая требования PaaS-платформы по части используемых языков, зависимостей, обработки состояния и т. д., вы можете запустить в PaaS-среде любое приложение, не волнуясь о том, какие машины используются на базовом уровне.

Но что произойдет, если вам нужно использовать специализированные зависимости, например, определенную версию языка Java? Сможете ли вы разместить в одной среде и бэкенд с сохранением состояния, и фронтенды без сохранения состояния? И насколько затратным будет развертывание большого количества приложений, у каждого из которых должно быть по несколько реплик? На определенном этапе ограничения PaaS-платформы могут оказаться непреодолимыми, и в этом случае вам придется отказаться от ее использования и начать все заново, что обычно является пугающей перспективой!

Традиционные PaaS-платформы, как правило, легки в освоении, но мало что могут предложить продвинутому пользователю, поэтому при использовании такой системы вы в итоге можете прийти до предела ее возможностей, после чего вам придется начинать все сначала. Kubernetes требует чуть больше времени на освоение, но предлагает все больше возможностей по мере вашего прогресса (рис. 1.9).

Если вы используете PaaS-платформу и не испытываете при этом никаких проблем, то, возможно, вам и не нужно переходить на Kubernetes. Однако мне не раз приходилось наблюдать ситуацию, когда команда разработчиков достигала того уровня сложности, при котором требования к их приложению начинали превосходить возможности PaaS-платформы. И что самое неприятное — вы не можете

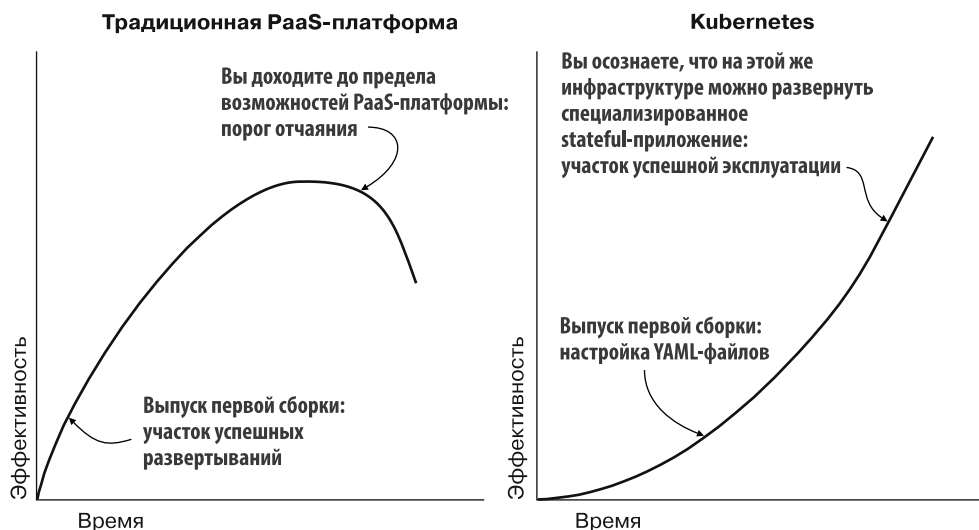


Рис. 1.9. Эффективность работы разработчика при использовании традиционной PaaS-платформы и Kubernetes

просто взять на себя часть тех функций, которые раньше выполняла платформа. В таком случае часто приходится менять всю архитектуру системы, отказываясь даже от тех компонентов, которые вполне вас устраивали, чтобы создать новые компоненты, которые необходимо добавить. В книге я постараюсь показать, что запустить в Kubernetes типичные для PaaS-среды рабочие нагрузки ненамного сложнее, чем в случае использования специально выделенной PaaS-платформы. Кроме того, вы узнаете, как запускать другие виды рабочих нагрузок, в том числе рабочие нагрузки с сохранением состояния, фоновую обработку и пакетные задания, что позволит вам в дальнейшем легко справляться с реализацией более сложных требований к продукту.

Некоторые более современные PaaS-платформы поддерживают контейнеры, что позволяет вам одновременно получить и гибкость, возможную благодаря контейнерам, и простоту процесса развертывания. Однако даже эти более продвинутые PaaS-платформы накладывают множество ограничений в отношении типов запускаемых рабочих нагрузок. Например, сможет ли такая платформа запустить stateful-приложение с блочным хранилищем, если вам потребуется произвести миграцию устаревшего приложения, или запустить специализированную базу данных, если вы не найдете ее аналог в виде управляемого сервиса? Я рекомендую внимательно рассмотреть ваши текущие и будущие потребности и выбрать ту платформу, которая будет способна расти и масштабироваться по мере развития вашего бизнеса.

ПАРУ СЛОВ О ПРОСТОТЕ

Я часто говорю своим ученикам: «Избегайте инструментов, которые делают простое более легким, а сложное — более тяжелым». Конечно, прекрасно, когда инструмент помогает вам быстрее начать работу, но каково ваше состояние после этого? Достаточно ли у вас знаний и инструментов для выполнения стоящей перед вами задачи? При выборе платформы нужно, прежде всего, следить за тем, чтобы выбранный инструмент позволял выполнять сложные задачи, а не делал простые задачи еще более простыми.

Kubernetes позволяет достаточно легко начать работу и в то же время предоставляет весьма мощные возможности для дальнейшего развития и расширения. Вы можете начать с запуска простого 12-факторного stateless-приложения, а затем мигрировать на заказное stateful-приложение, установленное ранее на виртуальной машине, или даже запустить собственную базу данных. Уровень абстракции, предлагаемый этой платформой, никак не ограничивает ваши возможности, в то же время позволяя вам начать работу, даже если у вас есть лишь минимальный необходимый набор элементов.

1.2.4. Когда стоит использовать Kubernetes, а когда — нет

Как и любой другой инструмент, эта платформа была создана затем, чтобы вы могли более эффективно выполнять свою работу — в данном случае управлять развертыванием приложений. Постарайтесь абстрагироваться от связанного с ней ажиотажа и подумайте о том, будет ли ее применение действительно полезным в вашем случае. Используя один из управляемых сервисов на базе Kubernetes, вы можете легко обеспечить бесперебойную работу компонентов своего кластера. Но помните о том, что использование такой универсальной платформы, как Kubernetes, влечет некоторые накладные расходы. Чтобы приложение работало, платформа должна выделять контейнерам ресурсы процессора и памяти, обновлять схемы развертывания, корректировать конфигурацию сети и поддерживать все в актуальном состоянии, не нарушая функционирование запущенных сервисов.

Допустим, вы можете точно оценить объем своих текущих и будущих бизнес-требований и не нуждаетесь в той гибкости и независимости от конкретного поставщика, которую предоставляет платформа Kubernetes и ее экосистема, а архитектура вашего приложения не выходит за рамки ограничений, накладываемых более специализированной платформой. В таком случае, конечно, лучше выбрать такую платформу. Возможно, в вашем случае это будет проще.

Обычно я не рекомендую использовать Kubernetes для запуска программного обеспечения, существующего в виде полностью управляемой версии. Например,

зачем разворачивать в Kubernetes реляционную базу данных, если эту задачу может решить поставщик облачных услуг? Конечно, из данного правила бывают исключения, поскольку иногда процессом лучше управлять самостоятельно, но в целом я рекомендую всегда по возможности использовать управляемый сервис.

В то же время Kubernetes *очень хорошо* справляется с такими задачами, как:

- запуск stateless-приложений с высокой плотностью размещения;
- совместный запуск разнородных рабочих нагрузок, например, современных stateless-приложений и устаревших монолитных stateful-приложений;
- миграция сервисов с устаревших систем на унифицированную платформу;
- выполнение высокопроизводительных вычислений, например, пакетных заданий для анализа данных и машинного обучения;
- и конечно, запуск большого набора микросервисов.

В каждом из этих случаев использование Kubernetes принесет много выгод, позволяя повысить эффективность работы за счет применения унифицированной платформы хостинга, автоматизации используемых систем и выполнения пакетных заданий.

Помните о том, что использование Kubernetes влечет дополнительный уровень накладных расходов на обслуживание системы. Если вы решите просто перебросить в Kubernetes текущую (нормально работающую) версию своей системы, то вместо одних проблем можете получить другие. В частности, переходить на Kubernetes, возможно, не стоит, если речь идет о переходе с платформы без сохранения состояния, которая и так удовлетворяет вашим требованиям, или переносе на Kubernetes стандартных рабочих нагрузок с сохранением состояния, для которых выработаны устоявшиеся схемы разворачивания, например, реляционных баз данных. В таких случаях переход на Kubernetes дает не так уж много преимуществ, поэтому вы должны тщательно взвесить все плюсы и минусы.

При принятии этого решения я рекомендую, с одной стороны, подумать о том, какие преимущества вам дает использование контейнеров и унифицированной платформы разворачивания, подходящей для самых разных видов рабочих нагрузок, а с другой — о том, какой объем дополнительных знаний вам придется усвоить, чтобы успешно управлять платформой Kubernetes. Если ваша отправная точка — набор сервисов, каждый из которых работает в отдельной виртуальной машине и требует определенной корректировки, то сделать этот выбор будет несложно. Не стоит колебаться и в том случае, если вы уже переросли возможности своей PaaS-платформы или ваша команда разработчиков (чрезвычайно компетентная) хочет ускорить процесс разворачивания за счет использования современных инструментов. Но если вы имеете дело с кластером MySQL, который работает в специально настроенной среде с практически стопроцентной степенью надежности, то, возможно, этот кластер лучше оставить в его текущем виде.

Переходить к использованию Kubernetes необязательно по принципу «все или ничего». Я рекомендую для начала выбрать те рабочие нагрузки, для которых такой переход будет наиболее оправдан, и постепенно выполнять их миграцию по мере освоения вами возможностей платформы.

РЕЗЮМЕ

- Контейнеры — современное средство запуска приложений, которое обеспечивает изоляцию между приложениями, запущенными на общем хосте, и влечет более низкий уровень накладных расходов по сравнению с виртуальными машинами.
- Kubernetes — платформа развертывания для контейнеризированных приложений.
- Данная платформа достаточно сложна в освоении, но зато позволяет представить самые разные варианты развертывания и берет на себя заботы о настройке инфраструктуры и поддержании приложений в рабочем состоянии.
- Такие управляемые сервисы, как Google Kubernetes Engine, снимают с вас бремя управления платформой Kubernetes, позволяя сосредоточить внимание на способе развертывания приложений.
- Разработчикам приложений достаточно корректно описать конфигурацию приложения в терминах платформы Kubernetes, после чего система проследит за тем, чтобы приложение работало должным образом.
- Главным преимуществом Kubernetes является то, что платформа позволяет дорабатывать приложение по мере необходимости. Вам вряд ли потребуется менять ее при изменении требований, например, если возникнет необходимость в том, чтобы приложение имело локальное состояние.
- Когда приложение необходимо масштабировать из-за увеличения потребности в ресурсах, Kubernetes позволяет сделать это наиболее эффективно.

2

Контейнеризация приложений

В этой главе

- ✓ Как контейнеризировать приложение.
- ✓ Локальный запуск контейнера.
- ✓ Выполнение команд в контексте контейнера.

Чтобы приложение можно было развернуть на платформе Kubernetes, его сначала нужно контейнеризировать, то есть упаковать вместе с зависимостями в исполняемый контейнер. Радует, что эта операция не только делает возможным развертывание приложения в Kubernetes, но и полезна сама по себе, позволяя запускать приложение на любой машине и не устанавливать на ней зависимости.

Какой бы способ развертывания вы ни использовали, контейнеризация приложения позволяет разработчикам работать над ним локально, используя Docker. Это означает, что настройка среды для каждого нового проекта сводится лишь к установке Docker. В свою очередь, это позволяет легко переключать контекст при разработке нескольких приложений, поскольку среда каждого приложения полностью изолируется от других сред (рис. 2.1). Это делает контейнеризацию эффективным методом повышения продуктивности разработчиков, даже если вы не будете использовать контейнеры при окончательном развертывании приложения в продуктивной среде (хотя обычно контейнеры используются и в ней).

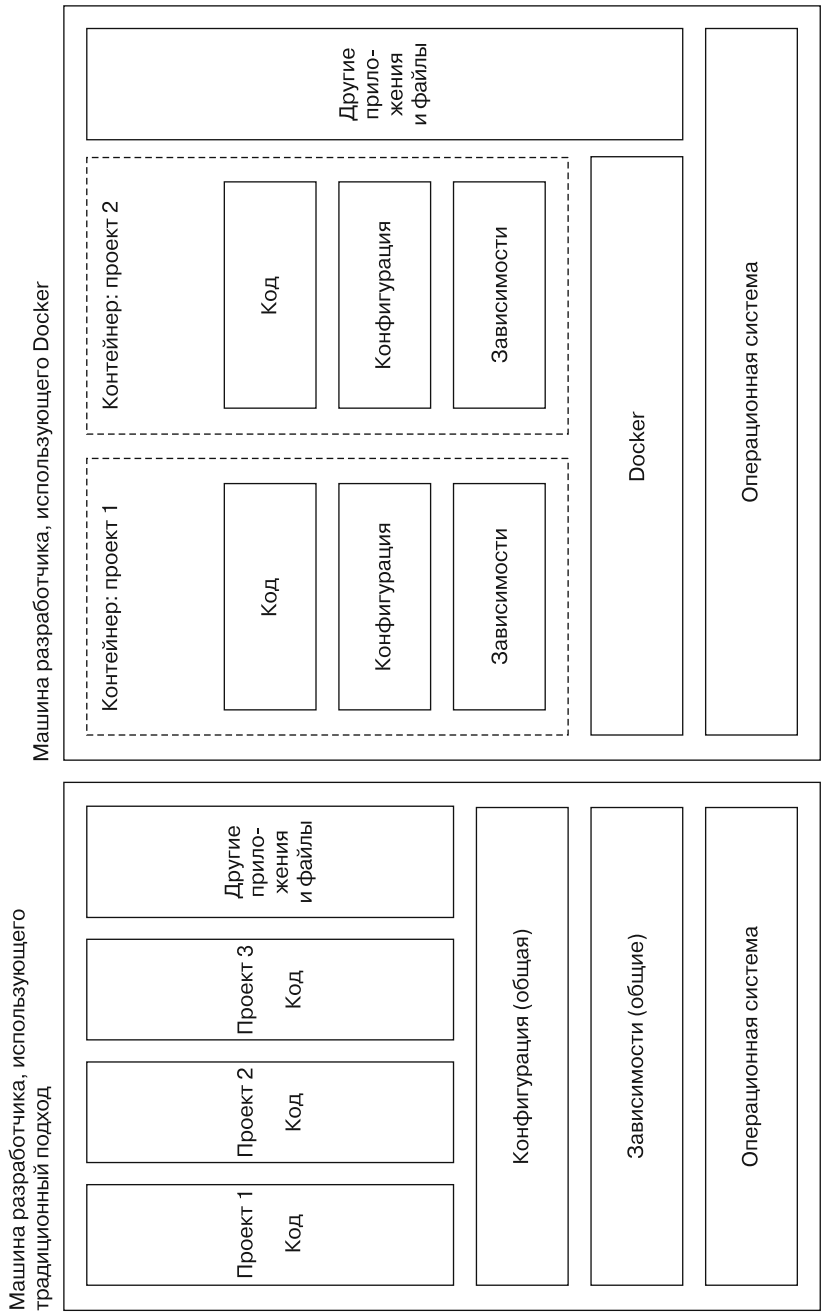


Рис. 2.1. Сравнение способов организации нескольких проектов на машине разработчика при традиционном подходе и при использовании контейнеров

При упаковке приложений в контейнеры вы определяете все свои зависимости и конфигурации в файле конфигурации контейнера — файле `Dockerfile` — вместо того, чтобы одновременно использовать для этого `bash`-скрипты, текстовые инструкции, какие-либо нестандартные системы настройки и просто возможности человеческой памяти. Это также позволяет вам развернуть множество приложений на одной хост-машине. При этом полностью исключается вероятность того, что они будут влиять друг на друга, и одновременно обеспечивается больший уровень производительности и меньший уровень накладных расходов, чем в случае полной виртуализации.

2.1. СОЗДАНИЕ DOCKER-КОНТЕЙНЕРОВ

Теперь поместим в контейнер какое-нибудь приложение.

2.1.1. Настройка среды разработчика

Docker представляет собой инструмент разработчика, доступный на большинстве платформ в виде приложения Docker Desktop (<https://www.docker.com/products/docker-desktop>), которое содержит такие полезные дополнения, как локальная Kubernetes-среда (о которой мы поговорим в главе 3). При использовании операционной системы Linux (в том числе с помощью подсистемы WSL в Windows) вы также можете установить отдельно только движок Docker Engine.

macOS

При использовании macOS нужно просто установить приложение Docker Desktop.

Windows

Если у вас ОС Windows, то я настоятельно рекомендую сначала установить подсистему WSL (Windows Subsystem for Linux, подсистема Windows для Linux) (<https://learn.microsoft.com/en-us/windows/wsl/install>). Для совместимости с Docker нужно установить вторую версию этой подсистемы, WSL 2. Затем установите такой дистрибутив Linux, как Ubuntu (<http://mng.bz/pP40>); это позволит вам использовать командную оболочку `bash` и упростит запуск представленных в этом разделе примеров. Затем установите приложение Docker Desktop.

Linux

Если вы используете Linux, то вам доступен и другой вариант установки Docker: в виде движка Docker Engine. Как это можно сделать в случае различных дистрибутивов, например Ubuntu, описано на <https://docs.docker.com/engine/install/ubuntu/>. Движок Docker Engine можно установить и при использовании Linux с помощью подсистемы WSL в Windows.

2.1.2. Выполнение команд в Docker

Чтобы посмотреть, как работает Docker, еще до создания собственного контейнера приложения запустим в Docker контейнеризированную версию командной оболочки Linux:

```
$ docker run -it ubuntu bash
root@18e78382b32e:/#
```

Эта команда скачивает базовый образ `ubuntu`, запускает контейнер и выполняет в нем команду `bash`. Параметр `-it` указывает, что оболочку `bash` нужно открыть в режиме интерактивного терминала. Теперь все вводимые нами команды будут выполняться внутри контейнера.

Мы собираемся выполнять сборку приложения в Ubuntu, поэтому установим пакет языка программирования. В нашем случае это будет пакет языка Python, поскольку данный язык будет использоваться во многих примерах текущей главы, но в случае применения других языков действовать нужно точно так же.

Выполните в консоли контейнера следующие две команды:

```
apt-get update
apt-get install -y python3
```

Теперь попробуйте выполнить команду языка Python в интерактивном режиме:

```
# python3
>>> print("Hello Docker")
Hello Docker
>>> exit()
#
```

Эту простую команду можно поместить в собственный Python-скрипт:

```
# echo 'print("Hello Docker")' > hello.py
# python3 hello.py
Hello Docker
```

Поэкспериментировав с контейнером, выйдите из него с помощью команды `exit`.

Приятным моментом здесь является то, что мы установили Python и выполнили команду этого языка в контейнере, а не в своей локальной системе. С помощью команды `run` платформы Docker мы создали *контейнер* на основе имеющегося у нас *образа*. Этот образ, `ubuntu`, представляет собой готовую файловую систему для запуска контейнеризированного процесса. При выходе из интерактивного терминала контейнер прекращает работать, но его можно легко запустить снова. Для этого нужно получить идентификатор контейнера с помощью команды `docker ps -a`,

запустить контейнер, используя команду `docker start $CONTAINER_ID`, и снова подключить командную оболочку с помощью команды `docker attach $CONTAINER_ID`:

```
$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
c5e023cab033   ubuntu   "bash"                  5 minutes ago   Exited (0) 1 second ago

$ CONTAINER_ID=c5e023cab033
$ docker start $CONTAINER_ID
$ docker attach $CONTAINER_ID
# echo "run more commands"
# exit
```

Запуск большого количества Docker-контейнеров в итоге приведет к тому, что у вас будет много остановленных контейнеров (которые будут занимать большой объем дискового пространства). Сохранять эти образы, как правило, не нужно, поэтому их в любой момент можно удалить с помощью следующей команды:

```
docker system prune -a
```

ОБРАЗ КОНТЕЙНЕРА И ЭКЗЕМПЛЯР КОНТЕЙНЕРА

В терминологии платформы Docker *образ контейнера* — это файловый артефакт (который можно скачать из реестра, как было сделано в данном разделе, или создать локально), а *экземпляр контейнера* (или просто контейнер) — это вызов (запуск) контейнера. В Kubernetes файлы конфигурации ссылаются только на образы контейнеров, а экземпляры контейнеров создаются во время выполнения и являются временными по своей природе (удаляются после остановки пода). Концепция экземпляра становится важной при локальном использовании Docker главным образом потому, что в таком случае каждый вызов создает долгоживущий экземпляр контейнера и их приходится в итоге удалять, чтобы освободить дисковое пространство.

Выполнение описанных выше действий позволяет получить Linux-среду, в которой можно будет тестировать и выполнять любые команды, не устанавливая на локальной машине что-либо еще, помимо Docker. Вы хотите получить две контейнеризированные Linux-среды с разной конфигурацией? Нет ничего проще — запустите еще один контейнер!

Если вам когда-либо приходилось выполнять начальную настройку виртуальной машины, то вы можете оценить, насколько быстро она выполняется в данном случае. Как мы видим, процесс создания контейнеров чрезвычайно прост. Как будет показано в следующем подразделе, так же прост и процесс их дальнейшей доработки и расширения.

2.1.3. Создание собственного образа

В предыдущем подразделе мы вручную запустили Linux-контейнер, установили язык Python, создали простой Python-скрипт и запустили его в контейнере. Теперь сделаем эту последовательность действий воспроизводимой автоматически. Для этого нужно определить конфигурацию контейнера (с установленным языком Python) и приложение (Python-скрипт) в собственном образе контейнера. Такой образ избавил бы нас от необходимости каждый раз вспоминать нужную последовательность действий и, кроме того, позволил бы другим людям выполнять сборку нашего замечательного приложения.

Хотя в данном примере используется простейший Python-скрипт, здесь с тем же успехом можно задействовать достаточно большое и сложное приложение. Кроме того, вместо Python можно использовать любой другой интерпретируемый язык (как следует поступать в случае компилируемых приложений, будет показано в подразделе 2.1.7). Для этого нужно просто заменить в конфигурации язык Python тем языком, который вы используете.

Сборка образа контейнера, позволяющего многократно воспроизводить необходимую схему развертывания приложения, выполняется с помощью конфигурационного файла `Dockerfile`, который содержит набор процедурных инструкций по сборке контейнера. В принципе, можно считать, что это что-то вроде `bash`-скрипта, который настраивает конфигурацию образа виртуальной машины с приложением и его зависимостями и выдает результат в виде образа контейнера.

ВЫПОЛНЕНИЕ ПРИМЕРОВ

Используемые в книге примеры Docker-приложений и конфигураций Kubernetes размещены в соответствующем репозитории исходного кода. Клонировать этот репозиторий и перейти в его корневой каталог можно с помощью следующих команд:

```
git clone https://github.com/WilliamDenniss/kubernetes-for-developers.git
cd kubernetes-for-developers
```

Примеры кода разбиты по главам и разделам. Например, примеры из главы 2 находятся в папке `Chapter02`, а примеры из подраздела 2.1.3 — в папке `2.1.3_Dockerfile`. При этом в заголовке каждого листинга в книге указывается путь к соответствующему файлу, чтобы вам было проще его найти.

Используемые в книге консольные команды должны запускаться из корневого каталога примеров кода (то есть каталога `kubernetes-for-developers`, если вы клонировали репозиторий, как было показано выше), поэтому вы должны возвращаться в этот каталог перед выполнением каждого следующего примера.

Мы начнем с той простейшей Python-программы, которую создали в предыдущем подразделе (листинг 2.1).

Листинг 2.1. Chapter02/2.1.3_Dockerfile/hello.py

```
print("Hello Docker")
```

Чтобы собрать образ контейнера для этого скрипта, мы должны создать файл Dockerfile, выбрать в качестве основы некий базовый образ контейнера, настроить конфигурацию языка Python и добавить программу. Пока в качестве основы мы будем использовать универсальный базовый образ `ubuntu`, который предоставляет контейнеризированную Linux-среду. В листинге 2.2 представлена базовая версия файла Dockerfile, определяющего эти шаги.

Листинг 2.2. Chapter02/2.1.3_Dockerfile/Dockerfile

```
FROM ubuntu
RUN apt-get update
RUN apt-get install -y python3
COPY . /app
WORKDIR /app
```

Определяем базовый образ контейнера (points to FROM ubuntu)

Задаем конфигурацию среды (points to RUN apt-get install -y python3)

Копируем приложение в контейнер (points to COPY . /app)

Задаем текущий рабочий каталог (points to WORKDIR /app)

Выполним сборку этого контейнера, присвоив ему имя `hello`:

```
cd Chapter02/2.1.3_Dockerfile/
docker build . -t hello
```

Теперь выполним внутри контейнера `hello` команду `python3 hello.py`:

```
$ docker run hello python3 hello.py
Hello Docker
```

Как можно заметить, в своем Dockerfile мы используем практически те же команды, которые применялись в предыдущем подразделе. Сначала, вместо того чтобы запускать образ контейнера `ubuntu` напрямую, мы выбираем его в качестве базового образа для файла Dockerfile. Затем, как и раньше, выполняем две команды `apt-get`, чтобы установить Python, копируем в образ свой Python-скрипт и, наконец, задаем расположение рабочего каталога по умолчанию, в котором будут выполняться команды. Кроме того, можно заметить, что для запуска своего кода мы по-прежнему используем команду `python3 hello.py`, просто указав с помощью префикса, что она должна выполняться внутри вновь созданного образа контейнера.

Таким образом, мы упаковали созданную нами среду и свой скрипт в аккуратный пакет, который сможем в дальнейшем запускать сами и предоставлять другим людям. Большим плюсом контейнеров является то, что они позволяют инкапсулировать операции настройки среды вместе с самой программой. Особенно приятно,

что если изменится базовый образ `ubuntu`, то можно будет собрать образ контейнера заново, просто еще раз выполнив приведенную выше команду `build`.

Сравним это с тем случаем, когда Python устанавливается на машину разработчика и все команды выполняются на локальной машине. Если бы мы поступили так, то нам пришлось бы установить на своей машине язык Python. Конечно, в этом нет ничего страшного, но что, если бы речь шла о более сложном приложении, которому требуются десятки различных инструментов и библиотек? Вы бы захотели засорять свою систему всеми этими дополнениями? И что, если речь будет идти о разработке нескольких разных приложений, которые имеют разные зависимости или, имея одну и ту же зависимость, нуждаются в разных ее версиях, что не позволяет устанавливать все их зависимости одновременно (то есть вы столкнетесь с так называемым «адом зависимостей»)?

Контейнеры избавляют вас от этих проблем, изолируя каждое приложение с его зависимостями в отдельном образе контейнера. Вы можете работать над несколькими проектами одновременно, делиться файлами `Dockerfiles` с другими разработчиками и загружать образы контейнеров в свою рабочую среду, ничего не меняя на своей машине разработчика.

При использовании другого языка, например Ruby, настройка образа выполняется практически так же (листинги 2.3, 2.4).

Листинг 2.3. `Chapter02-ruby/2.1.3_Dockerfile/hello.rb`

```
puts "Hello Docker"
```

Листинг 2.4. `Chapter02-ruby/2.1.3_Dockerfile/Dockerfile`

```
FROM ubuntu
RUN apt-get update
RUN apt-get install -y ruby
COPY . /app
WORKDIR /app
```

На этапе запуска контейнера по-другому будет выглядеть лишь передаваемая в контейнер команда:

```
$ cd Chapter02-ruby/2.1.3_Dockerfile
$ docker build . -t hello_ruby
$ docker run hello_ruby ruby hello.rb
Hello Docker
```

2.1.4. Использование базовых образов

В предыдущем подразделе мы настроили конфигурацию своего Linux-приложения, используя в качестве основы Linux-контейнер `ubuntu`. Такие дистрибутивы Linux, как `Ubuntu`, `CentOS` и `Alpine`, могут служить хорошей отправной точкой при определении конфигурации любого Linux-приложения. Но чтобы упростить вам жизнь, разработчики контейнеризированных решений создали несколько более

специализированных базовых образов, предполагающих использование конкретного языка и среды.

Вместо того чтобы устанавливать Python поверх базового образа `ubuntu`, можно просто взять за основу образ `python`, тем самым сократив количество необходимых действий. У такого подхода есть еще один плюс: этот базовый образ будет хорошо настроен для запуска Python-приложений, поскольку его создавали высококлассные специалисты. Настройка того же контейнера при использовании в качестве основы базового образа `python` показана в листинге 2.5.

Листинг 2.5. Chapter02/2.1.4_BaseImage/Dockerfile

```
FROM python:3
COPY . /app
WORKDIR /app
```

Не правда ли, так проще? Операции сборки и запуска кода при этом выполняются по-прежнему:

```
$ cd Chapter02/2.1.4_BaseImage
$ docker build . -t hello2
$ docker run hello2 python3 hello.py
Hello Docker
```

ЧТО ТАКОЕ БАЗОВЫЙ ОБРАЗ НА САМОМ ДЕЛЕ

Используемый в данном примере базовый образ `python` тоже был собран с помощью файла `Dockerfile` и выполняет настройку среды так, чтобы она содержала все, что необходимо для запуска программ на Python.

К каждому образу контейнера в репозитории Docker Hub прилагается соответствующий файл `Dockerfile`, поэтому вы всегда можете посмотреть, что содержит тот или иной образ. Внутри одного образа может быть другой, тот, в свою очередь, может содержать еще один и т. д., пока наконец вы не дойдете до совершенно пустого образа контейнера с именем `scratch`.

Если вы используете не Python, а Ruby, то процедура настройки будет выглядеть практически так же. Просто используйте базовый образ `ruby`, как показано в листингах 2.6 и 2.7.

Листинг 2.6. Chapter02-ruby/2.1.4_BaseImage/hello.rb

```
puts "Hello Docker"
```

Листинг 2.7. Chapter02-ruby/2.1.4_BaseImage/Dockerfile

```
FROM ruby
COPY . /app
WORKDIR /app
```


Собрать и запустить этот контейнер можно так:

```
$ cd Chapter02-ruby/2.1.4_BaseImage
$ docker build . -t hello_ruby2
$ docker run hello_ruby2 ruby hello.rb
Hello Docker
```

Базовый образ может быть рассчитан не только на использование определенной операционной системы и языка программирования. Например, если среда должна содержать сервер Apache, то за основу можно взять базовый образ `httpd`. В некоторых случаях для применения в качестве основы будут подходить сразу несколько базовых образов. В таком случае будет лучше взять за основу тот образ, который позволяет избавиться от наибольшего количества операций настройки (и скопировать оставшиеся операции из файлов `Dockerfile` других образов).

При желании можете найти базовый образ (или, по крайней мере, общедоступный пример такого образа) для практически каждого распространенного языка программирования, среды исполнения или приложения с открытым исходным кодом. Прежде чем приступить к сборке собственного образа с нуля, всегда старайтесь найти в Docker Hub или в Google пример образа для вашего варианта среды, который можно было бы использовать в качестве отправной точки.

2.1.5. Добавление команды по умолчанию

Обычно в контейнере всегда требуется выполнять одну и ту же команду (например, в рассмотренном выше примере в контейнере всегда выполняется команда `python3 hello.py`). Вместо того чтобы вводить эту команду снова и снова, ее можно определить в файле `Dockerfile` как команду по умолчанию (листинг 2.8).

Листинг 2.8. Chapter02/2.1.5_DefaultCommand/Dockerfile

```
FROM python:3
COPY . /app
WORKDIR /app
CMD python3 hello.py
```

Чтобы собрать и запустить этот контейнер, выполните в командной строке следующие команды:

```
$ cd Chapter02/2.1.5_DefaultCommand
$ docker build . -t hello3
$ docker run hello3
Hello Docker
```

В отличие от всех других команд, которые мы использовали в файле `Dockerfile` до сих пор, команда `CMD` не влияет на способ сборки контейнера. Она просто определяет команду по умолчанию, которая будет выполняться в том случае, если после команды `docker run` не будет указана никакая другая. При этом вы всегда можете проигнорировать это определение, указав во время выполнения другую команду. При использовании языка Ruby все выглядит точно так же: вы определяете команду

по умолчанию в файле Dockerfile, а затем выполняете сборку и запуск контейнера, используя команду `docker run $IMAGE_NAME`:

```
$ cd Chapter02-ruby/2.1.5_DefaultCommand
$ docker build . -t hello_ruby3
$ docker run hello_ruby3
Hello Docker
```

2.1.6. Добавление зависимостей

Любое более или менее сложное приложение будет иметь собственные зависимости, не внесенные в состав базового образа. Обеспечить их загрузку можно путем выполнения в ходе сборки контейнера команд, задающих необходимую конфигурацию образа. Именно так в предыдущем примере мы добавили Python в базовый образ на основе Linux, и этот метод можно использовать для установки любых других зависимостей, которые могут понадобиться вашему приложению. Например, если оно должно устанавливать соединение с базой данных MariaDB, то сборка контейнера может выполняться так (листинг 2.9).

Листинг 2.9. Chapter02/2.1.6_Dependencies/Dockerfile

```
FROM python:3
RUN apt-get update
RUN apt-get install -y mariadb-client
COPY . /app
WORKDIR /app
CMD python3 hello.py
```

С помощью команды `apt-get` добавьте в Linux-контейнер все, что нужно вашему приложению

Базовый образ `python` был создан на основе дистрибутива Debian. Этот дистрибутив Linux широко применяется для создания контейнеров и использует пакетный менеджер `apt`, что позволяет вам установить практически любую зависимость с помощью команды `apt-get`.

Однако устанавливать зависимости можно не только с помощью команды `apt-get`. Допустим, один из ваших сервисов создает PDF-файлы, и вам нужно добавить в образ шрифты стандарта Unicode. В таком случае можно добавить в образ бесплатные шрифты Noto от компании Google (листинг 2.10).

Листинг 2.10. Chapter02/2.1.6_Dependencies-2/Dockerfile

```
FROM python:3
RUN apt-get update
RUN apt-get install -y libarchive-tools
RUN mkdir -p ~/.fonts; cd ~/.fonts
RUN curl "https://noto-website-2.storage.googleapis.com/pkgs/Noto-hinted.zip" | bsdtar -xvf-
RUN fc-cache -f -v
COPY . /app
WORKDIR /app
CMD python3 hello.py
```

Устанавливаем утилиту `bsdtar`

Создаем новый каталог и переходим в него. Обратите внимание на то, как мы объединили две команды в одной строке

Скачиваем и распаковываем пакет со шрифтами

Устанавливаем шрифты

В контейнеры часто приходится добавлять большое количество зависимостей, поэтому вы можете таким образом добавлять любые нужные вам компоненты операционной системы, например, устанавливать шрифты или TLS-сертификаты.

2.1.7. Компиляция кода в Docker

А как следует поступать в случае компилируемых программ, написанных на Java, Swift, C++ или языках платформы .NET? В таком случае будет недостаточно просто добавить в файл `Dockerfile` команду `COPY`, если, конечно, вы не успели скомпилировать двоичные файлы заранее.

Конечно, вы можете заранее скомпилировать приложение на локальной машине, но с таким же успехом это можно сделать и непосредственно в Docker. Перепишем наш простейший пример на Java и скомпилируем его в контейнер (листинги 2.11, 2.12).

Листинг 2.11. Chapter02/2.1.7_CompiledCode/Hello.java

```
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Docker");
    }
}
```

Листинг 2.12. Chapter02/2.1.7_CompiledCode/Dockerfile

```
FROM openjdk
COPY . /app
WORKDIR /app
RUN javac Hello.java
CMD java Hello
```

Команда
компиляции



Этот файл `Dockerfile` мало чем отличается от всех предыдущих: как и раньше, мы берем за основу некий базовый образ (в данном случае это `openjdk`, содержащий набор инструментов OpenJDK) и копируем приложение. Однако затем мы выполним компиляцию приложения, используя команду `RUN`, предварительно указав с помощью директивы `WORKDIR`, в каком рабочем каталоге должна выполняться эта операция и все последующие.

Собрать и запустить этот контейнер можно так:

```
$ cd Chapter02/2.1.7_CompiledCode
$ docker build . -t compiled_code
$ docker run compiled_code
Hello Docker
```

В качестве еще одного примера в папке `Chapter02-swift/2.1.7_CompiledCode` представлен пример компиляции серверного приложения, написанного на языке Swift. Сборка и запуск этого кода выполняются аналогично.

2.1.8. Компиляция кода с помощью многоэтапной сборки

Использование команды RUN для выполнения компиляции и других действий — вполне допустимый подход, но у него есть недостаток: вам приходится добавлять в образ контейнера инструменты, необходимые для ее выполнения. И эти инструменты, и весь исходный код остаются в составе окончательного образа контейнера.

Например, мы можем попробовать отобразить содержимое того образа, который создали в предыдущем подразделе, с помощью команды `ls`:

```
$ docker run compiled_code ls
Dockerfile
Hello.class
Hello.java
```

В таком случае мы увидим, что этот образ содержит исходный код. К тому же здесь по-прежнему присутствует компилятор языка Java (`javac`), хотя он больше нам не потребуется (поскольку компилятор не нужен на этапе выполнения кода).

Образ контейнера выполняет сразу две функции — и сборку, и запуск кода, что не совсем правильно. Все эти дополнительные инструменты увеличивают не только его объем, но и добавляют способы атаки контейнера (ведь теперь компилятор может быть использован любым процессом, запущенным в контейнере). Конечно, мы можем очищать контейнер, выполняя в Docker еще несколько команд (например, для удаления исходного кода и деинсталляции уже ненужных инструментов), но такой подход не всегда удобен, особенно если дополнительные инструменты входят в состав базового образа.

Более удачный способ решения этой проблемы сводится к использованию многоэтапного процесса сборки контейнера (рис. 2.2). При этом сначала создается промежуточный контейнер, содержащий все, что требуется для *сборки* программы, а затем — окончательный контейнер, в котором есть все, что требуется для ее *запуска*. Это позволяет разделить обязанности и аккуратно изолировать их в отдельных контейнерах.

Попробуем реализовать с помощью многоэтапной сборки пример из предыдущего подраздела. В листинге 2.13 показано, как при этом будет выглядеть файл Dockerfile.

Листинг 2.13. Chapter02/2.1.8_MultiStage/Dockerfile

```
FROM openjdk:11 AS buildstage
COPY . /app
WORKDIR /app
RUN javac Hello.java

FROM openjdk:11-jre-slim
COPY --from=buildstage /app/Hello.class /app/
WORKDIR /app
CMD java Hello
```

← Присваиваем имя `buildstage` сборочному контейнеру, который отвечает за сборку кода

← В качестве основы для выполняемого контейнера используем сокращенную версию базового образа, которая не содержит инструментов компиляции

← С помощью выражения `--from=` ссылаемся на файлы из сборочного контейнера

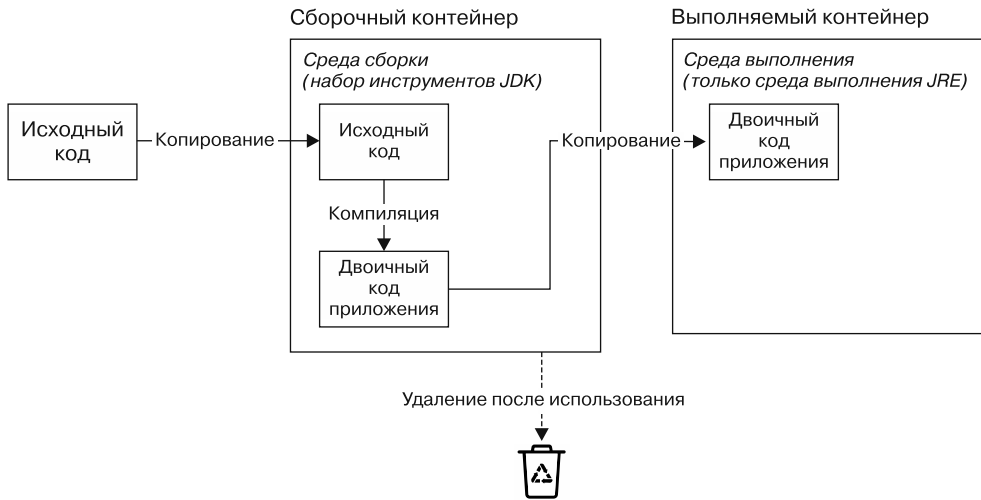


Рис. 2.2. Многоэтапная сборка контейнера, в ходе которой двоичные файлы собираются с помощью промежуточного контейнера

Как видно из этого примера, при таком подходе мы имеем как бы два файла Dockerfile в одном (две серии команд, начинающихся с команды FROM). Настройка и сборка первого контейнера выполняются исключительно в целях компиляции приложения, и при этом используется полный базовый образ `openjdk`, содержащий компилятор языка Java. Во второй контейнер добавляется только то, что требуется для запуска приложения, — и в этом случае используется базовый образ `jre`, содержащий лишь среду выполнения Java (Java Runtime Environment, JRE).

В качестве конечного результата этот файл Dockerfile выдает контейнер, который предназначен для рабочей среды и содержит скомпилированный Java-класс, а также необходимые для его запуска зависимости. Промежуточный первый контейнер, с помощью которого выполняется компиляция приложения, удаляется по завершении процесса компиляции (строго говоря, он сохраняется в кэше платформы Docker, однако ничто из его содержимого не добавляется в окончательный образ контейнера, используемый в продуктивной среде).

Запустить итоговый контейнер в данном случае можно так:

```
$ cd Chapter02/2.1.8_MultiStage
$ docker build . -t compiled_code2
$ docker run compiled_code2
Hello Docker
```

Если мы отобразим содержимое этого контейнера с помощью команды `ls`, то увидим, что он содержит только скомпилированный код:

```
$ docker run compiled_code2 ls
Hello.class
```

В качестве еще одного примера в папке `Chapter02-swift/2.1.8_MultiStage` представлен пример многоэтапной сборки серверного приложения, написанного на языке Swift. Сборка и запуск этого кода выполняются аналогично.

2.2. КОНТЕЙНЕРИЗАЦИЯ СЕРВЕРНОГО ПРИЛОЖЕНИЯ

Рассмотренные в предыдущем разделе примеры представляли собой простейшие программы, которые запускаются один раз и спустя некоторое время прекращают работу. Этот сценарий использования, в частности, применяется в случае контейнеризации консольных программ, пакетных рабочих нагрузок или FaaS-сервисов (Functions-as-a-Service, функция как услуга). Однако одним из наиболее распространенных видов рабочих нагрузок, развертываемых в Kubernetes, являются HTTP-сервисы — приложения, которые ожидают поступления запросов и обрабатывают их (то есть представляют собой веб-сервер).

С точки зрения платформы Docker серверные приложения ничем не отличаются от других. В их случае лишь немного иначе выполняется запуск и подключение к контейнеру, поскольку такой контейнер должен работать постоянно (чтобы иметь возможность обрабатывать запросы). Кроме того, обычно требуется перенаправить в контейнер обращения к портам локальной машины, чтобы с ним можно было связываться извне.

2.2.1. Контейнеризация сервера приложений

До сих пор в качестве примера мы использовали простейший скрипт Python «Hello World», но для наглядной демонстрации способа контейнеризации HTTP-серверов нам потребуется программа, которая, по сути, представляет собой HTTP-сервер. Минимально функциональный пример HTTP-сервера на языке Python, возвращающего текущие значения даты и времени, показан в листинге 2.14. Однако не стоит придавать слишком много значения самому этому коду. Содержимое данной книги не привязано к какому-либо конкретному языку, и язык Python используется здесь лишь в качестве примера. Те же принципы в равной мере применимы к любому другому HTTP-серверу.

Листинг 2.14. `Chapter02/timeserver/server.py`

```
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
from datetime import datetime

class RequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-type', 'text/plain')
        self.end_headers()
        now = datetime.now()
        response_string = now.strftime("The time is %-I:%M %p, UTC.")
```

```

        self.wfile.write(bytes(response_string, "utf-8"))

def startServer():
    try:
        server = ThreadingHTTPServer(('', 80), RequestHandler)
        print("Listening on " + ":".join(map(str, server.server_address)))
        server.serve_forever()
    except KeyboardInterrupt:
        server.shutdown()

if __name__ == "__main__":
    startServer()

```

Контейнеризация этого серверного приложения выполняется почти так же (листинг 2.15), как мы делали это ранее, когда работали с консольной программой.

Листинг 2.15. Chapter02/timeserver/Dockerfile

```

FROM python:3.12
ENV PYTHONUNBUFFERED 1
COPY . /app
WORKDIR /app
CMD python3 server.py

```

КОНТЕЙНЕРИЗАЦИЯ ВАШЕГО СОБСТВЕННОГО ПРИЛОЖЕНИЯ

В общем случае контейнеризация вашего собственного приложения выполняется так.

1. Найдите наиболее подходящий базовый образ, который содержит наибольшую часть необходимой вам конфигурации. В случае приложения на базе фреймворка Ruby on Rails начните с образа `ruby`, а не с более универсального образа `ubuntu`. Если вы используете Django, то начните с образа `python` и т. д.
2. Настройте все необходимые вам зависимости приложения (с помощью команды `RUN`, как мы уже делали ранее).
3. Скопируйте свое приложение.

В ходе этой процедуры очень полезно выполнять поиск аналогичных примеров в Интернете. Если вы не делаете нечто совершенно новое, то, возможно, кто-то другой уже делал это до вас и поделился примером файла `Dockerfile` для настройки приложения, созданного с помощью того же фреймворка. Так, если вы используете популярные фреймворки: Django, Ruby on Rails, WordPress, Node.JS или SpringBoot, — то легко найдете множество сходных примеров. Конечно, совершенно одинаковых приложений не бывает, и вы вряд ли найдете пример с точно такими же зависимостями, как у вас. И тем не менее такой пример может стать для вас хорошей отправной точкой.

Теперь, когда мы настроили свой HTTP-сервер, можно выполнить сборку конвейера, как мы уже делали ранее:

```
$ cd Chapter02/timeserver
$ docker build . -t timeserver
```

Операция запуска будет выглядеть немного иначе, поскольку для того, чтобы это приложение могло работать в браузере, мы должны перенаправить к контейнеру порты хост-машины. Перенаправим порт 8080 локальной машины на порт 80 контейнера, который будет прослушивать приложение:

```
$ docker run -it -p 8080:80 timeserver
Listening on 0.0.0.0:80
```

Теперь можно просмотреть страницу приложения, перейдя по адресу `http://localhost:8080` или воспользовавшись командой `curl`:

```
$ curl http://localhost:8080
The time is 1:30 PM, UTC.
```

Параметр `-it` (на самом деле это два параметра, которые обычно используются вместе) позволяет прекращать выполнение программы путем отправки сигнала `SIGTERM` (обычно с помощью сочетания клавиш `Ctrl+C/Command+C`). Это упрощает для разработчика работу по схеме «сборка — запуск — исправление ошибок — повторение» (сборка, запуск, нажатие клавиш `Ctrl+C`, исправление ошибок, повторение). Альтернатива состоит в том, чтобы запустить Docker в фоновом режиме, используя команду `docker run -d -p 8080:80 timeserver`. Здесь не используется параметр `-it`, поэтому остановку контейнера придется выполнять вручную: введите команду `docker ps`, чтобы отобразить список процессов, а затем — команду `docker stop $CONTAINER_ID`, чтобы остановить соответствующий процесс, либо остановите все запущенные контейнеры, используя команду `docker stop $(docker ps -q)`.

Еще больше упростить выполнение цикла разработки можно, записав команды сборки и запуска контейнера в одной строке, как показано ниже. В таком случае для повторной сборки нужно просто остановить приложение, нажав `Ctrl+C` (или эквивалентное сочетание клавиш), с помощью клавиши `↑` отобразить предыдущую команду и ввести ее снова, нажав клавишу `Enter`. Нужно лишь следить за тем, не возникают ли ошибки на этапе сборки, поскольку в таком случае запускается последний успешно собранный контейнер:

```
docker build . -t timeserver; docker run -it -p 8080:80 timeserver
```

Вот и все! Теперь у нас есть запущенное в Docker контейнеризированное приложение. В разделе 2.3 мы поговорим о том, как можно настроить и запустить локальную

среду отладки с помощью утилиты Docker Compose (что может оказаться полезным, если приложение состоит из нескольких разных контейнеров), а в следующей главе — о том, как можно развернуть это веб-приложение в Kubernetes.

2.2.2. Отладка

Если вам не удастся запустить приложение после настройки файла Dockerfile, то будет полезно запустить в среде контейнера командную оболочку и посмотреть, что же с ним не так. После запуска контейнера в соответствии с изложенными выше инструкциями запустите в нем командную оболочку, используя новое консольное окно:

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
6989d3097d6b   timeserver    "/bin/sh -c 'python3..." 2 minutes ago  Up 2 min

$ CONTAINER_ID=6989d3097d6b
$ docker exec -it $CONTAINER_ID sh
# ls
Dockerfile  server.py
# exit
$
```

Помимо команды `sh`, здесь можно использовать любую другую; например, в случае фреймворка Ruby on Rails можно воспользоваться командой `bundle exec rails console`, чтобы сразу запустить командную оболочку этого фреймворка, не совершая какие-либо промежуточные действия.

Я не буду приводить здесь полный список команд платформы Docker, поскольку его можно легко найти в документации, но в том, что касается отладки, я нахожу особенно полезной команду `docker cp`. С ее помощью можно копировать файлы из хост-машины в контейнер:

```
docker cp server.py $CONTAINER_ID:/app
```

Вдобавок ее можно использовать для копирования файлов из контейнера:

```
docker cp $CONTAINER_ID:/app/server.py .
```

Если вы сможете устранить проблему, исполняя команды с помощью `exec` или скопировав файлы, то обязательно отразите эти изменения в файле Dockerfile. Помните о том, что именно он, а не исполняемый экземпляр контейнера является вашей основной спецификацией. Если вы будете использовать ручную корректировку экземпляра контейнера, то этот способ будет ничем не лучше старого доброго «зайти командной строкой в виртуальную машину, чтобы поправить ее», от которого мы стремимся отказаться.

2.3. ЛОКАЛЬНОЕ ТЕСТИРОВАНИЕ С ПОМОЩЬЮ УТИЛИТЫ DOCKER COMPOSE

Итак, образ контейнера создан, и можно приступить к использованию платформы Kubernetes. При желании вы можете сразу перейти к следующей главе и начать развертывание созданного нами контейнера в облачной или локальной среде Kubernetes. В данном разделе я расскажу о том, как можно проводить локальное тестирование и разработку с помощью утилиты Docker Compose до развертывания контейнера в Kubernetes.

В предыдущем разделе мы запустили свое серверное приложение, используя Docker, и перенаправили к нему порты хост-машины, чтобы его можно было протестировать. Использование такого подхода к тестированию на этапе разработки имеет ряд недостатков. В частности, вам приходится каждый раз перенаправлять порты, и эта операция может быть достаточно сложной, если речь идет о приложении, содержащем несколько контейнеров.

В этом случае может помочь утилита Docker Compose. Это простейший оркестратор контейнеров, который позволяет запускать и останавливать несколько контейнеров в виде логической группы, сохраняя настройки среды исполнения между запусками, что существенно упрощает локальное тестирование. Чтобы запустить с помощью утилиты Compose контейнер веб-сервера из подраздела 2.2.1, необходимо настроить файл `docker-compose.yml` (листинг 2.16).

Листинг 2.16. Chapter02/2.3_Compose/docker-compose.yml

```
services:
  web:
    build: ../timeserver
    command: python3 server.py
    ports:
      - "8080:80"
```

Путь к каталогу, содержащему docker-контейнер, который требуется собрать

Команда, которая будет выполняться в контейнере. Эту настройку можно опустить, если вы используете команду CMD в файле Dockerfile

Перенаправляемые в контейнер порты локальной машины. В данном случае порт 8080 локальной машины перенаправляется на порт 80 контейнера

Собрать и запустить этот контейнер можно так:

```
cd Chapter02/2.3_Compose
docker compose build
docker compose up
```

Сам я в ходе разработки обычно объединяю эти две операции, чтобы сделать процедуру повторной сборки более компактной:

```
docker compose build; docker compose up
```

Выполнив эту простую настройку конфигурации, вы избавите себя от забот о том, какие именно команды Docker нужно выполнить, чтобы запустить и протестировать приложение, — все необходимые настройки будут храниться в файле утилиты Compose. В данном случае вы немного облегчаете свою работу, упрощая перенаправление порта локальной машины, однако по мере увеличения количества настроек и зависимостей преимущества этого подхода станут более очевидными.

2.3.1. Отображение локальных каталогов на каталоги контейнера

Ранее мы выполняли копирование файлов в контейнер и из него с помощью команды `docker cp`. Одной из действительно полезных свойств утилиты Compose является то, что она позволяет вместо этого подключить локальные каталоги вашей машины в контейнер. Вместо того чтобы записывать в контейнер копию приложения, можно просто сослаться на папку приложения на жестком диске. Это существенно упрощает процесс разработки, поскольку вы можете работать с файлами контейнера прямо на своей машине и вам не придется после каждого изменения копировать файлы в контейнер и заново выполнять его сборку.

Как вы помните, раньше мы копировали свое серверное приложение в расположенный внутри контейнера каталог `/app` (см. файл `Dockerfile`, представленный в листинге 2.15). Сейчас же вместо этого мы смонтируем локальный каталог в тот же каталог внутри контейнера, используя механизм привязки томов (листинг 2.17).

Листинг 2.17. Chapter02/2.3.1_VolumeMount/docker-compose.yml

```
services:
  frontend:
    build: .
    command: python3 server.py
    volumes:
      - type: bind
        source: .
        target: /app
    environment:
      PYTHONDONTWRITEBYTECODE: 1
    ports:
      - "8080:80"
```

Привязываем каталог сборки контейнера на локальной машине к каталогу `/app` контейнера (который раньше указывался в файле `Dockerfile`)

Определяем новую переменную среды, чтобы интерпретатор языка Python мог перезагружать исходный код

После того как мы обеспечим привязку томов, как показано выше, в контейнере будут использоваться те файлы, которые размещены на локальной машине, а не скопированы в контейнер. Если эти файлы будут изменяться на локальной машине,

то изменения будут немедленно отражаться и в контейнере, и его не придется собирать повторно. Используя интерпретируемые языки, такие как Python, Ruby и PHP, и языки разметки, такие как HTML и CSS, вы сможете существенно сократить цикл разработки, настроив среду так, чтобы для применения изменений было достаточно лишь выбрать в редакторе команду **Save** (Сохранить) и перезагрузить страницу в браузере.

ПРИМЕЧАНИЕ

Если код компилируемый, то данный подход часто не слишком полезен. Он позволяет заменить двоичный файл, расположенный в контейнере, двоичным файлом, компилируемым на локальной машине, однако в этом действии будет мало пользы, если вы предпочитаете выполнять весь процесс сборки в Docker (или если локальная среда имеет архитектурные отличия от среды контейнера). Поэтому, чтобы обеспечить компактный цикл разработки в случае компилируемого кода, я рекомендую использовать такие инструменты разработчика, как, например, Skaffold (<https://skaffold.dev/>).

Однако это еще не все действия, которые нужно выполнить для того, чтобы данный способ сработал в случае нашего приложения Python. Дело в том, что по умолчанию код Python не загружается заново с диска после внесения в него изменений. И даже если мы изменим исходный код, это никак не скажется на работе запущенного контейнера. Так будет происходить не только в той системе сборки, которую используем мы, но и во многих других.

Настроив монтирование локального каталога с помощью утилиты Compose, мы должны изменить свое Python-приложение сервера времени так, чтобы оно поддерживало перезагрузку уже запущенного кода. Эту настройку может задавать по-разному, в зависимости от того, какой язык или фреймворк вы используете. В случае языка Python можно использовать библиотеку `reloading`, чтобы функция GET заново загружалась с диска при поступлении каждого нового запроса (листинг 2.18).

Листинг 2.18. Chapter02/2.3.1_VolumeMount/server.py

```
from reloading import reloading
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
from datetime import datetime

class RequestHandler(BaseHTTPRequestHandler):
    @reloading
    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-type', 'text/plain')
        self.end_headers()
        now = datetime.now()
        response_string = now.strftime("The time is %-I:%M %p, UTC.")
        self.wfile.write(bytes(response_string, "utf-8"))
```

После добавления тега `@reloading` метод `do_GET` будет заново загружаться с диска при каждом его выполнении, что позволит изменять его после запуска приложения

```
def startServer():
    try:
        server = ThreadingHTTPServer(('',80), RequestHandler)
        print("Listening on " + ":".join(map(str, server.server_address)))
        server.serve_forever()
    except KeyboardInterrupt:
        server.shutdown()

if __name__ == "__main__":
    startServer()
```

Мы теперь используем дополнительную библиотеку, поэтому должны добавить эту зависимость в файл Dockerfile (листинг 2.19).

Листинг 2.19. Chapter02/2.3.1_VolumeMount/Dockerfile

```
FROM python:3
RUN pip install reloading
ENV PYTHONUNBUFFERED 1
COPY . /app
WORKDIR /app
CMD python3 server.py
```

Теперь, когда мы уже настроили приложение так, чтобы оно заново загружало файлы с диска, можно запустить его с помощью утилиты Compose, как мы делали ранее:

```
$ cd Chapter02/2.3.1_VolumeMount
$ docker compose build; docker compose up
Creating network "231_volumemount_default" with the default driver
Creating 231_volumemount_frontend_1 ... done
Attaching to 231_volumemount_frontend_1
```

Затем снова перейдем в браузере по адресу <http://localhost:8080/>. На этот раз откроем файл 2.3.1_VolumeMount/server.py (см. листинг 2.18) и изменим возвращаемый ответ. Например, вместо прежнего значения мы можем возвращать в строке 12 время в 24-часовом формате:

```
response_string = now.strftime("The time is %H:%M, UTC.")
```

Внесите это изменение в редакторе и сохраните файл программы, после чего перезагрузите страницу в браузере. Вы увидите, что выдаваемый программой ответ теперь выглядит по-другому:

```
$ curl http://localhost:8080
The time is 17:23, UTC.
```

В данном примере нам пришлось изменить код, чтобы перезагрузка файла заработала, но вносить такое изменение вряд ли потребуется при использовании стандартной среды разработки, поскольку ее можно будет настроить так, чтобы она автоматически выполняла перезагрузку.

Таким образом, локальные каталоги отображаются на каталоги контейнера, а цикл разработки сводится лишь к тому, чтобы выбрать в редакторе кода команду **Save** (Сохранить) и перезагрузить страницу в браузере. На мой взгляд, это одна из самых приятных возможностей утилиты **Docker Compose** и контейнеров в целом. Вы получаете все преимущества контейнеризации, где вам не нужно заниматься установкой инструментов разработчика на локальной машине, и такую же степень эффективности, как если бы код запускался на локальной машине без какой-либо операции сборки.

Следует отметить, что привязка томов работает и в обратном направлении. При внесении изменений внутри привязанного тома контейнера эти изменения отражаются в соответствующем локальном каталоге. Это может быть полезно в том случае, когда вы выполняете команды в контейнере и хотите сохранить выдаваемые ими результаты. На самом деле данный подход позволяет полностью исключить установку инструментов разработчика на локальной машине. Например, при использовании фреймворка **Rails** его нужно периодически обновлять с помощью выполнения команды **gem update rails** в каталоге проекта. Воспользовавшись привязкой томов, вы сможете выполнять эту команду в контейнере, получая в результате обновленный список пакетов на локальном диске, готовый для фиксации в системе контроля версий.

2.3.2. Добавление сервисных зависимостей

Если речь идет об абсолютно самостоятельном приложении, то на этом ваша работа будет закончена. Во всех остальных случаях вместе с приложением требуется установить несколько дополнительных сервисов, которые могут представлять собой созданные вами отдельные серверы или предлагаемые провайдером облачных услуг стандартные компоненты, такие как базы данных. В обоих вариантах вы можете добавить эти зависимости в **Compose**, чтобы создать локальную среду разработки.

Вы можете легко добавить в **Compose** несколько сервисов, как в виде ссылок на стандартные образы (в случае таких зависимостей, как база данных **MySQL**), так и в виде ссылок на другие проекты, размещенные на вашей машине. Структура проекта, как правило, содержит один корневой каталог для всех сервисов, подкаталоги для их проверки и файл утилиты **Docker Compose** со ссылками на все сервисы.

В листинге 2.20 показано, как может выглядеть файл утилиты **Compose** при использовании двух контейнеризированных сервисов: собираемого локально приложения и экземпляра базы данных, использующего общедоступный контейнер **MySQL**. Используемое здесь приложение на самом деле не использует базу данных **MySQL**, но я надеюсь, что данный пример хорошо показывает, насколько легко вы можете добавить нужные вам зависимости. В этом файле можно добавить все необходимые сервисы, в том числе и локально собираемые контейнеры, и внешние образы.

Эта простота, благодаря которой установить или удалить всю тестовую среду можно с помощью лишь одной команды, — один из главных доводов в пользу того, чтобы использовать для локального тестирования не только **Docker**, но и **Compose**.

ЧТО ЛУЧШЕ ИСПОЛЬЗОВАТЬ В СЛУЧАЕ ЛОКАЛЬНОЙ РАЗРАБОТКИ: COMPOSE ИЛИ KUBERNETES

Зачем вообще использовать Compose вместо Kubernetes для установки зависимых сервисов в среде разработки? Ведь для локальной разработки можно использовать Kubernetes, и это наилучший вариант, если нужно реплицировать рабочую среду (в главе 3 мы поговорим о локальной разработке более подробно). Однако многие разработчики предпочитают использовать для этой цели утилиту Compose, главным образом из-за ее простоты.

Утилиту можно легко настроить для локального использования, если требуются лишь несколько зависимых сервисов, как в случае большинства простых приложений. Если же речь идет о рабочей среде, которая представляет собой нечто более сложное, чем несколько экземпляров сервисов, запущенных на одной машине, то лучше использовать Kubernetes.

Из-за этой двойственности разработчики часто настраивают локальную среду разработки с помощью Compose, а продуктивную среду — с помощью Kubernetes. При этом они фактически дублируют конфигурацию времени выполнения, но поскольку она используется для двух разных целей: для разработки и развертывания в продуктивной среде, — то не будет выглядеть одинаково, даже если в обоих случаях будет использоваться Kubernetes. Я рекомендую выбирать тот подход, который позволяет максимально облегчить работу в конкретном случае.

Листинг 2.20. Chapter02/2.3.2_MultipleServices/docker-compose.yaml

```
services:
  frontend:
    build: ../timeserver
    command: python3 server.py
    environment:
      PYTHONDONTWRITEBYTECODE: 1
    ports:
      - "8080:80"

  db:
    image: mysql:5.7
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: super secret password
      MYSQL_DATABASE: my_database
      MYSQL_USER: dev_user
      MYSQL_PASSWORD: another secret password

volumes:
  db_data:
```

Собираемое локально приложение; у вас может быть несколько таких приложений

Сервис, запускающий общедоступный образ; у вас может быть несколько таких сервисов

Определение томов для базы данных, используемой на этапе разработки, которое обеспечивает сохранение данных между запусками

СОВЕТ

При настройке приложения для локальной разработки или для его использования в продуктивной среде все изменения конфигурации должны выполняться с помощью изменения переменных среды. В принципе, достаточно определить одну переменную, которой можно будет присваивать значение `prod` (рабочая среда) или `dev` (среда разработки) в зависимости от того, какой файл конфигурации вы хотите использовать. Не добавляйте в контейнер никакие настраиваемые элементы, которые нужно было бы изменять в зависимости от используемой среды. При таком подходе вы сможете использовать один и тот же контейнер в любой среде и будете уверены в том, что тестированию подвергается только тот продукт разработки, который вы собираетесь развернуть в продуктивной среде.

2.3.3. Имитация внешних зависимостей

Если до сих пор вы проводили тестирование, используя такие удаленные зависимости, как API облачного хранилища, то, возможно, пришла пора пересмотреть подход и попробовать заменить эти удаленные зависимости имитирующей их заглушкой. Используя в качестве заглушки легковесный локальный сервис, реализующий тот же внешний API, вы можете существенно ускорить процесс разработки и тестирования.

Раньше в качестве заглушки, как правило, нужно было использовать код, написанный на том же языке, что и приложение (по практическим соображениям, чтобы не нужно было поддерживать несколько разных сред для одного проекта).

Одно из преимуществ контейнеров состоит в том, что при их использовании становится неважным язык, на котором написаны заглушка и используемый вами облачный сервис: вам не нужно поддерживать среду заглушки, поскольку она работает в отдельном контейнере.

Вдобавок контейнеризация делает возможным использование высококачественных заглушек, представляющих собой более легковесную реализацию того же API, который вы собираетесь применять в рабочей среде.

В листинге 2.20 (см. выше) мы использовали в контейнере реальную базу данных MySQL (вместо того чтобы задействовать заглушку). С тем же успехом мы могли бы использовать на этапе тестирования реальный сервис объектного хранения, даже если бы в конечном счете планировали воспользоваться таким облачным сервисом, как Google Cloud Storage или Amazon S3.

Развивая пример с объектным хранением, предположим, что наше приложение обеспечивает облачное хранение с помощью S3-совместимого API (например, использует непосредственно сервис Amazon S3 или один из множества других сервисов объектного хранения, поддерживающих данный API, например Google Cloud Storage). Чтобы ускорить процесс разработки, мы могли бы симитировать облачный сервис, используя такой легковесный сервер, как Adobe S3Mock (<https://github.com/adobe/S3Mock>). Однако благодаря контейнеризации мы столь же

легко можем применить для этой цели и полнофункциональное S3-совместимое локальное хранилище наподобие MinIO (<https://min.io/>). Строго говоря, хранилище MinIO не является заглушкой — его можно развернуть и в продуктивной среде, если вы хотите использовать собственный сервис блочного хранения. Но в то же время его можно применять и как высококачественную заглушку, которая обеспечивает такие дополнительные преимущества, как удобный пользовательский интерфейс.

Ранее мы говорили о том, что использование контейнеров позволяет вам легко комбинировать различные сервисы, запуская каждый из них в отдельной среде. Теперь мы видим, что контейнеризация существенно упрощает и процесс разработки. Вместо того чтобы создавать заглушку самостоятельно или искать простейшую реализацию заглушки для своей среды, вы можете либо использовать локально тот же сервис, который собираетесь использовать в продуктивной среде (например, базу данных MySQL), либо заменить его другим полнофункциональным облачным сервисом (например, заменить облачное объектное хранилище сервисом MinIO). Как пример добавим сервис MinIO в качестве еще одного сервиса в файле утилиты Docker Compose (листинг 2.21).

Листинг 2.21. Chapter02/2.3.3_Fakes/docker-compose.yaml

services:

```
storage:
  image: minio/minio
  command: minio server /data
  volumes:
    - storage_data:/data
  restart: always
  environment:
    MINIO_ACCESS_KEY: fakeaccesskey
    MINIO_SECRET_KEY: fakesecretkey
  ports:
    - "9000:9000"
```

Дополнительный сервис хранения, используемый в качестве заглушки

```
frontend:
  build: ../timeserver
  command: python3 server.py
  environment:
    PYTHONDONTWRITEBYTECODE: 1
    S3_ENDPOINT: http://storage:9000
    S3_USE_PATH_STYLE: 1
    S3_ACCESS_KEY_ID: fakeaccesskey
    S3_SECRET_ACCESS_KEY: fakesecretkey
  ports:
    - "8080:80"
```

Настройка приложения на использование созданной нами заглушки

```
volumes:
  db_data:
  storage_data:
```

ПОВСЕМЕСТНОЕ ИСПОЛЬЗОВАНИЕ S3 API ДЛЯ РАБОТЫ С ОБЪЕКТНЫМИ ХРАНИЛИЩАМИ

Подобно тому как язык SQL в свое время стал стандартным языком запросов для работы с реляционными базами данных, сегодня поставщики объектных хранилищ стали повсеместно использовать S3 API. Его поддерживают такие крупные сервисы, как Google Cloud, Azure и AWS, большинство других облачных сервисов и многие системы хранения на базе «голового железа». Столь широкая распространенность этого API позволяет при необходимости легко сменить поставщика системы хранения и обеспечивает большой набор вариантов при выборе заглушки для локальной разработки.

Подобно тому как мы поступили здесь с сервисом MinIO, при использовании сервиса в качестве заглушки обычно можно сначала определить принимаемые им ключи доступа, а затем указать их при настройке приложения, используя переменные среды.

РЕЗЮМЕ

- Контейнеризация позволяет представить операцию сборки, среду и конфигурацию приложения в виде стандартного контейнера, при запуске которого степень изоляции будет такой же, как в случае использования виртуальных машин, а уровень накладных расходов — гораздо более низким.
- Контейнеризация — ключевое предварительное условие использования платформы Kubernetes, поскольку контейнеры являются тем видом исполняемой среды, который поддерживает Kubernetes.
- Использование контейнеров полезно не только в продуктивной среде, но и в среде разработки, позволяя вам работать над несколькими проектами одновременно, не создавая конфликты между разными средами и не выполняя сложный процесс настройки.
- Сборка образа контейнера выполняется с помощью конфигурационного файла Dockerfile, который содержит набор процедурных инструкций по сборке контейнера.
- При создании файла Dockerfile нужно сначала взять за основу базовый образ, который наилучшим образом удовлетворяет ваши потребности, а затем добавить настройки и зависимости, необходимые для работы вашего приложения.
- Многоэтапная сборка используется в случае компилируемых приложений.
- При многоэтапной сборке контейнера сначала создается промежуточный контейнер, содержащий все, что требуется для *сборки* программы, а затем —

окончательный контейнер, в котором есть все, что нужно для ее *запуска*. Это позволяет разделить обязанности и изолировать их в отдельных контейнерах.

- Утилита Docker Compose — это легковесный оркестратор контейнеров, позволяющий быстро создать контейнеризированную среду разработки в случае использования сразу нескольких сервисов.
- Отображение локальных каталогов на каталоги контейнера с помощью утилиты Compose дает возможность более оперативно редактировать некомпилируемые приложения, существенно сократив цикл разработки.
- На этапе тестирования контейнеризация позволяет заместить внешнюю зависимость высококачественной заглушкой, представляющей собой более легковесную реализацию того же API, который вы собираетесь использовать в продуктивной среде, например, контейнеризированную версию базы данных MySQL или реальный сервис объектного хранения.

3

Развертывание в Kubernetes

В этой главе

- ✓ Концепции платформы Kubernetes, относящиеся к конфигурации и размещению развертываемых приложений.
- ✓ Развертывание контейнеризированного приложения в Kubernetes на облачной платформе.
- ✓ Обновление развертываний до новых версий контейнера приложения.
- ✓ Запуск локальной версии Kubernetes для тестирования и разработки.

В предыдущей главе мы говорили о том, как можно контейнеризировать приложение. По завершении данного этапа вы уже будете иметь переносимую и воспроизводимую среду приложения, а также удобную среду разработки. Однако при этом у вас еще могут возникнуть проблемы с масштабированием приложения в продуктивной среде.

Если вы работаете по предельно простой схеме развертывания, позволяющей запускать каждый контейнер на отдельной виртуальной машине, то можете развернуть контейнеры непосредственно в виртуальных машинах и затем по мере

необходимости масштабировать последние. Это позволит вам получить лишь такое преимущество контейнеров, как более удобный способ упаковки. Однако в большинстве случаев вам нужно будет развернуть несколько различных сервисов, и потребуются более гибкий подход.

И вот тут вам может помочь такая система для оркестрации контейнеров, как Kubernetes. Под термином «оркестрация контейнеров» понимается просто планирование и отслеживание работы некоего множества различных контейнеров на некоем множестве различных машин. Такой инструмент позволяет вам работать главным образом в терминах развертывания приложения: вы можете определить контейнер и такие параметры развертывания, как количество необходимых реплик (экземпляров), требования по высокой доступности (распределение между доменами сбоя), сетевое взаимодействие сервисов и т. д., вместо того чтобы тщательно выверять конфигурацию используемых на базовом уровне вычислительных мощностей.

Предоставляя вам удобный способ распределения различных сервисов в общем пуле вычислительных ресурсов, система оркестрации позволяет обеспечить эффективное использование ресурсов при запуске нескольких приложений или использовании микросервисной архитектуры, где развертывание и управление различными компонентами приложения осуществляется по отдельности. Кроме того, вы можете одновременно применять различные схемы развертывания: от развертывания приложений без сохранения состояния до развертывания баз данных с сохранением состояния, пакетных заданий и многого другого. При этом вам не придется заботиться о том, на какой именно машине в итоге окажется каждый контейнер.

3.1. АРХИТЕКТУРА СИСТЕМЫ KUBERNETES

Система Kubernetes представляет собой слой абстракции, реализующий рабочие нагрузки, поверх таких базовых элементов вычислительной системы, как виртуальные или реальные машины и балансировщики нагрузки. Виртуальные (или реальные) машины называются *узлами* (nodes) и объединяются в *кластер* (cluster). Один или несколько контейнеров объединяются в так называемый *под* (Pod), который является в Kubernetes минимальной единицей планирования. Сетевое взаимодействие определяется с помощью *сервисов* (Service). Такие высокоуровневые элементы, как *развертывание* (Deployment), позволяют еще больше упростить работу с подами. Прежде чем приступить к развертыванию первой рабочей нагрузки, чуть подробнее изучим некоторые из этих базовых структурных элементов.

3.1.1. Кластер Kubernetes

Кластер Kubernetes — это набор узлов, представляющих собой те вычислительные мощности, на которых запускаются контейнеры. Обычно это виртуальные машины, однако могут использоваться и чисто аппаратные (невиртуализированные) машины. На каждом из этих узлов запускается *kubelet* — специальный процесс системы Kubernetes, который отвечает за взаимодействие с управляющим слоем (control plane) (процессом системы Kubernetes, осуществляющим оркестрацию) и управление жизненным циклом контейнеров, запускаемых на узлах с помощью среды запуска контейнеров. Помимо операционной системы, процесса kubelet и среды запуска контейнеров, вы можете запускать и другие процессы, например, свои рабочие нагрузки и системные компоненты, отвечающие за журналирование и мониторинг, делая это внутри контейнеров в соответствии со схемой, представленной на рис. 3.1.

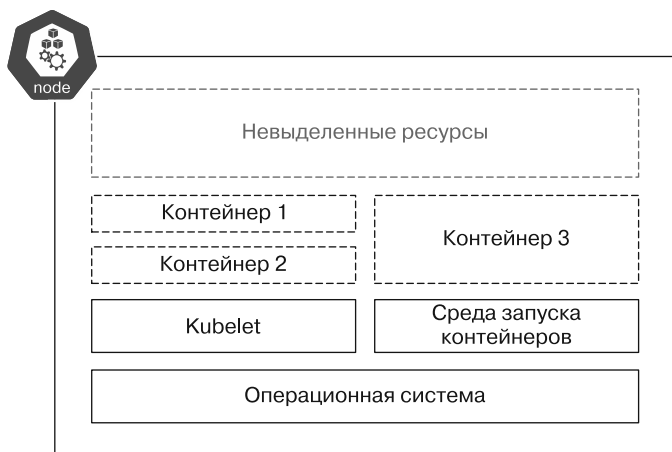


Рис. 3.1. Процессы, запускаемые на виртуальной машине, которая в терминологии Kubernetes называется узлом

Внутри кластера один или несколько (при использовании режима высокой доступности) узлов играют особую роль: они выполняют запуск самой программы оркестрации Kubernetes (рис. 3.2). Эти особые узлы составляют управляющий слой и отвечают за следующие действия:

- предоставление API, используемого для взаимодействия с кластером с помощью различных инструментов, таких как утилита командной строки Kubernetes;
- хранение состояния кластера;
- координацию работы входящих в кластер узлов для выполнения запланированных действий над контейнерами (запуска, остановки, повторного запуска).

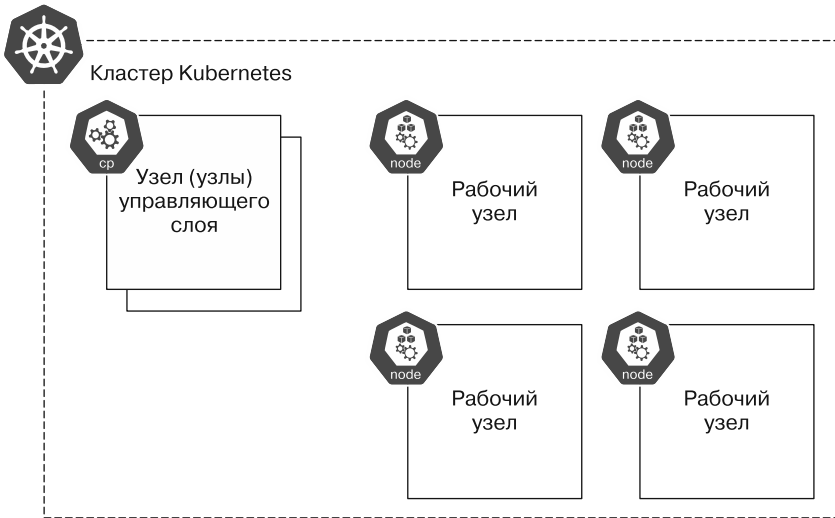


Рис. 3.2. Самоуправляемый кластер Kubernetes с управляющим слоем и рабочими узлами

В большинстве облачных сред управляющий слой предоставляется в виде управляемого сервиса. При этом узлы управляющего слоя обычно скрыты от пользователя, и то, как он работает на том или ином узле, является деталью реализации. В такой среде кластер фактически представляет собой управляющий слой с рабочими узлами (рис. 3.3).

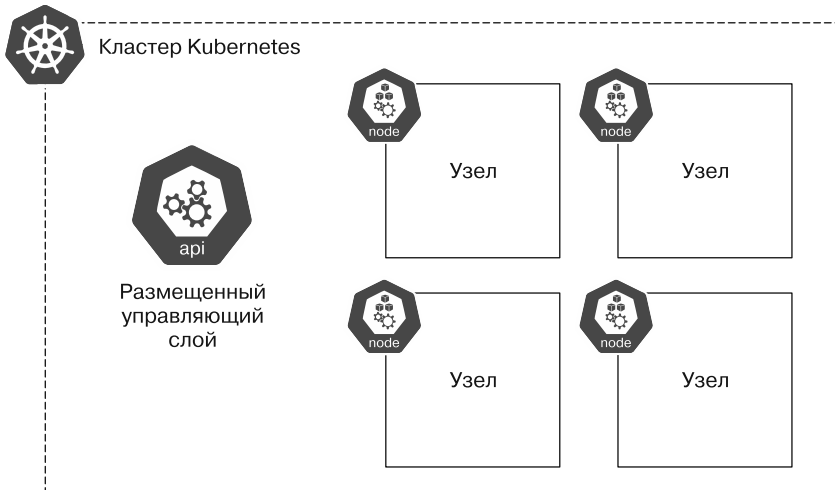


Рис. 3.3. Облачный кластер Kubernetes, узлы которого подключаются к размещенному управляющему слою

Рабочие узлы (которые мы будем далее называть просто *узлами*) отвечают за управление жизненным циклом запускаемых контейнеров, в том числе за их запуск и остановку. Управляющий слой дает узлу указание запустить определенный контейнер, а дальнейшую работу по запуску выполняет сам узел. Кроме того, узлы могут выполнять некоторые действия по собственной инициативе, например, перезапускать поврежденные контейнеры или освобождать дополнительную память, если имеющейся недостаточно.

Вместе управляющий слой и узлы составляют *кластер Kubernetes*, предоставляя вам платформу Kubernetes, на которой вы можете распределить свои рабочие нагрузки. Кластер предоставляется и управляется поставщиком платформы, используемой вами для запуска Kubernetes, который отвечает за создание таких ресурсов кластера, как узлы. Данная книга предназначена для разработчиков, поэтому основное внимание будет уделяться *использованию* кластера Kubernetes для запуска рабочих нагрузок, а не тому, как этот кластер предоставляется разработчикам поставщиком платформы (что относится к сфере деятельности поставщиков облачных услуг).

3.1.2. Объекты Kubernetes

После создания кластера взаимодействие с Kubernetes осуществляется в основном путем создания, проверки и изменения объектов Kubernetes с помощью Kubernetes API. Каждый из этих объектов представляет собой определенный структурный элемент общей схемы развертывания. Так, для представления нескольких контейнеров служит объект Pod, для представления нескольких подов — объект Deployment, для представления сетевых сервисов — объект Service и т. д. Даже отдельные узлы представляются в виде объектов, у которых можно запрашивать такую информацию о текущем состоянии, как количество используемых ресурсов. Чтобы развернуть в кластере типичное веб-приложение без сохранения состояния, нужно будет использовать следующие три объекта: Pod, Deployment (содержащий Pod) и Service.

Под

Под — просто набор контейнеров. Поды часто содержат только один контейнер, но могут содержать и несколько тесно связанных друг с другом контейнеров, если их требуется разворачивать совместно (рис. 3.4).

Под — базовая единица планирования в Kubernetes. В подах содержатся контейнеры вашего приложения. Kubernetes распределяет поды по узлам кластера с учетом того, какие ресурсы им необходимы. Например, если ваша рабочая нагрузка требует использования двух ядер процессора, то вы можете указать это в определении пода, и Kubernetes найдет для нее машину с двумя неиспользуемыми ядрами.



Рис. 3.4. Под Kubernetes, который может содержать один или несколько контейнеров

СКОЛЬКО КОНТЕЙНЕРОВ МОЖЕТ СОДЕРЖАТЬ ПОД

За исключением тех редких случаев, когда используемые контейнеры связаны друг с другом какой-то сильной зависимостью, контейнеры обычно развертываются по отдельности, и каждый контейнер размещается в отдельном поде. Поды с несколькими контейнерами обычно требуются лишь в том случае, когда второй контейнер выступает в роли «расширения» (паттерн Sidecar), выполняющего авторизацию, журналирование или какую-то иную служебную функцию, или когда совместное развертывание нескольких контейнеров становится целесообразным благодаря их тесной взаимосвязи.

Если мы посмотрим, какие процессы работают на соответствующем узле, то не увидим там сам под; мы увидим лишь процессы, исполняющие его контейнеры (рис. 3.5). Под — это просто логическая группа контейнеров. Kubernetes привязывает их друг к другу так, чтобы они имели один общий жизненный цикл: они вместе создаются, вместе перезапускаются в случае сбоя и вместе удаляются в итоге.

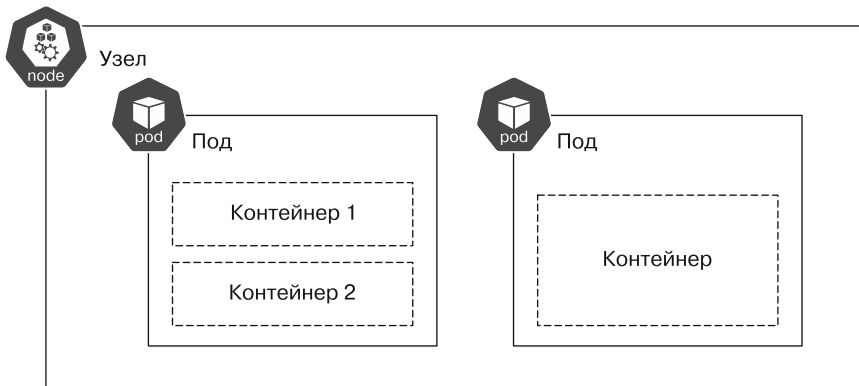


Рис. 3.5. Запуск нескольких подов на одном узле

Развертывание

Kubernetes позволяет запускать поды напрямую, но такой подход используется крайне редко. И приложения, и машины, на которых они работают, иногда дают сбой, и тогда поды приходится запускать или распределять по машинам заново. Поэтому вместо того чтобы выполнять распределение подов напрямую, их обычно обертывают в объект более высокого уровня, который управляет их жизненным циклом.

В случае таких непрерывно работающих приложений, как веб-серверы, этим объектом является *Deployment*. Другой вариант — объект *Job*, который используется для запуска пакетных процессов, исполняемых до их завершения, и о котором мы поговорим в главе 10. В объекте *Deployment* можно указать, сколько реплик пода необходимо запустить, как должны выполняться обновления и т. д.

Как и все другие объекты в Kubernetes, *Deployment* (рис. 3.6) представляет собой спецификацию целевого состояния системы, которое стремится обеспечить Kubernetes. Эта спецификация может содержать такую информацию, как необходимое количество реплик того или иного пода и детальные требования к способу распределения подов в пределах кластера (о которых мы поговорим подробнее в дальнейших главах).

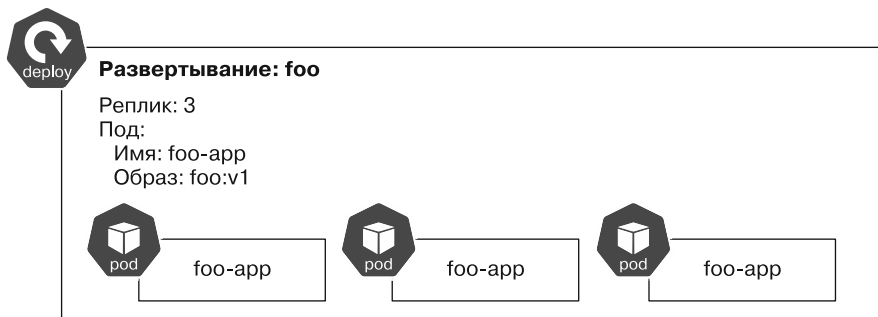


Рис. 3.6. Развертывание с тремя репликами пода *foo-app*

Kubernetes непрерывно сверяет наблюдаемое состояние с целевым и стремится предоставить вам то, что вы просили. Так, если определенный под станет недоступным спустя некоторое время после развертывания, например, из-за сбоя узла, на котором он был запущен, то Kubernetes заметит, что количество работающих подов меньше необходимого, и произведет распределение новых экземпляров пода, чтобы снова удовлетворить ваши требования. Такое автоматическое масштабирование и восстановление необходимого состояния — главный довод в пользу того, чтобы управлять жизненным циклом сервиса с помощью объекта *Deployment*, а не запускать поды напрямую.

Сервис

Объект *Service* описывает способ представления приложения, работающего в некотором наборе подов, в виде сетевого сервиса. Сервис предоставляет единый механизм адресации и распределяет нагрузку между подами (рис. 3.7). Каждый сервис имеет собственный внутренний IP-адрес и DNS-запись, на которые могут ссылаться другие поды, работающие внутри того же кластера. Кроме того, сервису можно присвоить и внешний IP-адрес.

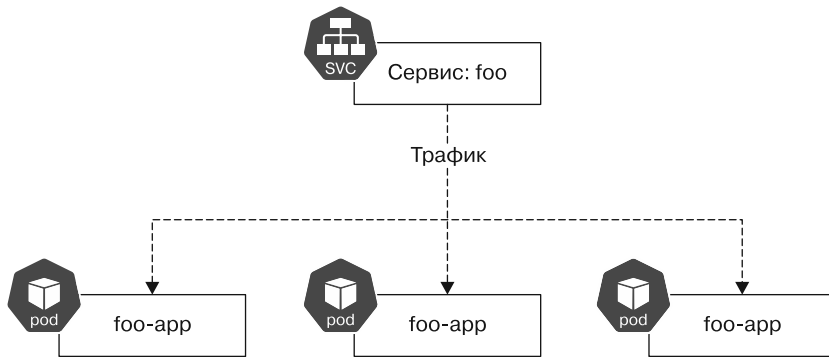


Рис. 3.7. Сервис Kubernetes

3.2. ПРОЦЕСС РАЗВЕРТЫВАНИЯ ПРИЛОЖЕНИЯ

Сначала произведем развертывание приложения, сделав его доступным в Интернете, а затем обновим его до новой версии. Это позволит увидеть, как в Kubernetes выполняется типичный цикл разработки, состоящий из этапов разработки, выпуска и обновления. В ходе данного процесса мы будем использовать рассмотренные в предыдущем разделе объекты Kubernetes: *Pod*, *Deployment*, с помощью которого мы будем управлять подом, и *Service*, благодаря которому мы сделаем этот под доступным для внешних пользователей.

3.2.1. Создание кластера

Чтобы развернуть свое приложение, вы сначала должны создать кластер Kubernetes. Я рекомендую использовать для этой цели общедоступное облако, поскольку в таком случае будет проще произвести начальную настройку и другие пользователи смогут ознакомиться с плодами вашей работы сразу после публикации IP-адресов развертываемых сервисов. Чтобы не тратить лишние деньги на этапе обучения, вы можете воспользоваться предложением тех поставщиков облачных услуг, которые предоставляют бесплатный пробный период.

На этапе разработки можно использовать локальный кластер Kubernetes, однако среда локального кластера отличается от среды облачного кластера в том, что касается таких процессов, как балансировка нагрузки. Я предпочитаю изучать ту среду, которую при желании можно будет в первый же день использовать в качестве рабочей, и потому рекомендую вам с самого начала использовать облачное решение.

ПРЕДПОЧИТАЕТЕ ИСПОЛЬЗОВАТЬ ЛОКАЛЬНЫЙ КЛАСТЕР?

Не стоит волноваться, если вы хотите использовать не облачный, а локальный кластер Kubernetes. В таком случае просто подключите утилиту `kubect1` к локальному кластеру, как описано в разделе 3.4, а затем продолжайте процесс развертывания в Kubernetes, начиная с информации, описанной в подразделе 3.2.3.

Вам нужно будет лишь учесть изложенные в разделе 3.4 соображения, чтобы на этапе развертывания собственного локально собранного образа контейнера система Kubernetes могла найти ваш образ. Кроме того, поскольку вы не сможете использовать общедоступные балансировщики нагрузки, то несколько иначе будет выглядеть и доступ к создаваемым вами сервисам (различия описываются в том же разделе 3.4).

Для запуска практически всех примеров кода из книги вам потребуется кластер Kubernetes, размещенный в том или ином облаке, и консольная утилита `kubect1` (произносится как *cube cuttle*), аутентифицированная для доступа к этому кластеру (описание процесса аутентификации можно найти в руководстве по началу работы с облачным сервисом). Далее я покажу, как эти кластер и утилиту можно получить при использовании облачной платформы Google Cloud, попутно объясняя, как те или иные шаги можно выполнить при использовании другой платформы.

Сервис Google Kubernetes Engine

Управляемый сервис Google Kubernetes Engine (GKE) в свое время стал первым таким продуктом на рынке, и в силу качества и простоты использования его часто выбирают те пользователи, которые хотят опробовать Kubernetes в деле. Как член команды разработчиков этого сервиса, я знаю его гораздо лучше других платформ и поэтому буду использовать его в тех местах книги, где будут описываться платформенно-ориентированные примеры.

При написании книги я старался сделать так, чтобы излагаемые здесь инструкции были применимы при использовании любой платформы на базе Kubernetes,

и надеюсь на то, что она будет в равной степени полезной вне зависимости от того, что вы используете: сервис GKE, OpenShift, Azure Kubernetes Service (AKS), Elastic Kubernetes Service (EKS) или какую-либо другую платформу либо дистрибутив системы Kubernetes. В тех редких случаях, когда выполняемые действия будут зависеть от платформы (как здесь, при создании кластера), я буду приводить вариант инструкций, ориентированный на сервис GKE, при этом указывая, где можно найти варианты инструкций, ориентированные на другие платформы.

СОЗДАНИЕ КЛАСТЕРА KUBERNETES В ЛЮБОМ ДРУГОМ ОБЛАКЕ

По завершении описываемого здесь этапа настройки кластера вам останется лишь настроить аутентификацию для утилиты `kubectl` с целью доступа к созданному вами кластеру Kubernetes. То есть для запуска описываемых в этой книге примеров достаточно создать кластер и настроить аутентификацию `kubectl`, что, как вы увидите на примере сервиса GKE, можно сделать с помощью двух команд. Если вы работаете на другой платформе, то вам следует использовать соответствующие аналоги этих команд.

Если вы решили запускать описываемые здесь примеры с помощью другого облачного сервиса, то создайте кластер, как это предлагает сделать ваш поставщик облачных услуг, а затем переходите к подразделу 3.2.2. Загрузка созданного контейнера в сеть тоже выполняется по-разному в зависимости от платформы, но я дам вам некоторые общие рекомендации касательно того, как это можно сделать при использовании любой другой платформы.

Для использования сервиса GKE вам потребуется аккаунт Google (если у вас есть адрес электронной почты, вторая половина которого выглядит как `@gmail.com`, то аккаунт Google у вас уже есть). Перейдите по адресу <https://console.cloud.google.com/>, выберите свой аккаунт и просмотрите условия использования. Если вы еще этого не сделали, то активируйте бесплатный пробный период или введите информацию об оплате, чтобы вы могли запускать примеры, представленные в книге (напомню, что если вы хотите запускать примеры локально, то вместо этого следует создать локальный кластер, как описано в разделе 3.4).

После настройки аккаунта перейдите в консоль сервиса GKE (прямая ссылка: <https://console.cloud.google.com/kubernetes>) и создайте кластер. Я рекомендую выбрать режим «Автопилот» (Autopilot), при котором выделение необходимых узлов и управление ими осуществляется автоматически. Выбрав этот режим, укажите имя кластера и регион, оставив без изменений сетевые и расширенные настройки (рис. 3.8).

Google Cloud Platform gke-autopilot-demo

← Create an Autopilot cluster HELP ASSISTANT

Create an Autopilot cluster by specifying a name and region. After the cluster is created, you can deploy your workload through Kubernetes and we'll take care of the rest, including:

- ✓ **Nodes:** Automated node provisioning, scaling, and maintenance
- ✓ **Networking:** VPC-native traffic routing for public or private clusters
- ✓ **Security:** Shielded GKE Nodes and Workload Identity
- ✓ **Telemetry:** Cloud Operations logging and monitoring

Name
kubernetes-for-developers

Region
us-west1

Networking

Define how applications in this cluster communicate with each other and how clients can reach them.

☒ Public cluster

☐ Private cluster

CREATE CANCEL Equivalent REST or COMMAND LINE

Рис. 3.8. Создание кластера сервиса GKE, работающего в режиме «Автопилот», с помощью пользовательского интерфейса

Затем следует настроить инструменты командной строки. Вам потребуется инструмент командной строки облачного сервиса (в данном случае утилита `gcloud` облачной платформы Google Cloud) для выполнения таких операций над кластером, как создание и аутентификация, и утилита `kubectl` для взаимодействия с API системы Kubernetes. Скачайте утилиту `gcloud` по адресу <https://cloud.google.com/sdk/install>, после чего следуйте представленным на этой странице инструкциям по установке.

После установки утилиты `gcloud` войдите в свой аккаунт с помощью команды `gcloud init`. Если у вас есть несколько аккаунтов Google, то выберите из них тот, в котором ранее был создан кластер:

```
gcloud init
```

Инструмент командной строки системы Kubernetes, `kubectl`, можно установить независимо (следуя инструкциям, представленным по адресу <https://kubernetes.io/docs/tasks/tools/>) или с помощью утилиты `gcloud`. Как вы это сделаете, не имеет

значения, но раз уж в примере используется утилита `gcloud`, то в данном случае установить `kubectl` будет удобнее с ее помощью:

```
gcloud components install kubectl
```

Итак, вы подготовили кластер к использованию и настроили утилиту `gcloud`. Теперь выберите в пользовательском интерфейсе команду **Connect** (Подключить) и скопируйте выведенную на экран команду утилиты `gcloud` (рис. 3.9) в свою командную оболочку, чтобы аутентифицировать утилиту `kubectl`. Либо выполните следующую команду, подставив свои параметры кластера:

```
CLUSTER_NAME=my-cluster
REGION=us-west1
gcloud container clusters get-credentials $CLUSTER_NAME --region $REGION
```

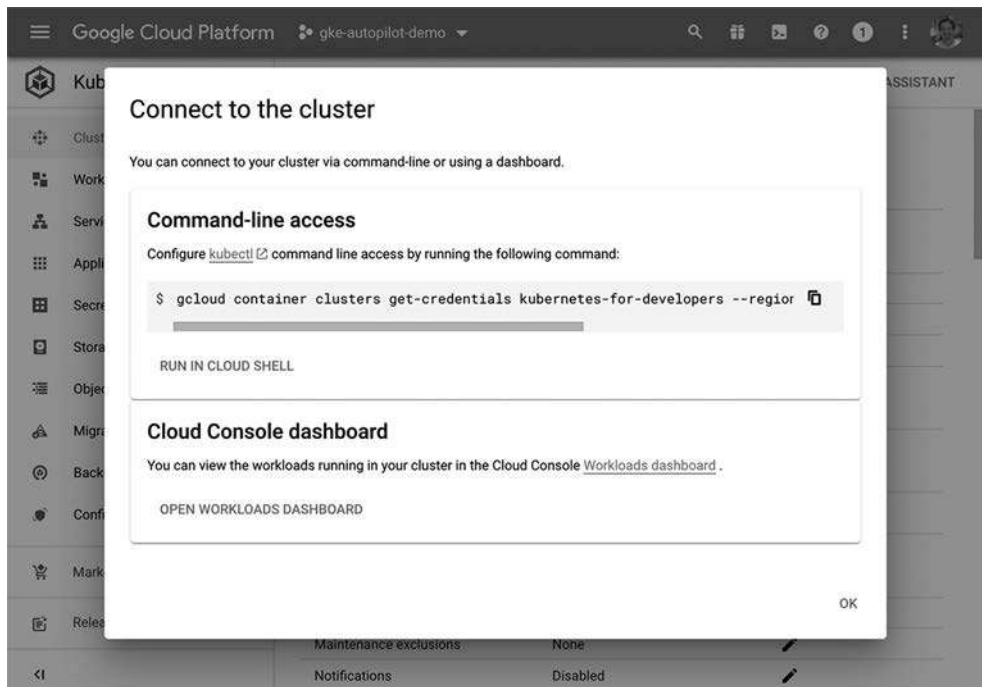


Рис. 3.9. Настройка подключения к кластеру сервиса GKE с помощью пользовательского интерфейса

Приведенная команда связывает платформу Google Cloud с системой Kubernetes, аутентифицируя с помощью соответствующих учетных данных инструмент командной строки `kubectl` для доступа к кластеру GKE.

СОЗДАНИЕ КЛАСТЕРА В КОМАНДНОЙ СТРОКЕ

Вы можете не использовать пользовательский интерфейс, выполнив оба шага: и создание кластера, и подключение к нему — в командной строке:

```
CLUSTER_NAME=my-cluster  
REGION=us-west1  
gcloud container clusters create-auto $CLUSTER_NAME --region $REGION  
gcloud container clusters get-credentials $CLUSTER_NAME --region $REGION
```

Таким образом, вы создали кластер, настроили аутентификацию утилиты `kubectl` и готовы к тому, чтобы развернуть свое первое приложение. В качестве проверки выполните команду `kubectl get pods`. Она должна сообщить, что у вас нет ресурсов (поскольку мы еще не развернули ни один под):

```
$ kubectl get pods  
No resources found in default namespace.
```

Если эта команда выдаст ошибку, значит, ваш кластер не был создан или аутентифицирован должным образом. Попробуйте выполнить предыдущие шаги заново или поищите информацию о том, что означает полученное вами сообщение об ошибке.

3.2.2. Загрузка в сеть собственного контейнера

До сих пор создаваемые нами контейнеры сохранялись и запускались на локальной машине. Чтобы развернуть контейнер в системе Kubernetes, работающей в облаке, сначала нужно загрузить свой образ контейнера в размещенный в сети реестр контейнеров. Это просто хранилище данных, которое хранит образы контейнеров и позволяет системе Kubernetes извлекать их. В большинстве реестров можно на выбор размещать либо общедоступные публичные образы (если имеют место проекты с открытым исходным кодом, такие как примеры кода из этой книги), либо требующие аутентификации приватные образы (в случае собственных проприетарных приложений).

При желании вы можете пропустить данный шаг и использовать общедоступные публичные образы, ссылки на которые будут предоставляться в последующих примерах. Однако я все же рекомендую создать и загрузить в сеть для дальнейшего использования собственный контейнер, чтобы вы не испытывали проблем, когда придет время развертывать собственные приложения.

Многие пользователи предпочитают использовать реестр контейнеров Docker Hub, особенно когда речь идет о размещении публичных образов контейнеров — базовых образов (как те, которые мы использовали в предыдущей главе), программного обеспечения с открытым исходным кодом наподобие MariaDB или каких-либо собственных программ или демоверсий, которыми вы хотите поделиться с миром.

К тому же в реестре Docker Hub (как и в любом другом реестре) можно разместить и приватные образы контейнеров, которые будут доступны для любой платформы Kubernetes, но потребуют некоторой дополнительной настройки учетных данных.

Для тех пользователей, которые хотят, чтобы их образы оставались *приватными*, более логичным вариантом является использование реестра контейнеров своего поставщика облачных услуг: это дает такие преимущества, как меньшее время скачивания образов, меньшие затраты на хранение данных в сети и более простой процесс аутентификации. Так, платформа Google Cloud предоставляет реестр Artifact Registry, платформа Amazon Web Services (AWS) — реестр Amazon Elastic Container Registry, платформа Azure — реестр Azure Container Registry и т. д.

Выбрав подходящий реестр контейнеров, выполните описываемый далее процесс загрузки контейнеров в сеть.

Настройка аккаунта

Зарегистрируйте аккаунт в выбранном вами сервисе, если вы еще этого не сделали, и создайте репозиторий, в который будете загружать свои образы. Если вы используете реестр Docker Hub, то перейдите по адресу <https://hub.docker.com/>, авторизуйтесь и перейдите по ссылке Create Repository (Создать репозиторий).

Если вы выбрали реестр Artifact Registry, то вам потребуется перейти по адресу <https://console.cloud.google.com/artifacts> и создать новый репозиторий типа Docker в указанном вами месте. Пометьте для себя сгенерированный сервисом путь, который будет выглядеть примерно так: `us-docker.pkg.dev/my-project/my-repository`.

Аутентификация

После этого нужно аутентифицировать инструмент командной строки платформы Docker, чтобы он мог загружать образы в только что созданный вами репозиторий. Для этого воспользуйтесь инструкциями, предлагаемыми для вашего реестра контейнеров.

Если вы выбрали реестр Docker Hub, выполните следующую команду:

```
docker login
```

Если же вы работаете с реестром Artifact Registry, воспользуйтесь сгенерированным ранее путем к репозиторию. Возьмите ту часть этого пути, которая представляет собой адрес хоста (в нашем примере это `us-docker.pkg.dev`), и с помощью приведенной ниже команды установите вспомогательную программу для консольной утилиты платформы Docker, чтобы вы могли загружать в сеть свои образы. Если вы используете несколько хостов, то эту команду можно выполнить для каждого из них:

```
HOST_NAME=us-docker.pkg.dev  
gcloud auth configure-docker $HOST_NAME
```

СОВЕТ

Выполнить аутентификацию платформы Docker, чтобы получить доступ к вашему облачному сервису, обычно несложно. Вам нужно просто найти подходящую для вашего облака команду настройки интерфейса командной строки платформы Docker, которая позволит ввести необходимые учетные данные. Попробуйте задать в поисковике запрос «аутентификация docker для доступа к реестру контейнеров [вашего облачного сервиса]».

Тег

При создании образов им присваивается случайное имя на основе хеша, которое выглядит примерно так: `82ca16cfe84`. В большинстве случаев рекомендуется заменять этот тег собственным, имеющим для вас какой-то смысл, чтобы вам было проще ссылаться на свои образы. В предыдущей главе мы использовали подобные теги для того, чтобы при запуске образов на локальной машине использовать в командах удобные имена (`docker run timeserver`), а не числовые идентификаторы (`docker run 82ca16cfe84`).

При загрузке контейнера в реестр контейнеров данный тег получает дополнительный смысл. Тег загружаемого образа должен представлять собой путь, составленный в соответствии с принятыми в реестре контейнеров правилами, чтобы он знал, в каком аккаунте и по какому адресу должен храниться образ (и чтобы ваш локальный клиент Docker знал, в какой реестр его нужно загружать). В этом случае нельзя использовать такое простое имя, как `timeserver`.

ТЕГ ВЕРСИИ

Тег версии (не путать с тегом образа контейнера. — *Примеч. науч. ред.*) — это неструктурированная строка, используемая для обозначения версии образа. В соответствии с принятым соглашением, он должен представлять собой номер версии (составленный по схеме: *основной_номер_версии.дополнительный_номер_версии.номер_патча*), к которому может быть добавлен необязательный суффикс. Например: `2.2.1`, `2.1.5`, `2.1.5-beta`. При запуске контейнеров может использоваться специальный тег версии `latest` для обозначения самого последнего образа, но данный тег нельзя использовать при загрузке образов в реестр, поскольку он присваивается автоматически реестром контейнеров.

В реестре Docker Hub используется следующее соглашение по именованию:

```
docker.io/$USERNAME/$REPOSITORY_NAME:$VERSION_TAG
```

где `$USERNAME` — ваше имя пользователя в системе Docker, `$REPOSITORY_NAME` — имя репозитория, созданного вами в реестре Docker Hub, а `$VERSION_TAG` — произвольная

строка (которая обычно содержит число). В моем случае этот путь выглядит как `docker.io/wdenniss/timeserver:1`, поскольку в системе Docker я использую имя пользователя `wdenniss`, мой репозиторий называется `timeserver` и в качестве произвольной строки я решил использовать `1`.

В каждом реестре используется свой формат тега. Например, в реестре Artifact Registry облачной платформы Google Cloud применяется следующий формат:

```
$LOCATION-docker.pkg.dev/$PROJECT_ID/$REPOSITORY_NAME/$IMAGE_NAME:$VERSION_TAG
```

После того как вы, используя графический интерфейс панели администрирования, создадите свой репозиторий в реестре Artifact Registry, на экран будет выведена первая часть этой строки (например, `us-docker.pkg.dev/wdenniss/ts`), которую вы можете скопировать через буфер обмена (или воспроизвести вручную, руководствуясь представленной выше формулой). К этой первой части можно добавить любое имя образа и тег версии, например `timeserver:1`. В моем случае результат выглядит так, как показано ниже; у вас должно получиться что-то похожее:

```
us-docker.pkg.dev/wdenniss/ts/timeserver:1
```

СОГЛАШЕНИЯ В ОТНОШЕНИИ ТЕГОВ, ИСПОЛЬЗУЕМЫЕ В РЕЕСТРАХ КОНТЕЙНЕРОВ

Каждый реестр частных контейнеров использует собственный способ составления тегов из отдельных строк, и каждый из этих способов немного отличается от других. Например, реестр платформы Azure (<http://mng.bz/o1YD>) использует тег вида `$REGISTRY_NAME.azurecr.io/$REPOSITORY_NAME:$VERSION_TAG`, а реестр платформы AWS (<http://mng.bz/nWod>) — тег вида `$AWS_ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/$REPOSITORY_NAME:$VERSION_TAG`. Единственное, что я могу сказать точно, — вы должны использовать тот способ составления тегов, который используется в вашем реестре контейнеров; в противном случае Docker не сможет правильно определить, куда следует поместить ваш образ. Выяснить, как именно нужно составлять теги, можно, вбив в поисковик запрос «тег реестра контейнеров [облачной платформы]».

После того как вы разберетесь с тем, как в вашем случае должен выглядеть тег образа (который в дальнейших примерах будет обозначаться как `$IMAGE_TAG`), вы можете присвоить теги всем тем Docker-образам, которые вы собрались загружать в реестр. Чтобы загрузить в реестр контейнеров один из тех образов, которые были созданы в предыдущей главе, вы можете сослаться на него с помощью предыдущего тега и дополнительно присвоить ему тег реестра контейнеров (каждый образ может иметь несколько тегов). Если вы произвели сборку контейнера с помощью

команды `docker build . -t timeserver`, как было показано в подразделе 2.2.1, то данный образ будет иметь тег `timeserver` и, соответственно, ему можно присвоить тег реестра контейнеров, как показано ниже:

```
IMAGE_TAG=us-docker.pkg.dev/wdenniss/ts/timeserver:1
docker tag timeserver $IMAGE_TAG
```

ПРИМЕЧАНИЕ

Если вы получили сообщение об ошибке `No such image` (Нет такого образа), то не волнуйтесь: сейчас мы произведем сборку этого образа заново.

Просмотреть полученный в результате список образов можно так:

```
$ docker images
REPOSITORY                                TAG      IMAGE ID      CREATED
timeserver                                latest   c07e34564aa0  2 minutes ago
us-docker.pkg.dev/wdenniss/ts/timeserver  1        c07e34564aa0  2 minutes ago
python                                     3.10     cf0643aafe49  1 days ago
```

Кроме того, присвоение тега можно выполнять, ссылаясь на существующие образы с помощью идентификатора образа (`docker tag $IMAGE_ID $IMAGE_TAG`), но во избежание путаницы я рекомендую присваивать какой-то тег еще на этапе сборки. На самом деле в большинстве случаев будет быстрее просто еще раз собрать образ с нуля, чем пытаться найти образ с нужным вам идентификатором.

Чтобы собрать и снабдить тегом контейнер `timeserver`, присвойте переменной `$IMAGE_TAG` имя вашего репозитория для образов и выполните команду сборки контейнера из его корневого каталога:

```
IMAGE_TAG=us-docker.pkg.dev/wdenniss/ts/timeserver:1
cd Chapter02/timeserver
docker build . -t $IMAGE_TAG
```

Загрузка в репозиторий

Итак, вы настроили репозиторий, аутентифицировали Docker и снабдили образ подходящим тегом. Теперь можно будет загрузить образ в репозиторий, используя следующую команду:

```
docker push $IMAGE_TAG
```

На предыдущем шаге аутентификации вы установили в конфигурацию Docker настройку, позволяющую платформе Docker общаться с реестром контейнеров той облачной платформы, которую вы решили использовать. Если вы получили сообщение об ошибке `Permission Denied` (Доступ запрещен), значит, вы либо не аутентифицировали Docker должным образом, либо неправильно составили строку тега образа. Проверьте, правильно ли была произведена аутентификация Docker для доступа к вашему репозиторию и правильно ли был составлен тег

образа, руководствуясь последней версией документации выбранного вами реестра контейнеров.

Если же все пройдет нормально, то на экран будут выведены представленные ниже результаты. Особое внимание следует обратить на последнюю строку, которая может содержать сообщения об ошибках аутентификации:

```
$ docker push $IMAGE_TAG
The push refers to repository [us-docker.pkg.dev/wdenniss/ts/timeserver]
9ab1337ca015: Pushed
3eaafa0b4285: Layer already exists
a6a5635d5171: Layer already exists
8c25977a7f15: Layer already exists
1cad4dc57058: Layer already exists
4ff8844d474a: Layer already exists
b77487480ddb: Layer already exists
cd247c0fb37b: Layer already exists
cfdd5c3bd77e: Layer already exists
870a241bfabd: Layer already exists
1: digest: sha256:edb99776ae47b...97f7a9f1864afe7 size: 2425
```

Таким образом, вы загрузили образ в сеть и теперь готовы к тому, чтобы произвести развертывание своего кода в Kubernetes!

3.2.3. Развертывание в Kubernetes

Теперь, когда у вас уже есть кластер и аутентифицированная утилита `kubectl`, можно приступить к развертыванию первого приложения. Для этого вы должны создать объект с очень подходящим для этого названием — `Deployment`. В Kubernetes используется декларативный способ определения конфигурации, при котором вы описываете (или «декларируете») необходимое вам состояние (например, «в кластере должны быть запущены три копии моего контейнера») в конфигурационном файле. После отправки этой конфигурации в кластер Kubernetes начинает следить за тем, чтобы состояние кластера соответствовало указанным вами требованиям.

В качестве таких конфигурационных файлов большинство разработчиков использует файлы формата YAML, поскольку их легче редактировать вручную. Можно использовать файлы формата JSON (преимущественно применяется для автоматизированной работы), и, кроме того, некоторые виды конфигурации могут создаваться императивно (о чем мы поговорим в разделе 3.3). В листинге 3.1 представлен простейший вариант определения развертывания для приложения сервера времени из главы 2. В нем мы ссылаемся на публичный образ контейнера, который я собрал на основе демонстрационного приложения и загрузил в реестр Docker Hub. Если у вас есть собственный образ, который вы загрузили в репозиторий контейнеров, как было описано выше, то отредактируйте этот файл, указав вместо моего образа свой.

Листинг 3.1. Chapter03/3.2_DeployingToKubernetes/deploy.yamll

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:1

```

Сколько реплик (экземпляров) пода необходимо развернуть

Какой образ контейнера необходимо развернуть и запустить

Данный манифест создаст три реплики контейнера. Далее будет показано, как настроить балансировщик нагрузки, чтобы поступающие запросы распределялись между этими тремя запущенными экземплярами. В этом минималистичном примере конфигурации развертывания самыми важными являются следующие три элемента: имя, которое будет использоваться для просмотра, изменения и удаления развертывания, количество реплик и имя контейнера. Остальное, по сути, лишь связующие элементы, необходимые для работы остальных (не волнуйтесь, в свое время мы разберемся и с тем, как работают эти связующие элементы).

Путь к образу контейнера представляет собой что-то вроде URL, указывающего, где находится контейнер. Если вы загрузили свой контейнер в реестр, как было описано в предыдущем подразделе, то у вас уже есть этот путь к образу. Мой образ контейнера с префиксом `docker.io` находится в реестре Docker Hub, который является популярным местом размещения публичных образов, в том числе базовых. Обратите внимание: если путь к образу не содержит название домена (то есть вы видите что-то вроде `ubuntu` или `wdenniss/timeserver`), то это просто сокращенный способ записи адреса образа, размещенного в реестре Docker Hub.

Таким образом, теперь вы знаете, как выглядит объект Deployment. Создадим его в кластере. Из корневого каталога нашего примера выполните следующую команду:

```

cd Chapter03/3.2_DeployingToKubernetes/
kubect1 create -f deploy.yamll

```

Команда дает системе Kubernetes указание создать объект, определенный в конфигурационном файле. Если вам нужно изменить этот объект после его создания (например, изменить номер версии образа), то внесите эти изменения в описание объекта в файле локально и обновите объект Deployment в кластере с помощью команды:

```

kubect1 apply -f deploy.yamll

```

Проверить состояние объекта Deployment можно с помощью команды:

```
$ kubectl get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
timeserver    3/3     3             3           36s
```

Как уже упоминалось, объект Deployment представляет собой декларативное описание установленных вами требований, например, требования запустить три реплики указанного пода. Когда в ответ на вашу команду Kubernetes сообщает об успешном создании объекта Deployment, это означает, что изменения в кластере запланированы, но не означает, что изменения осуществлены. Получая информацию по объекту Deployment с помощью команды `kubectl get`, можно видеть количество подов, готовых к обслуживанию трафика (число в столбце `READY`), и количество подов последней версии, если выполняется перевод подов на новую версию (число в столбце `UP-TO-DATE`). Для получения более подробных сведений о тех подах, которые содержит объект Deployment, можно получать информацию по подам напрямую:

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
timeserver-6df7df9cbb-7g4tx        1/1     Running   0           68s
timeserver-6df7df9cbb-kjg4d        1/1     Running   0           68s
timeserver-6df7df9cbb-lfq6w        1/1     Running   0           68s
```

ПРИМЕЧАНИЕ

Если ваши поды остаются в состоянии ожидания (**Pending**), это может говорить о том, что ваш кластер не имеет достаточного количества ресурсов. Если вы работаете в среде с динамическим предоставлением ресурсов, то обычно достаточно просто выждать еще одну-две минуты, после чего ваши поды уже будут распределены. Если же поды не выходят из состояния ожидания и после этого, то воспользуйтесь рекомендациями, изложенными ниже в пункте «Устранение неполадок: застревание в состоянии ожидания».

Команда `kubectl get pods` возвращает информацию о состоянии *всех* подов, находящихся в активном пространстве имен; если вы используете несколько объектов Deployment, эта информация может быть немного беспорядочной. В таком случае можно использовать более многословный формат этой команды, используя в качестве селектора метки интересующих вас подов (о метках мы поговорим подробнее в подразделе 3.2.4). Вот как выглядит этот полный вариант команды при использовании в качестве селектора метки нашего демонстрационного объекта Deployment:

```
$ kubectl get pods --selector=pod=timeserver-pod
NAME                                READY   STATUS    RESTARTS   AGE
timeserver-6df7df9cbb-7g4tx        1/1     Running   0           2m13s
timeserver-6df7df9cbb-kjg4d        1/1     Running   0           2m13s
timeserver-6df7df9cbb-lfq6w        1/1     Running   0           2m13s
```

Таким образом, мы запустили свой под и можем с ним взаимодействовать! Чтобы подключиться к только что созданному объекту Deployment и посетить развернутый нами сервер еще до создания публичного IP-адреса, можно просто перенаправить в контейнеры порт локальной машины:

```
$ kubectl port-forward deploy/timeserver 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Благодаря этому мы сможем взаимодействовать с объектом Deployment с локальной машины, перейдя по адресу <http://localhost:8080>. Чтобы убедиться в работоспособности нашего контейнеризированного приложения, откроем новое окно командной оболочки и выведем в нем записи журнала:

```
$ kubectl logs -f deploy/timeserver
Found 3 pods, using pod/timeserver-8bbb895dc-kg181
Listening on 0.0.0.0:80
127.0.0.1 - - [09:59:08] "GET / HTTP/1.1" 200 -
```

Команда `logs` с параметром `-f` (от follow, «продолжать») обеспечивает потоковый вывод записей журнала из одного из тех подов, которые содержит развертывание. В большинстве случаев бывает полезно вывести запись журнала в стандартный поток вывода (`stdout`) из своего приложения после его запуска, подобно тому, как в данном случае было выведено сообщение `Listening on 0.0.0.0:80` (Прослушивание порта 0.0.0.0:80). При таком подходе вы будете уверены в том, что ваш контейнер действительно запустился.

Большинство действий, которые вы будете выполнять в Kubernetes, не выполняется мгновенно. Как и многое другое, создание пода требует некоторого времени, поскольку нужно выделить дополнительную вычислительную мощность (размер которой зависит от того, какую платформу Kubernetes вы используете), скачать контейнер из реестра контейнеров и произвести его загрузку. Если все пройдет хорошо, то ваши контейнеры должны запуститься где-то через пару минут.

В случае успешного запуска поды вашего развертывания должны сообщить (при их опрашивании с помощью команды `kubectl get pods`), что они перешли в состояние выполнения (`Running`). Кроме того, вы можете столкнуться и с другими состояниями: состояние ожидания (`Pending`) говорит о том, что под ожидает выделения необходимой мощности, а состояние создания контейнера (`ContainerCreating`) — о том, что контейнер был распределен на имеющиеся у вас узлы и находится в процессе загрузки. Здесь может возникнуть такая проблема, как застревание пода в слегка неоднозначном состоянии ожидания, а также другие ошибки. Наиболее распространенные виды ошибок перечислены ниже.

Устранение неполадок: ошибка извлечения образа (ErrImagePull/ImagePullBackOff)

Данная ошибка говорит о том, что системе Kubernetes не удалось загрузить образ контейнера. Это может быть вызвано тем, что вы неправильно указали имя образа в своей конфигурации, указанный образ отсутствует в репозитории для контейнеров или учетные данные вашего кластера не позволяют получить доступ к репозиторию.

Проверьте, правильно ли записано имя образа и находится ли он в репозитории. Чтобы быстро запустить хоть какое-то развертывание, попробуйте вместо своего образа контейнера использовать публичный, например, тот, ссылка на который приводилась выше. Для применения внесенных вами изменений в конфигурации развертывания используйте команду `kubectl apply -f deploy.yaml`.

Устранение неполадок: застревание в состоянии ожидания

Если ваш под остается в состоянии ожидания (Pending) по прошествии одной-двух минут, это обычно означает, что планировщик системы Kubernetes не может найти в вашем кластере место для его развертывания. Решить эту проблему часто можно с помощью добавления в кластер дополнительных ресурсов, например, добавив дополнительный вычислительный узел или заменив имеющийся узел на более мощный.

Чтобы получить подробную информацию о поде, застрявшем в состоянии ожидания, воспользуйтесь командой `describe`:

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
timeserver-6df7df9cbb-7g4tx        1/1     Pending   0           1m16s

$ POD_NAME=timeserver-6df7df9cbb-7g4tx
$ kubectl describe pod $POD_NAME
```

Просмотрев результаты этой команды, вы увидите в разделе **Events (События)** список ошибок, с которыми столкнулась система Kubernetes. Если вы попытаетесь распределить развертывание, но количество ресурсов окажется недостаточным, то вы увидите предупреждение **FailedScheduling** (Не удалось выполнить планирование). Например, когда я попытался распределить под на недостаточное количество ресурсов, то система вывела следующее сообщение:

```
Warning FailedScheduling 26s (x2 over 26s) default-scheduler 0/2 nodes are
available:
➡ 2 Insufficient cpu.
```

Если при этом хотя бы один из ваших подов находится в состоянии выполнения (Running), то вам пока не о чем беспокоиться, поскольку для запуска своего сервиса

вам достаточно иметь хотя бы один под, способный отвечать на запросы. Но если в состоянии ожидания окажутся все ваши поды, то вам придется как-то исправить эту ситуацию, например, добавлять дополнительные вычислительные ресурсы.

Устранение неполадок: аварийное прекращение работы контейнера (CrashLoopBackOff)

Еще одна распространенная ошибка — аварийное прекращение работы контейнера, которое может объясняться разными причинами. В частности, иногда аварийный сбой происходит на этапе запуска контейнера (например, из-за ошибки в конфигурации), а иногда — спустя какое-то время после него.

В контексте развертываний Kubernetes *аварийным сбоем* считается любое прекращение работы контейнеризированного процесса, даже если он прекращает свою работу, выдав успешный код завершения. Развертывания служат для выполнения длительных процессов, а не разовых задач (для представления подов, которые должны выполняться как разовая задача, в Kubernetes служит объект Job, о котором пойдет речь в главе 10).

Эпизодические аварийные сбои контейнеров, которые, как в данном случае, представлены в виде пода, управляемого объектом Deployment, аккуратно устраняются с помощью их перезапуска. На самом деле, помимо прочего, команда `kubectl get pods` выводит и информацию о том, сколько раз уже перезапускался тот или иной контейнер. Если один из ваших контейнеров выходит из строя с интервалом примерно в час, то Kubernetes посчитает это совершенно нормальным и будет просто периодически перезапускать его и снова переходить к своей обычной работе.

Но если контейнер сбоит в ходе загрузки или вскоре после нее, то Kubernetes переводит его в экспоненциальный цикл перезапуска. Вместо того чтобы непрерывно пытаться перезапустить контейнер (расходуя на это системные ресурсы), Kubernetes вводит задержку между попытками перезапуска, размер которой увеличивается по экспоненциальному закону (то есть 10, 20, 40 секунд и т. д.).

При первом сбое контейнера ему присваивается статус `RunContainerError` (Ошибка запуска контейнера), если сбой происходит на этапе запуска, или `Completed` (Завершено), если он успел завершить работу. Если сбой повторяется, то контейнеру присваивается статус `CrashLoopBackOff` (Цикл аварийных перезапусков). В каждом таком контейнере, как правило, есть проблема, требующая вашего внимания. В частности, сбой может происходить из-за того, что контейнеру не предоставляется необходимая внешняя зависимость (например, база данных). В таком случае нужно проследить за тем, чтобы внешняя зависимость должным образом запускалась и была доступной для подключения.

При отладке сбойных контейнеров я всегда сначала выполняю команду `kubectl describe pod $POD_NAME`, как было сделано выше, чтобы поискать объяснение

проблем в описании событий. Кроме того, стоит просмотреть созданные контейнером записи журнала, отобразить которые можно с помощью команды `kubectl logs $POD_NAME`. Чтобы разобраться в причинах сбоя, будет полезно отобразить записи журнала, созданные для *предыдущего* экземпляра контейнера (который работал до аварийного перезапуска), и посмотреть, какие сообщения об ошибках выводились в момент сбоя. Для этого добавьте в команду для извлечения записей журнала параметр `--previous` (или просто `-p`):

```
kubectl logs -p $POD_NAME
```

3.2.4. Спецификация пода

Теперь чуть подробнее остановимся на структуре объекта Deployment. Важно понимать, что этот объект, по сути, является оболочкой объекта Pod, у которого тоже есть своя спецификация. Аналогично в Kubernetes устроены и другие виды высокоуровневых рабочих нагрузок, например объект Job. Этот момент важен еще и по той причине, что при публикации ввне объекта Deployment в виде объекта Service мы на самом деле будем ссылаться не на объект Deployment, а на поды.

Создавая объект Deployment с тремя репликами, вы в действительности даете указание контроллеру развертываний системы Kubernetes создать и обеспечить управление тремя подами. После этого контроллер развертываний обеспечивает управление жизненным циклом этих трех подов, в том числе их замену более новыми версиями при обновлении развертывания новым контейнером и перераспределение подов, вытесненных в ходе запланированных или незапланированных событий обслуживания. Как в данном случае выглядит структура объекта Deployment, показано на рис. 3.10.

В документации по API Kubernetes шаблон объекта Pod упоминается как шаблон PodSpec. В принципе, его можно «выдернуть» из определения объекта Deployment и запустить отдельно. Для этого вам потребуется создать отдельный конфигурационный YAML-файл с заголовком, указывающим, что это объект Pod, а не объект Deployment, и скопировать на корневой уровень нового файла все элементы, расположенные ниже слова `template` (шаблон) (листинг 3.2).

Листинг 3.2. Chapter03/3.2.4_ThePodSpec/pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: timeserver
  labels:
    pod: timeserver-pod
spec:
  containers:
  - name: timeserver-container
    image: docker.io/wdenniss/timeserver:1
```



Рис. 3.10. Объект Deployment с вложенным в него подом

При желании вы можете создать этот под напрямую, но в таком случае его жизненным циклом уже не будет управлять контроллер Kubernetes. Такие поды перезагружаются в случае сбоя, но не распределяются заново при их вытеснении вследствие наступления события обновления или отказа узла. Именно поэтому вместо того чтобы распределять поды напрямую, в большинстве случаев лучше делать это с помощью какого-либо высокоуровневого объекта — уже знакомого нам объекта Deployment и других, которые будут рассмотрены в дальнейших главах (объектов StatefulSet, Job и т. д.).

ПРИМЕЧАНИЕ

Важным следствием того, что объекты в Kubernetes организованы так, является то, что при размещении шаблона PodSpec внутри таких объектов, как Deployment, он всегда обладает теми же функциональными возможностями, что и объект Pod. То есть вы можете посмотреть в документации, какими полями обладает описание объекта Pod, и использовать любое из них в шаблоне пода.

Шаблон PodSpec содержит ключевую информацию о вашем приложении, в том числе сведения о том, какие контейнеры входят в его состав. Каждый из них получает собственное имя (чтобы можно было ссылаться на отдельные контейнеры внутри многоконтейнерного пода) и самое важное поле, содержащее путь к образу контейнера. Помимо этого, существует и много необязательных полей, часть которых играют важную роль при определении проверок работоспособности и необходимых ресурсов (эти поля будут рассмотрены в дальнейших главах).

Вы могли заметить в этом примере две одинаковые метки. И в разделе `selector → matchLabels` спецификации развертывания, и в разделе `metadata → labels` шаблона PodSpec содержится одна и та же пара «ключ — значение» (`pod: timeserver-pod`). Что же здесь происходит?

Объект Pod создается как независимый, находящийся под управлением контроллера развертываний, поэтому нам нужно как-то к нему обращаться. Чтобы решить эту проблему, Kubernetes требует, чтобы каждый под обладал меткой (которая может представлять собой произвольную пару «ключ — значение») и чтобы она указывалась (выбиралась) в определении развертывания. По сути, это «связующее звено», с помощью которого эти два объекта соединяются. Более наглядно это показано на рис. 3.11.

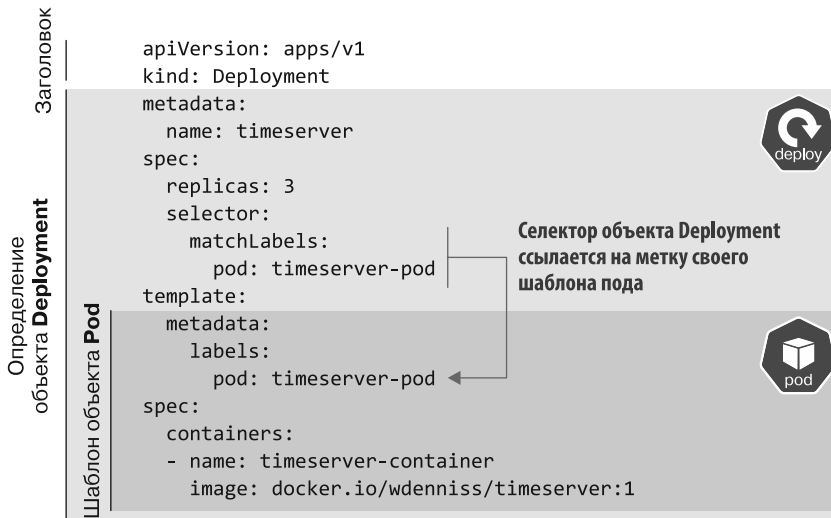


Рис. 3.11. Связь между селектором объекта Deployment и меткой шаблона пода

На первый взгляд, это кажется излишним: неужели Kubernetes не может связать эти два объекта, исходя из того факта, что объект PodSpec вложен в объект Deployment? Kubernetes требует, чтобы вы указывали эти метки вручную, поскольку они играют важную роль при непосредственном обращении к подам из других объектов. Так, например, в следующем подразделе мы займемся настройкой сетевого сервиса, который будет обращаться не к объекту Deployment, а непосредственно к его подам. То же самое справедливо и для других концепций, которые будут рассмотрены в книге далее, таких как «Квота неработающих подов» (Pod Disruption Budget, PDB). Указав метки для своих подов, вы будете знать, какие метки следует использовать для обращения к ним из других объектов. Необходимо помнить, что

под является в Kubernetes базовой единицей исполнения и планирования, в то время как объект `Deployment` — лишь один из множества возможных способов создания подов, управления ими и взаимодействия с ними.

Что касается самой метки, то она может представлять собой совершенно любую пару «ключ — значение», и даже просто заполняющий текст наподобие `foo: bar` — для Kubernetes это не имеет никакого значения. Я использовал здесь метку вида `pod: timeserver-pod`, поскольку нахожу такой способ обозначения удобным при выборе подов из других объектов. Кроме того, в документации часто используются метки вида `app: timeserver`. Я решил обойтись без использования имени объекта `Deployment (timeserver)` в качестве значения метки, чтобы вы ошибочно не подумали, что оно как-то связано с меткой пода (несмотря на то что здесь нет никакой связи).

Вот как устроен объект `Deployment` с вложенным шаблоном `PodSpec`. Я надеюсь, что понимание структуры этого объекта и механизма обращения к подам окажется для вас полезным. В следующем подразделе мы сделаем этот объект `Deployment` доступным для внешних пользователей, которые будут обращаться к подам, используя их метки.

3.2.5. Публикация сервиса

После успешного развертывания своего контейнера вы, конечно, захотите с ним повзаимодействовать. Каждый под получает собственный внутрикластерный IP-адрес, с помощью которого поды могут взаимодействовать в пределах кластера. Поды можно сделать доступными в Интернете напрямую или через IP-адрес узла (используя поле `hostPort`), но если речь не идет о создании сервера для игр, работающих в режиме реального времени, то такой подход используется крайне редко. В большинстве случаев и особенно при использовании объекта `Deployment` будет лучше объединить все поды в сервис, который предоставит пользователям единую точку доступа с внутренним (и, как вариант, внешним) IP-адресом и обеспечит распределение запросов между подами путем балансировки нагрузки. А если у объекта `Deployment` один под, то создание сервиса позволит предоставить пользователям стабильный адрес.

Помимо балансировки нагрузки, сервисы также следят за тем, какие поды в данный момент работают и могут принимать трафик. Например, если вы укажете в определении развертывания, что вам нужно запустить три реплики, это совсем не значит, что такое количество реплик будет доступно в любой момент. На этапе обновления узла могут оказаться доступными только две реплики, а на этапе загрузки новой версии развертывания — наоборот, больше трех. Сервис будет направлять трафик только к работающим подам (в следующей главе я расскажу, какие данные при этом нужно предоставить, чтобы все работало корректно).

Сервисы могут использоваться и внутри кластера для обеспечения взаимодействия различных приложений (такой подход часто называют «микросервисной архитектурой»). В таком случае они позволяют использовать такие удобные возможности, как, например, обнаружение сервисов. Данная тема будет подробно рассмотрена в главе 7, а пока сфокусируемся исключительно на том, как используются сервисы. Чтобы сделать доступным в Интернете свое новое приложение, определите сервис типа `LoadBalancer` (Балансировщик нагрузки), который будет предоставлять доступ к приложению конечным пользователям. Как и в случае объекта `Deployment`, мы должны начать с определения конфигурационного YAML-файла (листинг 3.3).

Листинг 3.3. Chapter03/3.2_DeployingToKubernetes/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: timeserver
spec:
  selector:
    pod: timeserver-pod
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
  type: LoadBalancer
```

В списке портов вы можете указать, какой порт должен быть доступен пользователям сервиса (`port`) и на какой порт пода будет направляться этот трафик (`targetPort`). Это позволяет вам, например, сделать определенный сервис доступным на порте 80 (который используется в качестве HTTP-порта по умолчанию) и подключить его к приложению в контейнере, работающем на порте 8080.

Поскольку каждый под и сервис в Kubernetes имеет собственный внутрикластерный IP-адрес, вы можете не бояться того, что ваши поды столкнутся с конфликтом портов. Вы можете запустить свое приложение на любом выбранном вами порте (например, на порте 80, если вы работаете с HTTP-сервисом) и для простоты указать его и в поле `port`, и в поле `targetPort`, как было сделано в приведенном выше примере. В таком случае можно вообще опустить поле `targetPort`, поскольку вместо значения этого поля по умолчанию используется значение поля `port`.

Все сервисы (за исключением Headless-сервисов, о которых мы поговорим в главе 9) получают внутрикластерный IP-адрес, который могут использовать входящие в кластер поды. Если вы укажете тип `LoadBalancer`, как было сделано в представленном выше примере, то сервис получит дополнительно и внешний IP-адрес.

Обратите внимание: у данного сервиса, как и у нашего объекта Deployment, есть раздел `selector`. Однако с помощью этого селектора выбирается не объект Deployment, о котором сервис фактически ничего не знает, а поды, имеющие указанную метку (в данном случае поды, созданные нашим объектом Deployment). Более наглядно это показано на рис. 3.12.

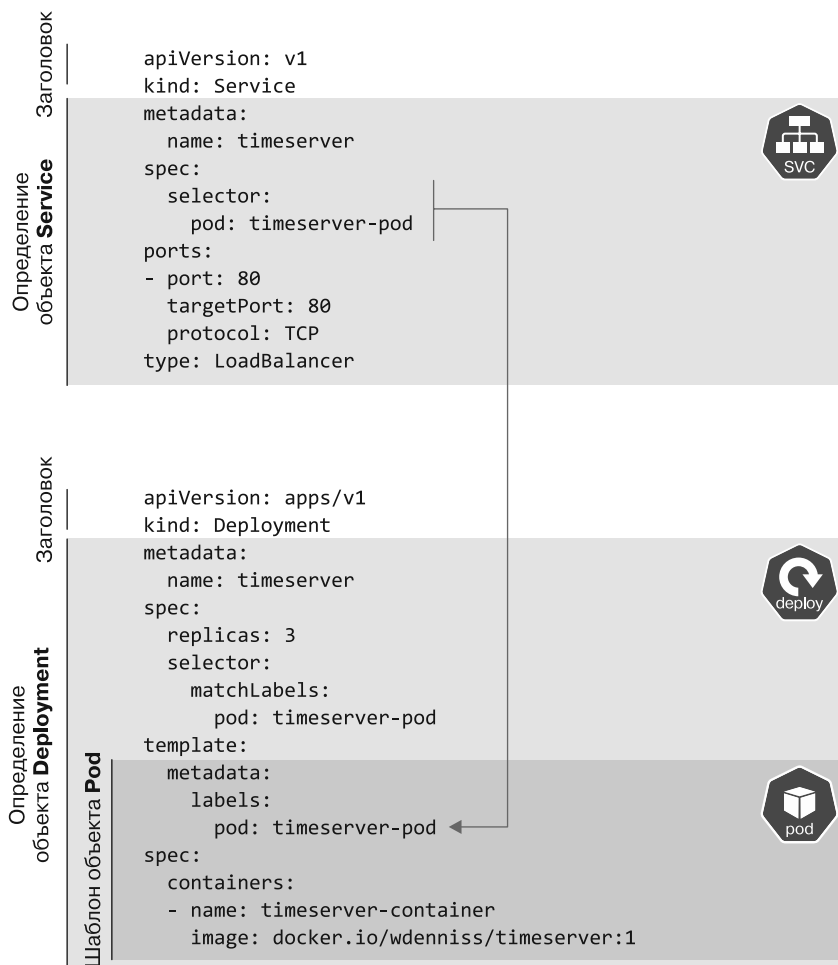


Рис. 3.12. Связь между сервисом и выбираемыми им целевыми подами

В отличие от раздела `selector` объекта Deployment, раздел `selector` объекта Service не имеет вложенного раздела `matchLabels`. Тем не менее эти два селектора эквивалентны, просто в Deployment используется более новый, более выразительный синтаксис платформы Kubernetes. И в Deployment, и в Service селектор

обеспечивает один и тот же результат, а именно определяет, с каким набором подов должен сопоставляться объект.

Чтобы создать объект Service в своем кластере, выполните следующие команды:

```
cd Chapter03/3.2_DeployingToKubernetes
kubectl create -f service.yaml
```

Обратите внимание: и для создания объекта Deployment, и для создания объекта Service используется одна и та же команда (`kubectl create`). Четыре базовые CRUD-операции — создание (`create`), чтение (`read`), обновление (`update`) и удаление (`delete`) — в Kubernetes выполняются для всех объектов с помощью четырех команд `kubectl`: `kubectl create`, `kubectl get`, `kubectl apply` и `kubectl delete`.

Чтобы отобразить информацию о состоянии объекта Service, следует вызвать команду `kubectl get`, указав соответствующий тип объекта:

```
$ kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
Kubernetes	ClusterIP	10.22.128.1	<none>	443/TCP	1h
Timeserver	LoadBalancer	10.22.129.13	203.0.113.16	80:30701/TCP	26m

Как можно заметить, помимо вашего объекта Service (в нашем случае имеющего имя `timeserver`), в этом списке есть и Service с именем `kubernetes`. Вы можете проигнорировать данную информацию, поскольку это сервис API Kubernetes, запущенный в вашем кластере. Кроме того, можно указать при вызове команды только тот объект Service, который вас интересует: `kubectl get service $SERVICE_NAME`.

Если в столбце `External IP` отображается состояние ожидания (`Pending`), это говорит лишь о том, что внешний IP-адрес ожидает, пока балансировщик нагрузки завершит подключение к сети. Обычно этот процесс завершается через одну-две минуты, поэтому не торопитесь считать состояние ожидания проблемой. Приступать к отладке следует лишь в том случае, если IP-адрес остается в таком состоянии в течение достаточно большого количества времени. Чтобы не вызывать многократно приведенную выше команду `get`, можно обеспечить потоковый вывод информации обо всех изменениях состояния, добавив в нее флаг `--watch/-w` (например: `kubectl get service -w`). Выполните эту команду, и через одну-две минуты на экране появится информация о том, что ваш объект Service уже имеет внешний IP-адрес.

ПРИМЕЧАНИЕ

Чтобы вам был предоставлен внешний IP-адрес, вы должны использовать систему Kubernetes, запущенную на облачной платформе, поскольку такая платформа за кулисами предоставляет и сетевой балансировщик нагрузки с возможностью доступа по внешнему IP. Если же вы используете локальную версию Kubernetes, то прочтите подраздел 3.4.3, в котором рассказывается о подключении с помощью таких инструментов, как команда `kubectl port-forward`.

После того как IP-адрес выйдет из состояния ожидания, попробуйте получить доступ к сервису, перейдя по его URL. Применительно к рассмотренному выше примеру это адрес `http://203.0.113.16` (вы должны использовать вместо него тот внешний IP-адрес, который выводит команда `kubectl get service` в вашем случае). Для проверки HTTP-запросов либо воспользуйтесь инструментом командной строки `curl` (`curl http://203.0.113.16`), либо попробуйте перейти по адресу сервиса в браузере:

```
$ curl http://203.0.113.16
The time is 7:01 PM, UTC.
```

Устранение неполадок: «Не удается подключиться»

Две наиболее частые причины ошибки «Не удается подключиться» (`Unable to Connect`) — некорректное определение селектора или портов. Еще раз тщательно проверьте, те ли метки выбирает селектор в шаблоне пода вашего развертывания. Убедитесь, что ваш контейнер действительно прослушивает тот порт, который указан в качестве целевого порта (это можно сделать, выведя на этапе загрузки отладочное сообщение с информацией о том, какой порт используется контейнером), и что вы правильно указали порт в адресной строке браузера.

Попробуйте подключиться напрямую к одному из своих подов через порт, указанный в `targetPort`, перенаправив порты с помощью утилиты `kubectl`. Если вам не удастся подключиться к поду напрямую, то, вероятно, проблема кроется в нем самом. Если вам удастся подключиться к поду напрямую, то ошибка может скрываться в определении сервиса, а если нет — в самом поде. Перенаправить порт к одному из подов развертывания можно с помощью следующей команды:

```
kubectl port-forward deploy/$DEPLOYMENT_NAME $FROM_PORT:$TO_PORT
```

где `$FROM_PORT` — порт, который вы будете использовать на локальной машине, а `$TO_PORT` — порт, указанный вами в объекте `Service` в качестве целевого порта (`targetPort`). Для рассмотренного выше примера это будет выглядеть так:

```
kubectl port-forward deploy/timeserver 8080:80
```

После выполнения этой команды перейдите в браузере по адресу `http://localhost:8080`. При этом будет автоматически выбран один из подов объекта `Deployment` (в обход объекта `Service`). Кроме того, вы можете подключиться напрямую к конкретно указанному поду, используя команду следующего вида:

```
kubectl port-forward pod/$POD_NAME $FROM_PORT:$TO_PORT
```

Устранение неполадок: External-IP застрял в состоянии Pending

Получение внешнего IP-адреса часто требует времени, поэтому попробуйте выждать еще несколько минут. Убедитесь в том, что ваш облачный провайдер предоставляет внешний IP-адрес сервисам типа `LoadBalancer`. Поищите в документации провайдера дополнительную информацию о настройке балансировщиков нагрузки в Kubernetes.

Если вы ведете разработку на локальной машине или просто хотите опробовать сервис, не дожидаясь получения внешнего IP-адреса, то перенаправьте к сервису порт локальной машины:

```
kubect1 port-forward service/$SERVICE_NAME $FROM_PORT:$TO_PORT
```

3.2.6. Взаимодействие с объектом Deployment

В ходе разработки полезно иметь возможность взаимодействовать с контейнером, чтобы копировать файлы в него и из него или выполнять команды. К счастью, в Kubernetes сделать это так же легко, как и в Docker.

Выполнение разовых команд

Подобно тому как в Docker можно выполнять разовые команды над образами с помощью команды `docker exec` (о которой мы говорили в главе 2), в Kubernetes можно выполнять разовые команды над подами, используя команду `kubect1 exec`.

Для диагностики возникающих в контейнере проблем часто используется команда `sh`, которая запускает в контейнере интерактивную командную оболочку (при условии, что в нем доступен командный интерпретатор `sh`). Запустив эту оболочку, вы можете затем выполнить все необходимые отладочные операции.

Хотя команда `exec` всегда выполняется над подом, вместо конкретного пода можно указать и развертывание; в таком случае утилита `kubect1` выполнит эту команду над случайно выбранным подом развертывания:

```
$ kubect1 exec -it deploy/timeserver -- sh
# echo "Testing exec"
Testing exec
```

Над контейнером можно выполнить и любую другую команду:

```
$ kubect1 exec -it deploy/timeserver -- echo "Testing exec"
Testing exec
```

Копирование файлов в контейнер и из него

Опять же, подобно тому, как это делается в Docker, вы можете копировать файлы между системой и контейнером с помощью команды `cp` утилиты `kubectl`. Эта команда требует, чтобы образ контейнера содержал утилиту `tar`. Данная возможность может оказаться полезной, когда требуется скачать создаваемые приложением записи журнала или другую диагностическую информацию. В качестве пути по умолчанию используется рабочий каталог контейнера; то есть если контейнер содержит файл `example.txt`, то его можно скопировать на свою машину так:

```
kubectl cp $POD_NAME:example.txt example.txt
```

Выполнить копирование в противоположном направлении можно так:

```
kubectl cp example.txt $POD_NAME:
```

3.2.7. Обновление приложения

После того как вы произведете развертывание своего приложения и сделаете его доступным для внешних пользователей, вам рано или поздно потребуется его обновить. Измените код приложения из нашего примера, а затем снова произведите сборку образа контейнера и загрузите его в репозиторий для контейнеров с новым тегом версии. Например, если раньше вы использовали тег `us-docker.pkg.dev/wdenniss/ts/timeserver:1`, то можно загрузить новый образ с тегом `us-docker.pkg.dev/wdenniss/ts/timeserver:2`. В принципе, вы можете использовать любые теги, однако общепринятая практика сводится к тому, чтобы указывать последовательно возрастающие номера версии.

После того как вы загрузите образ контейнера в репозиторий (как было показано в подразделе 3.2.2), обновите файл `deploy.yaml` из листинга 3.1, указав новое имя образа — например, как это было сделано в листинге 3.4 (в строке, выделенной жирным шрифтом).

Листинг 3.4. Chapter03/3.2.7_Updating/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
```

```

spec:
  containers:
  - name: timeserver-container
    image: docker.io/wdenniss/timeserver:2

```

Новая версия
образа

Сохраните файл и примените это изменение к своему кластеру:

```

$ kubectl apply -f deploy.yaml
deployment.apps/timeserver configured

```

Затем произойдет нечто интересное. Как вы помните, Kubernetes постоянно следит за соблюдением заданных вами требований и при необходимости корректирует наблюдаемое состояние системы. Вы только что объявили, что развертывание теперь использует образ версии 2, в то время как мы пока используем поды версии 1. Поэтому Kubernetes постарается скорректировать состояние так, чтобы развертывание содержало только поды текущей версии.

Мы можем посмотреть, как это происходит, выполнив команду `kubectl get deploy`. Вот как может выглядеть ее вывод:

```

$ kubectl get deploy
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
timeserver    3/3    1           3           10m

```

Содержимое столбца `READY` указывает, сколько подов готово к обслуживанию трафика и сколько было запрошено. В данном случае в состоянии готовности находятся все три пода. Однако содержимое столбца `UP-TO-DATE` говорит о том, что пока мы используем только один под текущей версии. Этому есть объяснение. Вместо того чтобы обновлять все поды одновременно, тем самым делая приложение на какое-то время недоступным, Kubernetes по умолчанию использует «скользящую» стратегию обновления, обновляя за раз только один или какую-то часть подов.

В следующей главе мы подробно рассмотрим эту и другие стратегии обновления, а также поговорим о том, какие проверки работоспособности следует настроить во избежание проблем в ходе обновления. Пока просто имейте в виду, что Kubernetes постарается применить внесенные вами изменения, заменив старые поды версии 1 новыми подами версии 2.

После того как количество подов, указанное в столбце `UP-TO-DATE`, станет равным количеству, указанному в столбце `READY`, процесс обновления будет завершен. Чтобы посмотреть, какие поды были созданы и замещены, можно воспользоваться командой `kubectl get pods`, которая отобразит список всех подов развертывания: и новых, и старых.

Отслеживание процесса обновления

Команда `kubectl get` выводит лишь мгновенный снимок состояния, а состояние развертывания подвержено частым изменениям, поэтому операторы кластера обычно автоматически отслеживают состояние развертывания, чтобы не выполнять одну

и ту же команду снова и снова. Обеспечить автоматическое отслеживание, в частности, можно с помощью вызова команды `kubectl` с флагом `--watch/-w`, например: `kubectl get pods -w` или `kubectl get deploy -w`. Добавление этого флага делает возможным потоковый вывод в консоль информации обо всех изменениях состояния.

Недостатком флага `watch` является то, что при его использовании консольный вывод становится немного хаотичным. Если вы используете достаточно большое количество подов, то на экран будут беспорядочно выводиться строки с описанием происходящих изменений, из которых будет трудно сделать вывод о текущем состоянии системы. Я предпочитаю вместо флага использовать команду `watch` операционной системы Linux. В отличие от него, эта команда обновляет всю информацию о состоянии, дополнительно указывая, что изменилось по сравнению с предыдущим состоянием. Эта команда доступна в большинстве дистрибутивов Linux, в macOS и WSL, вне зависимости от того, откуда вы загружаете свои пакеты.

Если в вашей системе доступна команда `watch`, то вы можете просто записать ее перед любой командой `kubectl`:

```
watch kubectl get deploy
```

Я предпочитаю вызывать эту команду с флагом `-d`, который выделяет все происходящие изменения:

```
watch -d kubectl get deploy
```

Используя лишь команды `watch` и `kubectl`, вы можете открыть сразу несколько окон терминала для отслеживания вывода разных команд (или воспользоваться менеджером терминалов `tmux`) и таким образом скомпоновать информационную панель, которая будет в режиме реального времени отображать всю необходимую информацию о состоянии.

Наблюдение за объектом Deployment

Уже рассмотренные нами команды `kubectl get deploy` и `kubectl get pods` возвращают информацию обо всех развертываниях и подах, находящихся в текущем пространстве имен. Если у вас много объектов `Deployment`, то можно отобразить информацию только об одном из них:

```
kubectl get deploy $DEPLOYMENT_NAME
```

В качестве имени интересующего вас объекта следует указать имя, указанное в начале файла развертывания в поле `name` раздела `metadata`. При желании вы также можете вывести и информацию обо всех подах определенного объекта `Deployment`; для этого нужно отобразить интересующий вас набор подов с помощью селектора меток:

```
kubectl get pods --selector=pod=timeserver-pod
```

где `pod=timeserver-pod` — селектор меток, указанный в определении объекта `Deployment`.

3.2.8. Освобождение ресурсов

Существует несколько способов удаления созданных объектов. Один из них сводится к тому, чтобы удалять объекты, указывая их тип и имя:

```
$ kubectl delete deploy timeserver
deployment.apps "timeserver" deleted
$ kubectl delete service timeserver
service "timeserver" deleted
$ kubectl delete pod timeserver
pod "timeserver" deleted
```

ПРИМЕЧАНИЕ

Достаточно удалить лишь те поды, которые вы создавали напрямую. Поды, которыми управляет такой объект, как Deployment, автоматически удаляются при его удалении.

Кроме того, вы можете удалять объекты, указывая в команде удаления соответствующий конфигурационный файл или каталог для конфигурационных файлов:

```
$ cd Chapter03
$ kubectl delete -f 3.2_DeployingToKubernetes
deployment.apps "timeserver" deleted
service "timeserver" deleted
$ kubectl delete -f 3.2.4_ThePodSpec/pod.yaml
pod "timeserver" deleted
```

Если после удаления вы передумаете, то можно будет просто создать эти файлы снова (с помощью команды следующего вида: `kubectl create -f 3.2_DeployingToKubernetes`). Этим и хорошо определение конфигурации в файлах: вам не нужно помнить, какие изменения вносились в текущее состояние, поскольку все изменения сначала отражаются в конфигурации.

Определенных накладных расходов требует и кластер, поэтому его тоже можно удалить по завершении работы с системой. Большинство облачных провайдеров позволяет это сделать с помощью графической панели администрирования. Если же вы используете сервис GKE из командной строки, то удалить кластер можно с помощью команды `gcloud container clusters delete $CLUSTER_NAME --region $REGION`. Даже если в кластере нет подов и сервисов, определенных накладных расходов требуют сами узлы (за исключением того случая, если вы используете что-то вроде сервиса GKE в режиме «Автопилот»); удаление кластера позволит высвободить и эти ресурсы. Если вы решили не удалять кластер и используете платформу с оплатой за каждый узел, то, помимо лишних объектов Kubernetes, вам следует удалить и лишние узлы.

СОВЕТ

Далее я буду исходить из предположения, что вы умеете освобождать ненужные вам ресурсы. При выполнении примеров, которые будут приводиться ниже (и любых других примеров), не забывайте об этом моменте и обязательно удаляйте объекты, которые вам уже не нужны, чтобы использовать меньше ресурсов и таким образом экономить свои деньги!

3.3. ИМПЕРАТИВНЫЕ КОМАНДЫ

Kubernetes предлагает два способа взаимодействия с системой: декларативный, при котором вы определяете (декларируете) в конфигурационных файлах необходимое вам состояние и применяете эти конфигурации к кластеру, и императивный, при котором вы последовательно выдаете API команды, чтобы в итоге добиться желаемого результата. Ориентированный на конфигурацию декларативный подход является более предпочтительным для большинства специалистов, хорошо знакомых с этой платформой (и меня в том числе), поэтому чаще используется на практике.

Однако на самом деле создать объект Deployment со своим контейнером и сделать его доступным в Интернете можно и с помощью императивных команд. Для полноты картины посмотрим, как это будет выглядеть в нашем случае (предположим, что мы удалили созданные ранее файлы, выполнив процедуру освобождения ресурсов, как было предложено в подразделе 3.2.8).

1. Создаем развертывание:

```
$ kubectl create deployment timeserver \
  --image=docker.io/wdenniss/timeserver:1
deployment.apps/timeserver created
```

2. Создаем сервис типа LoadBalancer и делаем его доступным на порте 80:

```
$ kubectl expose deployment timeserver --type=LoadBalancer --port 80
service/timeserver exposed
```

3. Проверяем результат:

```
$ kubectl get deploy,svc
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/timeserver	1/1	1	1	4m49s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	AGE
service/Kubernetes	ClusterIP	10.22.128.1	<none>	5m2
service/timeserver	LoadBalancer	10.22.130.202	<pending>	31s

4. Обновляем контейнер в развертывании до новой версии:

```
$ kubectl set image deployment timeserver timeserver=wdenniss/timeserver:2
deployment.apps/timeserver image updated
```

На первый взгляд, этот подход кажется более простым по сравнению с управлением системой Kubernetes с помощью конфигурационных файлов, которые, надо сказать, иногда бывают слишком многословными. Однако существует несколько веских причин, позволяющих сделать выбор в пользу подхода, ориентированного на конфигурацию. Первая из этих причин — воспроизводимость. Допустим, вам нужно воспроизвести определенную конфигурацию в другой среде, например,

в предпродуктивной или продуктивной, что достаточно часто требуется на практике. В случае декларативного подхода это можно сделать, применив в новой среде ту же самую конфигурацию (при необходимости внеся в нее небольшие поправки). Однако при императивном подходе вам придется каждый раз вспоминать всю необходимую последовательность команд, которую, вероятно, будет лучше сохранить в `bash`-скрипте.

Еще одна причина — при императивном подходе труднее вносить изменения. Чтобы изменить какие-либо настройки при использовании конфигурационных файлов, достаточно просто обновить и заново применить конфигурацию, после чего Kubernetes педантично реализует все, что в ней указано. При императивном подходе для каждого изменения придется использовать отдельную команду: `kubect1 set image` для изменения образа, `kubect1 scale` для изменения количества реплик и т. д. Кроме того, внесение изменений может закончиться неудачей, если какая-либо команда даст сбой, например, из-за превышения времени ожидания сети. А вот при декларативном подходе сбой обновления легко исправляется путем повторного применения изменений. В главе 11 мы поговорим о методологии GitOps, которая предполагает использование конфигурационных файлов подобно файлам исходного кода в программировании. Этот подход вообще несовместим с использованием императивных команд.

Однако не стоит волноваться, если вы столкнетесь с системой, при создании которой использовался императивный подход. В таком случае можно экспортировать конфигурацию кластера с помощью команды `kubect1 get -o yaml $RESOURCE_TYPE $RESOURCE_NAME`. Однако имейте в виду, что при этом нужно будет удалить ненужные поля, как будет показано в подразделе 11.1.2. К счастью, вы в любой момент можете перейти от одного подхода к другому, поскольку и при декларативном, и при императивном подходе Kubernetes сохраняет объекты одинаковым образом.

3.4. ЛОКАЛЬНЫЕ СРЕДЫ KUBERNETES

До сих пор в главе в качестве среды разработки мы использовали облачную платформу Kubernetes. Но с тем же успехом вы можете использовать и локальную версию Kubernetes. При описании процесса развертывания в Kubernetes я решил начать с облачной платформы, а не кластера для локальной разработки, поскольку конечная цель разработки обычно сводится к тому, чтобы опубликовать сервис, сделав его доступным за пределами своей машины. Если вы последовательно выполняли все приведенные ранее примеры, то вас уже можно поздравить: теперь вы знаете, как можно выполнить развертывание своих приложений в Kubernetes, сделав их доступными всему миру! В следующих главах вы узнаете, как эти приложения можно ввести в эксплуатацию, масштабировать и т. д.

НУЖНО ЛИ ИСПОЛЬЗОВАТЬ КЛАСТЕР KUBERNETES В ХОДЕ РАЗРАБОТКИ ПРИЛОЖЕНИЯ

Если вы собрались использовать Kubernetes при развертывании приложения в продуктивной среде, это еще не означает, что вы должны использовать Kubernetes и в ходе разработки приложения. Многие разработчики проводят локальную разработку и тестирование с использованием утилиты Docker Compose (о которой шла речь в разделе 2.3), а затем используют Kubernetes для развертывания окончательной версии приложения в продуктивной среде.

Утилита Docker Compose хорошо подходит для разработки приложений, имеющих ограниченное количество межсервисных зависимостей. При таком подходе приходится определять конфигурацию приложения дважды (для разработки с использованием Compose и для развертывания в продуктивной среде с помощью Kubernetes), но если приложение имеет ограниченное количество зависимостей, то реализация подхода потребует не слишком много дополнительных усилий. С другой стороны, такой подход позволяет использовать в ходе разработки некоторые полезные возможности Docker, например, монтировать в контейнер локальные каталоги, что в случае таких интерпретируемых языков, как Python и Ruby, исключает необходимость в пересборке контейнера при изменении кода. Кроме того, при таком подходе упрощается процедура настройки конфигурации, поскольку вам не нужно настраивать то, что относится к продуктивной среде, например, количество реплик или требования к ресурсам.

Невозможно переоценить, насколько полезными могут быть такие возможности утилиты Compose, как монтирование локального каталога приложения в виде тома, доступного для чтения и записи, редактирование кода без пересборки контейнера, запись вывода запускаемых в контейнере команд в виде log-файлов и обновление баз данных непосредственно в рабочем каталоге. У платформы Kubernetes тоже есть аналогичные инструменты, например, инструмент командной строки Skaffold, который позволяет обеспечить компактный цикл разработки с развертыванием результата в локальной или облачной версии Kubernetes. Однако большинство разработчиков вполне обоснованно предпочитает вести разработку с помощью платформы Docker.

Как я всегда говорю, используйте тот инструмент, который облегчит выполнение задачи. Выберите способ разработки приложения, который подходит вам больше: с использованием локального кластера Kubernetes или утилиты Docker Compose, — и используйте его. Эти два инструмента можно применять и одновременно: утилиту Compose — для разработки приложения, а локальный кластер Kubernetes — для тестирования развертывания.

Однако в некоторых случаях могут быть полезными и кластеры Kubernetes для локальной разработки. Например, такой кластер удобен, если вы хотите быстро выполнять развертывание и последовательное изменение кода с размещением приложения в кластере Kubernetes, особенно если приложение содержит несколько разных сервисов. Вполне подойдет такой кластер и в том случае, если вы хотите просто научиться использовать конструкции Kubernetes, не платя за облачный сервис, или протестировать свою конфигурацию развертывания на локальной машине.

Между использованием Kubernetes в локальной среде, имеющей ограниченное количество ресурсов, и использованием облачного сервиса продуктивного уровня с динамическим предоставлением ресурсов есть несколько важных различий. В облаке можно быстро масштабировать приложение, используя множество машин, распределенных в определенном географическом регионе, в то время как в локальной среде можно задействовать лишь ограниченное количество ресурсов. В облаке вы можете получить для своего сервиса публичный IP-адрес, а на локальной машине — нет.

В силу этих и многих других различий я считаю более эффективным тот подход к обучению, при котором вы учитесь работать с системой непосредственно в целевой рабочей среде. Соответственно, основной акцент в книге я делаю на использовании кластеров продуктивного уровня. В то же время следует сказать, что кластер для локальной разработки может оказаться очень полезным инструментом, если вы не будете забывать об указанных различиях.

Существует несколько способов запуска локального кластера Kubernetes. Два наиболее популярных способа сводятся к использованию приложения Docker Desktop и инструмента Minikube. Но если у вас установлено это приложение, то у вас уже есть локальный одноузловой кластер Kubernetes! Minikube, разработанный в рамках отдельного проекта компании Kubernetes, тоже прост в настройке и предлагает несколько расширенных возможностей наподобие использования нескольких узлов, которые могут пригодиться при тестировании более сложных конструкций Kubernetes, таких как политики предпочтения узлов и распределения подов, о которых мы поговорим в главе 8.

3.4.1. Кластер Kubernetes приложения Docker Desktop

Приложение Docker Desktop содержит собственную одноузловую среду Kubernetes. Если это приложение у вас установлено, то у вас уже есть локальная среда Kubernetes. Следуя инструкциям, представленным на сайте <https://docs.docker.com/desktop/kubernetes/>, подготовьте ее к работе, выполнив следующие два простых шага.

1. Активируйте поддержку Kubernetes в настройках приложения Docker Desktop и запустите Kubernetes.
2. Используя утилиту `kubectl`, переключите контекст на кластер приложения Docker Desktop.

ПРИМЕЧАНИЕ

Обратите внимание: локальная среда Kubernetes платформы Docker будет доступна только в случае использования приложения Docker Desktop. Если вместо него вы решили задействовать установленный в Linux движок Docker Engine, то данная функциональность вам не будет доступна.

После того как вы запустите приложение Docker Desktop с активированной поддержкой Kubernetes, вы можете отобразить информацию об этом контексте и переключиться на него, используя следующие две команды:

```
kubect1 config get-contexts
kubect1 config use-context docker-desktop
```

На самом деле с помощью них можно переключиться на любой из тех кластеров, которые вы уже успели подключить, в том числе на ту облачную платформу, использование которой рассматривалось в начале данной главы. В общем случае сменить кластер можно так:

```
kubect1 config get-contexts
kubect1 config use-context $CONTEXT
```

При частом переключении между кластерами использование этих двух команд становится несколько утомительным занятием, поэтому я настоятельно рекомендую задействовать вместо них утилиту `kubectx` (<https://github.com/ahmetb/kubectx>), которая существенно ускоряет этот процесс. Переключить контекст с ее помощью можно так:

```
kubectx
kubectx $CONTEXT
```

В случае каких-либо проблем с использованием приложения Docker Desktop попробуйте воспользоваться командами **Restart Kubernetes Cluster** (Перезапуск кластера Kubernetes) и **Clean/Purge Data** (Очистить/Удалить данные) из меню отладки.

3.4.2. Minikube

Еще одним инструментом, хорошо подходящим для локального тестирования, является Minikube, который обеспечивает более широкую проверку функциональности Kubernetes за счет предоставления многоузловой среды. Minikube — инструмент с открытым исходным кодом, поддерживаемый сообществом Kubernetes. Следуя инструкциям, представленным на сайте <https://minikube.sigs.k8s.io/docs/start/>, установите версию Minikube, подходящую для вашей системы.

По завершении установки выполните команду `minikube start` с параметром, указывающим необходимое количество узлов, чтобы загрузить виртуальный

многоузловой кластер (я настоятельно рекомендую использовать этот вариант среды, поскольку он в наибольшей степени напоминает рабочую среду Kubernetes, которую вы будете использовать в конечном счете):

```
minikube start --nodes 3
```

Команда `start` автоматически настроит утилиту `kubectl` на использование контекста `Minikube`; то есть все команды `kubectl` теперь будут применяться к кластеру `Minikube`. Чтобы снова переключить контекст на другой кластер, например, рабочей среды, воспользуйтесь командами `kubectl config` или `kubectx`, как было показано в предыдущем подразделе.

После запуска `Minikube` вы сможете использовать его кластер как обычный кластер Kubernetes, следуя инструкциям, изложенным в этой главе. Но прежде чем приступить к его использованию, следует убедиться, что все работает должным образом. Для этого попробуйте подключиться к кластеру с помощью команды `kubectl get nodes`:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	4m54s	v1.24.3
minikube-m02	Ready	<none>	4m32s	v1.24.3
minikube-m03	Ready	<none>	3m58s	v1.24.3

По завершении работы с кластером `Minikube` можно освободить используемые им ресурсы процессора и памяти, выполнив команду `minikube stop`. Если же вы хотите удалить все данные и освободить пространство под создание нового кластера `Minikube` с другими настройками (например, с другим количеством узлов), то воспользуйтесь командой `minikube delete`.

3.4.3. Использование собственного локального кластера Kubernetes

Настроив `kubectl` на работу с локальным кластером Kubernetes, вы можете произвести локальное развертывание своего приложения, используя те же команды `kubectl`, которые мы использовали в этой главе ранее для работы с облачной платформой. Однако есть важные различия в том, как получать доступ к сервису, и в том, как ссылаться на образы контейнеров, собранные локально. Выполните следующие команды, чтобы произвести развертывание демонстрационного приложения из его корневого каталога:

```
$ cd Chapter03/3.2_DeployingToKubernetes
$ kubectl create -f .
deployment.apps/timeserver created
service/timeserver created
```

ПРЕИМУЩЕСТВА ДЕКЛАРАТИВНОЙ КОНФИГУРАЦИИ

Во всех примерах этой книги мы будем использовать декларативный способ определения конфигурации, а не императивные команды. То есть каждый объект `Deployment` мы будем создавать с помощью определения и последующего применения его конфигурации, а не путем создания напрямую с помощью утилиты `kubectl`.

Одним из многочисленных преимуществ этого подхода является то, что вы можете протестировать свою конфигурацию на локальной машине и затем выполнить ее развертывание в рабочей среде, не вспоминая точную последовательность разовых команд. Обратите внимание, насколько элегантно мы можем развернуть в локальном кластере те же конфигурационные файлы, которые использовали в кластере продуктивной среды!

Получение доступа к сервису

В отличие от разработки с использованием облачной платформы Kubernetes, вы не получите внешний IP-адрес при создании сервиса типа `LoadBalancer` на локальной машине. При использовании приложения `Docker Desktop`, утилиты `Minikube` и фактически любого кластера Kubernetes можно выполнить дополнительную команду `kubectl` для перенаправления портов локальной машины к сервису внутри кластера. Это бывает полезно, когда нужно выполнить тестирование в локальном кластере Kubernetes и отладку своего облачного кластера. Получить доступ к сервису на локальной машине можно с помощью следующей команды:

```
kubectl port-forward service/$SERVICE_NAME $FROM_PORT:$TO_PORT
```

где `FROM_PORT` — порт локальной машины, на котором вы будете получать доступ к сервису, а `TO_PORT` — порт сервиса. В случае нашего демонстрационного приложения мы можем задать порт `8080` в качестве порта верхнего уровня, как показано ниже:

```
kubectl port-forward service/timeserver 8080:80
```

После этого можно будет подключиться к сервису, перейдя в браузере по адресу `http://localhost:8080`. Кроме того, в команде `port-forward` можно использовать несколько полезных флагов (<https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#port-forward>), в том числе `--address 0.0.0.0`, который выполняет привязку всех сетевых интерфейсов, чтобы к перенаправленному сервису можно было получать доступ с других устройств, подключенных к той же сети (если это позволяет делать ваш межсетевой экран). Перенаправление портов может быть полезным и при отладке сервисов, работающих на облачной платформе Kubernetes.

Minikube предлагает еще один способ (<https://kubernetes.io/docs/setup/learning-environment/minikube/#services>) перенаправления трафика к вашему сервису, который сводится к использованию следующей команды:

```
minikube service $SERVICE_NAME
```

В случае нашего демонстрационного приложения эта команда будет выглядеть так:

```
minikube service timeserver
```

ЛОКАЛЬНОЕ ПОЛУЧЕНИЕ ДОСТУПА К СЕРВИСАМ KUBERNETES ИЗ DOCKER

В некоторых случаях запущенный в системе Kubernetes сервис требуется напрямую использовать из Docker-контейнера, развернутого за пределами этой системы. Допустим, вы ведете в Docker последовательную доработку кода, и в ходе этого процесса вам нужен доступ к одному из тех сервисов, которые вы развернули в Kubernetes.

Обеспечить такой доступ достаточно просто. Перенаправьте сервис, как было описано выше, сделав его доступным на локальном порте. После этого его можно будет напрямую использовать в Docker-контейнере, указывая имя `host.docker.internal` на перенаправленном порте. Имя `host.docker.internal` обеспечивает в Docker связь между контейнером и сервисами локальной машины, и поскольку мы перенаправили сервис на локальный порт, такая ссылка позволяет успешно установить соединение.

В качестве примера допустим, что вы развернули в Kubernetes СУБД Redis (см. главу 9) и перенаправили порты с помощью команды `kubectl port-forward service/timeserver 6379:6379`. После этого вы хотите подключиться к этой СУБД из локального Docker-контейнера, используя в коде Python библиотеку `redis-py`. Это можно сделать так: `redis.Redis(host='host.docker.internal', port='6379')`.

Развертывание локальных образов

По умолчанию локальный кластер Kubernetes пытается извлечь образ контейнера из Интернета, как это делает кластер Kubernetes, развернутый в рабочей среде. Это не приведет к проблемам в случае такого публичного образа, как `ubuntu`, или нашего демонстрационного образа `docker.io/wdenniss/timeserver`. Но если нужно предоставить локальному кластеру собственный образ, который был собран на локальной машине, то потребуется выполнить некоторые дополнительные действия.

Конечно, в таком случае можно было бы загрузить свой образ в публичный реестр контейнеров и позволить локальному кластеру извлечь его, как это делается в рабочей среде.

Но если по мере разработки вы будете загружать в реестр каждый собираемый вами образ, то скорость разработки снизится, поскольку вам придется каждый раз ждать, пока образ загрузится в реестр и будет извлечен из него. Кроме того, речь идет о приватном образе, поэтому вам придется предоставлять локальному кластеру учетные данные, позволяющие получить доступ к образу (при извлечении приватных образов из реестра контейнеров облачной платформы Kubernetes эта операция обычно выполняется автоматически).

Чтобы заставить свой локальный кластер использовать локальный образ, необходимо внести в конфигурацию развертывания два изменения. Во-первых, вы должны добавить параметр `imagePullPolicy`, присвоив ему значение `Never`, а во-вторых, сослаться на свой образ, используя его локальное имя без какого-либо префикса в виде имени репозитория.

Путь к локально собранному образу содержит только имя репозитория и тег версии, без префикса в виде URL репозитория. Например, если образ был собран с помощью команды `docker build . -t timeserver`, как мы сделали в главе 2, то вы должны сослаться на него в файле конфигурации внутри спецификации пода, используя строку `image: timeserver:latest` (`latest` в качестве тега позволяет получить самую свежую версию образа). Отобразить список доступных локальных образов можно с помощью команды `docker images`. Как в конфигурации развертывания можно сослаться на локально собранный образ, показано в листинге 3.5.

Листинг 3.5. Chapter03/3.4.3_LocalDevelopment/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: timeserver:latest
          imagePullPolicy: Never
```

Ссылка на локально доступный образ

Политика извлечения образов дает Kubernetes указание не пытаться скачать этот локальный образ из какого-либо репозитория

СОВЕТ

Параметр конфигурации `imagePullPolicy: Never` следует использовать только в том случае, если вы собираетесь загружать образ с локальной машины. При использовании удаленного образа этот параметр сделает его извлечение невозможным и приведет к ошибке со статусом `ErrImageNeverPull`. Данная ошибка говорит о том, что объект `Deployment` был настроен на использование локального образа, но тот не был обнаружен.

Если вы используете `Minikube`, то после этого нужно будет сделать еще один шаг. Приложение `Docker Desktop` имеет доступ ко всем образам, которые были собраны на локальной машине с помощью платформы `Docker`, а вот `Minikube` не имеет такого доступа (этот инструмент использует отдельную среду запуска контейнеров и не имеет доступа к образам, доступным для локально установленной версии платформы `Docker`). Загрузить нужный вам образ в `Minikube` можно с помощью этой команды:

```
minikube image load $REPOSITORY:$TAG
```

Например:

```
minikube image load timeserver:latest
```

После этого нужно применить внесенные изменения, используя команду `kubectl`, как мы уже делали ранее:

```
kubectl apply -f deploy.yaml
```

РЕЗЮМЕ

- Кластер `Kubernetes` содержит управляющий слой (`control panel`) и узлы, на которых запускаются ваши контейнеры.
- Взаимодействие с кластером осуществляется через API `Kubernetes`, обычно с помощью инструмента командной строки `kubectl`.
- Чтобы выполнить развертывание приложения в `Kubernetes`, сначала нужно загрузить образ контейнера в репозиторий контейнеров.
- Рабочие нагрузки описываются такими объектами, как `Deployment`, которые инкапсулируют поды, определяющие ваши контейнеры.
- С помощью сервисов вы определяете конечные точки сети и делаете контейнеры доступными в Интернете.
- На поды можно ссылаться из таких объектов, как `Deployment` и `Service`, используя для этого метки.

- Kubernetes использует декларативную конфигурацию, обычно в виде конфигурационных файлов формата YAML.
- После того как вы определите свои требования в конфигурации, контроллер Kubernetes будет постоянно приводить состояние кластера в соответствие этим требованиям.
- Чтобы изменить приложение, достаточно указать в конфигурации новую версию контейнера и применить это изменение к кластеру.
- При внесении изменений Kubernetes сверяет версии всех конфигураций и распространяет на них внесенные изменения.

Автоматизированные операции

В этой главе

- ✓ Создание надежных развертываний приложений, рассчитанных на длительную непрерывную работу.
- ✓ Создание условий, при которых система Kubernetes будет без вашего вмешательства обеспечивать бесперебойную работу приложений.
- ✓ Обновление приложений без простоев.

Система Kubernetes может автоматически выполнять множество таких операций, как перезапуск сбойнувшего контейнера и миграция приложения в случае аппаратного сбоя. Это позволяет вам сделать свои развертывания более надежными, не нуждающимися в круглосуточном отслеживании их состояния. Автоматическое выполнение таких операций — одно из главных достоинств этой платформы, а понимание того, как этого достичь, — одно из необходимых условий максимально эффективного использования всего того, что вам может предложить Kubernetes.

Вдобавок Kubernetes делает возможным обновление приложений без каких-либо простоев и сбоев за счет загрузки новой версии и отслеживания ее состояния, чтобы она была готова к обслуживанию трафика к моменту удаления старой версии.

Чтобы система Kubernetes могла обеспечивать бесперебойную работу вашего приложения в ходе обычной эксплуатации и при проведении обновлений, вы должны предоставлять ей определенную информацию о его состоянии, используя так называемые проверки работоспособности. В следующих разделах вы узнаете, как

добавить в приложение несколько таких проверок и затем использовать их в сочетании со встроенными в Kubernetes стратегиями развертывания, позволяющими обновлять приложение без сбоев или простоев.

4.1. АВТОМАТИЧЕСКОЕ ОБЕСПЕЧЕНИЕ БЕЗОТКАЗНОЙ РАБОТЫ С ПОМОЩЬЮ ПРОВЕРОК РАБОТОСПОСОБНОСТИ

Некоторые виды проблемных ситуаций Kubernetes может выявлять и исправлять самостоятельно. Например, в случае сбоя приложения Kubernetes автоматически произведет его перезапуск. Точно так же при сбое или удалении одного из тех узлов, на которых запущен ваш контейнер, Kubernetes заметит, что ваш объект Deployment использует недостаточное количество реплик, и запустит дополнительные реплики в свободном пространстве кластера.

Но что можно сказать о тех случаях, когда приложение отказывается работать по такой причине, как зависание процесса, отказ веб-сервиса принимать соединения или зависимость приложения от внешнего сервиса, который вдруг стал недоступным? В Kubernetes встроен довольно изящный механизм, выявляющий такие ситуации и делающий все возможное для их устранения, но для этого вы должны предоставлять платформе информацию о работоспособности приложения и его готовности обрабатывать трафик. Для предоставления данной информации используются так называемые *проверки работоспособности*, которые в терминологии Kubernetes называются *проверками «жизнеспособности» (liveness probe)* и *«готовности» (readiness probe)*.

Kubernetes не может знать, что подразумевается под работоспособным состоянием или готовностью к обработке трафика в случае произвольно взятого приложения, поэтому приложения должны реализовать такую проверку сами. Далее Kubernetes просто зондирует состояние контейнера, периодически отправляя в него запросы, контейнер при обработке таких запросов выполняет внутреннюю проверку состояния, и если она показывает, что все в порядке, то возвращает код успешного результата. Если же приложение не успевает ответить за отведенное для этого время (например, из-за слишком большой нагрузки) или контейнер сам выявляет определенную проблему (такую как наличие проблемной зависимости), то результат проверки считается неудачным.

4.1.1. Проверки жизнеспособности и готовности

В Kubernetes работоспособность контейнера проверяется с помощью двух отдельных проверок: *жизнеспособности (liveness)*, определяющей, работает ли контейнер в данный момент, и *готовности (readiness)*, определяющей, готов ли он к обработке трафика. Обе проверки используют одинаковые способы получения информации о состоянии, но Kubernetes по-разному использует выдаваемые ими результаты (табл. 4.1).

Таблица 4.1. Различия между проверками жизнеспособности и готовности

Параметр	Проверка жизнеспособности	Проверка готовности
Семантическое значение	Работает ли контейнер?	Готов ли контейнер к принятию трафика?
Что происходит при превышении порогового количества неудачных результатов проверки?	Выполняется остановка и замещение пода	Система перестает направлять к поду трафик до получения успешного результата проверки
Время на восстановление после проваленной проверки	Медленно: не прошедший проверку под нужно распределить заново, что требует некоторого времени на его повторный запуск	Быстро: под уже работает и к нему можно сразу же направлять трафик при получении успешного результата проверки
Состояние по умолчанию после начальной загрузки контейнера	Успешный результат (работает)	Неудачный результат (не готов)

Использование двух типов проверки обусловлено несколькими причинами. Одной из них является состояние после начальной загрузки контейнера. Так, в случае проверки жизнеспособности изначально предполагается успешный результат (то есть считается, что контейнер работает, пока под не сообщит об обратном). А в случае проверки готовности изначально предполагается неудачный результат (то есть считается, что контейнер не готов к обслуживанию трафика, пока не сообщит об обратном).

Без проверки готовности у системы Kubernetes не было бы возможности определить, в какой именно момент контейнер будет готов к принятию трафика, и ей пришлось бы исходить из предположения, что он будет готов сразу после его запуска, и тут же добавлять его в набор контейнеров, используемых сервисом при балансировке нагрузки. Однако на самом деле большинство контейнеров приходит в состояние готовности лишь через несколько десятков секунд или даже пару минут, поэтому немедленное направление трафика к контейнеру привело бы к потере некоторой части трафика. Проверка готовности исключает вероятность такой потери, сообщая о готовности лишь после успешного прохождения контейнером внутренних проверок.

Сходным образом при проверке жизнеспособности требующее перезапуска состояние может отличаться от состояния неготовности контейнера к принятию трафика. Наглядным примером здесь может быть тот случай, когда контейнер ожидает подключения к какой-либо внешней зависимости, такой как база данных. Не имея к ней доступа, он не будет готов к обслуживанию трафика, в то время как его внутреннее состояние будет вполне нормальным. Вместо того чтобы тут же замещать такой контейнер другим, лучше дать ему какое-то время, чтобы он мог установить необходимое ему соединение с базой данных.

Еще одна причина наличия двух видов проверки — разные чувствительность и время восстановления. Проверка готовности обычно настраивается так, чтобы под

быстро исключался из ротации балансировщика нагрузки и столь же быстро снова в нее добавлялся после получения успешного результата (поскольку эти операции выполняются быстро и потребляют мало ресурсов). В отличие от этого, проверка жизнеспособности обычно настраивается так, чтобы под не удалялся слишком поспешно, поскольку повторный запуск контейнера занимает более длительное время.

4.1.2. Добавление проверки готовности

В случае веб-сервиса простейшая проверка работоспособности может проверять лишь то, обслуживает ли сервис трафик в данный момент. Прежде чем создавать специальную конечную точку для проверки работоспособности своего сервиса, попробуйте проверять его работоспособность с помощью любой конечной точки, выдающей код HTTP-статуса 200.

Для простоты можно использовать корневой путь, если он возвращает HTTP-статус 200 во всех ответах. Именно так себя ведет корневой путь в случае нашего демонстрационного контейнера, поэтому мы можем успешно использовать следующую проверку готовности (листинг 4.1).

Листинг 4.1. Chapter04/4.1.2_Readiness/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:1
          readinessProbe:
            initialDelaySeconds: 15
            periodSeconds: 30
            httpGet:
              path: /
              port: 80
              scheme: HTTP
            timeoutSeconds: 2
            failureThreshold: 1
            successThreshold: 1
```

Величина начальной задержки

Выполняется с интервалом 30 секунд

Выполняется этот HTTP-запрос

Тайм-аут ответа на запрос составляет 2 секунды

Признаком неготовности контейнера считается получение хотя бы одного ответа с кодом ошибки

Признаком возвращения контейнера в состояние готовности считается получение хотя бы одного ответа с кодом успешного завершения

Перейдите в корневой каталог и обновите объект Deployment с именем `timeserver`:

```
cd Chapter04/4.1.2_Readiness
kubectl apply -f deploy.yaml
```

Теперь каждый раз, когда какой-то контейнер будет не в состоянии пройти эту проверку готовности, проблемный под будет временно исключаться из используемого сервисом набора подов. Например, вы используете три реплики пода, и одна из них не смогла пройти проверку. В таком случае идущий к сервису трафик будет распределен между остальными двумя, по-прежнему работоспособными репликами. После того как проблемный под снова вернет успешный результат (в данном случае HTTP-статус 200), он будет снова добавлен в набор подов сервиса.

ПОЧУВСТВУЙТЕ РАЗНИЦУ

Если вы хотите прочувствовать разницу между проведением обновлений с использованием проверки готовности и без нее, то попробуйте выполнить следующий тест. В окне командной оболочки создайте объект Deployment, в котором не определена проверка готовности (пусть это будет объект, который мы использовали в главе 3):

```
cd Chapter03/3.2_DeployingToKubernetes
kubectl create -f
```

Дождитесь того момента, когда сервису будет присвоен внешний IP-адрес:

```
kubectl get svc -w
```

Теперь откройте еще одно окно консоли и, используя свой IP-адрес, настройте в нем отслеживание конечной точки сервиса:

```
EXTERNAL_IP=203.0.113.16
watch -n 0.25 -d curl "http://$EXTERNAL_IP"
```

Вернувшись в первое окно, запустите процесс обновления:

```
kubectl rollout restart deploy timeserver
```

В ходе процесса перезапуска подов в окне с командой `curl` будут периодически появляться сообщения о проблемах с подключением.

Теперь попробуйте обновить объект Deployment, в котором определена проверка готовности (например, развертывание из этого раздела):

```
cd ../../
cd Chapter04/4.1.2_Readiness
kubectl apply -f deploy.yaml
```

На этот раз в развертывание добавлена проверка готовности, поэтому вы не увидите в окне с командой `curl` никаких сообщений о проблемах с подключением.

Использовать такую проверку готовности особенно полезно при проведении обновлений, когда важно не допустить направления трафика к тем подам, которые еще не успели загрузиться (поскольку выполнение таких запросов будет заканчиваться неудачей). При корректной реализации проверок готовности вы сможете проводить обновления без каких-либо простоев, поскольку трафик будет направляться лишь к подам, находящимся в состоянии готовности, и не будет направляться к тем, которые еще только создаются.

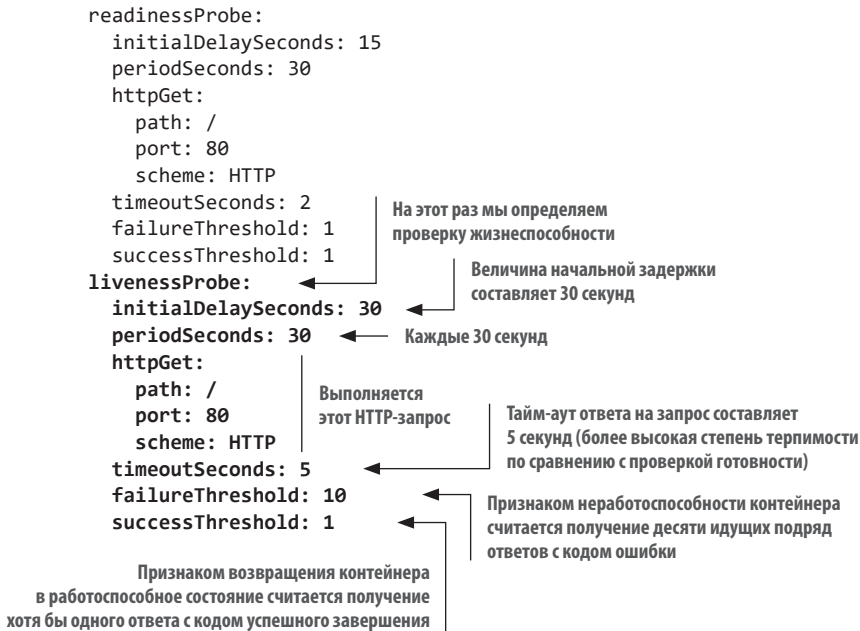
4.1.3. Добавление проверки жизнеспособности

Проверка жизнеспособности определяется практически так же, за исключением того, что в ней используется ключевое слово `livenessProbe`. Однако используется она совершенно по-другому. Если проверка готовности определяет, будет ли под получать трафик, то проверка жизнеспособности обеспечивает перезапуск пода (при достижении порогового количества неудачных результатов).

В предыдущем подразделе мы добавили в наш объект `Deployment` простейшую проверку готовности, которая вместо специально выделенной конечной точки использовала корневой путь сервиса. Мы можем использовать этот подход и в данном подразделе. В листинге 4.2 показано, как использовать для проверки жизнеспособности ту же конечную точку, которую мы использовали для проверки готовности, внося лишь незначительные изменения, которые позволяют снизить чувствительность к ошибкам. Когда проверка жизнеспособности выдает пороговое количество неудачных результатов, контейнер перезапускается и вновь становится работоспособным лишь спустя некоторое время, поэтому данный порог не должен быть слишком низким. Поэтому добавим в наш объект `Deployment` проверку жизнеспособности, которая будет выполнять перезапуск через 180 секунд после получения первого неудачного результата (после получения шести неудачных результатов с 30-секундным интервалом).

Листинг 4.2. Chapter04/4.1.3_Liveness/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:1
```

Обновите объект Deployment с именем `timeserver` до новой версии с внесенными нами изменениями:

```
cd Chapter04/4.1.3_Liveness
kubectl apply -f deploy.yaml
```

Теперь наш объект Deployment содержит и проверку готовности, и проверку жизнеспособности. Даже в таком начальном виде эти две проверки могут существенно повысить надежность вашего приложения. Если приложение простое, то на этом можно и остановиться. Если же вы хотите продолжить, то в следующем разделе я расскажу, как доработать эти проверки, чтобы обеспечить еще более высокий уровень надежности в рабочей среде.

4.1.4. Как создать эффективные проверки работоспособности

В принципе, для проверки состояния можно использовать уже имеющиеся конечные точки, как было сделано в двух предыдущих подразделах, но в большинстве случаев все же будет лучше добавить в приложение специально выделенные конечные точки, предназначенные для проверок работоспособности. Вы должны реализовать в этих проверках семантику готовности и жизнеспособности и постараться сделать их максимально легковесными. Если вы не будете понимать семантическую разницу между жизнеспособностью и готовностью, это может привести к нестабильной работе приложения из-за слишком частых перезапусков и возникновения каскадных отказов. Кроме того, уже существующая конечная

точка может оказаться слишком тяжеловесной. Зачем тратить ресурсы на отображение целой HTML-страницы, если вместо этого можно использовать простой ответ с HTTP-заголовком?

При создании конечных точек HTTP для реализации этих проверок важно учитывать, какие внешние зависимости при этом будут проверяться. В большинстве случаев проверка жизнеспособности не должна проверять внешние зависимости и может ограничиться лишь проверкой того, работает ли сам контейнер (при этом предполагается, что контейнер будет пытаться заново подключиться к своим внешним зависимостям). Нет никакого смысла в том, чтобы перезапускать вполне работоспособный контейнер из-за того, что он не может подключиться к другому сервису, в работе которого в данный момент есть проблемы. Это может привести к слишком частому выполнению перезапуска, что вызовет отток пользователей и каскадные отказы, особенно если ваше дерево зависимостей достаточно сложное. Однако у правила не проверять внешние зависимости в проверке жизнеспособности есть и одно исключение, о котором я расскажу чуть позже.

Проверка жизнеспособности проверяет лишь то, способен ли сервер выдавать ответы, поэтому ее результат может и должен быть предельно простым. Обычно он представляет собой простейший HTTP-ответ с кодом статуса 200, тело которого часто даже не содержит какого-либо текста. Если запрос сумел добраться до кода сервера, значит, он работает, и это все, что нам надо знать.

С другой стороны, проверка готовности в большинстве случаев должна проверять такие внешние зависимости, как подключение к базе данных (рис. 4.1). Если, например, вы используете три реплики пода и только две из них могут подключиться к базе, то будет разумным добавить в ротацию балансировщика нагрузки только эти две полнофункциональные реплики. В качестве проверки подключения проверка готовности может, например, пытаться извлечь строки из базы данных.

Таким образом, псевдокод проверки подключения к базе данных может выглядеть примерно так:

```
result = sql.execute("SELECT id FROM users LIMIT 1;")
if result:
    http_response(200, "Ready")
else:
    http_response(503, "Not Ready")
```

Успешное выполнение простейшего SQL-запроса — достаточное подтверждение того, что база данных подключена к приложению и способна выдавать ответы. Конечно, вместо запроса с оператором `SELECT` можно использовать и запрос с любым другим оператором, но я предпочитаю именно этот оператор как один из основных операторов языка SQL. Если сработает этот запрос, то сработают и все остальные.



Рис. 4.1. Проверки жизнеспособности и готовности и внешние зависимости

Наше демонстрационное приложение Python `timeserver` не требует подключения базы данных. Однако мы доработаем его код, чтобы добавить для проверок жизнеспособности и готовности специально выделенные пути `/healthz` и `/readyz`, поскольку такой подход является рекомендуемым (листинг 4.3).

Листинг 4.3. `Chapter04/timeserver2/server.py`

```

from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
from datetime import datetime

class RequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        match self.path:

```

```

    case '/':
        now = datetime.now()
        response_string = now.strftime("The time is %-I:%M %p, UTC.")
        self.respond_with(200, response_string)
    case '/healthz':
        self.respond_with(200, "Healthy")
    case '/readyz':
        dependencies_connected = True
        # TODO: проверить все зависимости
        if (dependencies_connected):
            self.respond_with(200, "Ready")
        else:
            self.respond_with(503, "Not Ready")
    case _:
        self.respond_with(404, "Not Found")

def respond_with(self, status_code: int, content: str) -> None:
    self.send_response(status_code)
    self.send_header('Content-type', 'text/plain')
    self.end_headers()
    self.wfile.write(bytes(content, "utf-8"))

def startServer():
    try:
        server = ThreadingHTTPServer(('', 80), RequestHandler)
        print("Listening on " + ":".join(map(str, server.server_address)))
        server.serve_forever()
    except KeyboardInterrupt:
        server.shutdown()

if __name__ == "__main__":
    startServer()

```

Новые пути
к проверкам
работоспособности

Теперь изменим конфигурацию нашего развертывания так, чтобы оно использовало новые конечные точки (листинг 4.4).

Листинг 4.4. Chapter04/4.1.4_GoodHealthChecks/deploy.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:

```

```

containers:
- name: timeserver-container
  image: docker.io/wdenniss/timeserver:2
  readinessProbe:
    initialDelaySeconds: 15
    periodSeconds: 30
    httpGet:
      path: /readyz
      port: 80
      scheme: HTTP
      timeoutSeconds: 2
      failureThreshold: 1
      successThreshold: 1
  livenessProbe:
    initialDelaySeconds: 30
    periodSeconds: 30
    httpGet:
      path: /healthz
      port: 80
      scheme: HTTP
      timeoutSeconds: 5
      failureThreshold: 3
      successThreshold: 1

```

Обновленные конечные точки

Теперь используется легковесная проверка жизнеспособности, поэтому можно снизить пороговое количество ошибок

Примените эту новую конфигурацию, как мы уже делали ранее. В зависимости от того, что представляет собой ваше приложение, вы можете использовать и более сложную логику готовности и жизнеспособности. Определенную здесь конечную точку `healthz` можно использовать и во многих других HTTP-приложениях (для которых достаточно убедиться в том, что HTTP-сервер отвечает на запросы).

Но если ваше приложение использует такие зависимости, как подключение к базе данных, то вам потребуется добавить в него проверку готовности, которая будет выявлять, действительно ли готово приложение к обслуживанию пользовательских запросов.

4.1.5. Перераспределение неготовых контейнеров

В предыдущем подразделе мы рассмотрели стандартный способ определения проверок жизнеспособности и готовности в Kubernetes, при котором зависимости сервисов проверяются только в проверке готовности. Но если мы не будем проверять зависимости в проверке жизнеспособности, это может привести к проблемам. Можно разделить ответственность между проверкой готовности, отвечающей на вопрос: «Готов ли контейнер к получению трафика?», и проверкой жизнеспособности, отвечающей на вопрос: «Работает ли контейнер?». Однако это может привести к ситуации, когда контейнер будет работать, но из-за ошибки в логике повторного доступа к зависимостям будет не в состоянии подключиться к ним. То есть контейнер будет всегда оставаться в состоянии неготовности, что, в принципе, можно было бы исправить, перезапустив его.

Как вы помните, то, что в проверке жизнеспособности обычно не проверяется готовность внешних зависимостей, объясняется тем, что при таком подходе под будет перезапускаться слишком быстро и у него не будет времени на разрешение внешних зависимостей. Но если под будет слишком долго оставаться в состоянии неготовности, то его все же будет лучше перезапустить. Иногда лучше просто «выключить» под и снова его «включить».

Kubernetes не позволяет выразить эту логику напрямую, но мы легко можем изменить свою реализацию проверки жизнеспособности в самом приложении так, чтобы она давала отрицательный результат, если под будет оставаться в состоянии неготовности в течение заданного периода времени. Для этого можно просто фиксировать момент последнего успешного прохождения проверки готовности и выдавать в проверке жизнеспособности отрицательный результат, если после этого момента будет проходить слишком много времени (например, 5 минут). В листинге 4.5 показано, как эту логику можно реализовать применительно к нашему контейнеру `timeserver`.

Листинг 4.5. `Chapter04/timeserver3/server.py`

```
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
from datetime import datetime, timedelta

last_ready_time = datetime.now()

class RequestHandler(BaseHTTPRequestHandler):

    def do_GET(self):
        global last_ready_time

        match self.path:
            case '/':
                now = datetime.now()
                response_string = now.strftime("The time is %-I:%M %p, UTC.")
                self.respond_with(200, response_string)
            case '/healthz':
                if (datetime.now() > last_ready_time + timedelta(minutes=5)):
                    self.respond_with(503, "Not Healthy")
                else:
                    self.respond_with(200, "Healthy")
            case '/readyz':
                dependencies_connected = True
                # TODO: проверить все зависимости
                if (dependencies_connected):
                    last_ready_time = datetime.now()
                    self.respond_with(200, "Ready")
                else:
                    self.respond_with(503, "Not Ready")
            case _:
                self.respond_with(404, "Not Found")
```

Задаем текущее время в качестве начального значения переменной, определяющей последний момент готовности, чтобы предоставить 5 минут после запуска

Если после получения последнего успешного прохождения проверки готовности (или с момента запуска) прошло 5 минут, то проверка жизнеспособности выдает отрицательный результат

Обновляем содержимое переменной с временным значением при каждом успешном прохождении проверки готовности

```

def respond_with(self, status_code: int, content: str) -> None:
    self.send_response(status_code)
    self.send_header('Content-type', 'text/plain')
    self.end_headers()
    self.wfile.write(bytes(content, "utf-8"))

def startServer():
    try:
        server = ThreadingHTTPServer(('', 80), RequestHandler)
        print("Listening on " + ":".join(map(str, server.server_address)))
        server.serve_forever()
    except KeyboardInterrupt:
        server.shutdown()

if __name__ == "__main__":
    startServer()

```

Теперь проверка жизнеспособности будет выдавать отрицательный результат в том случае, если контейнер не будет успевать перейти в состояние готовности за отведенное время, и эта ситуация будет разрешаться путем перезапуска. Тем самым мы убиваем двух зайцев, не отслеживая внешние зависимости в проверке жизнеспособности и отслеживая их в проверке готовности. Это значит, контейнер не будет принимать трафик до подключения внешних зависимостей и в то же время не будет сразу же перезапускаться, имея возможность за какое-то время привести себя в порядок. Но если контейнер будет оставаться в состоянии неготовности и по прошествии 5 минут, то проверка жизнеспособности выдаст отрицательный результат и таким образом заставит его перезапуститься.

Обеспечить перезапуск контейнера после продолжительного периода неготовности можно и с помощью использования конечной точки готовности и в проверке жизнеспособности, и в проверке готовности, с разным пороговым пределом. Например, проверка готовности может выдавать отрицательный результат через 30 секунд, а проверка жизнеспособности — через 5 минут. При этом вы по-прежнему даете контейнеру некоторое время на разрешение внешних зависимостей и в конечном счете перезапускаете его, если он долго не переходит в состояние готовности, что может говорить о наличии в нем какой-либо проблемы. Такой подход вполне допустим, несмотря на то что, проверяя зависимости в проверке жизнеспособности, мы идем вразрез с идиоматическим подходом к использованию платформы Kubernetes.

В заключение следует отметить, что эти проверки жизнеспособности и готовности играют чрезвычайно важную роль в процессе предоставления системе Kubernetes данных, которые ей нужны для автоматического обеспечения надежной работы приложения. Поэтому крайне важно реализовывать эти проверки, понимая разницу между ними и индивидуальные особенности вашего приложения.

4.1.6. Типы проверок

В приводившихся до сих пор примерах предполагалось использование HTTP-сервиса, и поэтому проверки были реализованы в виде HTTP-запросов. Однако Kubernetes можно использовать для запуска разных типов сервисов, а также пакетных заданий, у которых вообще нет каких-либо сервисных конечных точек. К счастью, мы можем использовать разные способы доступа к проверкам работоспособности.

Протокол HTTP

Протокол HTTP рекомендуется использовать во всех тех случаях, когда контейнер предоставляет HTTP-сервис. При этом сервис предоставляет некую конечную точку, например `/healthz`. Ответ с кодом HTTP-статуса 200 означает успешный результат; любой другой ответ (или превышение времени ожидания) означает неудачу.

Протокол TCP

Протокол TCP рекомендуется использовать в случае TCP-сервисов, не использующих протокол HTTP (например, SMTP-сервисов). Такая проверка выдает успешный результат при успешном открытии соединения:

```
readinessProbe:
  initialDelaySeconds: 15
  periodSeconds: 30
  tcpSocket:
    port: 25
  successThreshold: 1
  failureThreshold: 1
```

Определение
TCP-проверки

Bash-скрипт

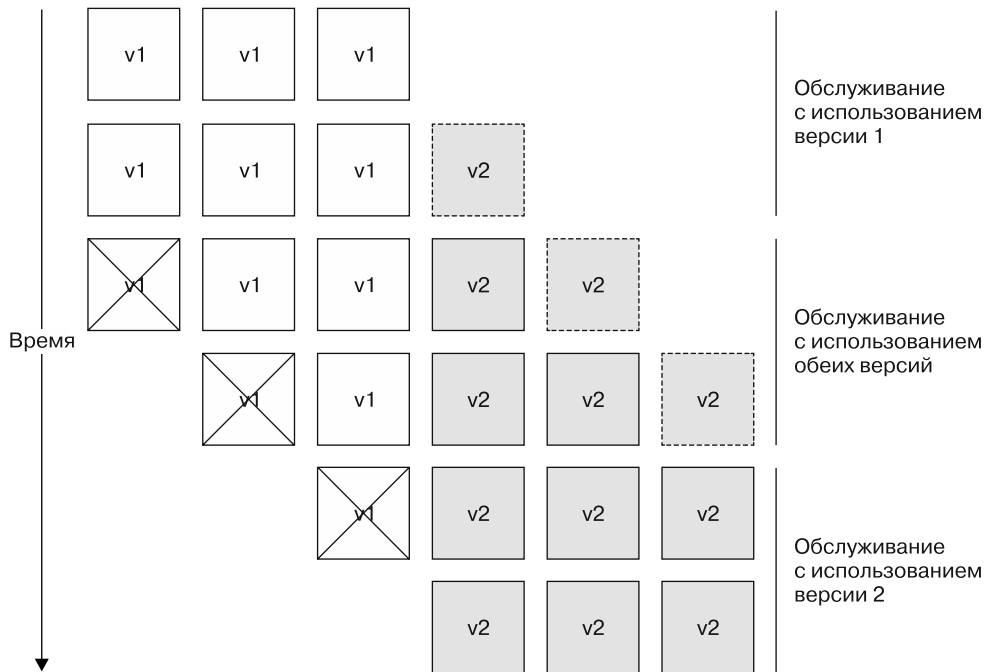
Bash-скрипт рекомендуется использовать всегда, когда контейнер не предоставляет HTTP- или TCP-сервис, например, в случае пакетных заданий, у которых нет сервисных конечных точек. Kubernetes будет выполнять предоставленный вами скрипт, в котором вы можете определить все нужные вам проверки. При этом ненулевой код завершения будет означать неудачный результат. Как может выглядеть определение проверки жизнеспособности в случае фоновой задачи, я покажу в разделе 10.4.

4.2. ОБНОВЛЕНИЕ РАБОТАЮЩИХ ПРИЛОЖЕНИЙ

Теперь, когда вы уже реализовали проверки работоспособности, можно обеспечить выкатывание обновлений приложения с нулевым временем простоя. При проведении обновлений Kubernetes с помощью проверки готовности определяет момент, когда новый под будет готов к получению трафика, и управляет скоростью выкатывания обновлений в соответствии с заданными вами параметрами. Вы можете использовать одну из нескольких возможных стратегий обновления, каждая из которых имеет свои особенности.

4.2.1. Скользящая стратегия обновления

В качестве стратегии обновления с нулевым временем простоя Kubernetes по умолчанию предлагает скользящую стратегию обновления. При этом подходе вы последовательно создаете группы подов с новой версией приложения (размер групп можно настраивать). Дождавшись момента, когда станет доступной новая группа подов, Kubernetes останавливает такое же количество подов со старой версией приложения. Этот процесс повторяется до тех пор, пока у вас не останется подов со старой версией (рис. 4.2).



Условные обозначения

	Контейнер находится в процессе создания		Версия 1
	Под обслуживает трафик		Версия 2
	Выполняется остановка пода		

Рис. 4.2. Состояние подов при проведении скользящего обновления. До завершения процесса обновления запросы могут обслуживаться и старой, и новой версией приложения

Данная стратегия имеет два преимущества. В ходе обновления:

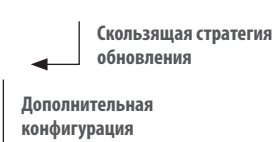
- обеспечивается непрерывная работа приложения;
- используется минимально возможное количество дополнительных ресурсов.

Обратите внимание: эта стратегия предполагает, что две версии приложения могут существовать одновременно, поскольку в течение какого-то времени здесь должны работать обе версии — и старая, и новая. То есть ваш бэкенд или любые другие зависимости должны работать с обеими версиями, а запросы пользователей могут попеременно обрабатываться и старой, и новой. Иными словами, после каждой перезагрузки браузер может отображать пользователю то старую, то новую версию страницы. В зависимости от используемого количества реплик процесс скользящего обновления (и, соответственно, отката к предыдущей версии) может быть достаточно долгим.

Настроим конфигурацию нашего развертывания, чтобы можно было использовать скользящую стратегию обновления (листинг 4.6).

Листинг 4.6. Chapter04/4.2.1_RollingUpdate/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 1
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:3
```



С помощью параметров `maxSurge` и `maxUnavailable` вы можете управлять скоростью процесса обновления.

Параметр `maxSurge`

Параметр `maxSurge` определяет, сколько дополнительных подов будет использовано при проведении обновления. Например, если используемых реплик пять, а параметр `maxSurge` — 2, то у вас может быть распределено до семи подов (разных версий).

Увеличение этого параметра позволяет ускорить процесс обновления, но в то же время ведет к потреблению в ходе этого процесса большего количества ресурсов. При жесткой экономии ресурсов параметр `maxSurge` следует сделать равным 0. С другой стороны, в случае крупномасштабного развертывания можно временно увеличить количество доступных ресурсов в кластере, добавляя дополнительные узлы и удаляя их по завершении обновления.

Параметр `maxUnavailable`

Параметр `maxUnavailable` определяет, сколько недоступных подов может быть при проведении обновления (помимо точного количества, вы можете использовать процентные значения, которые округляются вниз до ближайшего целого числа). Если указанное вами количество реплик позволяет справиться с ожидаемым трафиком, то данный параметр не должен быть намного больше 0; в противном случае в ходе обновления может снижаться уровень качества обслуживания. Увеличение этого параметра позволяет замещать большее количество подов за раз, тем самым ускоряя процесс обновления, но в то же время ведет к тому, что в ходе этого процесса к обслуживанию трафика будет готово меньшее количество подов.

Учитывая, что в ходе обновления теоретически может произойти какое-то другое событие, которое приведет к снижению степени доступности, например, отказ узла, в случае рабочих нагрузок продуктивного уровня я рекомендую делать параметр `maxUnavailable` равным 0. Но если этот параметр будет равен 0 и в вашем кластере не будет доступных для распределения ресурсов, то процесс обновления может застопориться и поды будут оставаться в состоянии ожидания (`Pending`), пока не появятся доступные ресурсы. Кроме того, когда параметр `maxUnavailable` равен 0, параметр `maxSurge` не может быть тоже равным 0, поскольку в целях сохранения полной доступности система должна временно увеличивать количество реплик, чтобы у новых подов было время достичь состояния готовности.

Рекомендации

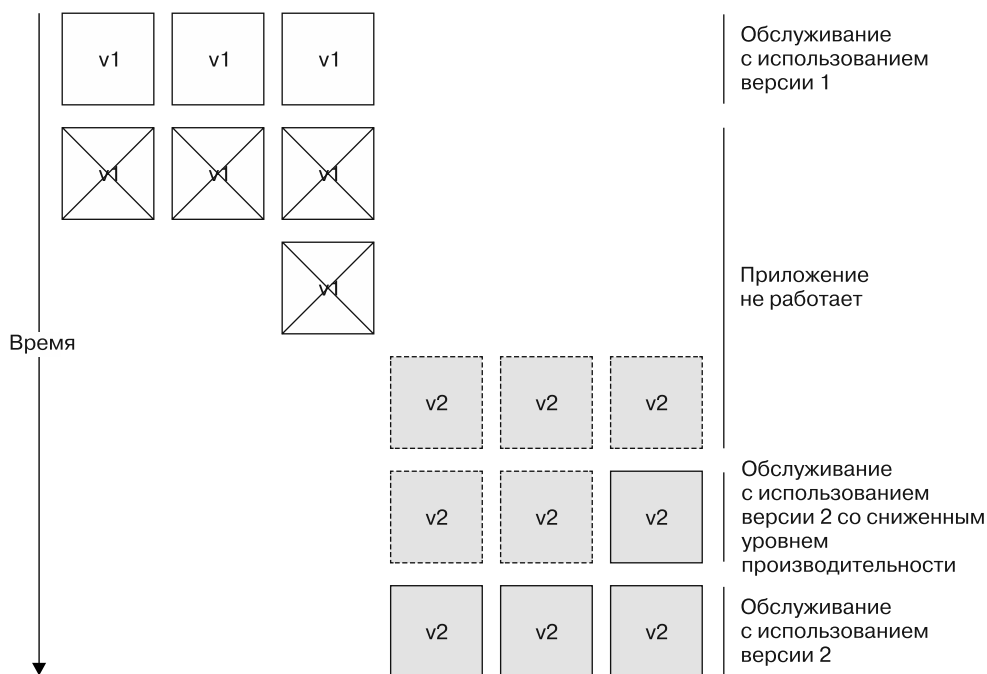
Скользящее обновление можно использовать применительно к большинству сервисов. В случае сервисов продуктивного уровня параметр `maxUnavailable` рекомендуется делать равным 0. Параметр `maxSurge` должен быть равен как минимум единице или большему значению, если у вас достаточно дополнительных ресурсов и вы хотите ускорить процесс обновления.

Развертывание изменений путем скользящего обновления

После того как вы настроите свой объект `Deployment` на использование скользящего обновления, для развертывания изменений будет достаточно обновить манифест развертывания (например, указав новую версию контейнера) и применить изменения с помощью команды `kubectl apply`. Такие изменения, как определение проверок готовности и жизнеспособности, тоже, как правило, подвергаются версионированию и развертываются точно так же, как новые версии образа контейнера. При необходимости запустить обновление можно и без внесения каких-либо изменений в объект `Deployment`, просто выполнив команду `kubectl rollout restart deploy $DEPLOYMENT_NAME`.

4.2.2. Стратегия повторного создания

Еще один подход, который иногда считают устаревшим, сводится к тому, чтобы просто переключиться на новую версию приложения, удалив все старые версии подов и распределив вместо них новые версии. В отличие от двух других стратегий обновления, рассматриваемых выше, данная стратегия *не является* стратегией с нулевым временем простоя. При ее использовании приложение практически всегда на какое-то время становится недоступным (рис. 4.3). При использовании корректной проверки готовности это время простоя можно сократить до минимума, сделав его равным времени загрузки первого нового пода, при условии, что он будет способен справиться с текущим клиентским трафиком. У данной стратегии



Условные обозначения

	Контейнер находится в процессе создания		Версия 1
	Под обслуживает трафик		Версия 2
	Выполняется остановка пода		

Рис. 4.3. Состояние подов при проведении обновления с использованием стратегии повторного создания. При такой стратегии приложение проходит через этапы полного простоя и сниженной производительности

есть преимущество: она не требует, чтобы старая и новая версии были совместимы друг с другом (поскольку они не будут работать одновременно) и чтобы у вас были дополнительные вычислительные мощности (ведь одни поды просто замещаются другими).

Данную стратегию можно использовать в среде разработки или в предпродуктивной среде, чтобы ускорить процесс обновления и обойтись без предоставления дополнительных вычислительных мощностей, необходимых для скользящего обновления. Однако во всех остальных случаях ее следует избегать. Чтобы ее использовать, определите поле `strategy`, показанное в листинге 4.6, следующим образом:

```
strategy:
  type: Recreate
```

4.2.3. Сине-зеленая стратегия

При сине-зеленой стратегии новая версия развертывается рядом со старой (рис. 4.4). Эти версии, соответственно, условно называются «синей» и «зеленой». Когда процесс развертывания, тестирования и подготовки новой версии к работе будет завершен, вы переключаете свой сервис на ее использование. При возникновении каких-либо проблем вы можете немедленно выполнить откат к старой версии. Убедившись в течение некоторого времени, что новая версия работает нормально, вы удаляете старую версию. В отличие от предыдущих двух стратегий, в данном случае старая версия остается готовой к обслуживанию и удаляется только после проверки работоспособности новой (и часто по усмотрению человека).

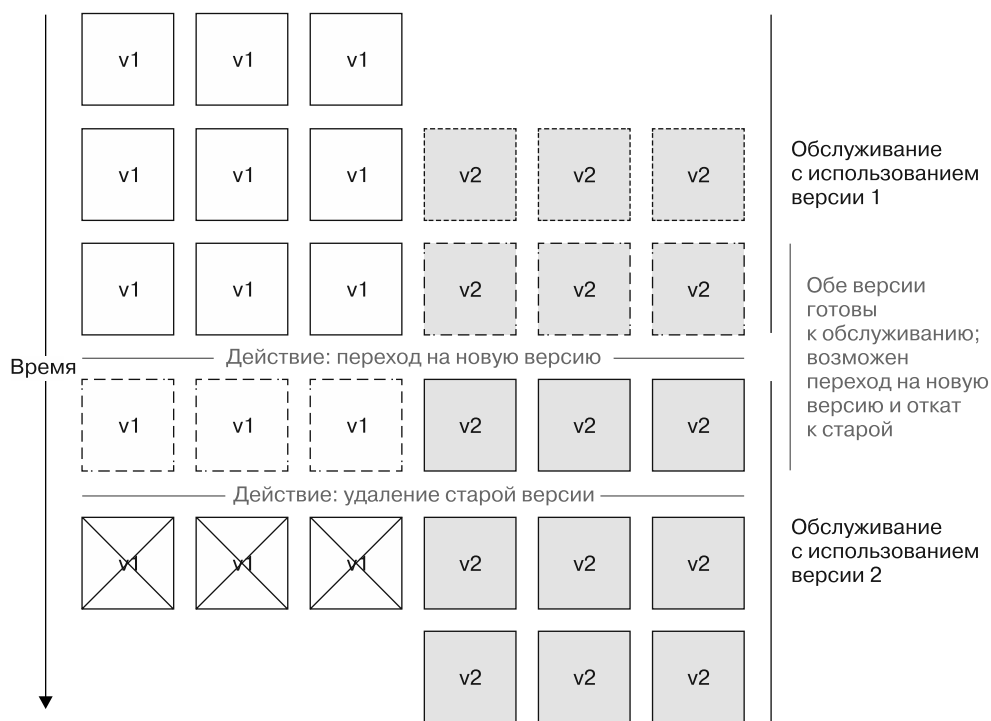
Данная стратегия обладает несколькими преимуществами:

- в тот или иной момент времени работает только одна версия приложения, благодаря чему обеспечивается единообразный пользовательский опыт;
- обновление выполняется очень быстро (за считанные секунды);
- так же быстро выполняется и откат к старой версии.

К недостаткам можно отнести следующие факторы:

- в течение какого-то времени требуется использовать удвоенное количество вычислительных ресурсов;
- данная стратегия не поддерживается напрямую развертываниями Kubernetes.

Эта стратегия обновления относится к передовым и часто применяется в случае крупных развертываний. Процедура обновления при этом может содержать и несколько дополнительных процессов. Например, после подготовки новой версии к работе она может сначала проверяться на некотором множестве внутренних пользователей, а затем — на некой доле внешнего трафика, после чего выполняется 100%-ное переключение на новую версию (этот процесс называется *канареечным тестированием*). Затем она снова оценивается, обычно в течение некоторого



Условные обозначения

	Контейнер находится в процессе создания		Выполняется остановка пода
	Под готов, но не обслуживает трафик		Версия 1
	Под обслуживает трафик		Версия 2

Рис. 4.4. Состояние подов при использовании сине-зеленой стратегии развертывания. В отличие от предыдущих стратегий, здесь есть две точки действия, в которых могут принимать решения другие системы, в состав которых потенциально может входить человек

периода времени (до нескольких дней), после чего начинается процесс сворачивания старой версии. Большой масштаб развертывания обеих версий, безусловно, требует удвоенного количества ресурсов, но зато в течение этого периода времени вы можете практически мгновенно выполнить откат к старой версии.

В отличие от предыдущих двух стратегий — стратегии скользящего обновления и повторного создания — сине-зеленая не поддерживается в Kubernetes напрямую. Чтобы упростить реализацию ее сложных элементов, пользователи обычно применяют дополнительные инструменты. Так, Istio позволяет выполнить тонкую

настройку разделения трафика, а Spinnaker — обеспечить автоматическую работу конвейера развертывания, содержащего канареечное тестирование и точки принятия решений.

Несмотря на отсутствие встроенной поддержки, сине-зеленую стратегию можно применять в Kubernetes. Конечно, если вы не будете использовать вышеупомянутые инструменты для настройки конвейера и разделения трафика, то вам придется выполнять много работы вручную и вы не будете иметь таких расширенных возможностей, как возможность проведения канареечного тестирования на крошечной доле эксплуатационного трафика. Однако реализовать эту стратегию не так уж сложно.

Вспомним объекты Deployment и Service, которые мы развернули в главе 3. Чтобы опробовать сине-зеленую стратегию на этом приложении, нам потребуется лишь создать еще один объект Deployment. Продублируйте развертывание и добавьте в один из двух экземпляров суффикс `-blue`, а во второй — суффикс `-green`. Добавьте эти суффиксы и в имени объекта Deployment, и в метке пода. Это позволит вам направлять трафик от своего сервиса либо к «синей», либо к «зеленой» версии пода.

При определении стратегии обновления в конфигурации развертывания укажите тип стратегии **Recreate** (Повторное создание). Поскольку в данном случае обновляются только поды неактивного развертывания, удаление старых версий и создание подов с новой версией приложения не ведет к простою и выполняется быстрее, чем в случае скользящего обновления.

Направление трафика к той или иной версии обеспечивается с помощью селектора сервиса (рис. 4.5). В данной системе с двумя развертываниями в любой момент времени одна версия является активной, а вторая — неактивной. Сервис выбирает поды активного развертывания с помощью селектора меток.

Чтобы развернуть новую версию с помощью сине-зеленой стратегии, вы должны выполнить следующие шаги.

1. Определите, какое развертывание является неактивным (то есть не выбрано сервисом).
2. Обновите путь к образу у неактивного развертывания, указав новую версию образа контейнера.
3. Дождитесь того момента, когда развертывание будет полностью выкачено (`kubectl get deploy`).
4. Обновите селектор сервиса так, чтобы он указывал на метку пода с новой версией.

Чтобы выполнить эти шаги, необходимо внести изменения в конфигурационный YAML-файл соответствующего ресурса и применить изменения с помощью команды `kubectl apply`. При следующем обновлении этого приложения нужно будет выполнить те же шаги, поменяв метки версий местами (то есть если в прошлый раз активной была синяя версия, то в этот раз будет зеленая).

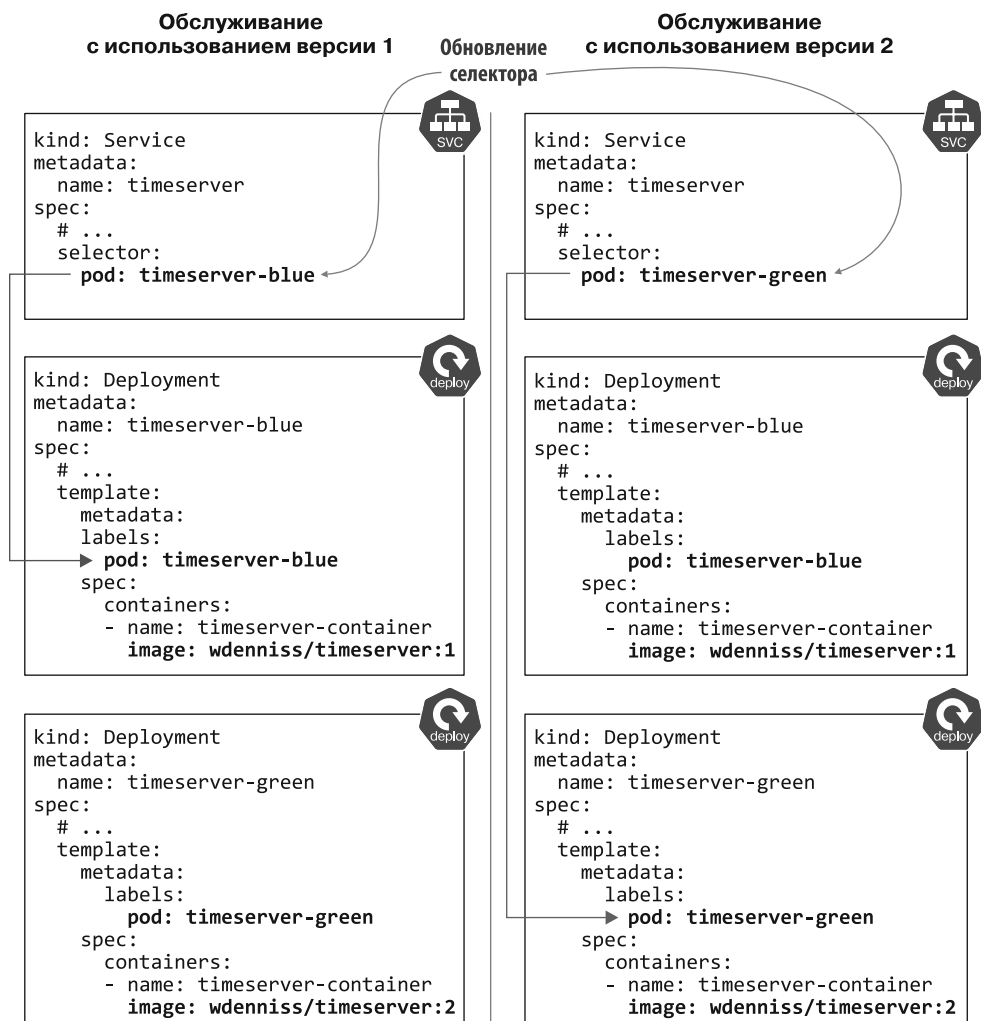


Рис. 4.5. Попеременное использование одним сервисом двух развертываний с разными версиями контейнера

Как я уже упоминал, данная стратегия удваивает количество подов, используемых развертыванием, тем самым существенно увеличивая количество потребляемых ресурсов. Чтобы минимизировать их потребление, вы можете уменьшать количество неактивных развертываний до нуля в те периоды времени, когда обновления не будут проводиться, снова делая его равным количеству активных развертываний непосредственно перед проведением обновления. При этом, вероятно, потребуются соответствующим образом регулировать и количество используемых кластером узлов (мы коснемся этой темы в главе 6).

4.2.4. Выбор стратегии развертывания

Для большинства развертываний вам хватит возможностей встроенных стратегий обновления. В качестве простейшего способа обеспечить проведение обновлений с нулевым временем простоя рекомендую использовать в Kubernetes скользящую стратегию обновления (**RollingUpdate**). Чтобы полностью исключить простои и сбои, потребуется реализовать проверку готовности; отсутствие такой проверки может привести к тому, что трафик будет направляться к контейнеру еще до его полной загрузки. Трафик может одновременно обслуживаться и старой, и новой версией приложения, что следует учитывать при разработке таких вещей, как форматы данных. В большинстве случаев рекомендуется предусматривать возможность использования как минимум текущей и предыдущей версии, поскольку это позволяет откатиться к предыдущей версии при возникновении каких-либо проблем.

Стратегию повторного создания (**Recreate**) можно с успехом применять в тех случаях, когда нет необходимости одновременно использовать две версии приложения. Например, она хорошо подходит для работы с унаследованными одноэкземплярными сервисами, которые могут существовать только в виде одной копии.

Сине-зеленое развертывание представляет собой стратегию более высокого уровня, которая требует использования дополнительных инструментов и процессов. Платой за это является возможность практически мгновенно переключиться на новую версию и получить преимущества двух других подходов, а именно использование в любой момент времени только одной версии приложения, но без каких-либо простоев стратегии повторного создания (**Recreate**). Я рекомендую начать со встроенных стратегий, отложив использование сине-зеленой стратегии до того момента, когда вам действительно потребуется нечто большее.

РЕЗЮМЕ

- Kubernetes предоставляет много инструментов, призванных помочь с обеспечением бесперебойной работы ваших развертываний и их обновления.
- Чтобы система Kubernetes могла обеспечивать бесперебойную работу вашего приложения с помощью перезапуска зависших и не дающих ответа контейнеров, необходимо определить проверки работоспособности.
- Проверка жизнеспособности позволяет системе Kubernetes определить, нуждается ли приложение в перезапуске.
- Проверка готовности определяет, какие реплики могут получать трафик от сервиса, что особенно важно при проведении обновлений, поскольку можно исключить потерю части запросов.
- Kubernetes позволяет исключить вероятность возникновения простоев в ходе обновления вашего приложения.

- По умолчанию в Kubernetes используется скользящая стратегия обновления (**RollingUpdate**), которая обеспечивает нулевое время простоя при минимальном потреблении дополнительных ресурсов.
- Стратегия повторного создания (**Recreate**) позволяет выполнять локальное обновление с определенным временем простоя, но без использования дополнительных ресурсов.
- Сине-зеленая стратегия обновления не поддерживается системой Kubernetes напрямую, но все же может быть реализована в ней с помощью ее стандартных конструкций.
- Сине-зеленая стратегия предлагает расширенные возможности, но является более сложной и временно удваивает количество ресурсов, потребляемых развертыванием.

5

Управление ресурсами

В этой главе

- ✓ Как Kubernetes выделяет ресурсы в кластере.
- ✓ Как настроить рабочую нагрузку, чтобы она запрашивала лишь необходимое ей количество ресурсов.
- ✓ Снижение затрат путем избыточного выделения ресурсов.
- ✓ Нахождение баланса между количеством реплик и степенью внутреннего параллелизма пода.

В главе 2 мы говорили о том, что контейнеры представляют собой дополнительный уровень изоляции с собственными ресурсами, а в главе 3 рассмотрели базовую единицу планирования платформы Kubernetes — под (набор контейнеров). В данной главе я расскажу о том, как поды распределяются по машинам в зависимости от их потребности в ресурсах и какую информацию нужно предоставить системе, чтобы под получил необходимые ему ресурсы. Зная, как поды распределяются по узлам, вы сможете принимать более эффективные архитектурные решения, касающиеся запросов ресурсов, их избыточного потребления и выделения, а также доступности и надежности.

5.1. РАСПРЕДЕЛЕНИЕ ПОДОВ

Распределение подов по узлам с учетом потребности в ресурсах в Kubernetes выполняет планировщик, который фактически представляет собой «мозг» вашей системы. Когда вы применяете свою конфигурацию в Kubernetes (как мы делали в главах 3 и 4), именно планировщик отыскивает в кластере узел с достаточным количеством ресурсов и дает ему указание загрузить и запустить контейнеры ваших подов (рис. 5.1).

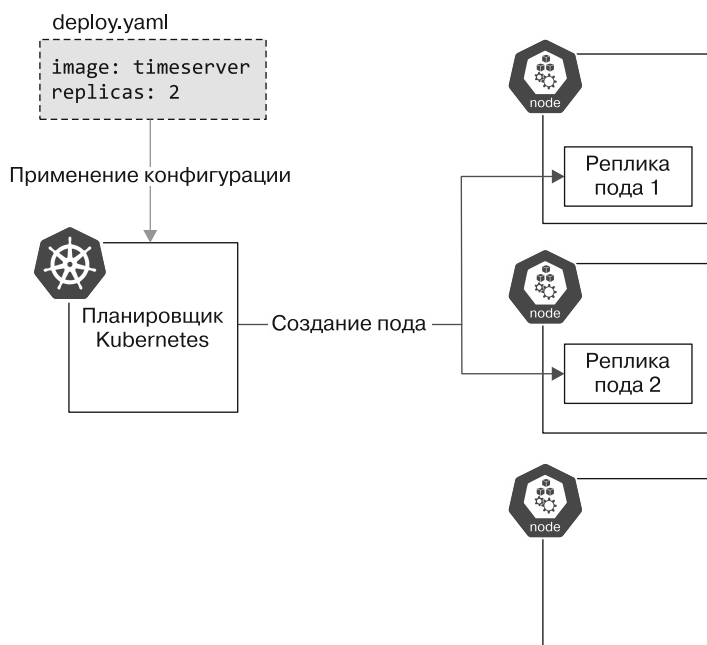


Рис. 5.1. Когда пользователь применил свою конфигурацию, планировщик создает в узлах реплики подов

Однако работа планировщика и связанных с ним компонентов на этом не заканчивается. Если вы используете объект Deployment (как мы уже делали в этой книге), то планировщик непрерывно следит за тем, чтобы состояние системы соответствовало указанным вами требованиям. То есть если ваш объект Deployment запрашивает две реплики пода, то планировщик не только создаст их, но и будет поддерживать в работающем состоянии. При возникновении какой-либо проблемы (например, при отказе узла из-за какого-либо сбоя) планировщик попытается найти для пода другое место, чтобы снова привести систему в соответствие с вашими требованиями (которые в данном случае сводятся к использованию двух реплик) (рис. 5.2).

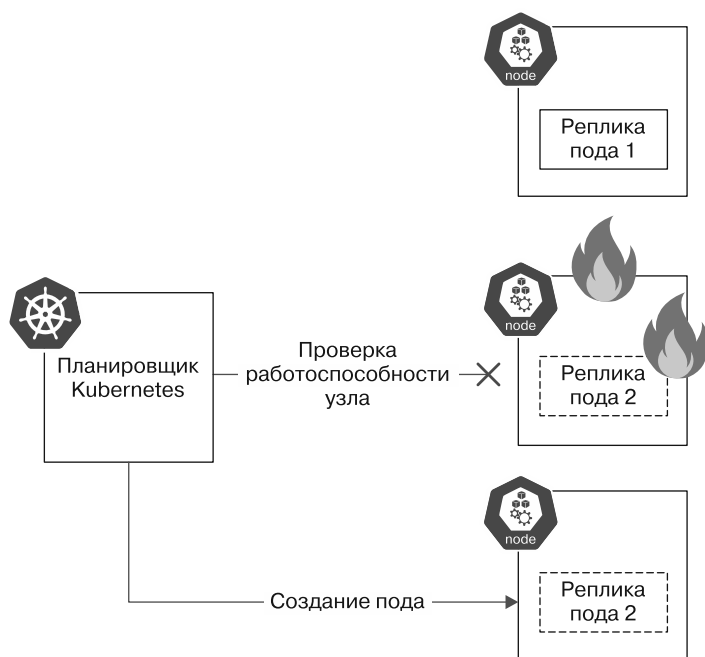


Рис. 5.2. При возникновении проблем на одном из узлов проверка работоспособности управляющего слоя Kubernetes выдает отрицательный результат. В таком случае и планировщик создает новую реплику пода на работоспособном узле

Повторное создание подов в случае отказа узлов — совсем не то же самое, что рассмотренная в предыдущей главе операция перезапуска подов. Перезапуск при выдаче отрицательного результата проверки жизнеспособности или готовности выполняет локально на узлах утилита kubelet, а отслеживание состояния узлов и повторное распределение подов при выявлении проблем осуществляет планировщик.

Узлы кластера имеют ограниченное количество ресурсов, а поды могут нуждаться в разном количестве ресурсов, поэтому важной задачей планировщика является поиск мест с достаточным количеством ресурсов для запуска того или иного пода (рис. 5.3). И в случае первоначального развертывания, и в случае повторного развертывания сбойного пода (см. рис. 5.2) планировщик подыскивает в кластере подходящее место для контейнеров пода, принимая во внимание множество разных параметров планирования.

Задача планировщика состоит в том, чтобы найти для пода подходящее место в кластере, исходя из его запросов ресурсов и всех других требований к размещению (о которых я расскажу в главе 8). Поды, которые не удастся разместить

в кластере, остаются в состоянии ожидания (Pending) (если некоторые из ваших подов слишком долго остаются в этом состоянии, то см. пункт «Устранение неполадок: застревание в состоянии ожидания» в главе 3).

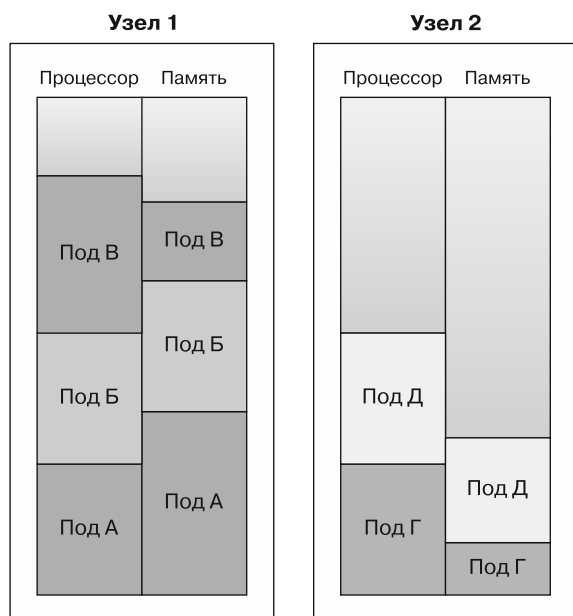


Рис. 5.3. Размещение пяти контейнеров на двух узлах с учетом их потребности в ресурсах

5.1.1. Определение ресурсов подов

Необходимую планировщику информацию можно предоставить, определяя запросы ресурсов в манифесте развертывания (или в других типах нагрузки, определение которых содержит определение пода). В предыдущих примерах не определялись запросы ресурсов, однако эту информацию всегда следует добавлять, когда речь идет о развертывании продуктивного уровня. В качестве примера в листинге 5.1 показано, как может выглядеть определение пода, которому требуется 20 % ресурсов ядра процессора и 200 МиБ памяти.

Обозначение `200m` в данном случае означает «200 миллиардер», то есть 20 % ресурсов одного ядра. Количество ресурсов процессора можно указывать с помощью чисел с плавающей запятой (например, `0.2`), однако очень многие пользователи платформы Kubernetes предпочитают указывать это количество в миллиядрах. Применительно к памяти буквы `mi` означают мебибайт (МиБ), `gi` — гигабайт (1024^3 байтов), `m` — мегабайт (1000^2), и `G` — гигабайт (1000^3 байтов) (для многих

разработчиков мегабайт — это как раз $1024^2 = 2^{20}$, однако разработчики Kubernetes воспользовались определениями единиц в системе СИ, где принят именно такой подход. — *Примеч. науч. ред.*).

Листинг 5.1. Chapter05/5.1.1_PodResources/deploy_requests.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:3
          resources:
            requests:
              cpu: 200m
              memory: 250Mi
```

Запросы ресурсов
данного контейнера

Определение необходимого количества ресурсов играет важную роль, позволяя Kubernetes подбирать для подов узлы в соответствии с их потребностью в ресурсах. Если вы не определите запросы ресурсов для какой-то части подов, то они будут распределены по произвольно выбранным узлам. Как этот случай выглядит в сравнении с тем, когда определяются все запросы ресурсов, показано на рис. 5.4. В представленном слева случае все пять подов распределяются по двум узлам с учетом их запросов ресурсов, а в случае, представленном справа, три пода не получили запросы, и система просто размещает их на том же узле, где были размещены первые два пода. Кроме того, можно заметить, что во втором случае для тех же пяти подов выделяется в два раза меньше вычислительных мощностей. При этом может случиться так, что подам без явно описанных запросов ресурсов не хватит выделенных мощностей или они будут вытеснены из-за того, что попытаются использовать больше памяти, чем доступно на узле.

На первый взгляд, в том, как система Kubernetes распределяет поды, нет ничего сложного: она просто выделяет то количество ресурсов, которое было указано в запросах. Однако на самом деле этот процесс не так прост, поскольку поды могут потреблять *очень много* ресурсов, то есть больше того количества, которое они запрашивают. Кроме того, многие процессы используют все запрашиваемые ресурсы лишь в течение какого-то времени, в остальное время нуждаясь в гораздо

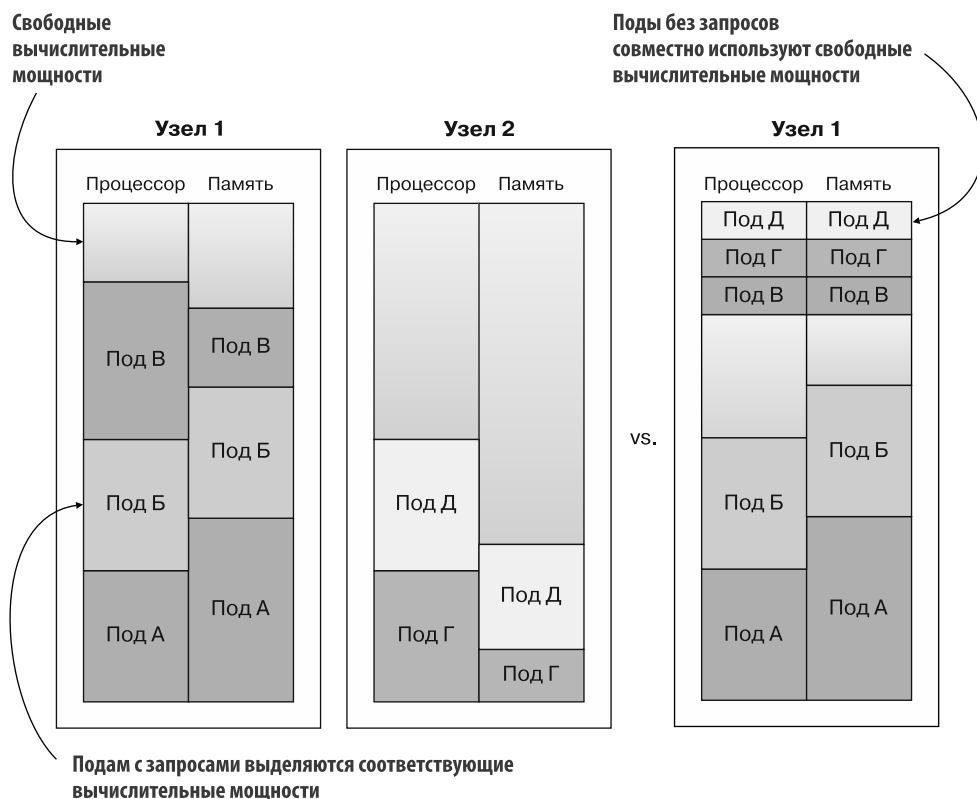


Рис. 5.4. Сравнение распределения подов при определении запросов ресурсов для всех подов и только для некоторой их части. Поды без запросов совместно используют свободные вычислительные мощности, которые предоставляются им по мере возможности, без учета их реальных потребностей

меньшем их количестве. Не будет ли разумным временно предоставлять неиспользуемые мощности другим подам, размещенным на том же узле? Именно так и будет поступать Kubernetes в том случае, если вы определите *лимиты*. При этом под получает и запросы ресурсов, с учетом которых планировщик ищет для него подходящее место, и лимиты, за которые он не должен выходить после своего запуска в выделенном для него месте (листинг 5.2).

Листинг 5.2. Chapter05/5.1.1_PodResources/deploy_requests_limits.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
```



```

matchLabels:
  pod: timeserver-pod
template:
  metadata:
    labels:
      pod: timeserver-pod
  spec:
    containers:
      - name: timeserver-container
        image: docker.io/wdenniss/timeserver:3
        resources:
          requests:
            cpu: 200m
            memory: 250Mi
          limits:
            cpu: 300m
            memory: 400Mi

```

Лимиты ресурсов данного контейнера.	Лимиты ресурсов данного контейнера.
Контейнер может использовать до 30 % ресурсов	Контейнер может использовать до 30 % ресурсов
ядра процессора и до 400 МиБ памяти	ядра процессора и до 400 МиБ памяти

При размещении подов на узлах планировщик учитывает только запросы ресурсов пода, не обращая внимания на его лимиты ресурсов. Однако и запросы, и лимиты влияют на производительность пода после его запуска.

В случае превышения *лимита* на ресурсы памяти система выполняет перезапуск пода, а в случае превышения лимита на ресурсы процессора просто регулирует выделенное поду процессорное время в соответствии с лимитом. Эти действия выполняются непосредственно на узле утилитой kubelet.

В тех случаях, когда несколько подов начинают конкурировать за ресурсы, поды, вышедшие за рамки своего *запроса* на ресурсы памяти, вытесняются (о том, как выполняется выбор вытесняемых подов, я расскажу в подразделе 5.1.3), а тем подам, которые потребляют процессор больше *запрошенного*, выделяется процессорное время не более запрошенного (а не лимита).

Учитывая, насколько важную роль эти параметры играют при распределении подов и управлении их работой, рекомендуется определять для контейнеров подов и запросы, и лимиты. Но как определить, чему должны равняться эти значения? В следующем разделе я расскажу, как запросы и лимиты совместно определяют класс качества обслуживания (quality of service, QoS) и как оценить уровень производительности приложения, позволяющий определить, какие значения вам следует использовать.

5.1.2. Качество обслуживания

В тех случаях, когда вы задаете лимиты, превышающие запросы, или вообще их не определяете, возникает еще одна проблема: что делать, когда такие поды начинают потреблять слишком много ресурсов (обычно слишком большое количество памяти) и подвергаются вытеснению в целях высвобождения ресурсов? Kubernetes решает эту проблему, ранжируя поды, чтобы в первую очередь удалять наименее важные из них.

Планируя рабочие нагрузки, подумайте о том, какой класс качества обслуживания им требуется. Kubernetes позволяет использовать три класса: гарантированный, с возможностью избыточного потребления и с предоставлением лучшего из возможного.

Гарантированный класс (Guaranteed)

У подов с данным классом обслуживания лимиты задаются равными запросам. Такая конфигурация наиболее стабильна, поскольку поду гарантированно предоставляется то количество ресурсов, которое он запрашивает, — ни больше ни меньше. Чтобы под можно было отнести к этому классу обслуживания, лимиты и запросы должны быть равными у всех его контейнеров.

Поды с гарантированным классом обслуживания при любых условиях имеют одинаковое количество ресурсов и никогда не вытесняются из узла, так как просто не могут потреблять больше того количества ресурсов, которое было им выделено при их распределении по узлам.

Класс с возможностью избыточного потребления (Burstable)

У подов с этим классом обслуживания лимиты превышают запросы, что позволяет им на какое-то время переходить в режим избыточного потребления, если в системе имеются дополнительные ресурсы (например, если другие поды не используют какую-то часть запрошенных ими ресурсов или если на узле еще остаются невыделенные ресурсы). С такими подами следует быть осторожными, поскольку при невнимательном их использовании можно ошибочно подумать, что они всегда будут работать в режиме избыточного потребления.

Допустим, изначально ваш под будет размещен на пустом узле, где ему будут доступны все имеющиеся ресурсы. Если в дальнейшем этот под будет перераспределен на другой узел с меньшим количеством ресурсов, то его производительность может заметно снизиться. Поэтому поды с возможностью избыточного потребления всегда следует тестировать в разных условиях. Под с несколькими контейнерами может быть отнесен к этому классу обслуживания, если в соответствии с требованиями не может быть отнесен к гарантированному классу и хотя бы один из его контейнеров имеет запрос ресурсов. Эти поды не подвергаются вытеснению в том случае, если не превышают свои запросы на такие неурезаемые ресурсы, как память.

Класс с предоставлением лучшего из возможного (Best Effort)

Поды, которые не получили ни запросы, ни лимиты, относятся к классу обслуживания с предоставлением лучшего из возможного и распределяются системой Kubernetes в любое удобное для нее место. Этот класс обслуживания — самый низкий, и я настоятельно не рекомендую его использовать. Вы можете добиться

того же результата, явно установив очень низкие запросы для подов с возможностью избыточного потребления, и лучше сделать так, чем надеяться на лучшее, пустив все на самотек.

Чтобы обеспечить стабильную работу пода, рекомендуется всегда запрашивать для него хотя бы минимально необходимое количество ресурсов и никогда не использовать поды без запросов ресурсов. В случае высокоприоритетных, критически важных рабочих нагрузок рекомендуется всегда делать лимиты равными запросам, обеспечивая тем самым гарантированный уровень производительности.

В случае конкуренции за ресурсы поды, относящиеся к классу обслуживания с предоставлением лучшего из возможного, вытесняются из узла в первую очередь.

5.1.3. Вытеснение, приоритет и приоритизация

При возникновении конкуренции за такие ограниченные ресурсы, как память (например, когда слишком большое количество подов пытается одновременно использовать избыточное количество памяти), Kubernetes высвобождает ресурсы, удаляя поды, которые потребляют больше запрошенного количества ресурсов, с помощью процесса, называемого *вытеснением* (eviction). Поэтому очень важно правильно определить необходимое для пода количество ресурсов. Если вытесненные поды входят в состав управляемой рабочей нагрузки, такой как развертывание, Kubernetes заново распределяет их в кластере, помещая обычно на другой узел. Но если какой-либо под вытесняется слишком часто, это обычно снижает степень доступности рабочей нагрузки и говорит о необходимости увеличения запросов ресурсов.

Вытеснение

Поды гарантированного класса никогда не подвергаются вытеснению в случае конкуренции за ресурсы, поэтому в тех случаях, когда требуется обеспечить сверхнадежное развертывание, всегда задавайте лимиты подов равными их запросам, тем самым обеспечивая им гарантированный класс обслуживания.

Далее я расскажу о том, как Kubernetes ранжирует поды негарантированных классов, чтобы определить, какие из них следует подвергнуть вытеснению, и как вы можете повлиять на этот процесс.

Принимая решение о том, какие поды следует подвергнуть вытеснению, Kubernetes в первую очередь рассматривает те, которые потребляют ресурсы в количестве, превышающем их запросы, сортируя такие поды по приоритету и по объему избыточного потребления (того вида ресурсов, который стал объектом конкуренции). Поды, относящиеся к классу обслуживания с предоставлением лучшего из возможного, не имеют запросов ресурсов, поэтому вытесняются самыми первыми (начиная

Часть I. Начало работы с Kubernetes

с тех, которые потребляют наибольшее количество ресурсов). По умолчанию все поды имеют одинаковый приоритет (0), и в таком случае Kubernetes ранжирует их исключительно по объему избыточного потребления (рис. 5.5).

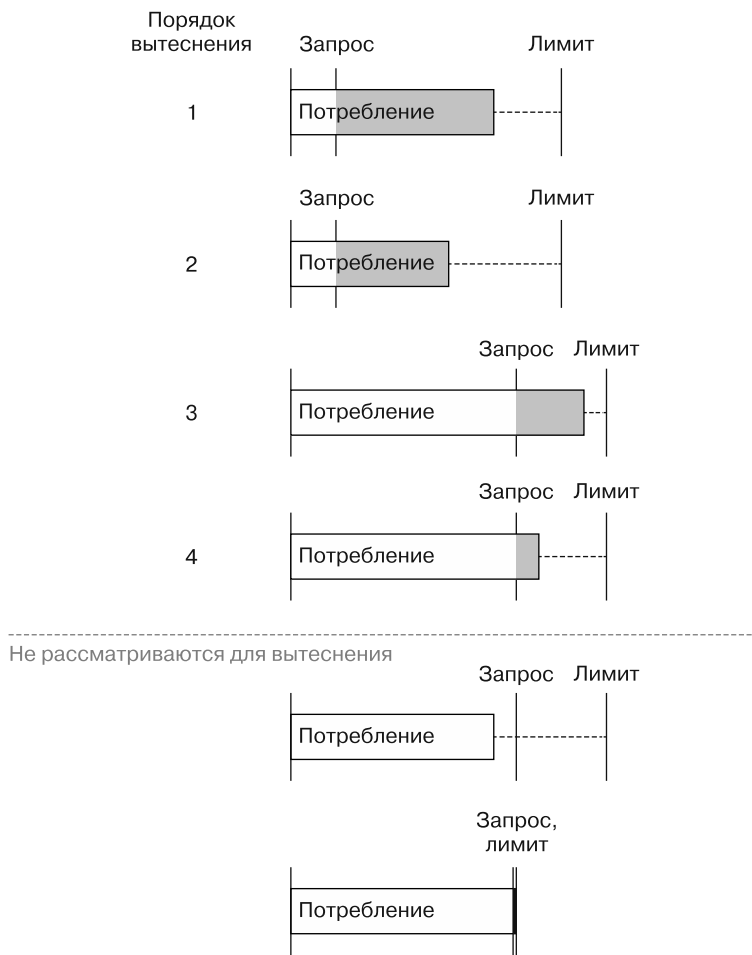


Рис. 5.5. Порядок вытеснения подов, имеющих одинаковый приоритет

Состояние ошибки Evicted

Если проверка состояния подов показывает, что некоторые из них находятся в состоянии `Evicted`, это говорит о том, что планировщик вытеснил их из-за избыточного потребления ресурсов. Это вполне допустимо, если такое вытеснение происходит эпизодически. Но если оно происходит слишком часто, то увеличьте запросы ресурсов своих контейнеров и подумайте о том, не стоит ли также добавить в кластер дополнительные вычислительные мощности.

Приоритет

Приоритет — это целочисленное значение в диапазоне от 0 до 1 000 000 000, которое вы можете присвоить подам в качестве класса приоритета, позволяющего изменять порядок ранжирования. На рис. 5.6 показано, в каком порядке будут вытесняться поды из примера, представленного на рис. 5.5, после присвоения им разного приоритета. Как видите, при вытеснении подов сначала учитывается их приоритет, а затем — объем избыточного потребления. Поды, которые не выходят за рамки своих запросов ресурсов, не рискуют быть вытесненными, вне зависимости от того, какой приоритет им присвоен.

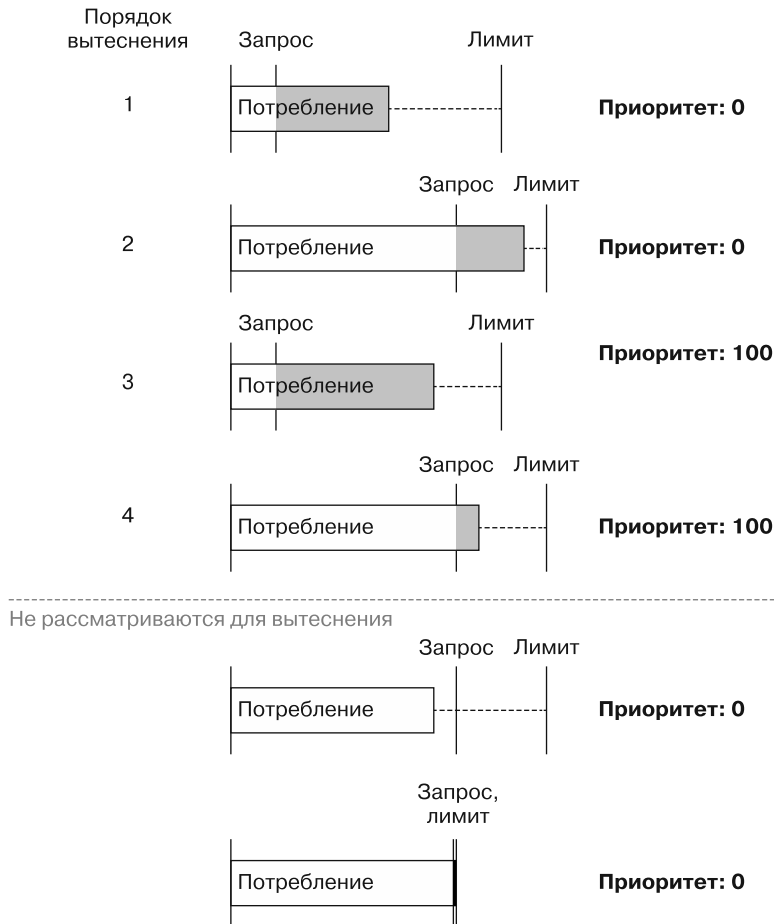


Рис. 5.6. Порядок вытеснения подов, имеющих разный приоритет

Чтобы создать собственный уровень приоритета, вы должны сначала создать объект `PriorityClass` (листинг 5.3).

Листинг 5.3. Chapter05/5.1.3_Priority/priorityclass.yaml

```

apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
  value: 1000000
preemptionPolicy: Never
globalDefault: false
description: "Critical services."

```

Целочисленное значение приоритета

Данный класс приоритета не будет вызывать вытеснение подов с более низким приоритетом при отсутствии в кластере доступных вычислительных мощностей

Должен ли данный класс приоритета использоваться по умолчанию?

Теперь присвойте поду объект PriorityClass (листинг 5.4).

Листинг 5.4. Chapter05/5.1.3_Priority/deploy.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      priorityClassName: high-priority
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:3
          resources:
            requests:
              cpu: 200m
              memory: 250Mi

```

Класс приоритета, который следует использовать для данного развертывания

Кроме того, планировщик использует приоритеты в ходе распределения подов. При большом количестве подов он в первую очередь распределяет те из них, которые имеют наибольший приоритет. Возможность управления порядком распределения может быть особенно полезной в тех случаях, когда требуется задать определенный порядок выполнения пакетных заданий (о которых мы поговорим подробнее в главе 10).

Приоритизация

Приоритеты могут использоваться сами по себе для ранжирования рабочих нагрузок так, чтобы наиболее важные из них распределялись первыми и вытеснялись последними. Однако при этом не исключена вероятность того, что при недостаточном количестве ресурсов в кластере высокоприоритетные поды будут долго оставаться в состоянии ожидания (Pending), в то время как низкоприоритетные будут работать.

Если вам нужно, чтобы вместо этого высокоприоритетные рабочие нагрузки проактивно вытесняли низкоприоритетные, а не ждали, пока освободятся необходимые им вычислительные мощности, то вы можете добавить поведение приоритизации (preemption), отредактировав поле `preemptionPolicy` объекта `PriorityClass` (листинг 5.5).

Листинг 5.5. Chapter05/5.1.3_Priority/priorityclass-preemption.yaml

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority-preemption
  value: 1000000
preemptionPolicy: PreemptLowerPriority
globalDefault: false
description: "Critical services."
```

Данный класс приоритета будет вытеснять низкоприоритетные поды в случае недостатка вычислительных мощностей в классе

К счастью, Kubernetes не забывает о тех подах, которые удаляются из узла в ходе вытеснения или приоритизации, при условии, что они входят в состав развертывания или другой управляемой рабочей нагрузки. Эти поды переводятся в состояние ожидания (Pending) и заново распределяются в кластере после того, как количество ресурсов снова становится достаточным. Это еще один довод в пользу использования таких конструкций для создания рабочих нагрузок, как развертывание, поскольку поды, созданные без развертывания, после вытеснения не распределяются заново.

Когда следует использовать приоритет и приоритизацию

Приоритет и приоритизация — очень полезные элементы системы Kubernetes, о которых следует иметь четкое представление, поскольку они влияют на процессы вытеснения и планирования. Но, прежде чем определять приоритеты для всех своих развертываний, проследите за тем, чтобы ваши поды получили корректные запросы и лимиты ресурсов, поскольку эта часть конфигурации обычно играет более важную роль.

Приоритет и приоритизация могут быть действительно полезными, если вы используете несколько развертываний и, стремясь сэкономить деньги, хотите по максимуму использовать вычислительные мощности кластера, выделяя подам ресурсов больше, чем фактически может предложить кластер. В таком случае необходимо как-то указать степень относительной важности каждого пода, чтобы разрешить конкуренцию за ресурсы. Однако я не рекомендую начинать определение своей конфигурации с этих элементов, поскольку так вы просто усложните ее. Проще сначала выделить кластеру такое количество ресурсов, которого будет с лихвой хватать для распределения всех ваших рабочих нагрузок, а затем выполнить более тонкую настройку, чтобы получить от этих ресурсов максимум эффективности. Еще раз напомним: самый простой способ обеспечить гарантированный уровень производительности критически важных сервисов сводится к тому, чтобы корректно определить запросы ресурсов подов и предоставить кластеру достаточное количество узлов для их распределения.

5.2. РАСЧЕТ РЕСУРСОВ ДЛЯ ПОДОВ

В предыдущем разделе я рассказал о том, почему для обеспечения высокого уровня надежности важно дать подам корректные запросы и лимиты ресурсов. Но как определить оптимальный размер запросов и лимитов? Это можно сделать, запустив поды и немного понаблюдав за их работой.

Для отслеживания уровня потребления ресурсов в Kubernetes можно использовать команду `kubectl top`, которая отображает информацию о том, сколько ресурсов используется подами и узлами. Нас будет интересовать только информация о подах, поскольку именно эти данные нужны для определения оптимальных запросов ресурсов.

Сначала попробуйте развернуть под, используя слишком большие запросы ресурсов. Возможно, он уже развернут таким образом в рабочей среде — завышенная оценка потребности в ресурсах обычно не вредит производительности (но может плохо сказываться на бюджете). Задача состоит в том, чтобы, предоставив поду избыточное количество ресурсов, оценить его реальный уровень потребления, а затем задать соответствующие запросы, исключив напрасную трату ресурсов.

Пока у вас еще нет четкого представления о том, сколько ресурсов требуется поду, лучше оставить его лимиты незадаанными (что позволит ему использовать все свободные ресурсы узла). Однако это не избавляет вас от необходимости задать *определенные* запросы ресурсов, поскольку изначально лучше выделить поду больше вычислительных мощностей, чем ему необходимо (листинг 5.6).

Листинг 5.6. Chapter05/5.2_ResourceUsageTest/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:3
          resources:
            requests:
              cpu: 200m
              memory: 250Mi
```

Количество реплик задано равным 1 для нагрузочного тестирования

На этапе тестирования мы не задаем лимиты ресурсов, чтобы можно было оценить уровень потребления

Выполните команду `kubect1 top pods` (возможно, при этом придется подождать результата в течение пары минут) и посмотрите, какой объем ресурсов, особенно памяти, используется на этапе запуска. Этот параметр полезно знать по той причине, что объем ресурсов, потребляемый подом на этапе запуска, можно использовать в качестве нижней границы при использовании класса обслуживания с возможностью избыточного потребления.

ПРИМЕЧАНИЕ

Если вы используете Minikube и получили сообщение об ошибке Metrics API not available (Недоступен API показателей), это можно исправить, включив поддержку показателей с помощью команды `minikube addons enable metrics-server`.

Теперь направьте к поду достаточно большую нагрузку, чтобы симитировать тот уровень потребления, который возможен на практике. Вам могут помочь такие инструменты для оценки производительности, как Apache Bench (эта утилита устанавливается вместе с Apache (<http://httpd.apache.org/docs/2.4/install.html>)). В качестве примера ниже показано, как с помощью этой утилиты можно сгенерировать 10 000 запросов, используя 20 потоков. Чтобы определить максимальный уровень потребления, обычно требуется протестировать под, задавая такую нагрузку в течение определенного времени (скажем, пяти минут).

```
kubect1 get svc
EXTERNAL_IP=203.0.113.16
ab -n 10000 -c 20 http://$EXTERNAL_IP/
```

Вдобавок вы можете понаблюдать за тем, как под справляется с реальной нагрузкой в рабочей среде. Как-либо ограничивать работу пода в рабочей среде следует только после того, как станет понятно, сколько ресурсов ему действительно необходимо. Поэтому сначала нужно задать завышенные запросы ресурсов и измерить реальный уровень потребления. Получив четкое представление о том, сколько ресурсов требуется поду, вы сможете скорректировать запросы, доведя их до адекватного уровня.

Направив к поду нагрузку, еще раз выполните команду `kubect1 top pods` (учитывая, что она может выдать результат только через одну-две минуты, проследите за тем, чтобы имитация нагрузки продолжала работать все это время). Результат будет выглядеть примерно так:

```
$ kubect1 top pod
NAME                                CPU(cores)    MEMORY(bytes)
timeserver-dd88988f5-tq2d9          145m          11Mi
```

По завершении тестирования пода вы получите несколько оценок уровня потребления, которые будут выглядеть примерно так, как показано в табл. 5.1 (приведенные в ней значения служат лишь в качестве примера).

Таблица 5.1. Уровень потребления ресурсов процессора и памяти на этапе запуска и под нагрузкой

Параметр	Процессор (в миллиардах)	Память (в мебибайтах)
На этапе запуска	20	200
При нормальном уровне нагрузки	200	400

Иногда бывает полезно повторить этот процесс еще несколько раз, чтобы оценить уровень потребления ресурсов подом при разном уровне нагрузки (например, при низком, среднем и высоком уровне трафика) и через разные промежутки времени. Оценки, полученные в последнем случае (например, сразу после загрузки, спустя один час после загрузки, спустя один день после загрузки), позволяют учесть, насколько может вырасти уровень потребления с течением времени (например, из-за утечек памяти). В конечном счете ваши результаты тестирования будут выглядеть примерно так, как показано в табл. 5.2.

Таблица 5.2. Итоговые результаты оценки уровня потребления ресурсов процессора и памяти

Параметр	Процессор (в миллиардах)	Память (в мебибайтах)
На этапе запуска	20	400
При нормальном уровне нагрузки	200	500
При высоком уровне нагрузки	850	503
Спустя один час	210	505
Спустя один день	200	600

5.2.1. Определение запросов и лимитов на объем памяти

Как же теперь определить запросы ресурсов на основе этих данных? Прежде всего, мы можем сделать вывод о том, что минимальный уровень потребления памяти в данном случае составляет 400 МиБ. Вы гарантированно получаете только тот объем памяти, который указан в запросе ресурсов, и вам известно, что под нагрузкой под потребляет как минимум 400 МиБ памяти, поэтому использование более низкого запроса приведет к возникновению ошибки OOMKilled (то есть прекращению работы пода из-за недостатка памяти). Возможно, вы не столкнетесь с этой ошибкой сразу же, если зададите более высокий лимит и будете располагать достаточным объемом свободной памяти. Но вам вряд ли стоит надеяться на наличие свободной памяти, если известно, что она все равно будет вам нужна.

Но значит ли это, что вы должны указать 400 МиБ в качестве запроса ресурсов? Нет. Во-первых, вам определенно потребуется некий буфер, скажем, в размере 10 %. Кроме того, полученные нами результаты показывают, что спустя один час работы

объем потребления составлял уже 505 МиБ, и поэтому в качестве исходной нижней границы будет лучше использовать данное значение (добавив к нему размер буфера). Но стоит ли в качестве нижней границы использовать значение 600 МиБ — тот уровень потребления, которого под достиг спустя один день, возможно, из-за наличия утечек памяти? Ответ на данный вопрос зависит от конкретной ситуации. Конечно, задав в качестве запроса более высокое значение, вы будете уверены в том, что ваш под без проблем проработает как минимум один день. Но поскольку Kubernetes может автоматически перезапускать сбойные контейнеры, то во многих случаях будет лучше сделать так, чтобы система спустя день перезагружала имеющий утечки процесс, не давая ему потреблять слишком много памяти.

В КАКИХ СЛУЧАЯХ ДОПУСТИМЫ УТЕЧКИ ПАМЯТИ

Общеизвестным фактом является то, что разработчики социальной сети Instagram (<https://instagram-engineering.com/dismissing-python-garbage-collection-at-instagram-4dca40b29172>) отключили сборку мусора в языке Python, тем самым заставив процессор работать на 10 % эффективнее. Этот подход вряд ли подойдет для среднего разработчика, но является достаточно интересным с точки зрения утечек памяти. Стоит ли их бояться, если процесс будет автоматически перезапущен при чрезмерном потреблении памяти, и при этом у нас тысячи реплик этого процесса? Вероятно, нет.

Kubernetes может автоматически перезапускать сбойные контейнеры (в том числе когда причина сбоя — недостаток памяти), и благодаря этому вы можете достаточно легко реализовать данный подход. Я не рекомендую использовать эту стратегию, не проведя предварительно тщательный анализ; тем не менее наличие в приложении небольшой утечки вряд ли стоит относить к числу серьезных проблем, которые необходимо устранить в первую очередь.

При этом важно проследить за тем, чтобы предоставленного контейнеру количества ресурсов было достаточно для его запуска и работы в течение определенного периода времени. В противном случае контейнер может циклически выдавать ошибку OOMKilled, что не понравится ни вам, ни вашим пользователям. Кроме того, во избежание заметных для пользователя отказов следует использовать достаточное количество реплик (о которых мы поговорим в следующем подразделе).

Используя полученные вами результаты тестирования, определите нижний предел потребления памяти, взяв за основу объем памяти, потребляемый подом под нагрузкой, и добавив к этому адекватный объем буфера (как минимум 10 %). Применительно к нашему примеру это значение может составлять $505 * 1,1 \approx 555$ МиБ. Мы знаем, что данного объема с лихвой хватит для как минимум часа работы под нагрузкой. В зависимости от бюджета и допускаемой степени риска вы можете

скорректировать это значение в большую или меньшую сторону (его увеличение снижает степень риска, но при этом возрастает уровень затрат).

Таким образом, ваши запросы должны как минимум обеспечивать стабильную работу пода. А что можно сказать о *лимите* на объем памяти? Допустим, что ваши результаты тестирования достаточно надежны и охватывают все возможные случаи (то есть у вас нет какого-либо случая с большим потреблением памяти, который не был охвачен тестированием). Тогда лимит не должен намного превышать уровень потребления, наблюдаемый спустя один день. Устанавливать более высокий лимит (например, в два раза превышающий это значение) не имеет смысла, поскольку ваши поды потребляют намного меньший объем памяти в течение первого дня. Если у вас есть утечка памяти, то будет лучше перезапускать под при достижении им лимита, не позволяя ему потреблять слишком много памяти.

Еще один вариант сводится к тому, чтобы просто установить лимит равным запросу, обеспечив тем самым гарантированный класс обслуживания для пода. Преимуществом этой стратегии является то, что она обеспечивает постоянный уровень производительности пода, вне зависимости от того, какие еще нагрузки запущены на узле помимо него. В таком случае нужно будет предоставить еще немного больше ресурсов, так как под будет уничтожен в тот самый момент, как только затребует больше ресурсов, чем было запрошено.

5.2.2. Определение запросов и лимитов на ресурсы процессора

В отличие от памяти, процессор является *сжимаемым* ресурсом. Если недостаток памяти ведет к сбою приложения, то недостаток процессорного времени вызывает лишь замедление работы процессора. Скорее всего, вы захотите предоставить приложению достаточное количество процессорного времени, чтобы исключить падение производительности, но при этом нет острой необходимости предусматривать дополнительный буферный запас, как в случае с памятью.

Если взять приложение, результаты тестирования которого были представлены в табл. 5.2 (см. выше), то можно увидеть, что в стабильном состоянии потребление ресурсов процессора составляет около 200 миллиард. Это значение будет хорошей отправной точкой при определении запроса на ресурсы процессора. Если вы хотите сэкономить деньги и готовы смириться с некой потерей производительности, то можете запросить даже чуть меньшее количество ресурсов.

Правильно установив лимит на ресурсы процессора, можно заставить Kubernetes более эффективно использовать эти ресурсы. Если этот лимит будет превышать соответствующий запрос, то при необходимости приложение сможет избыточно потреблять не используемые на узле ресурсы процессора. Как и в случае памяти, Kubernetes гарантированно предоставляет только то количество ресурсов процессора, которое указано в запросе, но часто бывает полезно позволить поду использовать не используемые на узле ресурсы процессора, чтобы он мог работать

немного быстрее. Что касается веб-приложений, которые проводят много времени в ожидании ответа от внешних зависимостей (например, от базы данных), то на узле часто бывает много неиспользуемых ресурсов процессора, которые можно предоставлять для обработки других активных запросов.

Как и в случае памяти, у варианта использования лимита, превышающего запрос (например, при определении класса обслуживания с возможностью избыточного потребления), есть недостаток: такой подход не позволяет обеспечить постоянный уровень производительности. Если вы запустите под с возможностью избыточного потребления на пустом узле, то у него будет гораздо больше ресурсов, чем когда вы запустите его на узле с большим количеством других подов. Часто бывает полезно позволить приложению потреблять неиспользуемые вычислительные мощности узла в случае всплесков трафика. Но если в вашем случае важно обеспечить постоянный уровень производительности, то, возможно, будет лучше сделать лимиты равными запросам.

5.2.3. Снижение затрат путем избыточного выделения ресурсов процессора

Один из возможных способов снижения затрат сводится к тому, чтобы выделять подам больше ресурсов процессора, чем их имеется на узле. При этом в запросе каждого пода указывается заниженное количество ресурсов процессора (которое меньше необходимого количества), что позволяет разместить на узле больше подов, чем при точном указании в запросах необходимого количества ресурсов.

Эта стратегия позволяет сэкономить деньги, но, очевидно, плохо сказывается на производительности. Несмотря на это, она может быть весьма полезной, когда рабочие нагрузки чрезвычайно склонны к *избыточному потреблению*. Допустим, вам нужно разместить сотни сайтов с низким уровнем трафика, каждый из которых обрабатывает только несколько запросов в час и потребляет ресурсы процессора лишь в эти редкие моменты. В таком случае запрос на ресурсы процессора для каждого приложения можно сделать равным 50 миллиадам (что позволит распределить по 20 подов на каждое ядро), а лимит на ресурсы процессора — равным 1000 миллиадам (что даст возможность временно выделять приложению целое ядро).

Чтобы успешно использовать стратегию избыточного выделения, необходимо четко понимать, какие еще процессы могут работать на машине. Если вы уверены в том, что большинство развертываемых вами сайтов будет почти всегда в незагруженном состоянии, то данный подход может быть очень эффективным. Но если в случае всплесков трафика сразу все поды будут пытаться перейти в режим избыточного потребления, это может привести к значительному падению производительности. При использовании такой конфигурации ваши контейнеры фактически перестают быть изолированными друг от друга, и поэтому вы должны четко понимать, что и как размещаете на узле. Однако в принципе эта проблема не является непреодолимым препятствием.

Конечно, самый надежный подход состоит в том, чтобы вообще не использовать избыточное выделение. Разумный компромисс сводится к тому, чтобы просто не злоупотреблять им. Позволив подам иногда задействовать чуть больше ресурсов процессора (путем установки лимита, превышающего запрос), вы можете обеспечить некоторое снижение времени реакции. Однако при этом нужно проследить за тем, чтобы *запросы* на ресурсы процессора были достаточно высокими, что позволит подам справиться с определенным базовым уровнем нагрузки и не слишком сильно полагаться на избыточные вычислительные мощности.

5.2.4. Нахождение баланса между количеством реплик и степенью внутреннего параллелизма пода

Теперь, уже зная о том, как запросы ресурсов сказываются на процессе распределения подов и количестве выделяемых им ресурсов, поговорим о таком явлении, как внутренний параллелизм пода. От степени параллелизма пода (количества одновременно работающих в нем процессов/потоков приложения) в значительной мере зависит то, сколько ресурсов ему будет необходимо. Поэтому вы должны обеспечить приемлемый компромисс между эффективностью и надежностью, найдя баланс между степенью параллелизма и количеством реплик.

Допустим, что раньше вы имели дело со средой, в которой установка приложения требовала больших затрат: денежных на серверы или временных на настройку экземпляров. В таком случае ваше приложение, скорее всего, обладает значительной степенью внутреннего параллелизма, который обеспечивается благодаря использованию параллельных потоков или веток и может быть выражен как количество параллельных рабочих потоков, задействуемых при обработке поступающих запросов. Использование параллельных рабочих потоков остается выгодным и в Kubernetes, поскольку они позволяют более эффективно применять ресурсы. Я никогда бы не посоветовал взять под с десятью параллельными рабочими потоками и развернуть его в виде десяти однопоточных реплик. Внутренний параллелизм контейнера очень эффективен с точки зрения использования памяти, так как параллельные ветки совместно используют значительную часть памяти, занятой двоичным кодом приложения, а потоки могут совместно использовать еще больший объем. Будет весьма полезно и совместное использование ресурсов процессора: веб-приложения проводят много времени в ожидании ответа от внешних зависимостей, что позволяет задействовать неиспользуемые мощности для параллельной обработки большого количества запросов.

Однако не стоит слишком сильно увлекаться использованием параллельных рабочих потоков внутри отдельных подов: чем больше реплик пода вы будете применять, тем надежнее он станет работать. Например, вы используете две реплики пода с 18 рабочими потоками в каждом, что в совокупности позволяет обрабатывать 36 параллельных подключений (рис. 5.7). Если одна из этих реплик даст сбой (или просто не сможет пройти проверку состояния, которая описывалась в главе 4), то вам придется ее перезапустить, и до завершения этой операции будет недоступна

половина ваших вычислительных мощностей. Более эффективный подход в данном случае сводится к тому, чтобы использовать шесть реплик пода с шестью рабочими потоками в каждом. Это позволит обеспечить достаточную степень избыточности, и внутреннего параллелизма.

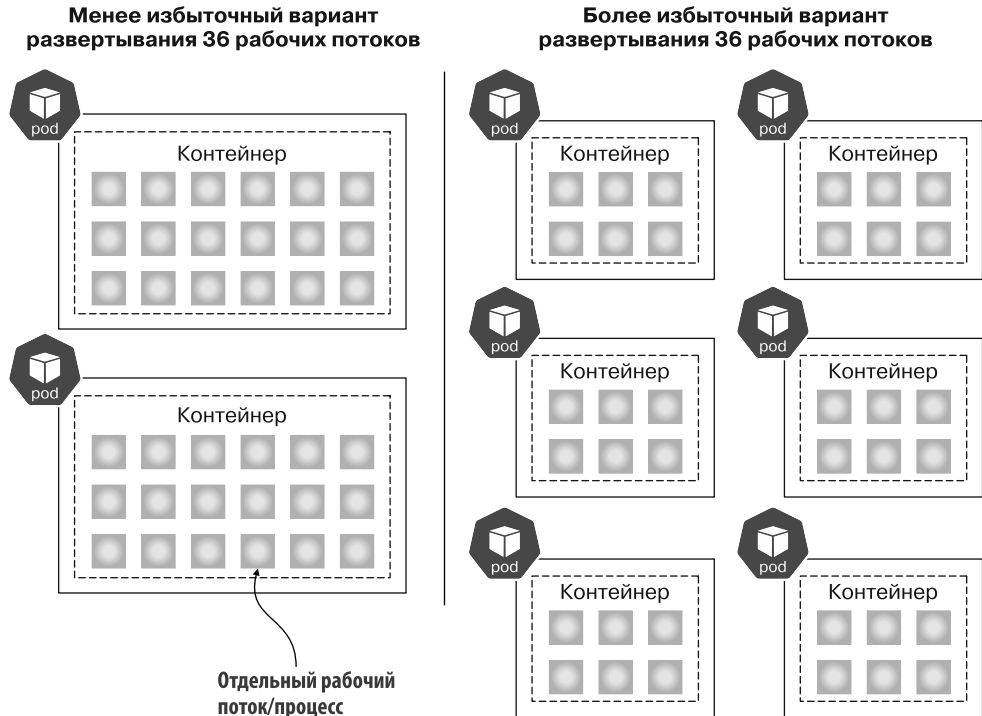


Рис. 5.7. Сравнение двух вариантов развертывания 36 рабочих потоков

Найти приемлемый баланс можно с помощью следующего метода приближенного расчета. Определите, сколько рабочих потоков вам нужно для обслуживания пользователей и какая часть от этого количества может стать недоступной в любой момент времени, не оказывая заметного влияния на пользовательский опыт. Количество потоков, которое может быть недоступным, вы и можете разместить в отдельном поде. Так, в рассмотренном выше примере предполагается, что незаметным для пользователей будет отключение 16 % из 36 рабочих потоков, то есть примерно шести, и поэтому именно шесть рабочих потоков должен содержать каждый отдельный под.

Иными словами, увеличение количества реплик повышает надежность вашего дизайна, но делает его менее эффективным с точки зрения потребления ресурсов. Поэтому вы должны найти приемлемый для вас баланс между надежностью и уровнем потребления ресурсов.

Помимо определения оптимального количества реплик, для повышения доступности важно проследить еще и за тем, чтобы поды были распределены по нескольким узлам. Если все ваши реплики будут запускаться на одном узле, то в вашей системе по-прежнему будет единая точка отказа в виде этого узла. К счастью, большинство облачных платформ Kubernetes (в том числе Google Kubernetes Engine) по умолчанию используют специальные политики, обеспечивающие распределение подов по всем доступным узлам и нескольким зонам (если кластер региональный). Чтобы сделать это предлагаемое по умолчанию поведение доступным для вас, достаточно проследить за тем, чтобы в вашем кластере было несколько узлов, расположенных в разных зонах. Более подробно топологии размещения узлов и распределения подов будут рассмотрены в главе 8.

РЕЗЮМЕ

- Планировщик Kubernetes — «мозг» вашей системы, который берет на себя всю работу по поиску в вашей инфраструктуре подходящего места для подов развертывания.
- Планировщик старается разместить максимально возможное количество подов на том или ином узле, при условии, что для контейнеров пода корректно заданы запросы ресурсов.
- Kubernetes руководствуется запросами и лимитами ресурсов пода при стандартном и избыточном выделении ресурсов, а также их освобождении.
- Избыточное выделение ресурсов в сочетании с возможностью их чрезмерного потребления позволяет сэкономить ресурсы, но приводит к колебанию уровня производительности.
- Задавая запросы и лимиты для своей рабочей нагрузки, вы тем самым определяете предоставляемый ей уровень качества обслуживания.
- При проектировании рабочих нагрузок вы должны обеспечить приемлемый компромисс между степенью доступности и уровнем потребления, найдя баланс между количеством реплик и рабочих потоков/процессов, запускаемых внутри пода.
- Чтобы в системе не было единой точки отказа, большинство облачных платформ по умолчанию выполняет распределение подов так, чтобы не допустить размещения реплик на одном узле. В целях повышения степени доступности проследите за тем, чтобы в вашем кластере было несколько узлов.

Часть II

*Переходим
в продуктивную среду*

Вы освоили такие основы работы с Kubernetes, как создание и развертывание контейнеров, установка лимитов ресурсов и настройка проверок жизнеспособности и готовности, позволяющих обеспечить автоматизацию. Теперь вы можете перейти на следующий уровень. Вторая часть будет посвящена созданию в Kubernetes продуктивных систем. Мы разберем, как масштабировать приложения вручную и автоматически, создавать масштабируемые приложения, настроить взаимодействие сервисов, поддерживаемых разными командами разработчиков, и сохранять конфигурации Kubernetes вместе с кодом, обеспечивая при этом надлежащий уровень безопасности и возможности обновления.

Кроме того, вы узнаете, какие еще виды рабочей нагрузки можно использовать в Kubernetes помимо рассмотренного в части I развертывания без сохранения состояния, в том числе рабочие нагрузки с сохранением состояния (которым требуется подключенный диск), фоновые очереди, пакетные задания и поды с демонами, которые должны исполняться на каждом узле. Вы научитесь информировать Kubernetes о таких требованиях к планированию, как необходимость в распределении подов по разным узлам или, наоборот, в их размещении на одном узле, а также как указать специфические требования к аппаратному обеспечению, такие как архитектура процессора ARM, графические ускорители и точечные вычисления (имеются в виду виртуальные машины, которые предлагаются на облачных платформах с существенной скидкой, но которые в любой момент могут быть выключены для использования вычислительных ресурсов для более приоритетных задач).

6

Масштабирование

В этой главе

- ✓ Масштабирование подов и узлов вручную.
- ✓ Динамическое масштабирование количества реплик пода с помощью метрики потребления процессорного времени и других показателей.
- ✓ Добавление и удаление узлов в зависимости от необходимого подам количества ресурсов при использовании управляемых платформ.
- ✓ Использование временных низкоприоритетных подов для обеспечения возможности избыточного потребления.
- ✓ Как сделать свое приложение масштабируемым.

Теперь, когда вы уже развернули приложение и добавили в него проверки работоспособности, чтобы оно могло работать без вашего вмешательства, пришла пора поговорить о том, как вы будете его масштабировать в дальнейшем. Центральное место в данной главе отводится масштабированию вверх (в сторону увеличения), поскольку для любого разработчика важно, чтобы архитектура его системы позволяла выполнить увеличение масштаба, если приложение станет чрезвычайно популярным и ему потребуется обслуживать резко возросшее количество пользователей. Кроме того, мы рассмотрим и масштабирование вниз

(в сторону уменьшения), которое позволяет экономить деньги в периоды относительного затишья.

Конечная цель состоит в том, чтобы обеспечить эксплуатацию своего развертывания с помощью автоматического масштабирования. Благодаря этому вы сможете спокойно заниматься своими делами, зная, что приложение сможет динамично отреагировать на любые всплески трафика. Чтобы добиться этой цели, нужно сделать приложение масштабируемым, составить четкое представление о том, как будут взаимодействовать поды и узлы в кластере Kubernetes при масштабировании, и правильно выбрать показатели, на основе которых автоскейлер (autoscaler) будет выполнять автомасштабирование.

6.1. МАСШТАБИРОВАНИЕ ПОДОВ И УЗЛОВ

Выполнив контейнеризацию своего приложения и развернув его в Kubernetes, вы уже сделали большой шаг в направлении реализации варианта развертывания, позволяющего выполнять масштабирование по мере роста популярности приложения. Теперь разберемся в том, как именно выполняется масштабирование вверх при возрастании трафика (и как выполняется масштабирование вниз, позволяющее сэкономить деньги в периоды затишья).

В Kubernetes вы фактически масштабируете два вида ресурсов: свое приложение в виде подов и предоставленные ему вычислительные ресурсы в виде узлов. Немного усложняет ситуацию то, что эти ресурсы требуется масштабировать по отдельности, несмотря на то что предъявляемые к ним требования в какой-то мере взаимосвязаны (как, например, в случае, когда приложению нужно выделить больше мощностей). Мы не можем масштабировать только поды, поскольку это приведет к недостатку вычислительных ресурсов, или только узлы, ведь из-за этого мы просто увеличим количество неиспользуемых мощностей. Что мы должны сделать, так это согласованно масштабировать и поды, и узлы, правильно определив необходимую степень увеличения (или уменьшения). К счастью, мы можем упростить себе жизнь, используя инструменты, упрощающие выполнение данной задачи (или полностью автоматизированные платформы, берущие ее выполнение на себя), о которых мы поговорим далее в текущей главе.

Чтобы справиться с возросшим трафиком, прежде всего, нужно увеличить количество реплик пода. Если масштабирование выполняется вручную, то нужно изменить конфигурацию развертывания, указав необходимое количество реплик (листинг 6.1).

Листинг 6.1. Chapter06/6.1_Replicas/deploy.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 6
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
      - name: timeserver-container
        image: docker.io/wdenniss/timeserver:3
        resources:
          requests:
            cpu: 200m
            memory: 250Mi

```

Укажите в поле replicas необходимое
вам количество реплик пода

После этого следует применить новую конфигурацию с помощью команды `kubectl apply -f deploy.yaml`, как мы уже делали ранее. Кроме того, утилита `kubectl` позволяет использовать для той же цели удобную императивную команду `scale`:

```
kubectl scale deployment timeserver --replicas=6
```

Но если вы попытаетесь добавить слишком большое количество реплик, то в вашем кластере очень скоро закончится место для размещения новых подов. Здесь свою роль должно сыграть масштабирование узлов. Выявить недостаток пространства можно, проверив состояние подов с помощью команды `kubectl get pods`. Если вы видите, что целая группа подов находится в состоянии ожидания (**Pending**), то им, скорее всего, не хватает места.

Поды могут переходить в это состояние по разным причинам, однако недостаток ресурсов — самая распространенная из них (в тех случаях, когда под остается в этом состоянии как минимум в течение минуты). При недостатке ресурсов поду не может быть предоставлено необходимое количество ресурсов, поэтому он остается в состоянии ожидания (**Pending**) до тех пор, пока его потребности в ресурсах не будут удовлетворены. При наличии у пода зависимостей он может иметь и другие невыполнимые требования (например, требование о том, чтобы он был развернут на узле вместе с другим подом, который еще не был создан). Чтобы исключить какие-либо сомнения относительно того, чем было вызвано состояние ожидания, отобразите

подробную информацию по интересующему вас поду с помощью команды `kubectl describe pod $POD_NAME` и проверьте содержимое раздела с описанием событий. Если вы увидите в нем что-то вроде ошибки `FailedScheduling` (Ошибка планирования) с сообщением `Insufficient CPU` (Недостаток ресурсов процессора), то это явный признак того, что вам нужно увеличить количество узлов.

БЕЗУЗЛОВЫЕ ПЛАТФОРМЫ KUBERNETES

Я хотел бы немного отвлечься и поговорить о безузловых платформах Kubernetes. На мой взгляд, в идеале облачная платформа Kubernetes совсем не должна беспокоиться о количестве узлов. Ведь вы все равно используете облако, так почему бы не задействовать платформу, которая будет предоставлять необходимое количество узлов, исходя из запросов ресурсов подов, позволив вам сосредоточиться на создании полезных приложений и сервисов?

Именно таким продуктом является режим «Автопилот» сервиса GKE, в разработке которого мне довелось принять участие в качестве менеджера по продуктам облачной платформы Google Cloud. В данном режиме сервис GKE освобождает разработчиков от переживаний о количестве узлов. Вам достаточно лишь создать стандартные рабочие нагрузки Kubernetes, такие как объекты `Deployment`, `StatefulSet` и `Job`, указав необходимое количество реплик и ресурсов процессора и памяти. После этого режим «Автопилот» берет на себя все заботы о предоставлении вашим подам необходимых вычислительных ресурсов и управлении этими вычислительными мощностями. Вы получаете два важных преимущества: во-первых, уменьшается трудоемкость процесса разработки, поскольку вам не нужно определять запросы ресурсов дважды (и в поде, и в узле), а во-вторых, упрощается процесс эксплуатации, так как существенно снижается объем работы по управлению узлами.

Режим «Автопилот» сервиса GKE уникален тем, что, в отличие от других безузловых платформ, при его использовании концепция узла Kubernetes по-прежнему остается в какой-то мере актуальной. Значительная часть связанной с узлами логики (в том числе такие параметры, как топологии распределения, предпочтение и антипредпочтение, о которых пойдет речь в главе 8) не теряет значимости и по-прежнему может применяться. Режим «Автопилот» сервиса GKE безузловой в том смысле, что позволяет не беспокоиться о предоставлении узлов и управлении ими, но в то же время не абстрагируется полностью

от узлов и не скрывает их. Ведь, в конце концов, ваш код все равно работает на определенной машине, и такой подход будет вполне уместным, когда речь идет о доменах отказов или совместном размещении подов в целях сокращения времени реакции.

На мой взгляд, данное решение совмещает в себе преимущества узлового и безузлового дизайна, предоставляя вам необходимую степень контроля над узлами и одновременно избавляя от забот об эксплуатации и администрировании этих узлов. Вам больше не нужно беспокоиться о количестве, размере и конфигурации используемых вами узлов, поддерживать их в работоспособном состоянии и следить за тем, чтобы они не простаивали без дела.

Если вы используете режим «Автопилот» сервиса GKE или аналогичную платформу, то можете проигнорировать все, что говорится в этой главе о масштабировании узлов, и сосредоточиться исключительно на масштабировании подов. При использовании данного режима вам достаточно обеспечить ручное или автоматическое (с помощью горизонтального автоскейлера) масштабирование подов, поскольку система будет автоматически предоставлять вам необходимые ресурсы узлов, обходясь без какой-либо дополнительной настройки.

Чтобы масштабировать узлы, вам нужно будет свериться с документацией используемой вами облачной платформы, поскольку сама система Kubernetes не позволяет выполнять оркестрацию узлов. Так, при использовании сервиса Google Kubernetes Engine в режиме «Автопилот» узлы предоставляются автоматически, и поэтому вы можете сразу же перейти к разделу 6.2. В случае кластеров сервиса GKE с пулами узлов нужно использовать команду следующего вида:

```
gcloud container clusters resize $CLUSTER_NAME \
  --node-pool $NODE_POOL_NAME \
  --num-nodes $NODE_COUNT
```

Масштабирование вниз обычно осуществляется с помощью той же команды, что и масштабирование вверх, но в случае узлов это зависит от используемой облачной платформы. Кластер сначала блокирует их, исключая возможность распределения на них новых подов, а затем опустошает их, вытесняя все поды, давая им некоторое время на корректное завершение работы. При этом поды, которые входят в состав развертывания или другой высокоуровневой конструкции для представления рабочей нагрузки, перераспределяются на другие узлы.

Теперь вы знаете, как вручную масштабировать поды и узлы. Далее я расскажу о том, как автоматизировать обе эти операции, используя горизонтальное автомасштабирование для подов и автомасштабирование кластера для узлов (если облачный провайдер дает такую возможность).

РУЧНАЯ БЛОКИРОВКА И ОПУСТОШЕНИЕ УЗЛОВ

Если вас интересует, что происходит при масштабировании узлов вниз, то попробуйте вручную заблокировать, опустошить и удалить узлы с помощью следующих команд:

```
$ kubectl get nodes
NAME                                     STATUS  ROLES  AGE  VERSION
gke-cluster-1-default-pool-f1e6b3ef-3o5d  Ready  <none>  7d   v1.27.3-gke.100
gke-cluster-1-default-pool-f1e6b3ef-fi16  Ready  <none>  7d   v1.27.3-gke.100
gke-cluster-1-default-pool-f1e6b3ef-yc82  Ready  <none>  7d   v1.27.3-gke.100

$ NODE_NAME=gke-cluster-1-default-pool-f1e6b3ef-3o5d

$ kubectl cordon $NODE_NAME
node/gke-cluster-1-default-pool-f1e6b3ef-3o5d cordoned

$ kubectl drain $NODE_NAME --ignore-daemonsets --delete-emptydir-data
node/gke-cluster-1-default-pool-f1e6b3ef-3o5d already cordoned
evicting pod default/timeserver-784d5485d9-mrspm
evicting pod kube-system/metrics-server-v0.5.2-66bbcd9bffc-78gl7
pod/timeserver-784d5485d9-mrspm evicted
pod/metrics-server-v0.5.2-66bbcd9bffc-78gl7 evicted
node/gke-cluster-1-default-pool-f1e6b3ef-3o5d drained
```

Обратите внимание: удаление узла с помощью утилиты `kubectl` не всегда ведет к удалению привязанной к нему виртуальной машины, и в таком случае вам по-прежнему придется платить за ее использование. Поэтому после удаления узла с помощью команды `kubectl delete node $NODE_NAME` всегда дополнительно проверяйте, была ли при этом удалена виртуальная машина. Если вы используете сервис GKE в режиме «Автопилот», то для того, чтобы исключить узел из числа используемых, достаточно его заблокировать и опустошить; после этого система автоматически выполнит весь дальнейший процесс удаления. Если же вы используете кластеры GKE, работающие в узловом режиме, то удаление виртуальной машины необходимо выполнять самостоятельно, с помощью такой команды:

```
$ gcloud compute instances delete $NODE_NAME --zone us-west1-c
```

Обычно при масштабировании вниз кластер удаляет лишние узлы автоматически, но эти команды могут оказаться полезными в том случае, если какой-либо узел начнет вести себя странно и вам нужно будет удалить его самостоятельно.

6.2. ГОРИЗОНТАЛЬНОЕ АВТОМАСШТАБИРОВАНИЕ ПОДОВ

Процесс масштабирования количества реплик пода с приложением называется в Kubernetes *горизонтальным автомасштабированием подов*. Это горизонтальное автомасштабирование, поскольку в данном случае вы пытаетесь справиться с возросшим трафиком, увеличивая количество реплик, а не ресурсов, доступных для каждой реплики, как в случае вертикального автомасштабирования. Масштабирование вверх обычно выполняется с помощью горизонтального масштабирования.

В состав Kubernetes входит так называемый горизонтальный автоскейлер подов (Horizontal Pod Autoscaler, HPA), который позволяет указать для пода целевое значение такого *показателя*, как потребление ресурсов процессора, и некоторые пределы масштабирования (минимальное и максимальное количество реплик). После этого HPA начинает следить за тем, чтобы выбранный вами показатель стремился к указанному значению, создавая или удаляя реплики пода. Так, если вы укажете, что потребление ресурсов процессора должно составлять 20 %, то HPA будет увеличивать количество реплик в том случае, когда среднее потребление ресурсов процессора (в рамках всех подов) будет подниматься выше 20 % (от количества ресурсов, запрашиваемого подами в своих ресурсных требованиях), и уменьшать количество реплик, когда этот показатель будет опускаться ниже 20 %. Эти действия будут выполняться в рамках заданных вами пределов и с определенной задержкой во избежание излишней поспешности. В листинге 6.2 показано, как создать горизонтальный автоскейлер для своего развертывания.

Листинг 6.2. Chapter06/6.2_HPA/hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: timeserver
spec:
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - resource:
      name: cpu
      target:
        averageUtilization: 20
        type: Utilization
      type: Resource
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: timeserver
```

Минимальное количество реплик

Максимальное количество реплик

Целевое значение показателя потребления ресурсов процессора. HPA будет создавать дополнительные реплики в том случае, если данный показатель будет подниматься выше указанного значения

Развертывание, которое будет подвергаться масштабированию

Создать автоскейлер можно и императивно. Я всегда рекомендую использовать конфигурацию, поскольку это упрощает внесение последующих изменений, но для полноты картины стоит сказать, что того же результата можно добиться и с помощью следующей императивной команды:

```
kubectl autoscale deployment timeserver --cpu-percent=20 --min=1 --max=10
```

Чтобы опробовать автоскейлер на практике, мы должны как следует нагрузить свой процессор. Дополнительно добавим в наше веб-приложение сервера времени HTTP-путь для сильно нагружающего процессор процесса вычисления числа пи. В листинге 6.3 показано, как определить метод для вычисления числа пи.

Листинг 6.3. Chapter06/timeserver4/pi.py

```
from decimal import *

# Рассчитайте число пи, используя ряд бесконечности Грегори – Лейбница
def leibniz_pi(iterations):

    precision = 20
    getcontext().prec = 20
    piDiv4 = Decimal(1)
    odd = Decimal(3)

    for i in range(0, iterations):
        piDiv4 = piDiv4 - 1/odd
        odd = odd + 2
        piDiv4 = piDiv4 + 1/odd
        odd = odd + 2

    return piDiv4 * 4
```

В листинге 6.4 показано, как добавить URL для вызова этого метода в программе `server.py`.

Листинг 6.4. Chapter06/timeserver4/server.py

```
from pi import *

# ...

case '/pi':
    pi = leibniz_pi(1000000)
    self.respond_with(200, str(pi))
```

Дополнительный
HTTP-путь

В листинге 6.5 представлена измененная версия развертывания, которая ссылается на новую версию контейнера, в которую дополнительно добавлен новый путь. Чтобы горизонтальный автоскейлер мог корректно работать, важно правильно задать запросы ресурсов; мы добавили их в главе 5 и оставляем здесь в том же виде.

Листинг 6.5. Chapter06/6.2_HPA/deploy.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:4
          resources:
            requests:
              cpu: 250m
              memory: 250Mi

```

Количество реплик изначально задается равным единице

Новая версия приложения

Чтобы горизонтальный автоскейлер мог корректно работать, важно правильно задать запросы ресурсов

Теперь мы можем создать развертывание, сервис и горизонтальный автоскейлер:

```

$ cd Chapter06/6.2_HPA
$ kubectl create -f deploy.yaml -f service.yaml -f hpa.yaml
deployment.apps/timeserver created
service/timeserver created
horizontalpodautoscaler.autoscaling/timeserver created
$ kubectl get svc -w

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.22.128.1	<none>	443/TCP	6m35s
timeserver	LoadBalancer	10.22.131.179	<pending>	80:32650/TCP	18s
timeserver	LoadBalancer	10.22.131.179	203.0.113.16	80:32650/TCP	26s

Пока вы дожидаетесь предоставления внешнего IP-адреса, можно начать отслеживать значения показателя потребления ресурсов процессора, выдаваемых для ваших подов, используя следующую команду (рекомендуется это делать в отдельном окне):

```
kubectl top pods
```

Получив внешний IP-адрес, сгенерируйте на новой конечной точке определенную нагрузку. Для этой цели, например, хорошо подходит утилита Apache Bench, которая может быть установлена в большинстве распространенных операционных систем. Ниже показано, как с ее помощью можно направить к нашей конечной точке 10 000 запросов, отправляя за раз по 50 запросов — по идее, этого должно хватить:

```

EXTERNAL_IP=203.0.113.16
ab -n 10000 -c 50 http://$EXTERNAL_IP/pi

```

Проследить за тем, как будет масштабироваться развертывание, можно с помощью следующей команды:

```
kubect1 get pods -w
```

Команда `watch` операционной системы Linux удобна тем, что дает возможность отслеживать все интересующие вас ресурсы с помощью одной команды (что не позволяет делать утилита `kubect1` сама по себе):

```
watch -d kubect1 get deploy,hpa,pods
```

Если вы все сделаете правильно, то в выводе команды `kubect1 top pods` будет наблюдаться рост показателя потребления ресурсов процессора, в соответствии с которым будут создаваться дополнительные реплики пода. После того как вы прекратите направлять нагрузку к конечной точке (прервав выполнение команды `ab` или дождавшись, пока она выполнится до конца), система начнет постепенно удалять дополнительно созданные реплики.

Вы можете заметить, что распределение дополнительных реплик при масштабировании вверх в ответ на увеличение количества запросов выполняется быстрее, чем их удаление при масштабировании вниз в ответ на сокращение количества запросов. Это объясняется тем, что система проявляет некоторую осторожность при удалении реплик, чтобы не действовать слишком поспешно при всплесках нагрузки. В моем случае весь процесс выглядел так:

```
$ kubect1 get deploy,hpa,pods
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/timeserver	2/6	6	2	7m7s

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
hpa/timeserver	Deployment/timeserver	100%/30%	1	6	6

NAME	READY	STATUS	RESTARTS	AGE
pod/timeserver-b8789946f-2b969	1/1	Running	0	7m7s
pod/timeserver-b8789946f-fzbnk	0/1	Pending	0	96s
pod/timeserver-b8789946f-httwm	1/1	Running	0	96s
pod/timeserver-b8789946f-vvnhj	0/1	Pending	0	96s
pod/timeserver-b8789946f-xw9zf	0/1	ContainerCreating	0	36s
pod/timeserver-b8789946f-zbzw9	0/1	ContainerCreating	0	36s

В данном случае НРА успешно справился со своей задачей, используя показатель потребления ресурсов процессора, но такой подход можно применять далеко не во всех случаях, поскольку ваша рабочая нагрузка может зависеть от чего-то другого. В отличие от данного примера, где мы использовали сильно нагружающий процессор запрос, многие HTTP-сервисы проводят много времени, ожидая ответа от внешних сервисов, таких как база данных. При масштабировании таких рабочих нагрузок вместо показателя потребления ресурсов процессора следует использовать такой показатель, как количество запросов в секунду (requests per second, RPS). Платформа Kubernetes позволяет использовать два встроенных

показателя: показатель потребления ресурсов процессора (который мы использовали в представленном выше примере) и показатель потребления памяти. Платформа не позволяет напрямую использовать такие показатели, как RPS, но может быть настроена на использование пользовательских и внешних показателей, предоставляемых вашим сервисом мониторинга. Именно об этом пойдет речь в следующем подразделе.

НЕСКОЛЬКО СЛОВ О ВЕРТИКАЛЬНОМ АВТОМАСШТАБИРОВАНИИ ПОДОВ

Под вертикальным автомасштабированием подов (Vertical Pod Autoscaling, VPA) понимается процесс, при котором поды масштабируются вертикально путем изменения количества предоставляемых им ресурсов процессора и памяти. В Kubernetes это реализуется путем отслеживания потребности подов в ресурсах и динамического изменения их запросов ресурсов с течением времени. В Kubernetes нет встроенной реализации вертикального автоскейлера, но вы можете использовать реализацию, созданную в рамках проекта с открытым исходным кодом (<https://github.com/kubernetes/autoscaler>). Кроме того, собственную реализацию вертикального автоскейлера предлагают и многие облачные платформы, в том числе сервис GKE.

Вертикальный автоскейлер подов автоматически отслеживает потребности подов в ресурсах, поэтому благодаря его применению вы сможете тратить меньше усилий и более эффективно использовать ресурсы. Кроме того, он будет полезен и в том случае, когда нужно обеспечить динамическое изменение запросов ресурсов подов с течением времени (при значительном колебании запросов подов).

Использование вертикального автоскейлера несколько увеличивает уровень сложности системы, к тому же он не всегда хорошо взаимодействует с горизонтальным автоскейлером. Поэтому я рекомендую не торопиться с его применением, начав с определения для подов корректных запросов ресурсов и использования горизонтального автоскейлера.

6.2.1. Внешние показатели

Многие разработчики предпочитают выполнять масштабирование на основе количества запросов в секунду (requests per second, RPS). Для использования этого показателя необходимо подсчитать, какое количество запросов в секунду может обрабатывать один экземпляр приложения (то есть оценить вычислительную мощность одной реплики). Зная это значение, вы сможете легко определить необходимое количество реплик в любой момент времени, разделив текущее количество запросов (значение RPS) на мощность одной реплики:

$$\text{количество_реплик} = \text{RPS} \div \text{мощность_реплики}$$

Данный подход имеет преимущество: убедившись в ходе тестирования, что приложение может справиться с определенным значением показателя RPS, вы можете не сомневаться в том, что оно будет успешно масштабировано при возрастании нагрузки, поскольку автоскейлер выделит ему достаточное количество мощностей.

На самом деле, даже если вы не используете *автоматическое* масштабирование, данный показатель можно с успехом применять и для ручного планирования мощностей. Просто оцените производительность своих реплик и прогнозируемый объем трафика и увеличьте количество реплик, исходя из этих значений. Однако Kubernetes позволяет автоматизировать данный процесс, настроив горизонтальный автоскейлер на использование показателя RPS.

В данном случае мы будем использовать свойство «внешняя метрика» (external metric) описания HPA. Недостатком этого подхода является то, что источник данных показателя находится за пределами кластера. Поэтому, если вы решили использовать какое-то другое средство мониторинга, а не то, которое использовал я, узнайте, как выглядит показатель RPS в вашем случае, сверившись с документацией этого инструмента. К счастью, RPS — достаточно распространенный показатель, и любой достойный инструмент мониторинга сможет ее предоставить.

Направлять трафик в кластер можно с помощью балансировщика нагрузки «уровня 4», работающего на уровне протокола TCP/IP, или объекта Ingress «уровня 7», работающего на уровне протокола HTTP. *Запрос* — это концепция протокола HTTP, поэтому для получения данного показателя следует использовать объект Ingress. Мы подробно рассмотрим этот объект в следующей главе, а пока будет достаточно сказать, что он отслеживает HTTP-трафик и поэтому может предоставить информацию о текущем количестве запросов.

В данном примере (листинги 6.6, 6.7) мы обратимся к тому же развертыванию, которое использовали ранее, но сделаем его доступным объекту Ingress с помощью сервиса типа **NodePort** (вместо сервиса типа **LoadBalancer**, который использовали в предыдущих главах).

Листинг 6.6. Chapter06/6.2.1_ExternalMetricGCP/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: timeserver-internal
spec:
  selector:
    pod: timeserver-pod
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
  type: NodePort
```

Имя данного
внутреннего сервиса

Для обеспечения доступа извне
используем тип NodePort

Листинг 6.7. Chapter06/6.2.1_ExternalMetricGCP/ingress.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: timeserver-ingress
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: timeserver-internal
                port:
                  number: 80

```

Ссылка на внутренний сервис из листинга 6.6

В случае использования платформы Google Cloud после этого можно заставить горизонтальный автоскейлер получать показатель RPS от объекта Ingress, определив его, как показано в листинге 6.8 (в этом коде вы должны подставить собственное имя правила перенаправления).

Листинг 6.8. Chapter06/6.2.1_ExternalMetricGCP/hpa.yaml

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: timeserver-autoscaler
spec:
  minReplicas: 1
  maxReplicas: 6
  metrics:
    - type: External
      external:
        metric:
          name: loadbalancing.googleapis.com|https|request_count
          selector:
            matchLabels:
              resource.labels.forwarding_rule_name: "k8s2-fr-21mgs2f1"
        target:
          type: AverageValue
          averageValue: 5
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: timeserver

```

Внешний показатель

С помощью свойства `forwarding_rule_name` вы сообщаете серверу показателей, какой объект Ingress он должен использовать. Если мы опустим поле `selector`, то сервер показателей выберет все объекты Ingress, а это нам совсем не нужно.

Немного усложняет ситуацию то, что в качестве имени правила перенаправления здесь используется зависящее от платформы имя ресурса, а не имя объекта Kubernetes (в данном случае это имя автоматически задается сервисом GKE). Чтобы узнать это зависящее от платформы имя ресурса, подождите несколько минут, пока система закончит настройку объекта Ingress, а затем отобразите информацию о нем с помощью команды `describe`:

```
$ kubectl describe ingress timeserver-ingress
Name:          timeserver-ingress
Namespace:     default
Address:       203.0.113.16
Default backend: default-http-backend:80 (10.22.0.202:8080)
Rules:
  Host          Path  Backends
  ----          -
  *
                /  timeserver-internal:80
                (10.22.0.130:80,10.22.0.131:80,10.22.0.196:80 + 1 more...)
Annotations:   ingress.kubernetes.io/backends:
                {"k8s -be-32730":"HEALTHY","k8s1-a5225067":"HEALTHY"}
                ingress.kubernetes.io/forwarding-rule: k8s2-fr-21mgs2f1
                ingress.kubernetes.io/target-proxy: k8s2-tp-21mgs2f1
                ingress.kubernetes.io/url-map: k8s2-um-21mgs2f1Events:
Type          Reason  Age          From          Message
----          -
Normal      Sync    6m28s (x31 over 5h6m)  loadbalancer-controller  Scheduled
for sync
```

Имя правила
перенаправления

←

Еще один способ получения этой информации, который может быть полезен при использовании автоматизированных инструментов, сводится к тому, чтобы, зная, как организованы данные внутри объекта Ingress, запросить это имя в команде `kubectl get`, используя формат `JsonPath`:

```
$ kubectl get ingress -o=jsonpath="{.items[0].metadata.annotations['ingress\
➡ kubernetes.io/forwarding-rule ']}"
k8s2-fr-21mgs2f1
```

СОВЕТ

При составлении представленного здесь выражения в формате `JsonPath` я сначала указал, что мне нужна JSON-версия объекта Ingress: `-o=json`. Дальнейшая часть этого выражения была получена методом проб и ошибок после изучения документации по `JsonPath` и советов на сайте Stack Overflow.

После того как вы подготовите все эти объекты, вам останется сделать лишь один последний шаг, а именно активировать набор инструментов Cloud Monitoring (Облачный мониторинг) для рабочих нагрузок своего кластера и установить вспомогательный инструмент, который сделает показатели доступными для горизонтального

автоскейлера. Используя предлагаемые по ссылке инструкции (<https://cloud.google.com/kubernetes-engine/docs/tutorials/autoscaling-metrics>), установите адаптер показателей Custom Metrics — Stackdriver Adapter.

Теперь, когда мы настроили все, что нам нужно: развертывание, сервис типа NodePort, объект Ingress, горизонтальный автоскейлер и адаптер показателей, — мы уже можем посмотреть, как это работает на практике. Направьте трафик к своему объекту Ingress (подставив IP-адрес своего объекта Ingress, полученный с помощью команды `kubectl get ingress`):

```
ab -n 100000 -c 100 http://203.0.113.16/
```

В отдельном окне наблюдайте за тем, как будет осуществляться масштабирование:

```
$ kubectl get hpa,ingress,pods
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
hpa/timeserver	Deployment/timeserver	94%/30%	1	6	4

NAME	CLASS	HOSTS	ADDRESS	PORTS
ingress/timeserver-ingress	<none>	*	203.0.113.16	80

NAME	READY	STATUS	RESTARTS	AGE
pod/timeserver-b8789946f-8dpmg	1/1	Running	0	5h51m
pod/timeserver-b8789946f-gsrt5	0/1	ContainerCreating	0	110s
pod/timeserver-b8789946f-sjvqb	1/1	Running	0	110s
pod/timeserver-b8789946f-vmhsw	0/1	ContainerCreating	0	110s

Как видите, теперь мы можем достаточно легко убедиться в том, что система работает ожидаемым образом. Сгенерировав с помощью утилиты Apache Bench множество параллельных запросов, вы можете определить, насколько быстро они обслуживаются (то есть подсчитать значение показателя RPS), и посмотреть, соответствует ли количество реплик этому значению. Схема работы выглядит немного сложнее, чем при использовании показателя потребления ресурсов процессора, где для проверки достаточно было как следует нагрузить под, что мы и сделали в предыдущем примере. Однако именно потому, что данный показатель позволяет выполнять масштабирование на основе *количества пользовательских запросов*, он и пользуется такой популярностью.

Отслеживание состояния и отладка

Отслеживать состояние горизонтального автоскейлера можно с помощью команды `kubectl describe hpa`. Особое внимание при этом следует уделять параметру `ScalingActive`. Если он равен значению `False`, это может говорить о том, что ваш показатель не активен. Такая ситуация может быть вызвана несколькими причинами: возможно, 1) у вас не установлен (или не аутентифицирован) адаптер показателей; 2) вы неправильно указали имя показателя или селектор; или 3) у вас просто еще

нет тестовых данных, позволяющих отслеживать заданный показатель. Следует иметь в виду, что даже при правильном определении конфигурации этот параметр будет равен значению `False`, если у вас не будет соответствующих тестовых данных (например, запросов). Поэтому, прежде чем приступать к более тщательному анализу, не забудьте направить к конечной точке некоторое количество запросов и выждать одну-две минуты, чтобы эти данные успели пройти через систему.

Среднее или абсолютное значение

В представленном выше примере мы использовали поле `targetAverageValue`, которое задает в качестве цели среднее значение показателя *для каждого пода*. Альтернативный подход сводится к тому, чтобы задействовать поле `targetValue`, которое задает в качестве цели абсолютное значение показателя. Значение показателя RPS рассчитывается для каждого пода, поэтому в данном случае необходимо использовать поле `targetAverageValue`.

Другие показатели

При работе с фоновыми задачами (которым будет посвящена глава 10) широко используется и такой внешний показатель, как длина очереди публикации/подписки (Pub/Sub в случае Google Cloud). Pub/Sub — это брокер сообщений, позволяющий создать очередь из подлежащих выполнению заданий, и вы можете настроить рабочую нагрузку Kubernetes на обработку такой очереди. При такой настройке вы скорее всего захотите реагировать на размер очереди добавлением или удалением реплик пода (обработчиков заданий из очереди). В документации по сервису GKE можно найти полностью работающий пример использования данного подхода (<https://cloud.google.com/kubernetes-engine/docs/tutorials/autoscaling-metrics#pubsub>), однако от рассмотренного выше примера он отличается фактически лишь тем, что HPA подов настроен на использование другого показателя:

```
metric:
  name: pubsub.googleapis.com|subscription|num_undelivered_messages
  selector:
    matchLabels:
      resource.labels.subscription_id: echo-read
```

Эта конфигурация содержит имя показателя и соответствующий идентификатор ресурса — в данном случае идентификатор подписки инструмента Pub/Sub платформы Google Cloud.

Другие инструменты мониторинга

Использование внешних показателей можно настроить *практически для любого* инструмента мониторинга, который применим в Kubernetes. В приведенном выше примере мы задействовали набор инструментов Cloud Monitoring облачной платформы Google Cloud, но использование системы Prometheus или какой-то другой облачной системы мониторинга ничем принципиально не отличается.

Для использования любой системы вам нужно будет выяснить следующее: 1) как установить адаптер показателей для вашей системы мониторинга; 2) какое имя показателя следует использовать для этой системы; 3) как выбрать ресурс показателя.

6.3. АВТОМАСШТАБИРОВАНИЕ УЗЛОВ И ПЛАНИРОВАНИЕ МОЩНОСТЕЙ

Использование горизонтального автомасштабирования подов — прекрасный способ обеспечить автоматическое масштабирование подов по мере необходимости. Но если при этом вы будете вручную масштабировать узлы в целях наращивания и сокращения мощностей, то управление процессом будет по-прежнему требовать постоянного участия человека. Используя распространенный инструмент автомасштабирования кластера (Cluster Autoscaler), вы также сможете обеспечить автоматическое масштабирование узлов по мере необходимости, что хорошо сочетается с использованием горизонтального автоскейлера для масштабирования подов.

6.3.1. Автомасштабирование кластера

Автомасштабирование кластера не относится к числу функций, предоставляемых базовым API Kubernetes, являясь необязательным компонентом. К счастью, большинство облачных провайдеров предоставляет эту функциональность (или нечто подобное) в виде встроенного свойства платформы, позволяющего обеспечить автоматическое масштабирование количества узлов, чтобы вы могли сосредоточиться только на своем приложении и необходимом для него количестве реплик. Данная функциональная возможность зависит от платформы, поэтому у каждого провайдера она реализована по-своему (а у некоторых провайдеров ее вообще нет). Чтобы найти нужную вам документацию, попробуйте ввести в браузере поисковый запрос «[Ваш облачный сервис] Cluster Autoscaler».

Что касается сервиса GKE, то в режиме «Автопилот» он автоматически обеспечивает предоставление и автомасштабирование узлов для ваших кластеров, и для этого не нужно задавать какие-либо дополнительные настройки. Если вы работаете с кластерами сервиса GKE с пулами узлов, то можете настроить автомасштабирование при создании нового или изменении существующего пула узлов.

При использовании функции автомасштабирования кластера вы можете сосредоточиться на масштабировании своих рабочих нагрузок, рассчитывая на то, что кластер будет автоматически реагировать на эти изменения (рис. 6.1). Это очень удобно, поскольку в таком случае вы избавлены от проблем с переходом подов в состояние ожидания (Pending) и при масштабировании уже существующих рабочих нагрузок, и при развертывании новых. При этом все же не помешает изучить особенности предлагаемой вашим провайдером реализации, чтобы понимать, что

будет происходить в тех или иных исключительных случаях (например, при добавлении подов, которые не могут поместиться в узлах с используемой в данный момент конфигурацией).

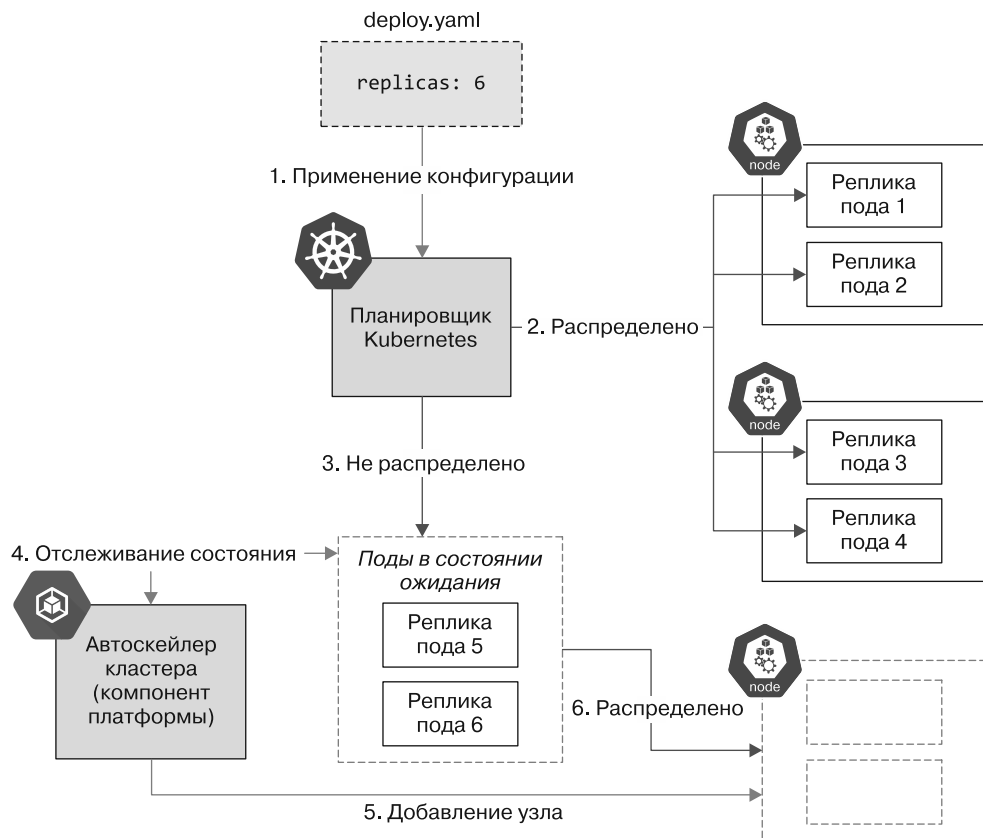


Рис. 6.1. Автоскейлер кластера отслеживает состояние подов, перешедших в состояние ожидания, и при необходимости создает новые узлы

Ознакомиться с документацией, в частности, стоит по той причине, что раньше автоскейлеры кластера могли добавлять только узлы с уже существующей, заранее определенной конфигурацией, и если вы имеете дело с таким случаем, то вам придется заранее определить все возможные типы узлов. Сервис GKE может добавлять узлы любого типа при его использовании в режиме «Автопилот» (это встроенное поведение, которое не требует дополнительных настроек) или в узловом режиме с настроенной функцией автоматического предоставления узлов.

Автомасштабирование кластера и другие инструменты провайдера, делающие возможным автоматическое добавление и удаление узлов, существенно облегчают вашу работу, позволяя практически не думать об узлах и сосредоточиться

исключительно на подах. А благодаря сочетанию с автомасштабированием подов, выполняемым с помощью горизонтального автоскейлера, автоматизированное развертывание становится процессом, который не требует вмешательства человека.

6.3.2. Свободные мощности при автомасштабировании кластера

Одним из недостатков автомасштабирования узлов по сравнению с добавлением узлов вручную является то, что иногда автоскейлер выполняет масштабирование *слишком точно*, не оставляя никаких свободных мощностей. Это большой плюс с точки зрения экономии затрат, в то же время такое поведение автоскейлера ведет к более медленному запуску новых подов, поскольку для них требуется выделять дополнительные мощности.

Запуск новых подов с предварительным добавлением новых узлов занимает больше времени, чем их запуск на уже существующих узлах. При этом приходится сначала выделять и запускать новые узлы, в то время как при распределении подов на уже существующие узлы достаточно просто извлечь контейнер и запустить под. Если контейнер уже находится в кэше, то к запуску пода можно будет приступить сразу же. Процесс запуска вновь распределенных подов с предварительным выделением дополнительных мощностей показан на рис. 6.2.

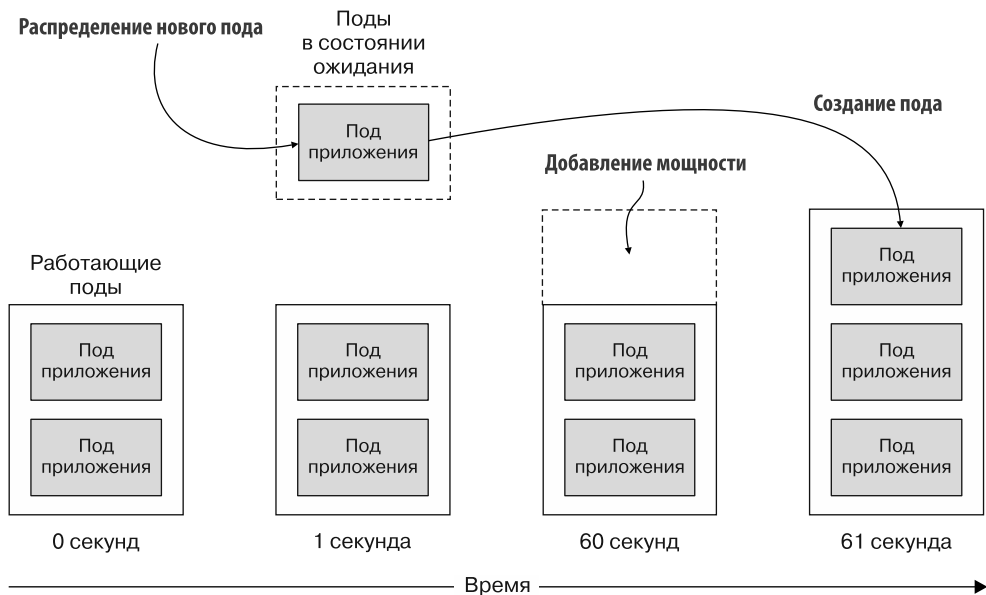


Рис. 6.2. Динамическое добавление мощности при автомасштабировании узлов в целях размещения вновь распределенных подов

Один из способов решения этой проблемы без отказа от автомасштабирования узлов сводится к использованию низкоприоритетных временных подов-заполнителей, единственная задача которых — резервирование мощностей (путем поддержания в состоянии готовности дополнительных узлов). Этим временным подам-заполнителям присваивается низкий приоритет, чтобы при масштабировании ваши рабочие нагрузки могли вытеснить их, подвергнув приоритизации, и воспользоваться освободившимися мощностями (рис. 6.3).

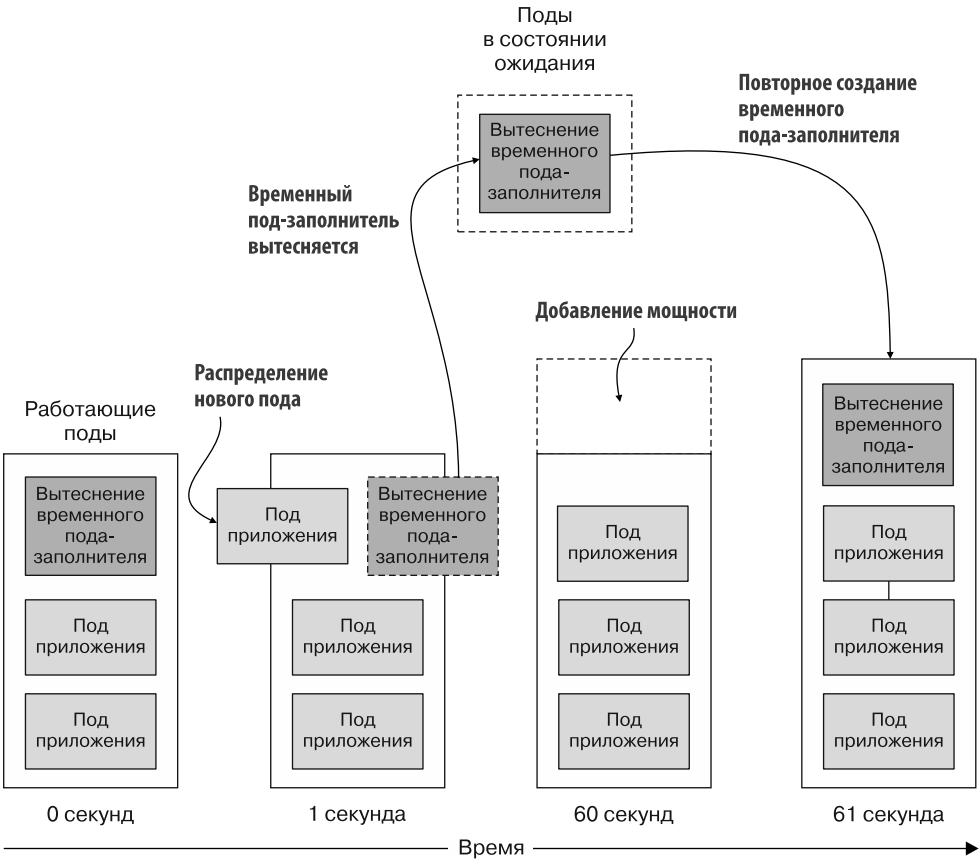


Рис. 6.3. Использование при автомасштабировании временного под-заполнителя, чтобы можно было быстро загружать новые поды, используя свободные мощности

Чтобы создать временное развертывание для своего пода, сначала нужно определить класс приоритета (объект `PriorityClass`). Значение этого класса должно быть меньше нуля (поскольку нам нужно, чтобы при выполнении приоритизации его мог вытеснить любой другой класс приоритета) (листинг 6.9).

Листинг 6.9. Chapter06/6.3.2_PlaceholderPod/placeholder-priority.yaml

```

apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: placeholder-priority
  value: -10
preemptionPolicy: Never
globalDefault: false
description: "Placeholder Pod priority."

```

Для временных подов-заполнителей
указываем низкое значение приоритета

Не подвергает приоритизации
остальные поды

Теперь мы можем создать развертывание с «ничего не делающим» контейнером (листинг 6.10).

Листинг 6.10. Chapter06/6.3.2_PlaceholderPod/placeholder-deploy.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: placeholder
spec:
  replicas: 10
  selector:
    matchLabels:
      pod: placeholder-pod
  template:
    metadata:
      labels:
        pod: placeholder-pod
    spec:
      priorityClassName: placeholder-priority
      terminationGracePeriodSeconds: 0
      containers:
        - name: ubuntu
          image: ubuntu
          command: ["sleep"]
          args: ["infinity"]
          resources:
            requests:
              cpu: 200m
              memory: 250Mi

```

Необходимое количество реплик. Это значение в сочетании с запросами
на ресурсы процессора и памяти определяет размер запаса мощности,
обеспечиваемого временным подом-заполнителем

Используем только что созданный
нами класс приоритета

Временный под-заполнитель будет прекращать свою
работу немедленно, без предоставления ему времени
для корректного завершения работы

Здесь мы даем временному поду-заполнителю
указание ничего не делать

Количество ресурсов, резервируемое для временного пода.
Эти значения должны соответствовать запросам ресурсов самого
большого пода и тех, которые будут использоваться вместо него

Если вы определяете такое развертывание самостоятельно, то нужно будет подумать о том, сколько реплик вам требуется и каким должен быть размер каждой из них (то есть ее запросы ресурсов на память и ресурсы процессора). Размер временного пода-заполнителя должен быть не меньше, чем размер самого большого из обычных подов; в противном случае ваша рабочая нагрузка может не поместиться в пространство, освобождающееся при вытеснении временного пода-заполнителя. В то же время этот размер не должен быть и слишком большим. Если вы хотите зарезервировать чуть больше мощностей, то лучше увеличьте количество реплик, вместо того чтобы использовать реплики, размер которых намного превышает размер стандартной рабочей нагрузки.

Чтобы мочь вытеснять временные поды-заполнители, подвергая их приоритизации, другие поды должны иметь класс приоритета с более высоким значением и политикой приоритизации (`preemptionPolicy`), не равной значению `Never`. К счастью, в Kubernetes по умолчанию используется класс приоритета со значением `0` и политикой приоритизации `PreemptLowerPriority`, благодаря чему становится возможным вытеснение временных подов-заполнителей всеми другими.

При желании вы можете явно определить этот используемый по умолчанию класс приоритета (листинг 6.11). Но поскольку на самом деле вам не нужно вносить никаких изменений в используемые по умолчанию настройки, то я рекомендую обойтись без этого определения и использовать данный листинг лишь в качестве образца при создании собственных классов приоритета (помня о том, что присваивать параметру `globalDefault` значение `true`, как было сделано здесь, требуется лишь в исключительных случаях). Напомню еще раз: чтобы ваши классы приоритета обеспечивали вытеснение временных подов-заполнителей, *никогда* не используйте политику приоритизации (`preemptionPolicy`) со значением `Never`.

Листинг 6.11. Chapter06/6.3.2_PlaceholderPod/default-priority.yaml

```

apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: default-priority
  value: 0
preemptionPolicy: PreemptLowerPriority
globalDefault: true
description: "The global default priority. Will preempt the placeholder Pods."
  
```

Инкапсулировав временные поды-заполнители в развертывание, как показано выше, вы можете предоставить постоянный запас пространства для масштабирования, чтобы при необходимости можно было быстро распределить поды на заданное количество готовых к использованию мощностей. Вдобавок вы можете инкапсулировать временные поды-заполнители в объект `Job`, если мощности нужно выделить только один раз, в объект `CronJob`, если мощности требуется выделять по расписанию, или создавать самостоятельные временные поды-заполнители.

6.4. КАК СДЕЛАТЬ СВОЕ ПРИЛОЖЕНИЕ МАСШТАБИРУЕМЫМ

Реализация процесса масштабирования — лишь часть уравнения. Помимо этого, нужно сделать свое приложение масштабируемым. Даже если у вас еще очень мало пользователей, чтобы волноваться по этому поводу, *начинать думать о том, как будет масштабироваться приложение, нужно задолго до того, как это действительно потребуется!*

Если вы не можете заранее предсказать, сколько пользователей будет у вашего приложения (например, если речь идет о старте с потенциально неограниченным количеством пользователей), то вам совершенно необходимо подумать

о масштабировании заранее, чтобы в итоге не пойти по сценарию «неудавшегося успеха», при котором приложение не может справиться со свалившейся на него популярностью. Вы не знаете, когда наступит этот момент взрывного роста, поэтому должны подготовиться к нему заранее. До данного этапа удастся дойти далеко не каждому стартапу, но если это случится с вами, то вы должны быть готовы превратить популярность в деньги, чтобы ваши труды не оказались напрасными.

К счастью, решив использовать Kubernetes для оркестрации своих контейнеров, вы уже заложили хорошую основу создания масштабируемого приложения. При разработке приложения необходимо принимать в расчет множество разных факторов, многие из которых никак не связаны с Kubernetes. Большинство принципов масштабируемого дизайна справедливы безотносительно того, используете ли вы Kubernetes, но я расскажу о практиках, особенно полезных при создании масштабируемого приложения, которое планируется развернуть в Kubernetes. Соблюдение некоторых принципов обеспечения масштабируемости на этапе разработки приложения может принести плоды в дальнейшем, когда наступит ваш момент популярности и вам нужно будет адаптировать его под невероятно возросшее количество пользователей.

6.4.1. Отказ от сохранения состояния

Один из наиболее важных принципов масштабируемости сводится к тому, чтобы не сохранять состояние локально. При этом каждая запущенная вами реплика (экземпляр) приложения может обслуживать любой поступающий запрос, не обращаясь к данным, сохраненным локально в каком-либо другом экземпляре. Для временной обработки данных может использоваться эфемерное локальное хранилище (быстрое виртуальное дисковое пространство контейнера, которое не сохраняется при перезапуске. — *Примеч. науч. ред.*), при условии, что оно не будет использоваться сразу несколькими репликами или в рамках нескольких запросов.

ПРИМЕЧАНИЕ

На мой взгляд, отказ от сохранения состояния — один из наиболее важных факторов в популярной методологии «12-факторного приложения» (Twelve-Factor App) (<https://12factor.net/processes>). Приложения без сохранения состояния (stateless-приложения) легче масштабировать и сопровождать, поскольку каждый экземпляр такого приложения может обрабатывать любой запрос независимо от других экземпляров.

В отличие от использования классической хост-машины, в Kubernetes все данные, которые контейнер записывает на диск, по умолчанию являются эфемерными (то есть удаляются при остановке или перезапуске контейнера). При желании вы можете создать приложение с сохранением состояния (stateful-приложение), используя персистентные тома и объект StatefulSet (см. главу 9), но по умолчанию контейнеры не сохраняют состояние, и в большинстве случаев этот параметр лучше не менять, чтобы при необходимости можно было выполнить масштабирование.

Вместо того чтобы сохранять состояние на дисках, развертываемых в Kubernetes, используйте для хранения данных внешние хранилища: реляционные и нереляционные базы для структурированных данных, объектные хранилища для файлов и размещаемые в памяти хранилища наподобие Redis для состояния пользовательской сессии. Чтобы обеспечить возможность масштабирования в облачной среде, всегда выбирайте управляемые сервисы (а не сервисы, предлагающие вам собственный сервер) и проверяйте, смогут ли они при необходимости справиться с возросшим количеством пользователей.

Сказанное выше вовсе не означает, что сохранение состояния — нечто плохое по самой своей сути. Вам все равно нужно *где-то* сохранять состояние, и иногда это приходится делать в приложении, имеющем собственный сервер. В тех случаях, когда приходится создавать такое приложение, старайтесь использовать легко масштабируемые решения, применяемые широко и успешно (например, Redis).

ВОЗМОЖНЫЕ ПРОБЛЕМЫ С ИСПОЛЬЗОВАНИЕМ РЕЛЯЦИОННЫХ БАЗ ДАННЫХ

При использовании таких реляционных баз данных, как MySQL или PostgreSQL, следует учитывать несколько возможных проблем.

Приведите в порядок свои SQL-запросы

Очевидно, что неэффективные обращения к базе данных плохо сказываются на масштабировании, снижая производительность по мере увеличения обрабатываемого объема данных и количества HTTP-запросов. Чтобы держать эту ситуацию под контролем, я рекомендую журналировать и анализировать свои SQL-запросы, и начать это делать лучше на самых ранних этапах разработки (не стоит откладывать до момента, когда ваше приложение станет хитом!).

Чтобы что-то улучшить, это сначала нужно измерить, поэтому важным первым шагом является журналирование скорости выполнения SQL-запросов при обработке каждого HTTP-запроса. Начните с тех HTTP-запросов, которые обрабатываются дольше всего, из-за того, что SQL-запросы выполняются слишком медленно, или потому, что их очень много.

Обе упомянутые СУБД — и MySQL, и PostgreSQL — позволяют использовать команду EXPLAIN, которая может помочь при анализе определенных SQL-запросов на предмет производительности. Широко применяемыми тактиками повышения производительности, в частности, являются индексирование часто используемых столбцов и сокращение количества выполняемых операторов JOIN. Подробное описание многих тактик оптимизации можно найти в документации по СУБД MySQL, в разделе, посвященном оптимизации операторов SELECT (<https://dev.mysql.com/doc/refman/8.0/en/select-optimization.html>).

Остерегайтесь проблемы N+1 запроса

Даже если ваши SQL-запросы будут сверхэффективными, выполнение каждого дополнительного запроса к базе данных будет чревато дополнительными затратами. В идеале все HTTP-запросы, обрабатываемые приложением, должны выполнять одинаковое количество SQL-запросов, вне зависимости от того, сколько данных при этом выводится на экран.

Если определенный HTTP-запрос должен выводить на экран список объектов, то в идеале он должен обслуживаться так, чтобы не генерировались отдельные SQL-запросы для каждого из этих объектов. Такую ситуацию принято называть «проблемой N+1 запроса» (поскольку вы используете один запрос для получения самого списка и N запросов для получения его элементов).

Этот антипаттерн особенно характерен для систем, использующих объектно-реляционное отображение (object-relational mapping, ORM) с отложенной загрузкой данных между родительским и дочерним объектами. Для извлечения содержимого дочерних объектов, которые связаны с родительским объектом отношением «один ко многим» с отложенной загрузкой, обычно требуется N+1 запрос (один запрос для родительского и N запросов — для дочерних объектов), что можно увидеть, проанализировав файлы журналов. К счастью, в большинстве таких систем можно заранее указать, что вы планируете обращаться к дочерним объектам, чтобы эти запросы можно было сгруппировать.

Такие ситуации с N+1 запросом обычно можно оптимизировать, получив постоянное количество запросов. Можно прибегнуть к двум способам: либо с помощью оператора JOIN возвращать дочерние объекты в запросе для получения списка, либо использовать два запроса: один для получения набора записей, второй для получения элементов дочерних объектов из этого набора. Помните, что в идеале каждый HTTP-запрос должен использовать небольшое постоянное количество SQL-запросов, которое не должно линейно возрастать при увеличении количества выводимых на экран записей.

Используйте реплики чтения для запросов с оператором SELECT

Один из лучших способов снижения нагрузки на основную базу данных сводится к созданию реплики чтения, что достаточно просто сделать в большинстве облачных сред. Отправляя реплике чтения все свои запросы на чтение, вы можете снять значительную часть нагрузки с основной реплики, доступной для чтения и записи.

Чтобы использовать этот паттерн в дизайне приложения еще задолго до того, как реплика чтения действительно станет необходимой, на первых порах можно симитировать эту реплику с помощью дополнительного соединения

с той же базой данных. Создайте это второе соединение, назначив ему отдельного пользователя с правами только на чтение. Когда придет пора развернуть реальную реплику чтения, достаточно будет просто изменить адрес экземпляра, присвоенный второму соединению.

Автоинкрементация первичного ключа

Если ваше приложение станет действительно популярным, то вы можете пожалеть о том, что изначально решили применить генерацию значений первичного ключа методом автоинкремента. Использование этой возможности становится проблемой при масштабировании, поскольку она предполагает использование единственного экземпляра базы данных с доступом для записи (что не позволяет выполнять горизонтальное масштабирование) и требует наложения блокировки для вставки записей, а это плохо сказывается на производительности (поскольку не позволяет вставлять больше одной записи за раз).

Это становится серьезной проблемой лишь в том случае, если количество пользователей сильно возрастет, но будет совсем не лишним учесть этот момент при разработке, чтобы потом не пришлось менять архитектуру приложения, если вам вдруг действительно понадобится выполнить значительное масштабирование. Распространенный способ решения этой проблемы сводится к использованию глобальных идентификаторов UUID — 128-битных чисел, обычно представляемых в виде шестнадцатеричной строки (вида 8fe05abe-e65d-11ea-b0da-00155d51dc33), которые могут быть сгенерированы уникальным образом любым клиентом (в том числе тем, который работает на устройстве пользователя).

Разработчики социальной сети Twitter в свое время решили использовать другой подход. Столкнувшись с необходимостью масштабирования, они решили создать собственный механизм инкрементации глобальных идентификаторов с тем, чтобы они по-прежнему поддавались сортировке и у более новых твитов номера были больше, чем у более старых (см. соответствующую публикацию в этой соцсети (https://blog.twitter.com/engineering/en_us/a/2010/announcing-snowflake.html)).

С другой стороны, возможно, вы решите пока не отказываться от инкрементации первичного ключа из чисто эстетических соображений, например, потому, что пользователь может видеть идентификаторы записей (как в случае с идентификаторами твитов в социальной сети Twitter), или просто чтобы не усложнять себе жизнь. Но даже в таком случае вы можете с самого начала постараться не использовать автоинкремент первичного ключа в тех местах, где это не несет какой-либо пользы, как, например, в объекте пользовательского сеанса — далеко не каждая таблица нуждается в использовании автоинкремента.

6.4.2. Микросервисная архитектура

Один из возможных подходов к разработке приложения сводится к тому, чтобы разбить каждый сервис на более мелкие, что принято называть «микросервисной архитектурой». При этом вы фактически просто создаете несколько внутренних сервисов, предназначенных для выполнения независимых задач, и используете удаленные вызовы процедур (обычно в виде HTTP-запросов) для вызова этих функций из других сервисов. Это разительно отличается от использования монолитной архитектуры с размещением всей логики программы в одном контейнере.

ЧТО ЛУЧШЕ: МИКРОСЕРВИСЫ ИЛИ МОНОЛИТНАЯ АРХИТЕКТУРА

Спорить о том, что лучше использовать: микросервисы или монолитную архитектуру, можно очень долго. В этой книге я ограничусь лишь тем, что приведу два авторитетных мнения по данному вопросу.

В своем посте, посвященном монолитной архитектуре (<https://m.signalnoise.com/the-majestic-monolith/>), Давид Хейнемейер Ханссон (David Heinemeier Hansson) высказал мнение о том, что микросервисы больше подходят для крупных IT-компаний, а небольшим командам разработчиков все же лучше использовать монолитную архитектуру. Он считает, что преимущества микросервисов проявляются лишь в определенных ситуациях и выражены не настолько ярко, чтобы прилагать ради них дополнительные усилия, особенно если команда небольшая.

Очень взвешенную и сбалансированную точку зрения на микросервисы изложили Джеймс Льюис (James Lewis) и Мартин Фаулер (Martin Fowler) в своей статье *Microservices* (<http://martinfowler.com/articles/microservices.html>).

По их мнению, одним из преимуществ этой архитектуры является иной подход к создаваемому продукту, при котором каждая отдельная команда разработчиков может сосредоточиться на разработке и поддержке своего компонента. Такая децентрализация позволяет каждой команде принимать собственные архитектурные решения.

Наряду с преимуществами данный подход имеет и некоторые недостатки, поэтому не стоит использовать его просто потому, что он вам больше нравится. К преимуществам можно отнести следующие факторы: сервисы могут создаваться независимо друг от друга (например, отдельными командами разработчиков) и с использованием разных языков программирования; кроме того, сервисы можно масштабировать независимо друг от друга. К недостаткам можно отнести то, что вам приходится использовать более сложный процесс отладки и интеграционного тестирования,

поскольку приложение содержит гораздо больше компонентов и вам нужно как-то отслеживать перемещение запросов внутри системы.

Безотносительно того, решите ли вы прибегнуть к микросервисам, я хотел бы обратить ваше внимание на то, что при использовании нескольких сервисов вы можете масштабировать их по отдельности. Такая возможность есть даже в том случае, когда вы применяете лишь один внутренний сервис помимо основного приложения, — чтобы воспользоваться преимуществами этой архитектуры, совсем не обязательно делать отдельным сервисом буквально каждую конечную точку. Например, вы взаимодействуете с веб-приложением, которое по большей части обрабатывает запросы, возвращающие данные в формате HTML или JSON, но имеет одну конечную точку для отображения графики в режиме реального времени, вследствие чего память будет использоваться более интенсивно по сравнению с обычными запросами. В таком случае имеет смысл создать отдельное развертывание (содержащее тот же контейнер) для обслуживания графической конечной точки, что позволит масштабировать ее независимо и в какой-то мере изолировать.

Это можно реализовать двумя способами. Вы можете использовать для всех пользователей общий фронтенд, который будет вызывать внутренний сервис (рис. 6.4). Либо же вы можете позволить конечным пользователям связываться с этим новым сервисом напрямую (рис. 6.5).



Рис. 6.4. Два HTTP-пути обслуживаются общим фронтендом, вызывающим внутренний сервис

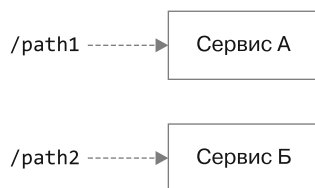


Рис. 6.5. Два HTTP-пути обслуживаются отдельными сервисами

Что бы вы ни решили: использовать микросервисную архитектуру или просто выделить один из сервисов в отдельное развертывание для его независимого масштабирования, использовать несколько языков программирования или некий собственный или свободно распространяемый код, позволяющий вам предоставить доступ к своему приложению, — вам в конечном счете придется создать в Kubernetes *внутренние сервисы*. Они получают приватный внутрикластерный IP-адрес и позволяют реализовать данную архитектуру за счет их вызова другими сервисами в пределах кластера. О том, как выполняется настройка таких внутренних сервисов, мы поговорим в следующей главе.

6.4.3. Фоновая обработка

Еще один способ облегчить масштабирование сводится к тому, чтобы по возможности отказаться от последовательного выполнения большого объема вычислений. Например, у вас есть конечная точка, которая возвращает эскиз изображения, генерируя этот эскиз, если его нет в кэше. В принципе, эти действия можно реализовать в виде последовательно выполняемой логики, где в ответ на запрос пользователя сервис возвращает эскиз из кэша или генерирует эскиз, если его там нет. Однако такой дизайн чреват проблемой: на создание эскиза обычно уходит гораздо больше времени, чем на его извлечение из кэша. Если сервер получит большое количество запросов, каждый из которых будет требовать создания эскиза, это может привести к заметному падению производительности или сбою сервера. Кроме того, такой дизайн трудно поддается масштабированию, поскольку запросы могут сильно различаться по ресурсоемкости. Даже масштабировав такой сервис, вы не сможете исключить вероятность того, что балансировщик нагрузки направит все ресурсоемкие запросы одному экземпляру.

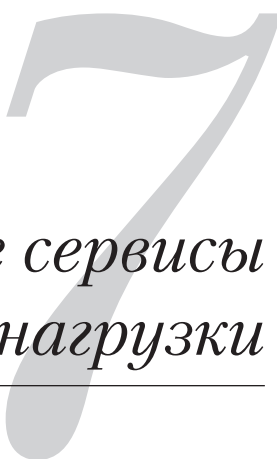
Решить эту проблему можно с помощью фоновой обработки, о чем мы поговорим в главе 10. Суть этого подхода сводится к следующему. При поступлении запроса, требующего ресурсоемких вычислений, вы не приступаете сразу же к их последовательному выполнению, а ставите их в очередь фоновых задач, возвращая клиенту код состояния, указывающий, что ему нужно отправить этот запрос еще раз. Обработку очереди фоновых задач при этом берет на себя отдельный контейнер, который можно более точно масштабировать, исходя из текущей длины очереди. При поступлении запроса, требующего создания эскиза, эта задача будет поставлена в очередь фоновых задач. Если все сложится удачно, то к моменту, когда клиент автоматически сгенерирует повторный запрос, очередь фоновых задач уже создаст эскиз, и его можно будет отправить клиенту. Таким образом, приложив немного дополнительных усилий для создания очереди фоновых задач и клиента с логикой отправки повторных запросов, мы получаем решение, которое позволяет получить тот же результат для конечного пользователя, но гораздо лучше поддается масштабированию.

РЕЗЮМЕ

- Kubernetes — хороший помощник по обеспечению масштабируемости; эту платформу используют многие из высоконагруженных приложений.
- Чтобы использовать Kubernetes эффективно, следует с самого начала предусмотреть в дизайне приложения возможность горизонтального масштабирования.
- Предоставлять по мере необходимости дополнительные поды можно с помощью горизонтального автоскейлера подов (объект `HorizontalPodAutoscaler`);

в сочетании с автомасштабированием кластера это позволяет сделать процесс масштабирования полностью автоматизированным.

- Масштабировать поды можно не только на основе показателя потребления ресурсов процессора, но и на основе любого другого показателя, предоставляемого вашим инструментом мониторинга.
- Для предоставления по мере необходимости дополнительных узлов можно использовать автоскейлер кластера (если эту функцию поддерживает ваш облачный провайдер).
- С помощью временных подов-заполнителей можно создать запас свободных мощностей даже при использовании автоскейлера кластера.
- Возможно, вам стоит разделить свое приложение на микросервисы или просто создать несколько развертываний одного и того же приложения, чтобы можно было масштабировать различные сервисы по отдельности.



Внутренние сервисы и балансировка нагрузки

В этой главе

- ✓ Создание внутренних сервисов.
- ✓ Маршрутизация пакетов в Kubernetes между виртуальными IP-адресами подов и сервисов.
- ✓ Определение IP-адреса внутренних сервисов.
- ✓ Настройка HTTP-балансировщика с помощью объекта Ingress.
- ✓ Предоставление TLS-сертификатов для создания конечных точек HTTPS.

Внутренние сервисы позволяют сделать процесс разработки и поставки приложения масштабируемым за счет разделения приложения на множество мелких сервисов. В Kubernetes можно запустить любой код, который можно подвергнуть контейнеризации, поэтому каждый из этих отдельных сервисов может создаваться в отдельном цикле разработки (возможно, отдельной командой разработчиков) и с использованием своего языка программирования или программной технологии. Вам больше не нужно беспокоиться о том, сможет ли выбранная вами платформа развертывания приложений запустить то, что вам нужно запустить.

В данной главе мы поговорим о том, как выполняется настройка и обнаружение внутренних сервисов в кластере и как Kubernetes выделяет каждому из них внутрикластерный IP-адрес и реализует внутреннюю сетевую маршрутизацию, чтобы сделать их доступными для других подов внутри кластера. Кроме того, вы узнаете, как предоставлять несколько сервисов на одном внешнем IP-адресе, используя объект Ingress, и как данный объект может обеспечить управление завершением TLS (TLS termination), чтобы вы могли предоставить конечные точки HTTPS, не настраивая TLS-сертификаты в приложении.

7.1. ВНУТРЕННИЕ СЕРВИСЫ

Необходимость в создании сервисов, доступных только внутри кластера, может быть вызвана разными причинами. Возможно, вы решили использовать микросервисную архитектуру, интегрировать сервис с открытым исходным кодом или просто хотите, чтобы два приложения, созданные с использованием разных языков программирования, взаимодействовали между собой.

В главе 3 мы уже пробовали использовать объект Service типа `LoadBalancer` (Балансировщик нагрузки) для получения внешнего трафика на публичном IP-адресе. Вдобавок объект Service дает возможность устанавливать связь с внутренними сервисами с помощью внутрикластерных IP-адресов. Kubernetes поддерживает несколько типов объекта Service, два из которых используются для создания внутренних сервисов `ClusterIP` и `NodePort`.

Тип `ClusterIP` предоставляет вам виртуальный IP-адрес, доступный для любого пода в пределах кластера Kubernetes (например, из вашего основного приложения). Тип `NodePort` в дополнение к этому резервирует порт с определенным номером на каждом узле кластера, что позволяет получать к нему доступ из-за пределов кластера (поскольку к IP узла можно адресоваться напрямую из сети). Внутрикластерное взаимодействие обычно обеспечивается с помощью типа `ClusterIP`, а тип `NodePort` используется для маршрутизации внешнего трафика, в том числе с использованием объекта Ingress. И в том и в другом случае Kubernetes настраивает сеть так, чтобы запросы перенаправлялись к поддерживающим сервис подам.

ПРИМЕЧАНИЕ

Строго говоря, внутрикластерным IP-адресом обладают три типа сервиса, а не только `ClusterIP`. `NodePort` и `LoadBalancer` предоставляют *дополнительные* методы доступа по сравнению с `ClusterIP`, будучи доступными также через внутрикластерный IP-адрес.

7.1.1. Сетевое взаимодействие в кластере Kubernetes

Теперь кратко поговорим о том, как в Kubernetes работают сети. Каждый под получает собственный IP-адрес и может напрямую связываться со всеми другими подами в кластере, обходясь без трансляции сетевых адресов (Network Address Translation, NAT). Контейнеры пода (помним, что в поде может быть несколько контейнеров) совместно используют один и тот же IP-адрес. В этом плане поды ведут себя так же, как виртуальные машины, поэтому вы можете не бояться, что размещенные на узле поды начнут конкурировать за порты (например, сразу будут одновременно пытаться запустить контейнер на порте 80). У каждого узла есть собственный IP-адрес, который присвоен сетевому интерфейсу виртуальной машины, в то время как IP-адрес пода использует виртуальный сетевой интерфейс с маршрутизацией трафика через сетевой интерфейс узла.

Каждому сервису (за исключением headless-сервисов, о которых речь пойдет в главе 9) присваивается виртуальный IP-адрес. Трафик, направленный на этот адрес, не направляется к отдельному поду или узлу, а распределяется между поддерживающими сервис подами с помощью размещенного на узле связующего сетевого компонента. Данный компонент предоставляется системой Kubernetes (с использованием iptables или виртуального сервиса IP-адресов (IP Virtual Service, IPVS)) и обеспечивает маршрутизацию трафика. Списки поддерживающих сервис подов и их IP сохраняются в актуальном состоянии на узле и используются для маршрутизации.

Когда с помощью внутрикластерного IP-адреса или порта узла под направляет сервису запрос, тот сначала обрабатывается размещенным на узле связующим сетевым компонентом, которому управляющий слой Kubernetes предоставляет обновленный список принадлежащих этому сервису подов (вместе с информацией о том, на каких узлах они размещены). Он случайным образом выбирает IP-адрес одного из этих подов и направляет к нему запрос через соответствующий узел (рис. 7.1). К счастью, все это происходит совершенно бесшовно, и, если ваше приложение, например, отправит на IP-адрес сервиса HTTP-запрос типа GET, поведение будет вполне ожидаемым.

Все вышесказанное означает следующее: при необходимости развернуть внутренний сервис можно создать сервис типа ClusterIP, которому будет назначен IP-адрес, доступный для использования другими подами в пределах вашего кластера (например, фронтенд вашего приложения). Направляемый к этому IP-адресу трафик будет автоматически распределяться между всеми репликами пода внутреннего сервиса. Разработчику приложения обычно не нужно знать, с помощью каких именно механизмов все это реализуется, поэтому будем надеяться, что вам будет достаточно изложенной здесь базовой информации.

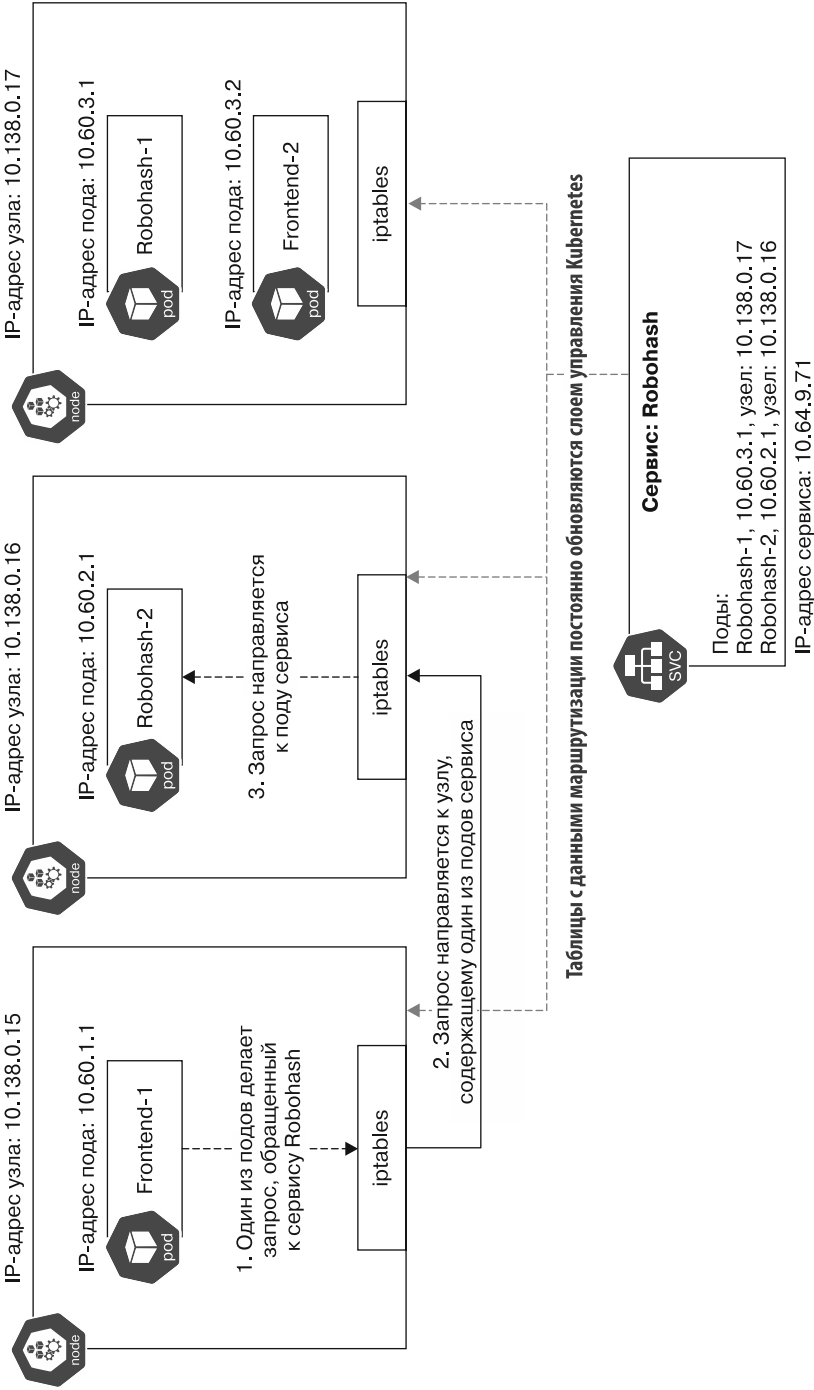


Рис. 7.1. Маршрутизация трафика, направляемого на IP-адрес внутреннего сервиса с именем Robohash. Под Frontend-1 делает внутренний запрос к сервису Robohash. Размещенный на узле связующий сетевой компонент на базе iptables сверяется с полученным от управляющего слоя Kubernetes списком принадлежащих сервису подов и случайно выбирает из него под Robohash-2. После этого запрос через соответствующий узел направляется к данному поду

7.1.2. Создание внутреннего сервиса

Теперь, когда у вас есть некоторое представление о том, как работают сети в Kubernetes под капотом, попробуем создать внутренний сервис, который смогут использовать другие поды в пределах кластера. В качестве примера добавим в наше демонстрационное приложение еще один сервис. Для этого я решил использовать библиотеку Robohash — изящный инструмент с открытым исходным кодом, который позволяет на основе хеш-кода (например, хеш-кода IP-адреса) генерировать для пользователей забавные аватары с изображением роботов. Внутренние сервисы могут представлять собой не только такие простые элементы, как генератор аватаров, но и что-то более сложное, например отдельные части приложения или полнофункциональные развертывания баз данных. В листинге 7.1 показано, как будет выглядеть развертывание этого нового контейнера.

Листинг 7.1. Chapter07/7.1_InternalServices/robohash-deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: robohash
spec:
  replicas: 1
  selector:
    matchLabels:
      app: robohash
  template:
    metadata:
      labels:
        app: robohash
    spec:
      containers:
        - name: robohash-container
          image: wdenniss/robohash:1
```



На этот раз вместо того чтобы делать сервис доступным для внешних пользователей, используя тип `LoadBalancer`, мы сделаем его внутренним с помощью типа `ClusterIP`. Как определить внутренний сервис для развертывания Robohash, показано в листинге 7.2.

Листинг 7.2. Chapter07/7.1_InternalServices/robohash-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: robohash-internal
spec:
  selector:
    app: robohash
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
  type: ClusterIP
```



В отличие от сервиса типа `LoadBalancer`, который мы использовали в главе 3, данный сервис не обладает внешним IP-адресом. Вы можете опробовать его на практике. Для этого, создав два представленных выше ресурса, произведите перенаправление портов с помощью утилиты `kubectl`:

```
kubectl port-forward service/robohash-internal 8080:80
```

Теперь вы сможете опробовать новый сервис, перейдя на локальной машине по адресу `http://localhost:8080`. Чтобы сгенерировать тестовый аватар, попробуйте ввести адрес `http://localhost:8080/example`. Вы должны увидеть автоматически сгенерированный аватар с изображением робота; пример такого аватара представлен на рис. 7.2.

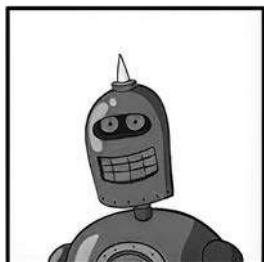


Рис. 7.2. Пример аватара с изображением робота (эти изображения komponуются сайтом Robohash.org из элементов, разработанных Зикри Кадером (Zikri Kader) и распространяемых по лицензии CC-BY)

Теперь попробуем использовать этот внутренний сервис из другого сервиса — фронтенда нашего приложения, таким образом реализовав микросервисную архитектуру (рис. 7.3).

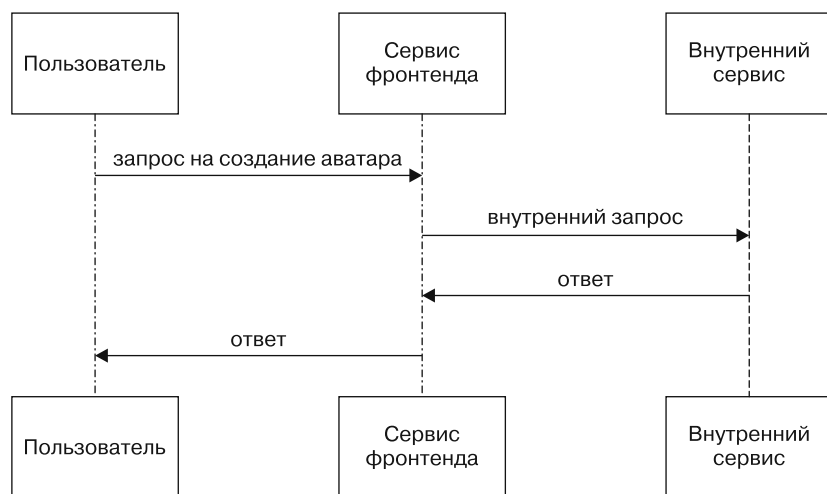


Рис. 7.3. Схема последовательности операций для простейшей микросервисной конфигурации

Для обращения к данному внутреннему сервису из других подов можно использовать его внутрикластерный IP-адрес. Узнать, какой внутрикластерный IP-адрес был присвоен сервису, можно, запросив информацию о сервисе с помощью такой команды:

```
$ kubectl get service
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)
robobash-internal   ClusterIP   10.63.254.218   <none>       80/TCP
```

В данном случае вы сможете обращаться к сервису из других подов, используя внутрикластерный IP-адрес **10.63.254.218** (например, отправлять HTTP-запросы типа GET по адресу <http://10.63.254.218/example>). Этот адрес будет доступен только для подов, находящихся в пределах кластера.

7.1.3. Обнаружение сервиса

В предыдущем примере мы узнали присвоенный сервису внутрикластерный IP-адрес, используя команду `kubectl get service`. Вы можете жестко задать этот IP-адрес в своем приложении, но с точки зрения переносимости это далеко не лучший вариант. Возможно, в дальнейшем вам потребуется развернуть то же приложение сразу в нескольких местах, например, в локальной среде разработки, в предпродуктивной и продуктивной среде (о настройке всех этих сред мы подробно поговорим в главе 11). Если вы жестко зададите этот IP-адрес в своем коде, то вам придется каждый раз его изменять.

Более разумный подход сводится к тому, чтобы динамически находить IP-адреса в поде, которому нужно обратиться к сервису, подобно тому, как в представленном выше примере мы выяснили IP-адрес с помощью утилиты `kubectl`. В Kubernetes обнаруживать сервис из пода можно с помощью переменных среды или DNS. DNS работает во всем кластере, в то время как переменная среды работает только для подов, размещенных в том же пространстве имен, что и сервис.

Обнаружение сервиса с помощью переменных среды

Kubernetes автоматически создает для каждого сервиса переменную среды, заносит в нее внутрикластерный IP-адрес и делает его доступным в каждом поде, создаваемом после создания сервиса. Для этих переменных принято соглашение по именованию, в соответствии с которым, например, для сервиса `robobash-internal` должна быть создана переменная среды `ROBOHASH_INTERNAL_SERVICE_HOST`.

Вместо того чтобы пытаться догадаться, как должно выглядеть это имя в вашем случае, вы можете просто отобразить список переменных среды, доступных для вашего пода, выполнив для него команду `env` с помощью команды `exec` (я немного сократил здесь вывод этой команды):

```
$ kubectl get pods
NAME                READY  STATUS   RESTARTS  AGE
robobash-6c96c64448-7fn24  1/1    Running   0          2d23h
```

208 Часть II. Переходим в продуктивную среду

```
$ kubectl exec robohash-6c96c64448-7fn24 -- env
ROBOHASH_INTERNAL_PORT_80_TCP_ADDR=10.63.243.43
ROBOHASH_INTERNAL_PORT_80_TCP=tcp://10.63.243.43:80
ROBOHASH_INTERNAL_PORT_80_TCP_PROTO=tcp
ROBOHASH_INTERNAL_SERVICE_PORT=80
ROBOHASH_INTERNAL_PORT=tcp://10.63.243.43:80
ROBOHASH_INTERNAL_PORT_80_TCP_PORT=80
ROBOHASH_INTERNAL_SERVICE_HOST=10.63.243.43
```

У такого подхода есть преимущество: он работает чрезвычайно быстро, поскольку переменные среды представляют собой обычные строковые константы, без каких-либо внешних зависимостей по отношению к поду. Кроме того, вы можете задать любой предпочитаемый вами DNS-сервер для обслуживания других запросов пода (например, DNS-сервер с адресом 8.8.8.8). Недостатком подхода является то, что переменные среды содержат информацию только о тех сервисах, которые находятся в том же пространстве имен и были созданы раньше самого пода.

Чтобы обновить информацию о сервисах, вы можете перезапустить поды своего развертывания с помощью этой команды (не внося никаких изменений в под):

```
kubectl rollout restart deployment $DEPLOYMENT_NAME
```

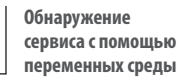
Один из часто используемых способов обеспечения доступа к этим переменным сводится к тому, чтобы создать отдельную конечную точку HTTP для внутреннего сервиса, задав ее как собственную переменную среды в определении развертывания. Благодаря такому подходу ваш контейнер становится еще более переносимым, и его можно запускать не только в Kubernetes, но и, например, в Docker Compose. В листинге 7.3 представлен пример того, как внести значение автоматически генерируемой переменной среды (такой как `ROBOHASH_INTERNAL_SERVICE_HOST`) в собственную переменную среды (`AVATAR_ENDPOINT`), которую будет использовать приложение в конечном счете.

Листинг 7.3. Chapter07/7.1_InternalServices/timeserver-deploy-env.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
```



```
spec:
  containers:
  - name: timeserver-container
    image: docker.io/wdenniss/timeserver:5
    env:
      - name: AVATAR_ENDPOINT
        value: http://$(ROBOHASH_INTERNAL_SERVICE_HOST)
```



Обнаружение сервиса с помощью переменных среды

Благодаря дополнительному уровню абстракции, на котором наша переменная среды ссылается на переменную среды, создаваемую системой Kubernetes, мы при желании можем выполнить независимый запуск контейнера в Docker (предварительно проследив за тем, чтобы переменная `AVATAR_ENDPOINT` уже содержала значение) или переключиться на использование DNS.

Итак, обнаружение сервиса с помощью переменных среды имеет два преимущества:

- очень высокая производительность (поскольку мы имеем дело со строковыми константами);
- отсутствие зависимости от компонентов Kubernetes для работы с DNS.

К недостаткам этого подхода можно отнести следующие два момента:

- сервис доступен только тем подам, которые находятся в том же пространстве имен, что и сам сервис;
- сервис доступен только тем подам, которые были созданы после его создания.

Обнаружение сервиса с помощью DNS

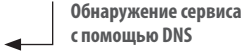
Альтернативный способ обнаружения сервисов сводится к использованию внутреннего DNS-сервиса кластера. Когда сервис и под запущены в разных пространствах имен, этот способ вообще является единственно возможным. При этом имя сервиса используется как имя хоста в DNS, что позволяет просто искать сервис в DNS по имени `robohash-internal` (или обращаться по адресу `http://robohash-internal`). При вызове сервиса из другого пространства имен к этому имени нужно будет добавить соответствующее пространство имен. Например, вызвать сервис `robohash-internal`, расположенный в пространстве имен `default`, можно, запросив имя `robohash-internal.default`.

Единственным недостатком данного подхода является то, что он работает чуть медленнее, поскольку получение нужного IP-адреса требует поиска в DNS. Во многих кластерах Kubernetes DNS-сервис запущен на том же узле, поэтому поиск работает достаточно быстро. Иногда может потребоваться обращение к DNS-сервису, запущенному на другом узле, или к управляемому DNS-сервису, в таком случае будет очень полезно кэшировать результат.

Мы уже определили URL конечной точки как переменную среды в своем разворачивании, поэтому можем легко изменить определение этой переменной, на этот раз используя в адресе имя сервиса (`http://robohash-internal`). В полном виде новая версия разворачивания будет выглядеть так (листинг 7.4).

Листинг 7.4. Chapter07/7.1_InternalServices/timeserver-deploy-dns.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
          env:
            - name: AVATAR_ENDPOINT
              value: http://robohash-internal
```



Обнаружение сервиса
с помощью DNS

Итак, обнаружение сервиса с помощью DNS имеет два преимущества:

- сервис можно вызывать из любого пространства имен в пределах кластера;
- вы не зависите от очередности создания объектов.

Недостатки этого подхода сводятся к следующим двум моментам:

- он работает чуть медленнее по сравнению с использованием переменных среды (которые представляют собой строковые константы);
- возникает зависимость от внутреннего DNS-сервиса.

Таким образом, использование переменных среды и DNS-поиска — это два способа, которые может использовать frontend-сервис для определения IP-адреса внутреннего пода внутреннего сервиса, вместо того чтобы использовать жестко заданный IP-адрес внутреннего сервиса. Данные методы обнаружения предполагают использование платформы Kubernetes, поэтому рекомендуется определять соответствующий путь как переменную среды контейнера, как было сделано в представленных выше примерах. В таком случае вы сможете легко указать совершенно другой путь, если нужно будет запустить контейнер, не используя Kubernetes.

Сводим все воедино

Теперь попробуем вызвать из приложения сервера времени созданный нами внутренний сервис, используя конечную точку `/avatar`. Эта конечная точка просто возвращает изображение, генерируемое внутренним сервисом (листинг 7.5).

Листинг 7.5. Chapter07/timeserver5/server.py

```
import urllib.request
import os
import random

# ...

case '/avatar':
    url = os.environ['AVATAR_ENDPOINT'] + "/" + str(random.randint(0, 100))
    try:
        with urllib.request.urlopen(url) as f:
            data = f.read()
            self.send_response(200)
            self.send_header('Content-type', 'image/png')
            self.end_headers()
            self.wfile.write(data)
    except urllib.error.URLError as e:
        self.respond_with(500, e.reason)

# ...
```

Теперь, когда наше приложение надлежащим образом использует внутренний сервис, мы можем развернуть все это в Kubernetes:

```
$ cd Chapter07/7.1_InternalServices
$ kubectl create -f robohash-deploy.yaml
deployment.apps/robohash created
$ kubectl create -f robohash-service.yaml
service/robohash-internal created
$ kubectl create -f timeserver-deploy-dns.yaml
deployment.apps/timeserver created
$ kubectl create -f timeserver-service.yaml
service/timeserver created
```

```
$ kubectl get svc/timeserver
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
timeserver	LoadBalancer	10.22.130.155	203.0.113.16	80:32131/TCP	4m25s

```
$ open "http://203.0.113.16/avatar"
```

Дождитесь того момента, когда вам будет предоставлен внешний IP-адрес, и попробуйте перейти по адресу `/avatar`. Вас должен поприветствовать аватар с изображением робота. После этого опробуйте альтернативный метод обнаружения сервиса, используя

вместо файла `timeserver-deploy-dns.yaml` файл `timeserver-deploy-env.yaml`; при этом в итоге вы должны получить тот же результат.

Фактически мы используем здесь микросервисную архитектуру! С помощью этого подхода вы можете создать несколько внутренних сервисов и осуществлять их развертывание и раздельное управление ими (например, привлекая отдельные команды разработчиков). В качестве независимых сервисов можно добавлять свободно распространяемые инструменты или просто отдельные компоненты приложения, созданные с использованием других языков программирования.

7.2. ОБЪЕКТ INGRESS: БАЛАНСИРОВКА HTTP(S)-ЗАПРОСОВ

В рассмотренных ранее примерах мы создавали внешний IP-адрес, используя сервис типа `LoadBalancer`. При этом вам предоставляется балансировщик нагрузки «уровня 4», который осуществляет балансировку нагрузки запросов на сетевом уровне и может работать с несколькими сетевыми протоколами (такими как TCP, UDP или SCTP). Вы настраиваете сервис на использование нужного вам протокола и порта — и получаете IP-адрес, который обеспечивает равномерное распределение трафика между вашими подами. При экспонировании HTTP-сервиса через балансировщик нагрузки вы должны реализовать собственный механизм управления TLS-терминацией, который будет обеспечивать настройку сертификатов и запуск конечной точки HTTPS, и весь трафик, идущий к этой конечной точке, будет направляться к определенному набору подов (благодаря использованию правил `matchLabels`). При этом нельзя экспонировать два или больше независимых сервисов через один и тот же балансировщик нагрузки (хотя в принципе один сервис может перенаправлять запросы другому сервису внутри себя).

При публикации приложения, специально предназначенного для обработки HTTP-трафика, часто полезнее использовать балансировщик нагрузки «уровня 7», выполняющий балансировку нагрузки на уровне HTTP-запросов. Этот тип балансировщика предоставляет такие расширенные возможности, как терминация HTTPS-соединений (с автоматическим управлением параметрами HTTPS-соединения), и использует маршрутизацию на основе пути, вследствие чего можно с помощью нескольких сервисов обслуживать однодоменный хост. В Kubernetes для создания такого балансировщика нагрузки используется объект `Ingress` (Точка входа).

Объект `Ingress` позволяет разместить несколько внутренних сервисов за одним внешним IP-адресом с балансировкой нагрузки. При этом вы можете направлять HTTP-запросы разным backend-сервисам в зависимости от используемого URI-адреса (`/foo`, `/bar`), имени хоста (`foo.example.com`, `bar.example.com`) или сразу обоих этих параметров (рис. 7.4). На одном IP-адресе можно запускать несколько сервисов, что позволяет обслуживать несколько разных путей, используя одно

доменное имя. Это уникальная особенность объекта Ingress, ведь если вы решите предоставлять несколько сервисов с помощью объекта Service типа `LoadBalancer`, как мы делали в предыдущих главах, то каждому из них будет присвоен отдельный IP-адрес и вам придется использовать отдельные домены (например, `foo.example.com` — для одного сервиса, `bar.example.com` — для другого и т. д.).

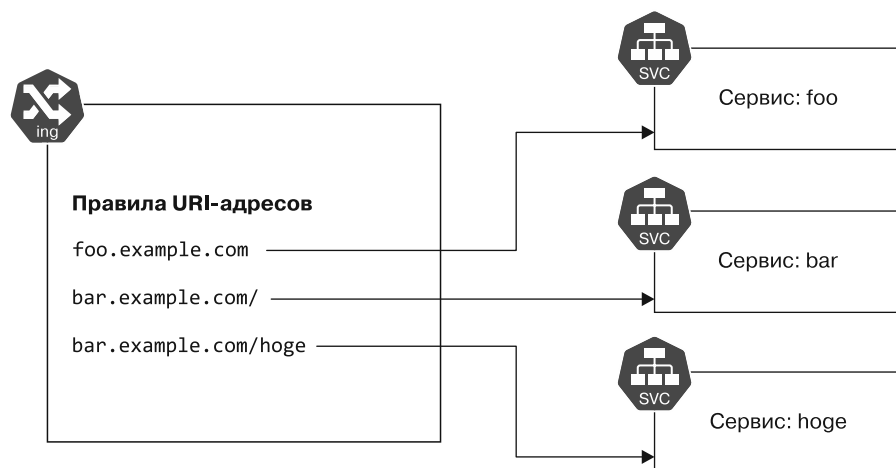


Рис. 7.4. Список правил, или отображений URL, объекта Ingress позволяет одному HTTP-балансировщику распределять трафик на несколько сервисов

Таким образом, объект Ingress позволяет размещать несколько сервисов на одном хосте. Эта возможность может быть полезной и при масштабировании приложения. Допустим, вам нужно разделить приложение на несколько сервисов, чтобы сделать более эффективным процесс разработки (например, за счет использования отдельными командами разработчиков собственных циклов разработки) или процесс масштабирования (например, за счет независимого масштабирования определенных аспектов приложения). Вы можете использовать объект Ingress для перенаправления запросов, при этом доступные пользователю URL не изменятся. Например, один из используемых вашим приложением путей служит для обработки запросов с повышенным потреблением ресурсов процессора. В таком случае эту часть приложения можно выделить в отдельный сервис, чтобы ее можно было масштабировать независимо. Объект Ingress позволяет вносить такие изменения незаметно для конечных пользователей.

Пример того, как можно определить объект Ingress, обеспечивающий обслуживание нескольких маршрутов с помощью разных бэкендов, представлен в листинге 7.6. В этом примере на корневом пути (/) предоставляется внутренний сервис Timeserver, а на пути `/robohash` — внутренний сервис Robohash.

Листинг 7.6. Chapter07/7.2_Ingress/ingress_path.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: timeserver-ingress
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: timeserver-internal
                port:
                  number: 80
          - path: /robohash
            pathType: Prefix
            backend:
              service:
                name: robohash-internal
                port:
                  number: 80

```

Первый путь, обслуживаемый сервисом timeserver-internal

Второй путь, обслуживаемый сервисом robohash-internal

В листинге 7.7 представлен еще один вариант определения объекта Ingress, который предполагает использование нескольких хостов. Для каждого из них также можно определить несколько путей, используя формат, представленный в листинге 7.6.

Листинг 7.7. Chapter07/7.2_Ingress/ingress_host.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: timeserver-ingress
spec:
  rules:
    - host: timeserver.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: timeserver-internal
                port:
                  number: 80
    - host: robohash.example.com
      http:

```

Первый хост, обслуживаемый сервисом timeserver-internal

Второй хост, обслуживаемый сервисом robohash-internal

```

paths:
- path: /
  pathType: Prefix
  backend:
    service:
      name: robohash-internal
      port:
        number: 80

```

Объект Ingress ссылается на внутренние сервисы, которые должны быть определены как сервисы типа **NodePort**. Как может выглядеть определение такого сервиса, показано в листинге 7.8.

Листинг 7.8. Chapter07/7.2_Ingress/timeserver-service-internal.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: timeserver-internal
spec:
  selector:
    pod: timeserver-pod
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
    type: NodePort

```

← Чтобы объект Ingress мог использовать внутренний сервис, его следует определить как сервис типа NodePort

С помощью свойства **pathType** объект Ingress можно настроить на обеспечение полного или частичного совпадения с указанным путем: в первом случае к сервису направляются только запросы с полностью совпадающим путем, а во втором — с совпадающим префиксом. Я не буду объяснять эту тему более подробно, поскольку с данной задачей прекрасно справляется официальная документация. Приведу лишь фрагмент документации, который касается обработки множественных совпадений:

*«В некоторых случаях запросу будет соответствовать сразу несколько путей, указанных в объекте Ingress. В таких случаях большим приоритетом наделяются пути с большей длиной совпадающего фрагмента. Если после применения этого правила остаются пути с равным приоритетом, то большим приоритетом наделяются пути с типом **Exact**, а не с типом **Prefix**» (<https://kubernetes.io/docs/concepts/services-networking/ingress/#multiple-matches>).*

Как вы могли заметить в листинге 7.6, мы определили два пути: **/** и **/robohash**. Если пользователь запросит путь **/robohash**, то будет направлен ко второму сервису,

несмотря на то что этому запросу соответствует и первый путь. Это может показаться необычным, если раньше вы использовали другой механизм маршрутизации (например, метод перезаписи URL, используемый сервером Apache), где большим приоритетом наделяется *первое* совпадение. В Kubernetes, напротив, большим приоритетом наделяется *более длинное* совпадение. Я нахожу используемый в Kubernetes подход более удобным, поскольку именно такой дизайн обычно и требуется разработчику.

Чтобы выполнить развертывание данного примера, удалите предыдущий пример, если он еще работает (`kubect1 delete -f Chapter07/7.1_InternalServices`), и выполните следующие команды:

```
$ cd Chapter07/7.2_Ingress
$ kubect1 create -f robohash-deploy.yaml
deployment.apps/robohash created
$ kubect1 create -f robohash-service.yaml
service/robohash-internal created
$ kubect1 create -f timeserver-deploy-dns.yaml
deployment.apps/timeserver created
$ kubect1 create -f timeserver-service-internal.yaml
service/timeserver-internal created
$ kubect1 create -f ingress_path.yaml
ingress.networking.k8s.io/timeserver-ingress created

$ kubect1 get ing -w
NAME                CLASS    HOSTS    ADDRESS          PORTS    AGE
timeserver-ingress  <none>  *       203.0.113.20    80      4s
timeserver-ingress  <none>  *       203.0.113.20    80      100s
```

После того как объекту Ingress будет предоставлен IP-адрес, к нему можно будет перейти в браузере. Попробуйте ввести в адресной строке путь `/robohash`, чтобы подключиться к сервису Robohash через объект Ingress. Обратите внимание, что на выделение объекту Ingress всех необходимых ресурсов может уйти чуть больше времени. Даже после того как этому объекту будет предоставлен IP-адрес и к нему можно будет перейти в браузере, вы можете столкнуться с ошибкой 404. В таком случае попробуйте выждать еще минут пять, дав облачному провайдеру время на завершение настройки объекта Ingress.

Для отладки проблем с объектом Ingress можно использовать команду `kubect1 describe ingress`. Вот, например, что она выдала при ее вызове после того, как объект Ingress получил IP-адрес, но до его перехода в состояние готовности:

```
$ kubect1 describe ingress
Name:                timeserver-ingress
Namespace:           default
Address:             203.0.113.20
Default backend:     default-http-backend:80 (10.22.0.130:8080)
```



```

Rules:
  Host      Path  Backends
  ----      -
  *
    /      timeserver-internal:80 (10.22.0.135:80)
  /robohash robohash-internal:80 (10.22.1.4:80)
Annotations: ingress.kubernetes.io/backends: {"k8s-be-32730--a52250670846a599": "Unknown",
                                                "k8s1-a5225067--default-timeser ...
ingress.kubernetes.io/forwarding-rule: k8s2-fr-21mgs2f1
ingress.kubernetes.io/target-proxy: k8s2-tp-21mgs2f1
ingress.kubernetes.io/url-map: k8s2-um-21mgs2f1

Events:
  Type      Reason      From      Message
  ----      -
  Normal    Sync        loadbalancer  UrlMap "k8s2-um-21mgs2f1" created
  Normal    Sync        loadbalancer  TargetProxy "k8s2-tp-21mgs2f1" created
  Normal    Sync        loadbalancer  ForwardingRule "k8s2-fr-21mgs2f1" created
  Normal    IPChanged   loadbalancer  IP is now 203.0.113.20
  Normal    Sync        loadbalancer  Scheduled for sync

```

Бэкенд находится
в неизвестном
состоянии (Unknown)

А вот как выглядел вывод этой команды спустя еще несколько минут. Вместо состояния Unknown в разделе аннотаций здесь указано состояние HEALTHY. На данном этапе я уже смог получить доступ к сервису, перейдя по соответствующему IP-адресу в браузере:

```

$ kubectl describe ing
Name:          timeserver-ingress
Namespace:     default
Address:       203.0.113.20
Default backend: default-http-backend:80 (10.22.0.130:8080)
Rules:
  Host      Path  Backends
  ----      -
  *
    /      timeserver-internal:80 (10.22.0.135:80)
  /robohash robohash-internal:80 (10.22.1.4:80)
Annotations: ingress.kubernetes.io/backends: {"k8s-be-32730--a52250670846a599": "HEALTHY",
                                                "k8s1-a5225067--default-timeser...
ingress.kubernetes.io/forwarding-rule: k8s2-fr-21mgs2f1
ingress.kubernetes.io/target-proxy: k8s2-tp-21mgs2f1
ingress.kubernetes.io/url-map: k8s2-um-21mgs2f1

Events:
  Type      Reason      From      Message
  ----      -
  Normal    Sync        loadbalancer  UrlMap "k8s2-um-21mgs2f1" created
  Normal    Sync        loadbalancer  TargetProxy "k8s2-tp-21mgs2f1" created
  Normal    Sync        loadbalancer  ForwardingRule "k8s2-fr-21mgs2f1" created
  Normal    IPChanged   loadbalancer  IP is now 203.0.113.20
  Normal    Sync        loadbalancer  Scheduled for sync

```

Бэкенд находится
в работоспособном
состоянии (HEALTHY)

СОВЕТ ПО ЭКОНОМИИ РАСХОДОВ: СОКРАЩЕНИЕ КОЛИЧЕСТВА IP-АДРЕСОВ С ПОМОЩЬЮ ОБЪЕКТА INGRESS

Одним из преимуществ объекта Ingress является то, что благодаря использованию маршрутизации на основе хоста вы можете присваивать нескольким сервисам один и тот же внешний IP-адрес. Каждый тип сервиса LoadBalancer получает отдельный IP-адрес и не выполняет никакой маршрутизации на основе HTTP-запроса. В отличие от этого, объект Ingress проверяет значение заголовка Host в HTTP-запросе и соответственно маршрутизирует трафик.

Многие облачные провайдеры рассчитывают размер взимаемой платы, исходя из количества используемых правил балансировки нагрузки, что приблизительно соответствует количеству выделяемых внешних IP-адресов с балансировкой нагрузки. В таком случае вы можете в какой-то мере сократить свои денежные расходы, объединив несколько сервисов на одном IP-адресе с помощью объекта Ingress, вместо того чтобы предоставлять каждый сервис с помощью отдельного IP-адреса.

Ваш облачный провайдер может по отдельности группировать балансировщики нагрузки, которые работают на уровне HTTP-запросов (объекты Ingress) и на уровне сети (сервисы типа LoadBalancer). Кроме того, он может давать возможность работать на минимальном тарифе за использование правил (как, например, это делала платформа Google Cloud на момент написания книги). В таком случае вы можете на первых порах обойтись минимальным тарифом, используя исключительно первый или второй вид балансировщиков.

Еще один возможный подход, от которого я рекомендую воздержаться, сводится к использованию собственного *Ingress-контроллера*. Этот метод (который мы не будем подробно рассматривать в данной книге) подразумевает развертывание в качестве балансировщика нагрузки свободно распространяемого компонента, который реализует функциональность объекта Ingress, переопределяя реализацию по умолчанию, предоставленную облачным провайдером. При таком подходе объекты Ingress и сервисы типа LoadBalancer учитываются как один и тот же тип правил при расчете взимаемой платы, что позволяет экономить деньги, если вам нужно и то и другое, но требует дополнительных усилий по самостоятельному управлению этим компонентом. Вы уверены, что легко справитесь с отладкой Ingress-контроллера в Kubernetes? Как показывает мой опыт, более разумный подход все же сводится к тому, чтобы работать со стандартными объектами Ingress и снижать расходы путем использования только одного вида балансировщиков.

7.2.1. Обеспечение безопасных соединений с помощью протокола TLS

Еще одним полезным свойством объекта Ingress является то, что он выполняет TLS-шифрование. Современные веб-приложения обычно размещаются в сети в виде HTTPS-приложений, защищенных с помощью протокола TLS. Это важно с точки зрения безопасности, но накладывает дополнительную нагрузку на сервер приложений. В зависимости от используемого промежуточного программного обеспечения сервера вы можете получить некий прирост производительности, позволив объекту Ingress управлять TLS-соединением (выступая в роли «TLS-терминатора»), взаимодействуя с бэкендом только по протоколу HTTP — конечно, делая это через такую защищенную сеть, как сеть вашего облачного провайдера (рис. 7.5). При желании объект Ingress можно настроить так, чтобы он заново шифровал трафик, взаимодействуя с вашими сервисами по протоколу HTTPS, однако он не может передавать зашифрованный трафик напрямую от клиента к бэкенду, не внося в него никаких изменений. Для этой цели следует использовать сервис типа LoadBalancer, как мы сделали в главе 3.

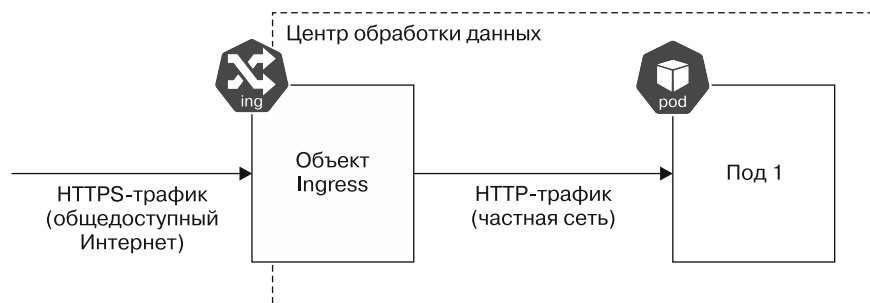


Рис. 7.5. Объект Ingress завершает HTTPS (TLS)-трафик и может направлять его к сервисным подам с помощью простых HTTP- или новых HTTPS-соединений

Объект Ingress завершает ваши TLS-соединения, поэтому вы должны выдать ему соответствующие сертификаты. Если вам, как и мне, приходилось многократно выполнять эту задачу в других системах, то она может вызывать у вас невольный страх. К счастью, в Kubernetes выполнить ее не составляет большого труда.

Вам нужно лишь импортировать свой сертификат и ключ как секрет Kubernetes, а затем сослаться на этот секрет в определении объекта Ingress. Секреты Kubernetes — это просто объекты с данными, используемые для хранения в кластере таких элементов, как TLS-ключи.

Чтобы это сделать, обычно нужно создать сертификат в соответствии с инструкциями, предоставленными вашим центром сертификации, в результате чего вы получите два необходимых файла: созданный вами приватный ключ и сертификат от центра сертификации.

В демонстрационных целях мы можем создать вместо доверенного сертификата собственный самоподписанный сертификат. Это позволит нам обеспечить такое же шифрование соединения, но без проверки достоверности сертификата, и поэтому вы увидите в браузере несколько пугающих предупреждений. Создать такой сертификат можно с помощью следующих команд:

```
# создаем приватный ключ
openssl genrsa -out example.key 2048

# создаем запрос сертификата для домена 'example.com'
openssl req -new -key example.key -out example.csr \
    -subj "/CN=example.com"

# выдаем самоподписанный сертификат
openssl x509 -req -days 365 -in example.csr -signkey \
    example.key -out example.crt
```

Создав приватный ключ и сертификат, как показано выше или в соответствии с инструкциями от вашего центра сертификации, вы должны создать секрет Kubernetes:

```
kubectl create secret tls my-tls-cert --cert example.crt --key example.key
```

Как вы могли заметить, мы используем здесь императивную команду `kubectl create`. Это один из тех немногих случаев, когда я рекомендую использовать императивную команду вместо определения конфигурации в файле, поскольку это проще, чем вручную создавать этот объект и переводить все данные в формат Base64. Если вы хотите посмотреть, какую конфигурацию создали с помощью этой команды, то можете легко ее отобразить, используя команду `kubectl get -o yaml secret my-tls-cert`.

Наконец, последний шаг сводится к тому, чтобы сослаться на созданный нами секрет в объекте Ingress (листинг 7.9).

Листинг 7.9. Chapter07/7.2.1_TLS/ingress_tls.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: timeserver-tls
spec:
  tls:
    - secretName: my-tls-cert
  rules:
    - host: example.com
      http:
```

Ссылка
на TLS-секрет



```

paths:
- path: /
  pathType: Prefix
  backend:
    service:
      name: timeserver-internal
      port:
        number: 80
- http:
  paths:
  - path: /
    pathType: Prefix
    backend:
      service:
        name: robohash-internal
        port:
          number: 80

```

Чтобы сослаться на сервисы типа **NodePort**, созданные в предыдущем разделе, теперь можно создать новую версию объекта **Ingress**, в которой используется **TLS**-секрет:

```

$ cd Chapter07/7.2.1_TLS/
$ kubectl create -f ingress_tls.yaml
ingress.networking.k8s.io/timeserver-tls created
$ kubectl get ing
NAME           CLASS      HOSTS          ADDRESS          PORTS          AGE
timeserver-tls <none>     example.com    203.0.113.15    80, 443       9m15s
$ open "https://203.0.113.15"

```

Как вы помните, в случае объекта **Ingress** процедура предоставления ресурсов длится чуть дольше и продолжается еще некоторое время после получения им **IP**-адреса. Если вы используете самоподписанный сертификат, то в браузере появится несколько пугающих предупреждений.

Чтобы протестировать маршрут в виде указанного в объекте **Ingress** доменного имени (в данном случае **example.com**), необходимо научить систему **DNS** сопоставлять этот домен с **IP**-адресом объекта **Ingress**. Кроме того, можно провести тестирование на локальной машине, добавив **IP**-адрес и доменное имя объекта **Ingress** в файле **hosts** (инструкции по редактированию этого файла можно найти, выполнив поиск в Интернете по запросу «изменение файла **hosts** в <ваша операционная система [номер версии]>»). Узнать, какой **IP**-адрес был присвоен объекту **Ingress**, можно с помощью команды **kubectl get ingress**. Ниже показано, как выглядел вывод этой команды в моем случае и какую запись я добавил в локальном файле **hosts**:

```

$ kubectl get ingress
NAME           CLASS      HOSTS          ADDRESS          PORTS          AGE
timeserver-tls <none>     example.com    203.0.113.15    80, 443       82m
$ cat /etc/hosts
# ...
203.0.113.15 example.com

```

Должным образом настроив свой хост, вы сможете перейти в браузере по адресу <https://example.com>. Если вы используете самоподписанный сертификат, то в браузере могут появиться устрашающие сообщения об ошибках, которые можно просто закрыть. Если вы решили реально опубликовать сервис в Интернете, то вернитесь к предыдущему шагу и запросите сертификат от центра сертификации, чтобы затем использовать его при создании TLS-секрета.

Здесь, опять же, следует отметить приятную особенность Kubernetes: благодаря тому, что вся эта конфигурация определяется в виде объектов данной платформы, а не в виде случайного набора файлов, эту среду можно очень просто воспроизвести в другом месте.

ПОЛЬЗУЕТЕСЬ GKE? ПОПРОБУЙТЕ ИСПОЛЬЗОВАТЬ УПРАВЛЯЕМЫЕ СЕРТИФИКАТЫ!

Изложенные выше инструкции предполагают использование в объекте Ingress сертификата от центра сертификации (Certificate Authority, CA). Однако пользователи сервиса Google Kubernetes Engine (GKE) могут еще больше упростить себе жизнь, используя вместо этого управляемые сертификаты.

Если вы решите воспользоваться этой возможностью, то вам нужно будет пропустить шаг получения сертификата от центра с последующим сохранением приватного ключа и сертификата в Kubernetes в виде секрета. Вместо этого потребуется сначала подтвердить свое право собственности на домен (в консоли управления платформы Google Cloud), а затем создать используемый сервисом GKE объект ManagedCertificate со списком (под)доменов, которым вы хотите выдать сертификаты, и сослаться на этот объект в своем объекте Ingress. После этого сервис GKE будет автоматически предоставлять необходимые сертификаты и обеспечивать управление ими. Вся эта процедура достаточно проста и подробно описана в официальной документации (<https://cloud.google.com/kubernetes-engine/docs/how-to/managed-certs>).

РЕЗЮМЕ

- Если ваши требования выходят за рамки того, что можно разместить в одном контейнере, то Kubernetes предлагает несколько инструментов, позволяющих создать, обнаружить, подключить и опубликовать несколько сервисов.
- Внутренние сервисы дают возможность подключать большое количество рабочих нагрузок, которые могут создаваться с использованием разных языков программирования, выпускаться с помощью разных графиков выпуска или просто нуждаться в независимом масштабировании.

- Внутренние сервисы могут публиковаться через внутрикластерный IP-адрес с возможностью их вызова из других подов в пределах кластера.
- Kubernetes позволяет использовать два способа обнаружения сервисов для определения IP-адреса внутренних сервисов: с помощью переменных среды и DNS.
- Объект Ingress позволяет публиковать несколько внутренних сервисов в Интернете, используя один IP-адрес и маршрутизацию на основе пути и/или имени хоста.
- Объект Ingress представляет собой балансировщик нагрузки уровня HTTP(S)-запросов, который может быть настроен на завершение TLS (TLS termination) с использованием нескольких TLS-сертификатов.
- Завершение TLS на уровне балансировщика нагрузки позволяет сделать менее трудоемким процесс настройки приложения и немного разгрузить центральные процессоры.

8

Выбор параметров узлов

В этой главе

- ✓ Выбор узлов с заданными аппаратными характеристиками.
- ✓ Управление распределением рабочих нагрузок на узлы с заданными аппаратными характеристиками с помощью ограничений и допусков.
- ✓ Распределение рабочих нагрузок по разным узлам.
- ✓ Недопущение возникновения единой точки отказа благодаря использованию стратегии развертывания, обеспечивающей высокую степень доступности.
- ✓ Группирование определенных подов на одном узле с исключением узлов, содержащих некоторые другие поды.

До сих пор в этой книге предполагалось, что входящие в кластер вычислительные узлы — машины, на которых в конечном счете запускаются ваши контейнеры, — ничем не отличаются друг от друга. Различные поды могут запрашивать разное количество ресурсов процессора, но все они в итоге запускаются на одинаковых узлах.

Одной из главных особенностей облачных вычислений является то, что даже при использовании абстрагированной платформы, которая автоматически выполняет

низкоуровневые операции выделения вычислительных ресурсов, как это могут делать облачные платформы Kubernetes, вас может все равно в какой-то мере волновать вопрос о том, на каких серверах в конечном счете запускаются ваши рабочие нагрузки.

Как бы привлекательно ни выглядела концепция бессерверных вычислений в теории, на практике ваша рабочая нагрузка все равно запускается на некой машине, и в некоторых случаях вы не можете, да и не должны абстрагироваться от ее свойств.

Здесь вам может помочь выбор узлов с заданными свойствами. Такие управляемые платформы, как Google Kubernetes Engine, позволяют учитывать самые разные аппаратные и конфигурационные параметры узлов. Вы можете указать, что узел должен иметь процессор с архитектурой x86 или Arm либо процессор от компании AMD или Intel. Можно указать, что узел должен содержать дорогие аппаратные компоненты наподобие графического процессора или использовать низкобюджетный режим спотового выделения ресурсов, позволяющий экономить деньги за счет менее жестких требований в плане обеспечения бесперебойной работы. Учитывать такие вещи требуется не так уж часто, однако эта возможность может оказаться полезной и даже критически важной, если вам нужно будет сэкономить деньги с помощью спотового режима или использовать графический процессор для выполнения вычислений, связанных с искусственным интеллектом и машинным обучением.

Следует учитывать и тот факт, что Kubernetes всегда старается разместить как можно больше подов на одном узле, используя метод «корзинной упаковки» (bin-packing). Он позволяет экономить деньги и особенно эффективен при использовании режима избыточного потребления, когда один под может временно использовать неиспользуемые вычислительные мощности другого пода. Недостатком метода упаковки является то, что его использование может приводить к возникновению единой точки отказа. К счастью, Kubernetes исключает вероятность размещения всех реплик пода на одном и том же узле за счет использования встроенной *топологии распределения подов*. В этой главе вы научитесь выбирать узлы с заданными свойствами, группировать поды в одном месте и распределять их по разным местам.

8.1. КАК ВЫБРАТЬ ПАРАМЕТРЫ УЗЛОВ

Используемые вами вычислительные узлы могут отличаться друг от друга. Некоторые из ваших рабочих нагрузок могут требовать, чтобы узел содержал такие аппаратные компоненты, как высокопроизводительный центральный или графический процессор, или обладал такими свойствами, как использование спотовой модели выделения ресурсов. Одни узлы могут использовать операционную

систему Linux, а другие — Windows. На одних узлах может работать процессор с архитектурой x86, а на других — с архитектурой Arm и т. д. Раньше мы могли размещать рабочие нагрузки на машинах с определенными свойствами. Теперь же мы можем делать то же самое и в Kubernetes, используя селекторы узлов и правила предпочтения.

8.1.1. Селекторы узлов

Дифференциация свойств узлов в Kubernetes выполняется с помощью меток узлов. Настроить под на использование узлов с заданными свойствами можно с помощью селектора узлов или предпочтений узлов (node affinity); это просто разные способы определения того, какие метки (и, соответственно, свойства) узлов требуются вашему поду.

Допустим, что один из ваших подов должен работать на узле с Arm-процессором. Такие узлы получают общепринятую метку `kubernetes.io/arch: arm64` (подобные метки определяет сообщество разработчиков свободно распространяемого ПО, чтобы обеспечить единообразие при использовании разных облачных платформ). Используя селекторы или предпочтения узлов, мы указываем эту метку, чтобы быть уверенными в том, что под будет запускаться только на узлах с Arm-процессором. В листинге 8.1 показано, как сделать так, чтобы рабочая нагрузка выбирала узлы с архитектурой `arm64`, исключив вероятность распределения пода на узлы с другой архитектурой центрального процессора.

Листинг 8.1. Chapter08/8.1.1_NodeSelection/deploy_nodeselector.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      nodeSelector:
        kubernetes.io/arch: arm64
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
```

Выбор узлов
с архитектурой arm64

Немного более многословный способ определения того же требования сводится к использованию правил предпочтения узлов (листинг 8.2).

Листинг 8.2. Chapter08/8.1.1_NodeSelection/deploy_nodeaffinity.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/arch
                    operator: In
                    values:
                      - arm64
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5

```

Альтернативный
способ выбора узлов
с архитектурой arm64

Обе представленные выше конфигурации развертывания дают в итоге одинаковый результат, а именно обеспечивают размещение пода только на узлах с Arm-процессором (чтобы посмотреть, где в итоге оказался ваш под, вы можете запросить информацию о подах с помощью команды `kubectl get pods -o wide`, а затем проверить состояние узлов, используя команду `kubectl describe node $NODE_NAME | grep arch`). Преимуществом механизма предпочтения узлов является то, что он позволяет использовать более выразительную логику; более подробно об этом мы поговорим в следующем разделе.

Однако, запросив в своем шаблоне PodSpec метку узла с нужными вам свойствами, вы сделаете лишь первый необходимый шаг. Помимо этого, вам нужно еще как-то обеспечить реальное выделение узлов с такой функциональностью (то есть узлов, имеющих указанную вами метку). Выделение узлов и связанных с ними возможностей всегда осуществляется на уровне платформы. Если вы используете такую полностью управляемую платформу, как сервис GKE в режиме «Автопилот», то для получения узла с заданными свойствами будет достаточно определить селектор узлов с соответствующими метками, конечно, при условии, что платформа может предоставить такой узел. Если же вы работаете на более традиционной платформе Kubernetes, то вам придется дополнительно позаботиться о предоставлении таких узлов, например создав пул или группу узлов с нужными свойствами.

Узнать, какие узлы может предоставить ваш провайдер, можно в его официальной документации. Но если ваш кластер уже содержит узел с нужными вам параметрами, то вы можете просто отобразить сведения об этом узле и посмотреть, какие метки он позволяет использовать:

```
kubectl describe nodes
```

Например, для узла с Arm-процессором, входящего в кластер сервиса GKE, неполный список доступных меток выглядит так:

```
Labels:      cloud.google.com/compute-class=Scale-Out
             cloud.google.com/gke-boot-disk=pd-standard
             cloud.google.com/gke-container-runtime=containerd
             cloud.google.com/gke-image-streaming=true
             cloud.google.com/gke-max-pods-per-node=32
             cloud.google.com/gke-nodepool=nap-19wjaxds
             cloud.google.com/gke-os-distribution=cos
             cloud.google.com/machine-family=t2a
             kubernetes.io/arch=arm64
             kubernetes.io/hostname=gk3-autopilot-cluster-4-nap-19wja
             kubernetes.io/os=linux
             node.kubernetes.io/instance-type=t2a-standard-4
             node.kubernetes.io/masq-agent-ds-ready=true
             topology.gke.io/zone=us-central1-f
             topology.kubernetes.io/region=us-central1
             topology.kubernetes.io/zone=us-central1-f
```

Метка узла,
используемая
в листингах 8.1 и 8.2



8.1.2. Предпочтение и антипредпочтение узлов

Правила предпочтения узлов представляют собой очень выразительное средство определения конфигурации, которое позволяет не ограничиваться указанием нужного вам набора меток. Так, используя оператор In, вы можете задать список возможных значений. Если, например, требуется выбрать только узлы с архитектурой x86 или Arm, это можно сделать, предоставив список возможных значений с помощью оператора In (листинг 8.3).

Листинг 8.3. Chapter08/8.1.2_NodeAffinity/deploy_nodeaffinity_multi.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 6
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
```

```

спес:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/arch
                operator: In
                values:
                  - arm64
                  - amd64
    containers:
      - name: timeserver-container
        image: docker.io/wdenniss/timeserver:5
        resources:
          requests:
            cpu: 500m

```

Данный под может быть запущен либо на архитектуре arm64 (Arm), либо на архитектуре amd64 (x86)

Поле `nodeSelector`, используемое в листинге 8.1 (см. выше), позволяет указать несколько условий, при этом под распределяется на узел, если все эти условия выполняются. В отличие от этого, используемый здесь оператор `In` распределяет под на узел с любой из указанных меток, чего можно достичь только с помощью правил предпочтения узлов. Вы можете потребовать соблюдения нескольких условий, разместив несколько выражений в поле `matchExpressions`.

Используя в поле `operator` оператор `NotIn`, эти выражения можно превратить в правила антипредпочтения, *запрещающие* размещение пода на узлах с указанными метками (табл. 8.1).

Таблица 8.1. Описание операторов

Оператор	Описание
In	Значение метки узла должно совпадать с одним из указанных вариантов
NotIn	Значение метки узла не должно совпадать с указанными вариантами
Exists	На узле должна быть метка с указанным ключом (и любым значением)
DoesNotExist	На узле не должно быть метки с указанным ключом
Gt	Указанное значение должно быть больше, чем значение метки узла
Lt	Указанное значение должно быть меньше, чем значение метки узла

У правил предпочтения узлов есть еще одно преимущество: вы можете создать правила *предпочтительного*, а не *необходимого* выбора, таким образом выразив некоторый набор предпочтений. Например, если ваш контейнер поддерживает несколько архитектур, позволяя использовать и архитектуру x86, и архитектуру Arm, но вы хотите по возможности использовать архитектуру Arm (например, потому, что это обходится более дешево), это можно выразить следующим образом (листинг 8.4).

Листинг 8.4. Chapter08/8.1.2_NodeAffinity/deploy_nodeaffinity_preferred.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 6
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: kubernetes.io/arch
                    operator: In
                    values:
                      - arm64
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
          resources:
            requests:
              cpu: 500m

```

Предпочтительный
выбор узлов
с архитектурой arm64,
с использованием
любого другого узла
при недоступности
таких узлов

ВОЗМОЖНЫЕ ПРОБЛЕМЫ ПРИ ИСПОЛЬЗОВАНИИ ПРЕДПОЧТЕНИЙ

Использование оператора `preferredDuringSchedulingIgnoredDuringExecution` иногда приводит к неожиданным результатам. Механизм предпочтительного выбора обычно работает должным образом при наличии в кластере невыделенных вычислительных мощностей, но может некорректно взаимодействовать с автоскейлером кластера в том случае, когда требуется добавить дополнительный узел при отсутствии невыделенных мощностей предпочтительного типа. Если при этом в кластере еще будут оставаться узлы с невыделенными мощностями (непредпочтительного типа), то Kubernetes попытается распределить под на один из этих узлов, прежде чем добавлять дополнительные узлы предпочтительного типа.

Это объясняется тем, что отвечающий за распределение подов планировщик Kubernetes и предоставленный облачной платформой автоскейлер узлов (стандартный компонент облачной платформы, отвечающий за добавление дополнительных узлов) работают в какой-то мере независимо друг от друга. На уровне облачной платформы автоскейлер узлов стремится найти поды, которые находятся в состоянии ожидания и могут быть распределены после добавления дополнительных мощностей. Однако еще до того как автоскейлер успевает что-либо сделать, планировщик Kubernetes распределяет под на доступные мощности непредпочтительного типа.

При использовании облачной платформы вы обычно можете лишь запросить нужную вам функциональность и понадеяться на то, что платформа всегда будет способна предоставить вычислительные мощности необходимого типа.

8.1.3. Наложение на узлы ограничений, запрещающих распределение по умолчанию

При использовании группы узлов с особыми характеристиками часто требуется сделать так, чтобы поды не распределялись на эти узлы по умолчанию. Примером такой ситуации, в частности, может служить использование архитектуры Arm: не все образы контейнеров ее поддерживают, поэтому часто требуется настроить кластер так, чтобы поды распределялись на узлы с архитектурой Arm, только когда в определении рабочей нагрузки явно указано, что она поддерживает их. Еще одним примером подобной ситуации может служить использование узлов с графическим процессором, которые нужно зарезервировать лишь для тех подов, которые нуждаются в этих компонентах. Еще пример — узлы, которые используют режим спотового выделения ресурсов и могут быть остановлены в любой момент. Они подходят далеко не всем рабочим нагрузкам.

Теоретически в таком случае можно было бы заставить почти все поды избегать особых узлов, используя правило антипредпочтений узлов (то есть правило предпочтения узлов с оператором `NotIn`), но это потребовало бы слишком большого объема усилий. Вместо этого Kubernetes позволяет наложить на узел ограничение (taint), запрещающее распределять на него поды по умолчанию. При этом вы сначала накладываете такое ограничение на узлы с особыми свойствами, на которые поды не должны распределяться по умолчанию, а затем добавляете допуск (toleration), позволяющий обходить это ограничение, в шаблоне PodSpec тех рабочих нагрузок, которые можно запускать на этих узлах.

В качестве примера мы можем добавить ограничение на отдельные узлы и посмотреть, к чему это приведет. Вы никогда не будете делать так в продуктивной

среде, но это действие вполне уместно на этапе изучения возможностей платформы. Воспользуемся инструментом Minikube (с которым вы познакомились в главе 3) и наложим ограничение на один из узлов:

```
$ minikube start --nodes 3
Done! kubectl is now configured to use "minikube" cluster
```

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	77s	v1.24.3
minikube-m02	Ready	<none>	55s	v1.24.3
minikube-m03	NotReady	<none>	19s	v1.24.3

```
$ NODE_NAME=minikube-m02
$ kubectl taint nodes $NODE_NAME spot=true:NoSchedule
node/minikube-m02 tainted
```

СОВЕТ

Удалить наложенное ограничение можно с помощью команды `kubectl taint nodes $NODE_NAME spot-`.

В данном случае выражение `spot=true` содержит ключ и значение ограничения, которыми мы воспользуемся в дальнейшем при определении допуска, позволяющего подам обходить это ограничение. Ключевое слово `NoSchedule` определяет производимый ограничением эффект (который сводится к запрету распределения подов без допуска). Здесь доступны и другие варианты, но я не рекомендую вам их использовать. На первый взгляд, в некоторых случаях может быть полезным ключевое слово `PreferNoSchedule`, которое создает мягкий запрет на распределение. На самом же деле ограничение накладывается с целью не допустить распределения подов на определенные виды узлов, и, не позволяя добиться этой цели, мягкий запрет может только усложнить процесс отладки. Иногда лучше оставить нераспределенными те поды, которые нуждаются в выделении дополнительных ресурсов, чем допустить их распределение на особые узлы, имеющие ограничение, и тем самым вызвать ненужные проблемы.

При использовании облачной платформы Kubernetes вы вряд ли будете накладывать ограничение на отдельные узлы, как было сделано в представленном выше примере. Обычно ограничение накладывается сразу на целую группу узлов с одинаковыми характеристиками, поскольку одни узлы регулярно заменяются другими при выполнении операций обновления и устранения сбоев, и при наложении ограничений на отдельные узлы такие узлы рано или поздно будут удалены. Поищите в документации вашей облачной платформы информацию об API, позволяющих накладывать ограничения так, чтобы они охватывали целую группу узлов и не прекращали свое действие после обновлений.

НАЛОЖЕНИЕ ОГРАНИЧЕНИЙ НА УЗЛЫ ПРИ ИСПОЛЬЗОВАНИИ СЕРВИСА GKE

При использовании сервиса GKE в режиме «Автопилот» наложение ограничений на узлы выполняется в полностью автоматическом режиме. Когда вы выбираете узлы с каким-то особым (не используемым по умолчанию) параметром, например, узлы, использующие спотовый режим или архитектуру Arm, выделяемые вам узлы этого типа автоматически получают ограничение. Поды тоже автоматически получают допуск, позволяющий обойти это ограничение, и поэтому вам остается лишь выбрать необходимый параметр узла. Автоматическое изменение подов осуществляется с помощью контроллера доступа (см. главу 12), который устанавливается и контролируется облачной платформой.

При использовании кластеров сервиса GKE с пулами узлов вы можете накладывать ограничение на пулы узлов при их создании. Например, при создании пула узлов, содержащего виртуальные машины, вы можете наложить ограничение на все эти узлы, как показано ниже:

```
gcloud container node-pools create $NODE_POOL_NAME --cluster $CLUSTER_NAME \
  --spot --node-taints spot=true:NoSchedule
```

Если кластер будет состоять исключительно из узлов, использующих спотовый режим, это ограничение будет не нужно, поскольку в таком случае не придется проводить различие между узлами.

Если вы попытаетесь распределить рабочую нагрузку после наложения ограничений на отдельные узлы, то увидите, что она не будет распределена на эти узлы (посмотреть, на каких узлах в итоге окажутся поды, можно с помощью команды `kubectl get pods -o wide`). Чтобы рабочая нагрузка могла быть распределена на получивший ограничение узел, ей необходимо выдать допуск, позволяющий обходить это ограничение (листинг 8.5).

Листинг 8.5. Chapter08/8.1.3_Taints/deploy_tolerate_spot.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
```

```

pod: timeserver-pod
spec:
  tolerations:
    - key: spot
      value: "true"
  containers:
    - name: timeserver-container
      image: docker.io/wdenniss/timeserver:5

```

Данная рабочая нагрузка получила допуск, позволяющий обходить ограничение `spot=true`, и может быть распределена на узел, имеющий такое ограничение

Выдача допуска подам не ведет к их обязательному распределению на узлы с ограничением, а лишь *делает это возможным*. На каком именно узле в итоге окажется такой под, зависит от многих факторов, в том числе от количества доступных на узле ресурсов. То есть поды с допуском могут быть размещены и на узлах, не имеющих ограничения, и на узлах с ограничением, которое они могут обходить (рис. 8.1).



Рис. 8.1. Данный кластер содержит два стандартных узла и спотовую виртуальную машину с пониженной степенью доступности. Поды пакетной рабочей нагрузки получили допуск, позволяющий использовать спотовые узлы, и могут быть распределены на оба вида узлов. В отличие от этого, поды приложения не получили допуск и могут быть распределены только на узлы, не имеющие ограничения

Ограничения и допуски часто используются в сочетании с селекторами узлов или правилами предпочтений узлов, чтобы определенный вид узлов мог применяться только определенным набором подов. Хороший пример такой ситуации — использование графических рабочих нагрузок: они должны запускаться только на узлах с графическим процессором, и эти дорогостоящие ресурсы не следует растрачивать на неграфические рабочие нагрузки (рис. 8.2).

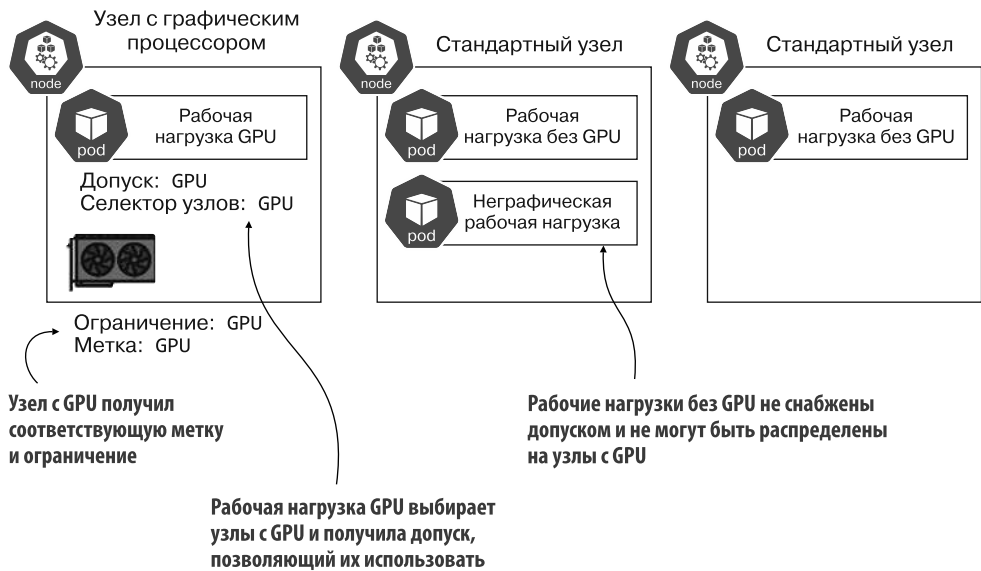


Рис. 8.2. Данный кластер содержит два стандартных узла и узел с графическим процессором. Этот узел получил ограничение, запрещающее распределение на него стандартных рабочих нагрузок. Рабочая нагрузка GPU получила допуск, позволяющий обходить это ограничение, и с помощью селектора узлов исключает вероятность ее распределения на другие узлы

Допуск, позволяющий обходить все ограничения

Некоторые рабочие нагрузки (в частности, наборы демонов (DaemonSets), о которых мы поговорим в главе 12) должны запускаться на каждом узле и поэтому должны разрабатываться с расчетом на все виды конфигураций, используемых в кластере. Такие рабочие нагрузки обычно получают допуск, позволяющий обходить все ограничения (листинг 8.6).

Листинг 8.6. Chapter08/8.1.3_Taints/daemonset_tolerate_all_taints.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: example-ds
spec:
  selector:
    matchLabels:
      pod: example-pod
  template:
    metadata:
      labels:
        pod: example-pod
```

```

спес:
  tolerations:
    - effect: NoExecute
      operator: Exists
    - effect: NoSchedule
      operator: Exists
  containers:
    - image: ubuntu
      command: ["sleep", "infinity"]
      name: ubuntu-container

```

Допуск, позволяющий
обходить все
ограничения

При использовании такого допуска следует иметь в виду, что получивший его под должен работать на всех типах узлов, которые используются в кластере сейчас и будут использоваться в дальнейшем. Это может стать проблемой при добавлении узлов с особыми характеристиками, например, узлов с Arm-процессором, которые требуют использования контейнеров, специально предназначенных для архитектуры Arm. Допустим, вам потребуется обеспечить распределение пода на все виды узлов, невзирая на ограничения, *за исключением* узлов с определенной меткой, например узлов с Arm-процессором. Вы сможете сделать это, используя универсальный допуск в сочетании с правилом антипредпочтения (листинг 8.7).

Листинг 8.7. Chapter08/8.1.3_Taints/daemonset_tolerate_antiaffinity.yaml

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: example-ds
spec:
  selector:
    matchLabels:
      pod: example-pod
  template:
    metadata:
      labels:
        pod: example-pod
    spec:
      tolerations:
        - effect: NoExecute
          operator: Exists
        - effect: NoSchedule
          operator: Exists
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/arch
                    operator: NotIn
                    values:
                      - arm64

```

Допускается наличие
любых ограничений...

...но запрещается
распределение на узлы
с Arm-процессором

```
containers:
- image: ubuntu
  command: ["sleep", "infinity"]
  name: ubuntu-container
```

8.1.4. Разделение рабочих нагрузок

Ограничения, допуски и селекторы узлов могут использоваться и для разделения рабочих нагрузок. В рассмотренных выше примерах выбор узлов выполнялся исключительно в целях получения узлов с определенными особенностями, такими как использование архитектуры Arm, спотовых вычислений или графического процессора, но это можно делать и для того, чтобы отделить рабочие нагрузки друг от друга путем их размещения на разных узлах.

Вы можете исключить вероятность совместного размещения определенных подов, используя правило антипредпочтения подов (как будет показано в подразделе 8.2.3), но иногда лучше просто размещать рабочие нагрузки в отдельных группах узлов.

О необходимости такого разделения мне, в частности, много раз приходилось слышать от людей, которым требовалось запускать пакетные рабочие нагрузки, содержащие два вида подов: координирующие, которые управляют выполнением работы, и рабочие, непосредственно выполняющие работу. Эти два вида подов лучше размещать на отдельных узлах, чтобы более скачкообразный процесс автомасштабирования узлов для рабочих подов не влиял на более стабильный процесс автомасштабирования узлов для координирующих подов. Еще один пример — «проблема шумного соседа», при которой два пода могут конкурировать за ресурсы в случае их размещения на одном узле и будут работать лучше, если разместить их на разных узлах.

Обеспечить разделение рабочих нагрузок можно, объединив ранее рассмотренные методы и определив собственную метку узла. Узел получает метку и ограничение, а рабочая нагрузка — допуск и селектор с той же меткой. В совокупности это объединение делает возможным распределение рабочей нагрузки на собственную группу узлов (которая, в принципе, может использоваться совместно с другими рабочими нагрузками, получившими такие же селектор и допуск).

В листинге 8.8 представлен пример того, как обеспечить разделение рабочих нагрузок, добавив в развертывание произвольно выбранный допуск и селектор узлов. Для удобства и в допуске, и в селекторе узлов используются одинаковые пары «ключ — значение» ("group=1"), однако не следует забывать, что они представляют разные концепции платформы Kubernetes.

Листинг 8.8. Chapter08/8.1.4_WorkloadSeparation/deploy_group1.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver1
```

```

spec:
  replicas: 5
  selector:
    matchLabels:
      pod: timeserver1-pod
  template:
    metadata:
      labels:
        pod: timeserver1-pod
  spec:
    tolerations:
      - key: group
        operator: Equal
        value: "1"
        effect: NoSchedule
        nodeSelector:
          group: "1"
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5

```

Допуск, позволяющий
обходить ограничение
group=1

Выбор метки
group=1

В демонстрационных целях можно скопировать это развертывание в файл `deploy_group2.yaml` и использовать в копии допуск и селектор с другой парой «ключ — значение» ("group=2"):

```

tolerations:
  - key: group
    operator: Equal
    value: "2"
    effect: NoSchedule
  nodeSelector:
    group: "2"

```

Допуск, позволяющий
обходить ограничение
group=2

Выбор метки
group=2

Чтобы поды этих развертываний развертывались на отдельные наборы узлов, необходимо иметь узлы с ограничением, которое запрещает распределение на них других подов, и метку, позволяющую выбирать их нашим рабочим нагрузкам. Если узлы не получают метку, эти развертывания вообще не будут распределены, поскольку не будет ни одного узла, удовлетворяющего требованию селектора узлов. Если узлы получают метку, но не ограничение, эти рабочие нагрузки будут распределены и отделены друг от друга за счет использования селектора узлов. Однако на тех же узлах могут оказаться и любые другие поды, поскольку их не будет удерживать от этого ограничение.

Если используемая вами система Kubernetes возлагает ответственность за управление узлами на пользователя, то для разделения рабочих нагрузок вам также потребуется предоставить узлы с соответствующими ограничениями и метками. В качестве примера мы можем создать кластер Minikube и напрямую добавить

в узлы ограничения и метки. Однако имейте в виду, что при использовании управляемой платформы такие манипуляции выполняются на уровне пула или группы узлов, при предоставлении узлов с помощью API платформы, поэтому вам нужно будет посмотреть, с помощью каких параметров в этом API определяются метки и ограничения.

```
$ minikube start --nodes 3 ← Создаем новый кластер Minikube
```



```
$ kubectl get nodes ← Отображаем информацию об узлах
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	67s	v1.24.3
minikube-m02	Ready	<none>	46s	v1.24.3
minikube-m03	Ready	<none>	24s	v1.24.3


```
$ kubectl taint nodes minikube-m02 group=1:NoSchedule
```

```
$ kubectl label node minikube-m02 group=1
```



```
$ kubectl taint nodes minikube-m03 group=2:NoSchedule
```

```
$ kubectl label node minikube-m03 group=2
```

Снабжаем узел m02 меткой и ограничением для его использования группой 1

Снабжаем узел m03 меткой и ограничением для его использования группой 2

Мы должны добавить в узлы и ограничение, и метку (подобно тому, как ранее мы добавили в развертывание и допуск, и селектор узлов), поскольку они служат разным целям. Ограничение запрещает размещение на узле рабочих нагрузок, которые не имеют соответствующего допуска, а метка позволяет обеспечить гарантированное размещение на узле рабочей нагрузки, исключив вероятность ее попадания на другие узлы (например, не имеющие ограничений). Для удобства я использовал и в ограничении, и в метке одинаковые пары «ключ — значение» ("group=1"), но в принципе при желании я мог бы использовать здесь и разные пары.

РАЗДЕЛЕНИЕ РАБОЧИХ НАГРУЗОК ПРИ ИСПОЛЬЗОВАНИИ СЕРВИСА GKE В РЕЖИМЕ «АВТОПИЛОТ»

Если вы развернете представленную выше рабочую нагрузку, используя сервис GKE в режиме «Автопилот», то узлы с соответствующими метками и ограничениями будут предоставлены автоматически! Это объясняется тем, что данная платформа является безузловой и исполняет определенные в подах требования. Помимо прочего, сюда входит и предоставление подходящих узлов, поэтому вам не требуется делать что-либо еще. При использовании традиционной платформы Kubernetes, где вы сами управляете узлами, вам потребуется создавать подходящие узлы самим.

Теперь, когда у нас есть настроенный кластер, развернем наши объекты Deployment, имеющие разделение рабочих нагрузок, и посмотрим, на какие узлы при этом будут распределены поды:

```
$ kubectl create -f Chapter08/8.1.4_WorkloadSeparation
deployment.apps/timeserver1 created
deployment.apps/timeserver2 created
```

							Поды развертывания timeserver1 запускаются на узле minikube-m02
NAME	READY	STATUS	RESTARTS	AGE	NODE		
timeserver1-75b69b5795-9n7ds	1/1	Running	0	2m2s	minikube-m02		
timeserver1-75b69b5795-kft64	1/1	Running	0	2m2s	minikube-m02		
timeserver1-75b69b5795-mnc4j	1/1	Running	0	2m2s	minikube-m02		
timeserver1-75b69b5795-msg9v	1/1	Running	0	2m2s	minikube-m02		
timeserver1-75b69b5795-r8r9t	1/1	Running	0	2m2s	minikube-m02		
timeserver2-6cbf875b6b-6wm7w	1/1	Running	0	2m2s	minikube-m03		
timeserver2-6cbf875b6b-dtnhm	1/1	Running	0	2m2s	minikube-m03		
timeserver2-6cbf875b6b-fd6vh	1/1	Running	0	2m2s	minikube-m03		
timeserver2-6cbf875b6b-q6fk8	1/1	Running	0	2m2s	minikube-m03		
timeserver2-6cbf875b6b-zvk72	1/1	Running	0	2m2s	minikube-m03		

Поды развертывания timeserver2
запускаются на узле minikube-m03

Завершив работу с кластером Minikube, удалите его:

```
minikube delete
```

8.2. РАЗМЕЩЕНИЕ ПОДОВ

Учитывая, что любой из ваших подов может потребовать перезапуска из-за утечки памяти или неудачного результата проверки работоспособности, рекомендуется всегда использовать несколько реплик пода. Однако важно понимать, что степень доступности зависит не только от того, сколько реплик вы используете (о чем мы подробно говорили в подразделе 5.2.4), но и от того, *где* размещаются эти поды.

Если у вас будет десять реплик пода, но все они будут находиться на одном узле, то ваш под перестанет работать при выходе из строя данного узла. Применительно к типичной облачной топологии это означает следующее: если все ваши *узлы* будут находиться в одной зоне доступности, то вы столкнетесь с проблемами при аварийном ее отключении. Сколько времени и денег следует потратить для предотвращения подобных ситуаций, вы должны решить сами, исходя из того, какую степень доступности вам нужно обеспечить и каким бюджетом вы располагаете, ведь, как известно, нет предела совершенству (например, для сверхнадежности можно выполнить развертывание на нескольких облачных платформах).

В данном разделе я ограничусь описанием нескольких наиболее разумных и мало-затратных стратегий распределения подов по уже имеющимся узлам. Используя эти стратегии, вы можете обеспечить некоторое увеличение степени доступности, не неся при этом дополнительных расходов.

8.2.1. Создание высокодоступных развертываний

Ранее мы уже говорили о том, что при размещении подов на узлах необходимо учитывать запросы ресурсов. Однако в ходе этого процесса следует учитывать и другие факторы. Сделать ваше приложение высокодоступным вы можете, позаботившись о том, чтобы все его реплики не были размещены на одном узле. Допустим, вы используете три реплики небольшого пода (которому требуется 100 миллиард процессора и 100 МиБ памяти). Они легко могут поместиться на одном узле, но в случае его сбоя ваше развертывание станет недоступным.

Будет лучше, если планировщик распределит эти реплики по всему кластеру. К счастью, Kubernetes позволяет добиться этого с помощью наложения *ограничений на топологию распределения* (рис. 8.3). Такое ограничение обеспечивает распределение узлов по нескольким доменам отказов, которые могут представлять собой отдельный узел или целую зону; вдобавок можно использовать сразу несколько ограничений, чтобы обеспечить одновременное распределение по узлам, зонам и другим видам доменов отказов, определенным вашим провайдером.

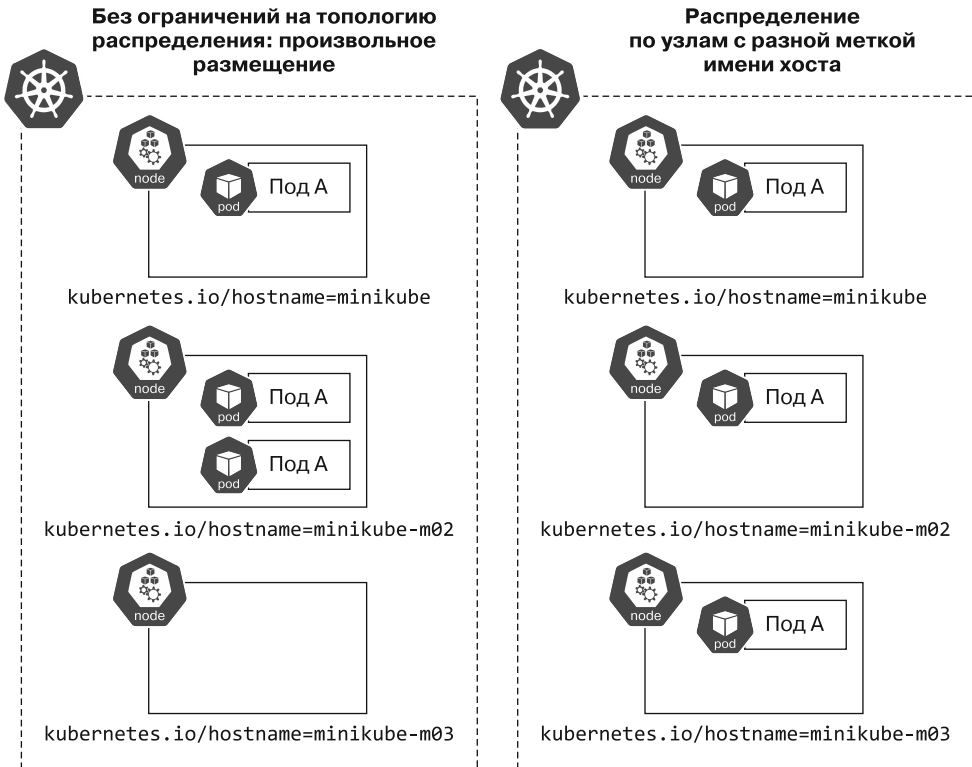


Рис. 8.3. Размещение подов рабочей нагрузки с использованием и без использования ограничений на топологию распределения

ПРИМЕЧАНИЕ

Многие провайдеры Kubernetes, в том числе сервис GKE, используют определенную топологию распределения по умолчанию. Если вы уверены, что используемый по умолчанию способ распределения будет хорошо работать в большинстве случаев, то можете пропустить данный подраздел. Тем не менее я посчитал необходимым привести здесь эту информацию, поскольку уверен, что вы должны понимать, почему вещи происходят тем или иным образом, в том числе и то, почему поды распределяются по разным узлам. Кроме того, с помощью изложенных в этой главе подходов вы можете изменить политики, используемые по умолчанию, например, чтобы использовать более строгие ограничения в случае критически важных развертываний. Либо же эти подходы помогут вам обеспечить применение топологии распределения к тем объектам, к которым она не применяется по умолчанию, например к объектам Job (о которых мы поговорим в главе 10).

Чтобы переопределить топологию распределения для определенной рабочей нагрузки, вы можете добавить поле `topologySpreadConstraints`, как это сделал я в листинге 8.9.

Листинг 8.9. Chapter08/8.2.1_TopologySpread/deploy_topology.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              pod: timeserver-pod
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
          resources:
            requests:
              cpu: 200m
              memory: 250Mi
```

Добавленное ограничение на топологию

Максимальный уровень дисбаланса реплик

Используемая в топологии метка узла

Дополнительный селектор меток, настроенный на метку метаданных данного шаблона

Действие, выполняемое в том случае, когда невозможно удовлетворить требование топологии

В данном примере мы с помощью параметра `topologyKey` настраиваем развертывание на использование топологии `kubernetes.io/hostname`. Это означает, что Kubernetes будет считать одинаковыми ключи топологии для узлов с одинаковым значением метки `kubernetes.io/hostname`. Обычно каждый узел обладает уникальным именем хоста, и благодаря этому становится возможным распределение по отдельным узлам.

Важно отметить: чтобы эта конфигурация работала, необходимо проследить за тем, чтобы в узлах кластера действительно имелась метка, указанная в поле `topologyKey` (в данном случае метка `kubernetes.io/hostname`). Существует несколько часто используемых меток, одну из которых я использовал здесь, однако нет никакой гарантии того, что эти метки будет использовать ваша платформа Kubernetes. Поэтому лучше запустите команду `kubectl describe node` и посмотрите, какие метки есть у ваших узлов.

Далее следует параметр `maxSkew`, которому я присвоил минимально возможное значение 1. Это означает, что уровень дисбаланса не может быть больше единицы, то есть количество подов, размещенных на любом узле, может быть лишь на единицу больше по сравнению с другими узлами.

Параметр `whenUnsatisfiable` определяет, какие действия будут выполняться в том случае, если ограничение будет невозможно выполнить (например, если узел будет до отказа заполнен другими подами). Здесь возможны два варианта: `ScheduleAnyway` и `DoNotSchedule`, смысл которых очевиден без каких-либо объяснений. Вариант `DoNotSchedule` может пригодиться на этапе тестирования, когда нужно посмотреть, работает ли правило, но в продуктивной среде безопаснее использовать вариант `ScheduleAnyway`. Вариант `ScheduleAnyway` «смягчает» правило, но при этом Kubernetes все равно делает все возможное для удовлетворения ваших требований, что, на мой взгляд, лучше по сравнению с тем, чтобы вообще не распределять реплику, особенно если вы хотите обеспечить максимально возможную степень доступности.

Наконец, последним параметром является поле `labelSelector` с дочерним элементом `matchLabels`, с которым мы уже знакомы начиная с главы 3. Здесь немного удивляет, почему Kubernetes не может выполнить простейшую операцию сверки с уже имеющейся информацией: зачем вообще нужен этот элемент, если мы уже добавили такой же элемент в определение пода? Как бы то ни было, данный элемент `matchLabels` должен выглядеть точно так же, как элемент `matchLabels`, добавленный в определение развертывания.

Теперь попробуем развернуть этот пример и посмотрим, будут ли поды размещены так, как мы ожидаем. Для этой демонстрации потребуется кластер, который будет содержать несколько узлов и не будет использовать какую-либо стратегию распределения по умолчанию. Сервис GKE по умолчанию выполняет распределение по узлам и зонам, поэтому при использовании данной платформы вам фактически не потребуется выполнять такую настройку. Но даже в этом случае будет полезно знать, как платформа реализует данный способ распределения, чтобы при необходимости в него можно было внести некоторые коррективы. Чтобы опробовать

244 Часть II. Переходим в продуктивную среду

данный пример и посмотреть, чем отличаются друг от друга разные топологии, я предлагаю использовать кластер Minikube с тремя узлами:

```
minikube start --nodes 3
cd Chapter08/8.2.1_TopologySpread
kubectl create -f deploy_topology.yaml
kubectl get pods -o wide
```

Содержимое столбца **NODE** должно показывать (рис. 8.4), что ваши поды размещены на трех отдельных узлах (при условии, что используемый вами кластер состоит из трех узлов).

```
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
timeserver-754665f45f-6n6bc	1/1	Running	0	39s	10.244.2.46	minikube-m03
timeserver-754665f45f-dkvvp	1/1	Running	0	39s	10.244.0.27	minikube
timeserver-754665f45f-dmqpq	1/1	Running	0	39s	10.244.1.47	minikube-m02

Рис. 8.4. Результат использования развертывания с элементом `topologySpreadConstraints`. Выделенный рамкой столбец показывает, что поды размещены на разных узлах

ПРИМЕЧАНИЕ

Ограничения на топологию распределения действуют только на этапе планирования, то есть при размещении подов на узлах. Если вы измените топологию (например, добавите еще один узел) уже после того, как будут запущены все реплики, то данное изменение не приведет к перемещению уже запущенных подов. При необходимости вы можете заново развернуть поды, обновив развертывание, что приведет к повторному применению правил распределения с учетом изменений в топологии.

Представленная на рис. 8.4 информация о состоянии подов показывает, что каждый из трех подов был распределен на отдельный узел. Для сравнения попробуйте развернуть то же развертывание, убрав из него поле `topologySpreadConstraints`, и увидите, что при этом все поды могут оказаться на одном узле. Если же поды распределяются по отдельным узлам даже без настройки топологии, то это, скорее всего, объясняется тем, что кластер использует такой способ распределения по умолчанию.

Поле `topologySpreadConstraints` можно использовать с любой меткой узла, поэтому еще одна распространенная стратегия сводится к тому, чтобы распределять рабочие нагрузки по разным зонам (если ваш кластер содержит несколько зон). Чтобы опробовать этот способ, вы можете еще раз развернуть рассмотренный выше пример, на этот раз используя зональный ключ, который в стандартном общепринятом виде выглядит как `topology.kubernetes.io/zone`. При этом опять же нужно проследить, чтобы у вас действительно были узлы с такой меткой; в противном случае в зависимости от того, как было настроено поле `whenUnsatisfiable` (Если невыполнимо), ее использование не даст никакого эффекта или приведет к тому, что рабочие нагрузки вообще не будут распределены. В поле `topologySpreadConstraints` можно в виде массива определить сразу несколько топологий, что позволяет обеспечить распределение и по узлам, и по зонам.

8.2.2. Совместное размещение взаимозависимых подов

В некоторых случаях у вас будет несколько тесно связанных друг с другом подов, которые лучше разместить на одной физической машине (рис. 8.5). В частности, для использования этой архитектуры хорошо подходят чрезвычайно «болтливые» сервисы (которые делают много межсервисных вызовов). Допустим, у вас есть фронтенд и бэкэнд, которые активно взаимодействуют друг с другом, и вы хотите обеспечить их размещение на одном узле, чтобы уменьшить сетевую задержку и объем сетевого трафика.

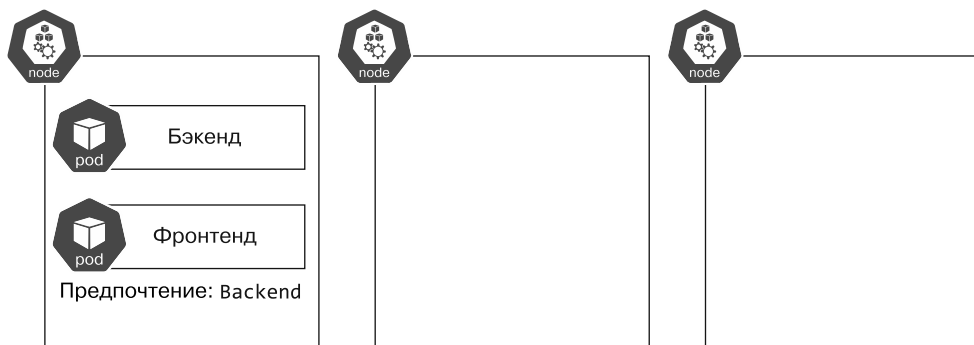


Рис. 8.5. С помощью правил предпочтения подов обеспечено размещение подов фронтенда на одном узле с подом бэкэнда

Данный способ разворачивания реализуется с помощью правил предпочтения подов. При этом вы просто добавляете в одно из разворачиваний — пусть в данном случае это будет фронтенд — правило предпочтения, которое предписывает планировщику размещать данные поды только на узлах, содержащих какой-то другой под, в данном случае под бэкэнда. Пусть, например, разворачивание бэкэнда выглядит следующим образом (листинг 8.10).

Листинг 8.10. Chapter08/8.2.2_Colocation/backend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: mariadb-pod
  template:
    metadata:
      labels:
        pod: mariadb-pod
    spec:
```

Метка, которая будет использоваться
в правиле предпочтения

```
containers:
- name: mariadb-container
  image: mariadb
  env:
  - name: MARIADB_RANDOM_ROOT_PASSWORD
    value: "1"
```

В данном развертывании нет ничего необычного; здесь все сделано так же, как мы делали до сих пор. Данный под будет размещен в любом из доступных в кластере свободных мест.

Теперь в развертывании фронтенда необходимо потребовать, чтобы его поды размещались только на узлах, содержащих под развертывания бэкенда. Это можно сделать, определив правило предпочтения подов (листинг 8.11).

Листинг 8.11. Chapter08/8.2.2_Colocation/frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
      - name: timeserver-container
        image: docker.io/wdenniss/timeserver:5
        affinity:
          podAffinity:
            requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                - key: pod
                  operator: In
                  values:
                  - mariadb-pod
              topologyKey: "kubernetes.io/hostname"
```

Правило
предпочтения
подов

Данное развертывание дает планировщику указание разместить его под в пределах заданной топологии на узле, содержащем запущенный под с меткой `pod: mariadb-pod`. Если вы попытаете создать эти два развертывания в таком же кластере Minikube, какой мы использовали в предыдущем разделе, то увидите, что все четыре пода (одна реплика бэкенда и три реплики фронтенда) будут размещены на одном узле (на который изначально будет распределен под бэкенда). В этом примере применяется узловая топология (с использованием общепринятой метки имени

хоста), поэтому данное приложение будет распределено только на узел, содержащий указанный под. Если бы мы применили зональную топологию (с помощью упомянутой в подразделе 8.2.1 общепринятой зональной метки), то данный под был бы размещен на любом узле в зоне, содержащей запущенный под с указанной меткой.

Можно сделать такое совместное размещение мягким требованием (выполняемым по принципу предоставления лучшего из возможного), чтобы ваши поды подвергались распределению даже в том случае, когда оно не может быть выполнено. Для этого можно использовать элемент `preferredDuringSchedulingIgnoredDuringExecution` вместо элемента `requiredDuringSchedulingIgnoredDuringExecution`. При использовании правила предпочтения в определении потребуется добавить еще несколько полей, например, поле для весового коэффициента правила (с помощью которого определяется порядок приоритетности при использовании нескольких предпочтений).

Как видите, Kubernetes обеспечивает значительную степень гибкости, позволяя определять правила распределения подов как обязательные к исполнению или желательные и выбирать множество разных вариантов предпочитаемой топологии (наиболее распространенные из которых — узловая и зональная). Имеющаяся здесь широта выбора может даже дезориентировать. На мой взгляд, в большинстве случаев правила предпочтения подов *не стоит* применять сразу же; лучше отложить их использование до того момента, когда вам потребуется решить какую-то особую задачу (например, разместить поды на одном узле для уменьшения сетевой задержки).

8.2.3. Избегание определенных подов

В подразделе 8.2.1 мы говорили о том, как с помощью ограничений на топологию распределения можно обеспечить распределение по разным узлам подов *одного* развертывания, чтобы исключить возникновение единой точки отказа. А что делать, когда поды связаны друг с другом, но по тем же соображениям их лучше распределить по разным узлам и развертываются они по отдельности (что не позволяет использовать ограничения на топологию распределения)? Например, вы создали отдельные развертывания для бэкенда и кэширующего сервиса, и вам нужно, чтобы они были распределены по разным узлам.

В таком случае можно использовать правило антипредпочтения подов. Вы должны просто инвертировать правило предпочтения подов (с которым вы уже знакомы), чтобы поды распределялись по другим узлам (или иным элементам топологии) (листинг 8.12).

Все эти конструкции можно использовать совместно. Например, можно обеспечить распределение подов по разным узлам, используя ограничения на топологию, и внести в это некоторые коррективы с помощью правил предпочтения. Следите лишь за тем, чтобы ваши правила были выполнимыми; иначе ваши поды могут остаться нераспределенными. Как и в случае обычного правила предпочтения, описываемого в предыдущем разделе, вы всегда можете «смягчить» свое правило, используя элемент `preferredDuringSchedulingIgnoredDuringExecution` вместо элемента `requiredDuringSchedulingIgnoredDuringExecution`. При этом лучше сначала протестируйте

правило в его обязательной версии, чтобы убедиться, что вы правильно настроили поле `labelSelector`, и уже после этого переключайтесь к его «мягкой» версии. В следующем разделе я озвучу еще несколько советов по отладке таких правил.

Листинг 8.12. Chapter08/8.2.3_PodAntiAffinity/frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 3
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: pod
                    operator: In
                    values:
                      - mariadb-pod
              topologyKey: "kubernetes.io/hostname"
```

Мы инвертировали правило предпочтения подов из предыдущего примера, чтобы данный под явно избегал узлов, содержащих под с меткой `pod: mariadb-pod`

8.3. ОТЛАДКА ПРОБЛЕМ С РАЗМЕЩЕНИЕМ

Размещение подов — достаточно сложная задача, поэтому не удивляйтесь, если столкнетесь с проблемами. Наиболее частая ошибка — указание метки, которой нет ни у одного из узлов, или, в случае безузловой платформы, указание метки свойства узла, не поддерживаемого платформой. Такие поды остаются нераспределенными. В следующих двух подразделах я расскажу, с какими ошибками вы будете сталкиваться чаще всего и как их можно исправить.

8.3.1. Отказываются работать правила размещения

Если в ходе тестирования создается впечатление, что ваши правила размещения не работают, то в первую очередь следует проверить, не используете ли вы «мягкие» правила (то есть предпочтения). Планировщик игнорирует такие правила в случае их невыполнимости, поэтому убедиться в их работоспособности не так

просто. В связи с этим лучше всегда тестируйте свои правила в их обязательном виде еще до того, как сделаете их «мягкими».

Попробуйте провести развертывание в небольшом кластере, содержащем лишь несколько узлов, не используя при этом «мягких» правил. Это позволит увидеть, какой эффект производят конструкции, определяющие способ размещения подов. Чтобы убедиться в функциональности ваших правил, специально попытайтесь распределить не соответствующие им поды. Эти поды должны остаться в состоянии ожидания (Pending), поскольку наложенные вами ограничения будут для них невыполнимыми.

8.3.2. Поды остаются в состоянии ожидания

Если ваши поды остаются в состоянии ожидания (Pending), это говорит о том, что планировщик не может найти для них подходящее место. Мы уже сталкивались с этой проблемой в главе 3, когда говорили о ситуации, в которой у кластера недостаточно ресурсов для размещения пода. Вы определили правила размещения, поэтому не исключена вероятность того, что под не удастся распределить потому, что ваши правила будут для него невыполнимы. Чтобы выяснить, чем именно вызвана данная проблема (то есть какое правило не может быть выполнено), отобразите подробную информацию о поде с помощью команды `describe`. Это нужно сделать именно на уровне пода, поскольку, запросив информацию о развертывании, вы не увидите никаких сообщений об ошибках, а лишь заметите, что развернуто недостаточное количество реплик:

```
kubect1 get pods
kubect1 describe Pod $POD_NAME
```

В качестве примера ниже показано, как может выглядеть информация о поде в том случае, когда у него нет допуска, который позволяет обходить ограничение, наложенное на доступные узлы. В такой ситуации нужно либо дать поду соответствующий допуск, либо увеличить количество узлов, у которых нет ограничения:

```
Events:
  Type      Reason             Age          From                    Message
  ----      -
Warning    FailedScheduling   4s           default-scheduler       0/1 nodes
are available: 1 node(s) had taints that the pod didn't tolerate.
```

В следующем примере показано, какую информацию вы можете получить, когда для пода не могут быть выполнены правила предпочтения или антипредпочтения. В таком случае проверьте и исправьте свои правила, после чего попробуйте выполнить развертывание еще раз:

```

Type      Reason             Age          From                    Message
----      -
Warning    FailedScheduling   17s (x3 over 90s) default-scheduler       0/1 nodes
are available: 1 node(s) didn't match pod affinity/anti-affinity, 1 node(s)
didn't match pod anti-affinity rules.
```

РЕЗЮМЕ

- В определении своих подов вы можете указать, что они должны выбирать узлы с определенными аппаратными характеристиками или, наоборот, избегать их.
- В узлы с особыми аппаратными характеристиками можно добавить ограничение, которое запрещает распределять на них поды по умолчанию. Подам, которые должны запускаться на этих узлах, можно дать допуск, позволяющий обходить это ограничение.
- Совместно используя ограничения, допуски, метки узлов и селекторы, можно отделить друг от друга определенные рабочие нагрузки.
- Создавайте высокодоступные развертывания, используя несколько реплик и хорошо настроенную политику топологии распределения.
- Если определенные поды полезно разместить в одном месте, этого можно достичь с помощью правил предпочтения подов.
- Если определенные поды нежелательно размещать в одном месте, то вы можете исключить вероятность их совместного размещения, используя правила анти-предпочтения подов.



Stateful-приложения

В этой главе

- ✓ Конструкции Kubernetes, используемые для представления дисков и состояния.
- ✓ Подключение к подам постоянного хранилища.
- ✓ Использование объекта StatefulSet для развертывания многоподовых stateful-приложений с лидирующей ролью.
- ✓ Перемещение и восстановление данных с помощью переподключения объектов Kubernetes к дисковым ресурсам.
- ✓ Подключение к подам больших эфемерных томов.

Свое место в Kubernetes есть и для stateful-приложений (то есть рабочих нагрузок с подключенным хранилищем). Сейчас многие превозносят stateless-приложения за их преимущества: легкость развертывания, высокая масштабируемость, отсутствие необходимости подключать хранилище и обеспечивать контроль над ним. Тем не менее это совсем не значит, что stateful-приложения не имеют права на существование. И если вы хотите выполнить развертывание сложной базы данных или перенести старое stateful-приложение с виртуальной машины, то Kubernetes позволит вам легко справиться с этой задачей.

Используя постоянные тома, вы сможете добавить stateful-хранилище в любой ваш под. Что касается stateful-нагрузок с несколькими репликами, то для высокоуровневого управления stateless-приложением в Kubernetes можно использовать объект Deployment, а управлять stateful-приложением можно с помощью объекта StatefulSet.

9.1. ТОМА, ПОСТОЯННЫЕ ТОМА, ЗАЯВКИ И КЛАССЫ ХРАНЕНИЯ

Разговор о сохранении состояния в Kubernetes мы начнем с рассмотрения концепций этой платформы, связанных с управлением томами (дисками), а затем перейдем к высокоуровневой конструкции StatefulSet. Подобно тому как виртуальные машины Kubernetes представляет в виде узлов, свое представление эта система использует и для дисков.

9.1.1. Тома

Kubernetes предоставляет подам функциональность, позволяющую им смонтировать том. Что же такое *том*? В документации можно найти следующее описание.

По сути, том — это просто каталог, который доступен контейнерам пода и может содержать некоторые данные. Как создается этот каталог, на каком носителе он размещается и какие данные содержит, зависит от того, какой тип тома вы используете (<https://kubernetes.io/docs/concepts/storage/volumes/>).

Kubernetes предлагает несколько встроенных типов тома; кроме того, администратор вашей платформы может добавить дополнительные типы с помощью драйверов хранилищ. К наиболее часто используемым типам можно отнести:

- тома `emptyDir`, представляющие собой эфемерные тома, привязанные к жизненному циклу узла;
- тома `ConfigMap`, которые позволяют определить файлы в манифестах Kubernetes и предоставить их приложению в виде файла на диске;
- предоставляемые облачным провайдером диски для постоянного хранения.

Рассмотрим каждый тип тома более подробно.

Том `emptyDir`

Встроенный тип тома `emptyDir` служит для представления эфемерных томов, пространство для которых выделяется на загрузочном диске узла. Если под будет удален или перемещен на другой узел либо сам узел станет неработоспособным, то все содержимое такого тома будет утрачено. А в чем же преимущества этого типа?

Если под содержит несколько контейнеров, то смонтированный в нем том `emptyDir` может использоваться ими совместно. Поэтому, если вам нужно сделать данные доступными для всех контейнеров пода, это можно осуществить, определив и смонтировав том `emptyDir` для каждого контейнера пода (листинг 9.1). Содержимое такого тома сохраняется и при *перезапуске* контейнеров, хотя этого не происходит

в случае многих других ранее рассмотренных событий. Это удобно в случае таких временных данных, как дисковый кэш, когда выгодно сохранять данные при перезапуске пода, но не нужно обеспечивать их длительное хранение.

Листинг 9.1. Chapter09/9.1.1_Volume/emptydir_pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
  labels:
    pod: timeserver-pod
spec:
  containers:
    - name: timeserver-container
      image: docker.io/wdenniss/timeserver:5
      volumeMounts:
        - name: cache-volume
          mountPath: /app/cache/
  volumes:
    - name: cache-volume
      emptyDir: {}
```

Путь, в который монтируется каталог

Определение тома

Но почему же этот тип тома получил название `emptyDir`? Данные в нем сохраняются на узле в изначально пустом каталоге. На мой взгляд, это крайне неудачное название, но с этим уже ничего не поделаешь.

СОВЕТ

Если вам нужно предоставить под рабочую нагрузку временное рабочее пространство, то прочтите раздел 9.4, посвященный использованию универсальных эфемерных томов. Это более современный способ предоставления временного хранилища, при котором вы не ориентируетесь на размещаемые на хостах тома.

В качестве демонстрации того, как это выглядит на практике, в подразделе 9.2.2 будет представлен пример. В нем я покажу, как с помощью тома `emptyDir` обеспечить совместный доступ к данным для двух контейнеров одного пода, один из которых является контейнером инициализации, выполняющим необходимые операции настройки перед запуском основного контейнера.

Томы ConfigMap

Еще одна полезная конструкция — объекты `ConfigMap`. Они позволяют определить пары «ключ — значение» в одном месте, чтобы их можно было использовать из множества других объектов, причем так можно сохранять даже целые файлы! Обычно эти файлы представляют собой различные конфигурационные файлы наподобие файлов `my.cnf` для MariaDB, `httpd.conf` для Apache, `redis.conf` для Redis и т. д. Если вы смонтируете объект `ConfigMap` как том, то определяемые им файлы станут доступными для чтения из контейнера. Томы `ConfigMap` всегда доступны только для чтения.

С помощью данного подхода особенно удобно определять конфигурационный файл, используемый для образов контейнера из публичных репозиториях, поскольку вы можете предоставить конфигурацию, не дополняя сам образ. Например, если вам нужно использовать базу данных Redis, то вы можете воспользоваться ссылкой на официальный образ СУБД Redis и просто смонтировать собственный конфигурационный файл в виде тома ConfigMap в том месте, где СУБД Redis его ожидает, — нет никакого смысла создавать собственный образ лишь для того, чтобы добавить этот файл. Примеры того, как настроить базу данных Redis с помощью собственного конфигурационного файла, определенного в виде тома ConfigMap, будут представлены в подразделах 9.2.1 и 9.2.2.

Томы облачных провайдеров

Для *stateful*-приложений не слишком характерно использование эфемерных томов или томов, доступных только для чтения, поэтому более распространенный подход в их случае сводится к тому, чтобы смонтировать в виде томов диски облачного провайдера. На какой бы платформе вы ни использовали Kubernetes, ваш провайдер должен был разместить в кластере драйвера, позволяющие вам смонтировать постоянное хранилище: либо NFS, либо блочное (а часто оба эти типа хранилища).

В качестве примера в листинге 9.2 представлена спецификация пода с базой данных MariaDB, который запускается на платформе Google Kubernetes Engine (GKE). При этом в каталог `/var/lib/mysql` монтируется постоянный диск сервиса GCE, что позволяет обеспечить постоянное хранение данных по схеме, представленной на рис. 9.1.

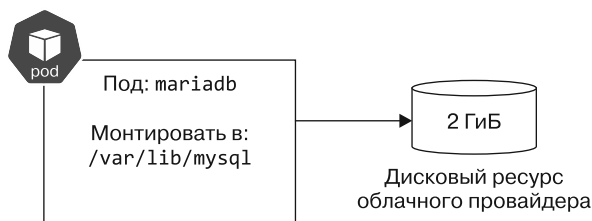


Рис. 9.1. Под со смонтированным томом облачного провайдера

В отличие от более автоматизированных и не зависящих от используемой облачной платформы подходов, которые мы рассмотрим далее, данный метод зависит от того, услугами какого облачного провайдера вы пользуетесь, и требует ручного создания диска. Вы должны убедиться, что диск с указанным именем существует (предварительно создав его за пределами кластера Kubernetes, используя инструменты вашего облачного провайдера) и что этот диск находится в той же зоне, что и ваш под. В данном случае я выбираю для пода зону с диском, используя элемент `nodeSelector`; это необходимо делать в любом кластере Kubernetes, содержащем несколько зон, чтобы под не был распределен в зону, отличную от зоны, в которой создан диск.

Листинг 9.2. Chapter09/9.1.1_Volume/mariadb_pod.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: mariadb-demo
  labels:
    app: mariadb
spec:
  nodeSelector:
    topology.kubernetes.io/zone: us-west1-a
  containers:
  - name: mariadb-container
    image: mariadb:latest
    volumeMounts:
    - mountPath: /var/lib/mysql
      name: mariadb-volume
    env:
    - name: MARIADB_ROOT_PASSWORD
      value: "your database password"
  volumes:
  - name: mariadb-volume
    gcePersistentDisk:
      pdName: mariadb-disk
      fsType: ext4

```

nodeSelector выбирает зону, в которой находится диск, чтобы под был создан в той же зоне

Каталог, в который монтируется диск

Имя ресурса платформы Google Cloud, представляющего постоянный диск

Создать диск за пределами системы в этом примере можно с помощью данной команды:

```
gcloud compute disks create --size=10GB --zone=us-west1-a mariadb-disk
```

ПРИМЕЧАНИЕ

Данный пример с его инструкциями, зависящими от платформы, приводится здесь лишь для полноты картины. Он показывает, как развивались подходы к использованию томов, но не является примером рекомендуемого подхода к их применению. Далее мы рассмотрим намного более эффективный, не зависящий от платформы способ создания дисков, который предполагает использование объектов PersistentVolume и StatefulSet.

Создавая диск вручную, обратите особое внимание на то, в каком месте создается ресурс. И в представленной выше команде, и в элементе nodeSelector должна быть указана одна и та же зона. Если вы заметите, что ваш под зависает в состоянии ContainerCreating, то попробуйте найти причину проблемы, просмотрев журнал событий. Вот, например, какое предупреждение я получил, когда создал диск не в той зоне:

```
$ kubectl get events -w
```

```

0s Warning FailedAttachVolume pod/mariadb-demo
AttachVolume.Attach failed for volume "mariadb-volume" : GCE persistent disk
not found: diskName="mariadb-disk" zone="us-west1-a"

```

Подход с ручным подключением томов имеет недостаток: эти диски приходится создавать за пределами Kubernetes. Это значит, что:

- пользователь, создающий под, должен иметь разрешение на создание диска, что не всегда соответствует истине;
- действия, которые не отражаются в конфигурации Kubernetes, требуется запоминать и выполнять вручную;
- дескрипторы томов являются платформенно-зависимыми, вследствие чего соответствующие YAML-конфигурации Kubernetes становятся непереносимыми и их нельзя использовать на другой платформе.

Естественно, у платформы Kubernetes есть средство, позволяющее решить эту проблему с переносимостью. Используя предлагаемые данной платформой абстракции для представления постоянных томов, вы можете просто запросить нужные вам дисковые ресурсы и затем получить их, не выполняя какие-либо действия за пределами системы. Об этом всем мы и поговорим далее.

9.1.2. Постоянные тома и заявки

Более платформенно-независимый способ управления томами в Kubernetes реализуется с помощью высокоуровневых примитивов *PersistentVolume* (PV) и *PersistentVolumeClaim* (PVC). Вместо того чтобы подключаться к тому напрямую, под ссылается на объект *PersistentVolumeClaim*, который определяет необходимые поду дисковые ресурсы в платформенно-независимых терминах (например: «место для хранения данных объемом 1 гигабайт»). Дисковые ресурсы в Kubernetes представлены объектами *PersistentVolume*, подобно тому, как виртуальные машины представлены как узлы. После того как заявка *PersistentVolumeClaim* будет создана, Kubernetes стремится предоставить запрашиваемые в ней ресурсы, создавая или находя соответствующий том *PersistentVolume* и связывая эти два объекта друг с другом (рис. 9.2). Как правило, будучи однажды связанными, PV и PVC остаются связанными до тех пор, пока не будет удален соответствующий диск.

Этот процесс сопоставления заявки на ресурсы с объектом, представляющим доступные ресурсы, сильно напоминает распределение подов, в ходе которого под запрашивает определенное количество ресурсов процессора и памяти и кластер находит для него узел с соответствующим их количеством. Такой подход дает возможность определять запросы на хранилище в платформо-независимым способом. В отличие от подхода с непосредственным подключением диска облачного провайдера, при использовании заявки *PersistentVolumeClaim* ваши поды могут быть развернуты на любой платформе, предоставляющей постоянное хранилище.

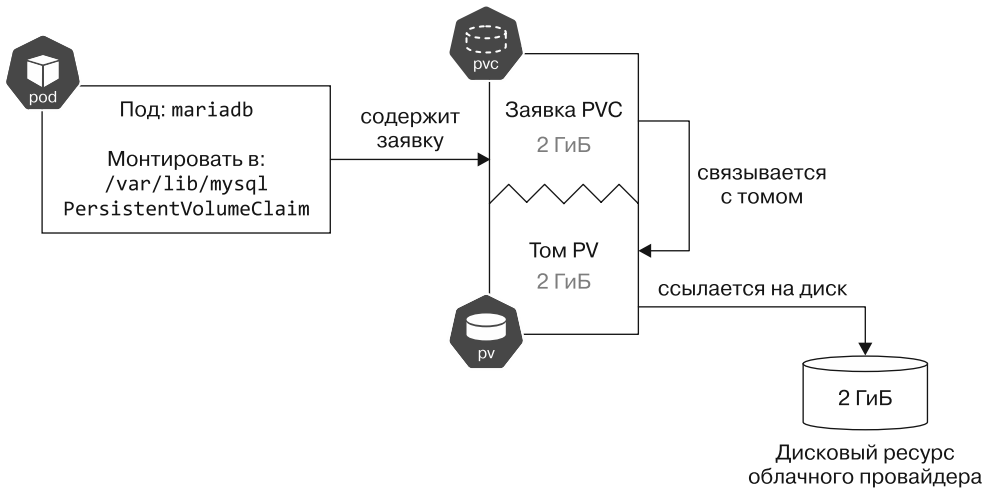


Рис. 9.2. Под ссылается на заявку `PersistentVolumeClaim`, которая привязывается к тому `PersistentVolume`, который ссылается на диск

Теперь перепишем определение пода из предыдущего подраздела так, чтобы он запрашивал для себя том `PersistentVolume` с помощью заявки `PersistentVolumeClaim`. Этот внешний диск будет монтироваться в каталог `/var/lib/mysql`, в котором сохраняет свои данные база данных MariaDB (листинг 9.3).

Листинг 9.3. `Chapter09/9.1.2_PersistentVolume/pvc-mariadb.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: mariadb-demo
  labels:
    app: mariadb
spec:
  containers:
    - name: mariadb-container
      image: mariadb:latest
      volumeMounts:
        - mountPath: /var/lib/mysql
          name: mariadb-volume
      resources:
        requests:
          cpu: 1
          memory: 4Gi
      env:
        - name: MARIADB_ROOT_PASSWORD
          value: "your database password"
```

Каталог базы данных MariaDB, в который будет смонтирован том, привязанный к PVC

Запрашиваемые подом вычислительные ресурсы

```

volumes:
- name: mariadb-volume
  persistentVolumeClaim:
    claimName: mariadb-pv-claim
---
apiVersion: v1
kind: PersistentVolumeClaim  ← Объект PersistentVolumeClaim
metadata:
  name: mariadb-pv-claim
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi

```

Вместо того чтобы ссылаться на дисковый ресурс, ссылаемся на объект PersistentVolumeClaim

Запрашиваемые подом ресурсы хранилища

В определении объекта PersistentVolumeClaim мы запрашиваем 2 ГиБ пространства в хранилище и указываем требуемый режим доступа (**accessMode**). Вариант **ReadWriteOnce**, используемый наиболее часто, предназначен для тех томов, которые ведут себя как традиционный жесткий диск, и позволяет монтировать том для чтения/записи в одном поде. Здесь возможны еще два варианта. Режим **ReadOnlyMany** служит для монтирования уже содержащего данные тома, который предоставляет нескольким подам доступ на чтение. А режим **ReadWriteMany** позволяет монтировать файловое хранилище (такое как NFS), в котором несколько подов могут выполнять чтение и запись одновременно (этот достаточно специфический режим поддерживается далеко не всеми драйверами хранилищ). Поскольку **stateful**-приложения используют традиционные блочные хранилища, то в этой главе мы будем использовать только режим **ReadWriteOnce**.

Если вы используете платформу с динамическим выделением ресурсов, то для удовлетворения запроса на хранилище, определенного в объекте PersistentVolumeClaim, будет автоматически создан объект PersistentVolume, представляющий дисковый ресурс, после чего эти два объекта будут связаны друг с другом. Поведение тома PersistentVolume при динамическом выделении ресурсов определяется с помощью StorageClass (Класс хранения), о котором мы поговорим в следующем подразделе. Сервис GKE и почти все остальные облачные платформы предоставляют возможность динамического выделения ресурсов с использованием класса хранения по умолчанию, поэтому представленный выше под из листинга 9.3 может быть развернут практически на любой платформе.

Если ваша платформа все же *не обеспечивает* динамическое выделение ресурсов, то вам (или администратору кластера) потребуется вручную создать том PersistentVolume, имеющий достаточно ресурсов для удовлетворения заявки PersistentVolumeClaim (рис. 9.3). После этого Kubernetes все так же автоматически свяжет заявку с созданным вручную томом PersistentVolume.

Объект PersistentVolumeClaim в том виде, в котором он определен в предыдущем примере, представляет собой заявку на предоставление ресурсов. Она

становится обязательным требованием позже, когда Kubernetes находит подходящий том PersistentVolume и привязывает PVC к PV. Таким образом, объект PersistentVolumeClaim является заявкой в начале своего жизненного цикла и становится требованием после связывания с томом.

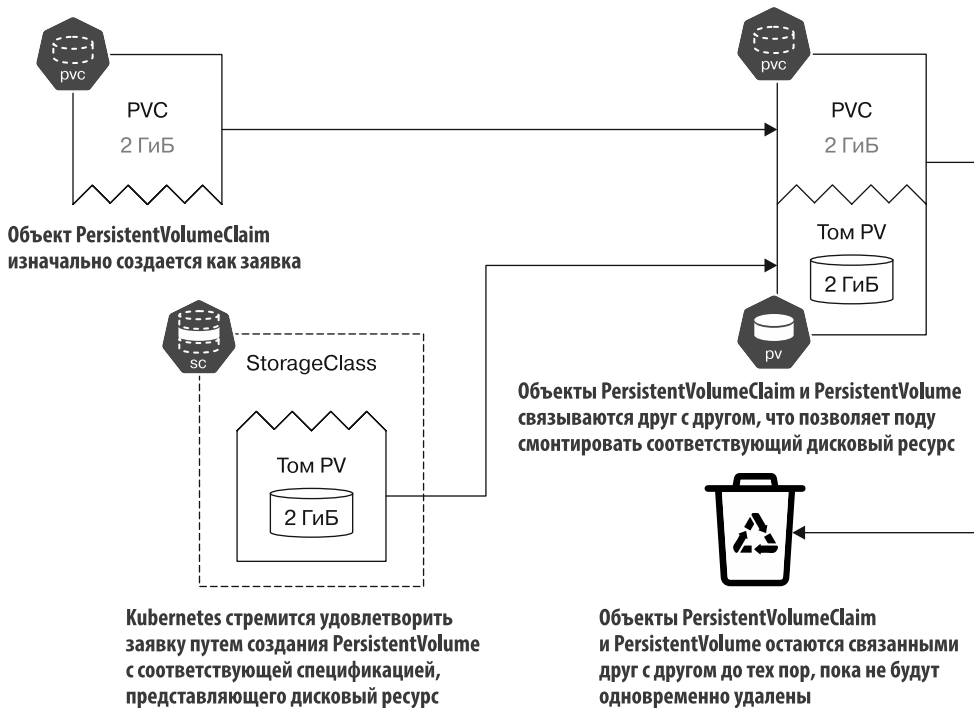


Рис. 9.3. Жизненный цикл объектов PersistentVolumeClaim и PersistentVolume в системе с динамическим выделением ресурсов

На этом можно было бы и остановиться, однако на этих дисках будут храниться ценные для вас данные, поэтому мы зайдем чуть дальше и посмотрим, как осуществляется эта операция связывания. Если вы создадите объект PersistentVolumeClaim (был представлен в листинге 9.3, см. выше) и затем посмотрите, как выглядит его YAML-код *после* выполнения операции связывания, то увидите, что в нем появилось поле `volumeName`, содержащее имя того тома PersistentVolume, который был привязан к данному объекту и теперь должен предоставляться в обязательном порядке. Вот как это может выглядеть на практике (я опустил часть информации, чтобы сделать код более читабельным):

```
$ kubectl create -f Chapter09/9.1.2_PersistentVolume/pvc-mariadb.yaml
pod/mariadb-demo created
persistentvolumeclaim/mariadb-pv-claim created
```

260 Часть II. Переходим в продуктивную среду

```
$ kubectl get -o yaml pvc/mariadb-pv-claim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mariadb-pv-claim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
  storageClassName: standard-rwo
  volumeMode: Filesystem
  volumeName: pvc-ecb0c9ed-9aee-44b2-a1e5-ff70d9d3823a
status:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 2Gi
  phase: Bound
```

Том PersistentVolume, с которым теперь связан данный объект

Если мы запросим информацию о том PersistentVolume, указанном в данной конфигурации, используя команду `kubectl get -o yaml pv $NAME`, то увидим, что он тоже связан с заявкой PVC. Вот как это выглядело в моем случае:

```
$ kubectl get -o yaml pv pvc-ecb0c9ed-9aee-44b2-a1e5-ff70d9d3823a
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pvc-ecb0c9ed-9aee-44b2-a1e5-ff70d9d3823a
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 2Gi
  claimRef:
    apiVersion: v1
    kind: PersistentVolumeClaim
    name: mariadb-pv-claim
    namespace: default
  csi:
    driver: pd.csi.storage.gke.io
    fsType: ext4
    volumeAttributes:
      storage.kubernetes.io/csiProvisionerIdentity: 1615534731524-8081-pd.csi.storage.gke.io
    volumeHandle:
      projects/gke-autopilot-test/zones/us-west1-b/disks/pvc-ecb0c9ed-9aee-44b2-a1e5-ff70d9d3823a
    persistentVolumeReclaimPolicy: Delete
    storageClassName: standard-rwo
    volumeMode: Filesystem
status:
  phase: Bound
```

Заявка PersistentVolumeClaim, с которой связан данный том PersistentVolume

Указатель на соответствующий дисковый ресурс

Том находится в состоянии Bound (Связано)

Чтобы составить более четкое представление об этих объектах, сравним их определения, разместив их рядом друг с другом (рис. 9.4).

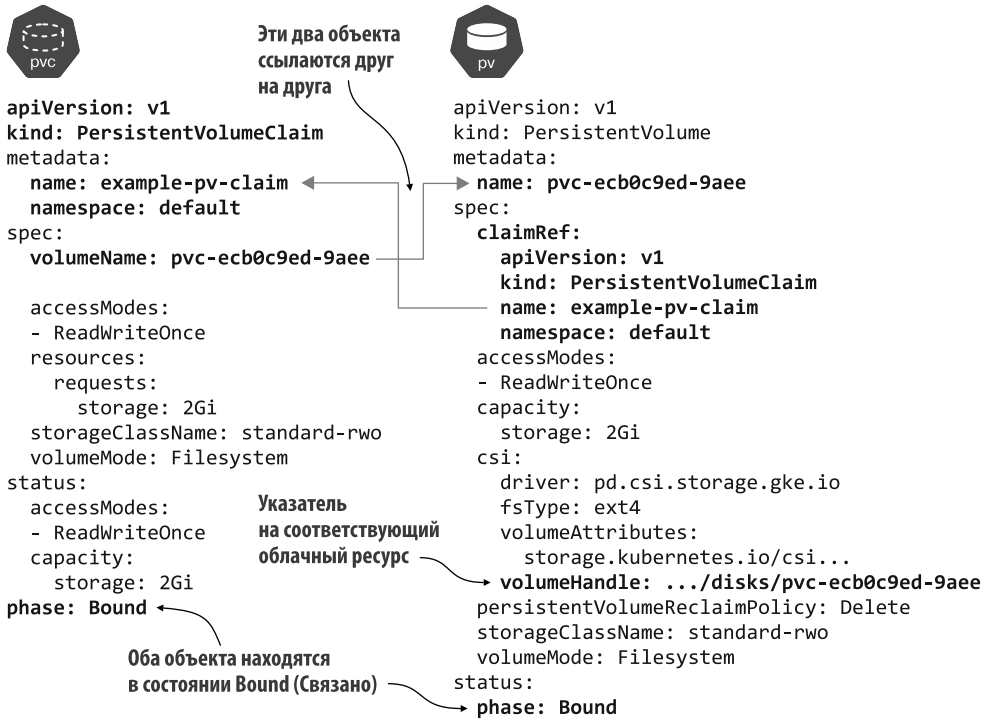


Рис. 9.4. Объекты PersistentVolumeClaim и PersistentVolume после предоставления тома и связывания этих объектов друг с другом

Объект PersistentVolumeClaim подвергся серьезным метаморфозам, превратившись из заявки на предоставление ресурсов в требование предоставить конкретный дисковый ресурс для сохранения ваших данных. Так не ведет себя ни один из известных мне других объектов Kubernetes. Платформа часто добавляет в объекты дополнительные поля и выполняет над ними определенные действия, однако они почти никогда не изменяются столь серьезно, как здесь, где создаваемая изначально универсальная заявка на ресурсы в итоге превращается в привязанный объект, имеющий сохраненное состояние.

Объект PersistentVolumeClaim не следует описанной выше схеме жизненного цикла лишь тогда, когда вам нужно подключить к поду некоторые уже существующие данные. В таком случае объекты PersistentVolumeClaim и PersistentVolume изначально указывают друг на друга и связываются уже на этапе создания. Данный сценарий будет рассмотрен в разделе 9.3, где мы поговорим о перемещении и восстановлении дисков и среди прочего рассмотрим полностью работающий пример восстановления данных.

ЛОКАЛЬНОЕ ТЕСТИРОВАНИЕ ПОДА С БАЗОЙ ДАННЫХ MARIADB

Вам нужно подключить базу данных MariaDB и убедиться, что все было настроено правильно? Никаких проблем. Просто перенаправьте порт контейнера базы данных mariadb на свою машину:

```
kubect1 port-forward pod/mariadb-demo 3306:3306
```

Затем подключитесь к контейнеру из своего локального клиента mysql. У вас нет такого клиента? Запустите контейнер с помощью Docker!

```
docker run --net=host -it --rm mariadb mariadb -h localhost -P 3306 -u root -p
```

Пароль базы данных будет указан в соответствующей переменной среды в определении пода (см. листинг 9.3 выше). Подключившись к базе данных, убедитесь в ее работоспособности, выполнив какой-нибудь SQL-запрос:

```
MariaDB [(none)]> SELECT user, host FROM mysql.user;
```

```
+-----+-----+
| User      | Host      |
+-----+-----+
| root      | %         |
| healthcheck | 127.0.0.1 |
| healthcheck | ::1       |
| healthcheck | localhost |
| mariadb.sys | localhost |
| root      | localhost |
+-----+-----+
6 rows in set (0.005 sec)
```

```
MariaDB [(none)]> CREATE DATABASE foo;
Query OK, 1 row affected (0.006 sec)
```

```
MariaDB [(none)]> exit
Bye
```

9.1.3. Классы хранения

До сих пор мы задействовали механизм динамического выделения ресурсов, используемый облачной платформой по умолчанию. Но что, если мы захотим указать, чтобы на этапе связывания предоставлялись диски другого типа или что с данными при удалении объекта PersistentVolumeClaim должно происходить нечто конкретное? В этом нам помогут классы хранения.

Они позволяют определять различные типы *динамических* хранилищ, доступных для заявок в объектах PersistentVolumeClaim, и запрашиваемые таким образом тома должны быть настроены. В вашем кластере Kubernetes наверняка уже определено несколько таких классов. Отобразим информацию о них с помощью команды `kubect1 get storageclass` (чтобы улучшить читабельность, я опустил несколько столбцов):

```
$ kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY
premium-rwo	pd.csi.storage.gke.io	Delete
standard	kubernetes.io/gce-pd	Delete
standard-rwo (default)	pd.csi.storage.gke.io	Delete

При создании пода с заявкой `PersistentVolumeClaim` в предыдущем подразделе был использован класс хранения по умолчанию (в данном случае `standard-rwo`). Если вы вернетесь назад и еще раз посмотрите, как выглядел связанный объект `PersistentVolumeClaim`, то увидите, что этот класс хранения был указан в его конфигурации в поле `storageClassName`.

Это вполне нормальное начало, которое практически не требует изменений, но все же один момент здесь, вероятно, следует скорректировать. Если вы посмотрите на содержимое столбца `RECLAIMPOLICY` в приведенном выше выводе команды `kubectl get storageclass`, то сможете заметить, что в нем указана политика `Delete`. Это значит, что при удалении заявки `PVC` будет удаляться и связанный с ней том `PV` вместе с соответствующим дисковым ресурсом. Это вполне допустимо, если почти все ваши `stateful`-нагрузки просто кэшируют на диске маловажные данные. Но если ваши рабочие нагрузки сохраняют уникальные и ценные данные, то такой подход будет неуместен.

Кроме того, Kubernetes позволяет использовать политику восстановления `Retain`, которая дает возможность сохранять соответствующий дисковый ресурс при удалении заявки `PVC`. Благодаря этому вы можете оставить диск у себя и связать его с новыми объектами `PV` и `PVC`, которые даже могут находиться в другом кластере (что может потребоваться при перемещении рабочих нагрузок в другой кластер). Данная политика восстановления имеет недостаток: при ее использовании требуется вручную удалять уже ненужные диски, что неудобно на этапах тестирования и разработки и в случае применения рабочих нагрузок, сохраняющих такие временные данные, как кэши; именно поэтому она обычно и не используется по умолчанию.

Создать собственный объект `StorageClass` проще всего, взяв за основу какой-либо из уже существующих классов хранения, например, тот, который в данный момент используется по умолчанию. Экспортировать класс хранения `standard-rwo`, который я уже упоминал, можно с помощью представленной ниже команды. Если вы хотите взять за основу какой-то другой класс хранения, то укажите его в этой команде вместо класса `standard-rwo`:

```
kubectl get -o yaml storageclass standard-rwo > storageclass.yaml
```

Теперь мы можем скорректировать этот класс так, чтобы он использовал политику восстановления `Retain`, подходящую для важных данных. Мы собираемся создать новый класс хранения, поэтому нужно указать новое имя и удалить поле `uid` и другие ненужные поля с метаданными. После внесения этих изменений мой класс хранения стал выглядеть так (листинг 9.4).

Листинг 9.4. Chapter09/9.1.3_StorageClass/storageclass.yaml

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
  name: example-default-rwo
parameters:
  type: pd-balanced
provisioner: pd.csi.storage.gke.io
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true

```

Необязательная аннотация, задающая этот класс в качестве используемого по умолчанию

Платформо-зависимое значение, задающее тип хранилища

Политика восстановления Retain позволяет сохранять диск при удалении тома PV

Теперь вы можете напрямую сослаться на новый класс хранения в любом объекте PersistentVolumeClaim или шаблоне с полем `storageClassName`. Такой подход, в частности, будет очень уместен в том случае, когда политику восстановления Retain нужно использовать только для отдельных рабочих нагрузок.

При желании вы можете установить новый класс хранения в качестве используемого по умолчанию, добавив аннотацию `is-default-class` (является классом по умолчанию), как было показано в листинге 9.4 (см. выше). Если вам нужно, чтобы этот класс заменил текущий класс, используемый по умолчанию, пометьте текущий класс как не используемый по умолчанию. Это можно сделать, открыв текущий класс по умолчанию для редактирования, используя команду `kubectl edit storageclass standard-rwo`, или отредактировав его с помощью команды, представленной ниже. Опять же, если в вашем случае по умолчанию используется какой-то другой класс хранения, укажите его имя вместо имени класса `standard-rwo`:

```

kubectl patch storageclass standard-rwo -p '{"metadata": {"annotations":
➤ {"storageclass.kubernetes.io/is-default-class": "false"}}}'

```

Закончив редактирование, создайте новый класс хранения с помощью команды `kubectl create -f storageclass.yaml`. Если вы задали этот класс в качестве нового класса по умолчанию, то он будет использоваться всеми томами PersistentVolume, которые будут создаваться в дальнейшем.

В большинстве случаев требуется использовать несколько классов хранения, обеспечивающих разный уровень производительности и политику восстановления для разных типов данных. Например, у вас могут быть:

- база данных с критически важными данными рабочей среды, которой лучше предоставить высокопроизводительное хранилище с сохранением данных;
- кэши, которым не помешает высокая производительность, но не требуется сохранение данных;
- эфемерное хранилище для пакетной обработки, которое может обойтись дисками средней производительности без сохранения данных.

Установите один из этих классов хранения в качестве класса по умолчанию, исходя из того, какой вариант наиболее предпочтителен для большинства нагрузок, и ссылайтесь на другие классы вручную, используя поле `storageClassName` заявки `PersistentVolumeClaim`.

9.1.4. Одноподовые развертывания для stateful-нагрузок

При использовании заявок `PersistentVolumeClaim` вы можете обеспечить развертывание одной реплики stateful-нагрузки, просто обернув свой под в развертывание. Плюсом использования развертывания для всего одной реплики пода является то, что такой под автоматически создается заново в случае его остановки (листинг 9.5).

Листинг 9.5. Chapter09/9.1.4_Deployment_MariaDB/mariadb-deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mariadb
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-container
          image: mariadb:latest
          volumeMounts:
            - mountPath: /var/lib/mysql
              name: mariadb-volume
          resources:
            requests:
              cpu: 1
              memory: 4Gi
          env:
            - name: MARIADB_ROOT_PASSWORD
              value: "your database password"
      volumes:
        - name: mariadb-volume
          persistentVolumeClaim:
            claimName: mariadb-pvc
---
apiVersion: v1
kind: PersistentVolumeClaim
```

Используем стратегию Recreate, чтобы исключить вероятность того, что при обновлении несколько реплик попытается смонтировать один и тот же том

Используем такой же шаблон спецификации пода, как в подразделе 9.1.2

```

metadata:
  name: mariadb-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi

```

Вот, в общем-то, и все. Мы получили одноподовое развертывание для базы данных MariaDB с подключенным диском, которое не будет удалено, даже если будет удален весь кластер Kubernetes, благодаря использованию политики восстановления *Retain* в классе хранения по умолчанию, который мы создали в предыдущем подразделе. Если вы хотите попробовать использовать эту базу данных, то создайте для нее сервис (Chapter09/9.1.4_Deployment_MariaDB/service.yaml). После этого попробуйте подключиться к базе данных из локального клиента (см. врезку «Локальное тестирование пода с базой данных MariaDB» в подразделе 9.1.2) или воспользоваться контейнеризированной версией приложения phpMyAdmin (которая размещена в репозитории с примерами кода из этой книги внутри каталога Bonus/phpMyAdmin).

ЗАПУСК БАЗ ДАННЫХ В KUBERNETES

Прежде чем приниматься за развертывание в Kubernetes собственной базы данных MariaDB, поинтересуйтесь, не позволяет ли ваш облачный провайдер использовать для этой цели управляемое решение. На первый взгляд, развертывание своей базы кажется самым простым решением, однако нужно учитывать, что впоследствии вам придется нести операционные издержки, связанные с обеспечением защиты этих данных, их обновлением и управлением ими. Обычно возможности Kubernetes по работе со stateful-нагрузками стоит использовать только в случае каких-либо уникальных сервисов или сервисов, которые ваш облачный провайдер не предоставляет в виде управляемого решения.

Как было показано в этом разделе, рабочую нагрузку можно сделать сохраняющей состояние (то есть stateful-нагрузкой), подключив к ней тома с помощью заявок PersistentVolumeClaim, используя для этой цели объекты Pod и Deployment, но в этом случае мы ограничены одной репликой пода. Во многих случаях большего и не требуется, но что, если вам понадобится развернуть достаточно сложную stateful-нагрузку наподобие поисковой системы Elasticsearch или in-memory базы данных Redis в нескольких репликах? Конечно, в таком случае можно попытаться объединить несколько развертываний, но, к счастью, у платформы Kubernetes есть высокоуровневая конструкция StatefulSet, которая служит для представления именно таких рабочих нагрузок.

9.2. НАБОР С СОХРАНЕНИЕМ СОСТОЯНИЯ (STATEFULSET)

Теперь мы уже знаем, как в Kubernetes можно подключить постоянное хранилище к поду. Это очень полезная возможность, поскольку поды являются в Kubernetes базовым структурным элементом таких конструкций для представления рабочих нагрузок, как развертывание (см. главу 3) и задание (см. главу 10). Теперь вы сможете подключить постоянное хранилище к любой такой конструкции и создать stateful-поды в любом нужном вам месте, установив идентичную спецификацию тома для всех экземпляров.

У таких конструкций, как развертывание, есть ограничение: они требуют использования для всех подов одной и той же спецификации, что порождает проблемы в случае традиционных томов с методом доступа `ReadWriteOnce`, которые могут монтироваться только в один экземпляр пода. Вы не столкнетесь с проблемами, если развертывание будет содержать только одну реплику. Но если попытаетесь создать вторую, то процесс ее создания завершится неудачей, поскольку запрашиваемый ею том уже будет смонтирован.

К счастью, в Kubernetes предусмотрена специальная высокоуровневая конструкция на тот случай, когда требуется использовать несколько подов, каждый из которых работает со своим отдельным диском (что является достаточно распространенной ситуацией). Напомню, что объект `Deployment` специально предназначен для управления непрерывно работающими сервисами (обычно представляющими собой stateless-сервисы). Точно так же объект `StatefulSet` специально предназначен для управления stateful-сервисами.

Этот объект имеет несколько свойств, помогающих в создании таких сервисов. Вместо того чтобы ссылаться на один и тот же том в `PodSpec`, вы можете определить шаблон тома. И в таком случае для каждого пода Kubernetes будет создавать отдельный `PersistentVolumeClaim (PVC)` (что избавляет вас от проблемы подключения томов с помощью развертывания, где все экземпляры используют одну и ту же заявку `PVC`). `StatefulSet` присваивает каждому поду стабильный идентификатор, который привязывается к отдельной заявке `PVC` и гарантирует упорядоченность при выполнении операций создания, масштабирования и обновления. Таким образом, `StatefulSet` позволяет создать несколько подов и координировать их работу с помощью стабильного идентификатора, при необходимости назначая им разные роли.

9.2.1. Процесс развертывания `StatefulSet`

Чтобы посмотреть, как это выглядит на практике, попробуем развернуть в виде `StatefulSet` два популярных вида stateful-нагрузок: базы данных `MariaDB` и `Redis`. Пока ограничимся более простым случаем с одноподовым `StatefulSet`, где

не нужно будет беспокоиться о различных ролях. Далее мы попробуем добавить дополнительные реплики с разными ролями, чтобы задействовать все возможности StatefulSet.

База данных MariaDB

Сначала преобразуем одноподовое развертывание базы данных MariaDB из предыдущего раздела в объект StatefulSet и воспользуемся шаблоном для заявки PVC, чтобы нам не нужно было создавать отдельные заявки PVC вручную (листинг 9.6).

Листинг 9.6. Chapter09/9.2.1_StatefulSet_MariaDB/mariadb-statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mariadb
spec:
  selector:
    matchLabels:
      app: mariadb-sts
  serviceName: mariadb-service
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb-sts
  spec:
    terminationGracePeriodSeconds: 10
    containers:
      - name: mariadb-container
        image: mariadb:latest
        volumeMounts:
          - name: mariadb-pvc
            mountPath: /var/lib/mysql
        resources:
          requests:
            cpu: 1
            memory: 4Gi
        env:
          - name: MARIADB_ROOT_PASSWORD
            value: "your database password"
    volumeClaimTemplates:
      - metadata:
          name: mariadb-pvc
        spec:
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 2Gi
```

StatefulSet использует тот же шаблон сопоставления меток, что и развертывание (см. главу 3)

Это ссылка на headless-сервис, определяемый в конце данного файла

StatefulSet требует, чтобы был задан период корректного завершения работы. Это количество времени в секундах, предоставляемое поду для самостоятельного завершения работы перед его принудительной остановкой

Монтируемый том для базы данных MariaDB теперь определяется в разделе volumeClaimTemplates

С помощью StatefulSet мы можем определить шаблон для заявок PersistentVolumeClaim, подобно тому, как определяется шаблон для реплик пода. Этот шаблон используется для создания заявок PersistentVolumeClaim с их привязкой к каждой из реплик пода

```
---
apiVersion: v1
kind: Service
metadata:
  name: mariadb-service
spec:
  ports:
    - port: 3306
  clusterIP: None
  selector:
    app: mariadb-sts
```

Headless-сервис,
используемый
данным
StatefulSet

Чем же это определение StatefulSet отличается от рассмотренного в предыдущем разделе определения развертывания для того же пода с базой данных MariaDB? Помимо использования других метаданных объекта, здесь можно заметить два важных различия. Первое состоит в том, что здесь по-другому определяется заявка PersistentVolumeClaim. Если в предыдущем разделе она определялась как отдельный объект, то в данном случае она определяется с помощью шаблона volumeClaimTemplates, подобно тому, как в развертывании используется шаблон для подов. На основе этого шаблона StatefulSet будет создавать отдельные заявки PersistentVolumeClaim для каждого пода. В случае одноподового StatefulSet этот шаблон обеспечивает практически такой же результат (лишь с тем отличием, что нам не нужно определять отдельный объект PersistentVolumeClaim), а вот когда мы перейдем к использованию многоподовых StatefulSet, он начнет играть очень важную роль. Для наглядности я разместил на рис. 9.5 определение заявки PersistentVolumeClaim из объекта Deployment рядом с шаблоном volumeClaimTemplates из объекта StatefulSet.

<pre>apiVersion: v1 kind: PersistentVolumeClaim metadata: name: mariadb-pvc spec: accessModes: - ReadWriteOnce resources: requests: storage: 2Gi</pre>	<pre>... volumeClaimTemplates: - metadata: name: mariadb-pvc spec: accessModes: - ReadWriteOnce resources: requests: storage: 2Gi</pre>
--	---

Рис. 9.5. Сравнение объекта PersistentVolumeClaim и шаблона volumeClaimTemplates

Если мы запросим информацию о заявках PVC после создания StatefulSet, то увидим, что данный шаблон создал одну заявку (чтобы улучшить читабельность, я удалил некоторые столбцы):

```
$ kubectl get pvc
NAME                                STATUS  VOLUME    CAPACITY  ACCESS MODES
mariadb-pvc-mariadb-0              Bound   pvc-71b1e  2Gi       RW0
```

Основное различие здесь состоит в том, что имя созданной шаблоном заявки PVC дополнительно содержит имя пода (в случае первого пода это `mariadb-0`). То есть вместо того чтобы просто использовать имя шаблона для заявок, `mariadb-pvc`, система соединяет друг с другом имя шаблона и имя пода: `mariadb-pvc-mariadb-0`.

Еще одним различием по сравнению с развертыванием является то, что с помощью строки `serviceName: mariadb-service` мы ссылаемся здесь на сервис, который определяется так:

```
apiVersion: v1
kind: Service
metadata:
  name: mariadb-service
spec:
  ports:
    - port: 3306
    clusterIP: None
  selector:
    app: mariadb-sts
```

Он немного отличается от тех сервисов, с которыми мы имели дело до сих пор, и представляет собой так называемый `headless-сервис` (на что указывает строка `clusterIP: None` в его спецификации). В отличие от сервисов других типов, `headless-сервисам` не назначаются виртуальные внутрикластерные IP-адреса для распределения трафика между подами. Если вы попытаетесь отобразить DNS-запись данного сервиса (например подключившись к поду с помощью команды `exec` и выполнив команду `host mariadb-service`), то увидите, что она представляет собой A-запись, то есть IP-адрес самого пода, а не виртуальный внутрикластерный IP-адрес. Если вы запросите ту же информацию в случае `headless-сервиса` с несколькими подами (какой, например, используется набором `StatefulSet` для базы данных Redis, о котором мы будем говорить далее), то вам будет предоставлено несколько A-записей (по одной для каждого пода).

Вторым полезным различием `StatefulSet` является то, что он выдает каждому поду стабильный сетевой идентификатор. Каждый под в данном наборе является уникальным и обладает собственным подключенным томом, поэтому полезно иметь возможность адресовать каждый из них по отдельности. Этим `StatefulSet` сильно отличается от развертывания, которое специально предназначено для того, чтобы поды были идентичными, и поэтому неважно, какой из них будет использоваться для выполнения того или иного запроса. Чтобы было проще выполнять непосредственное подключение к подам, в `StatefulSet` им присваиваются последовательно возрастающие порядковые номера (0, 1, 2 и т. д.). При этом в случае перезапуска под сохраняет за собой свой прежний порядковый номер, в то время как в развертывании перезапускаемые поды получают новое случайно выбранное имя.

Обратиться к поду набора StatefulSet можно, используя его порядковый номер в конструкции `$ИМЯ_НАБОРА_STATEFULSET-$ПОРЯДКОВЫЙ_НОМЕР_ПОДА.$ИМЯ_СЕРВИСА`. В нашем случае к единственному поду можно обратиться, используя имя `mariadb-0.mariadb-service`. При обращении из-за пределов пространства имен к этому нужно будет добавить пространство имен (как при работе с любым сервисом). Например, если наш под находится в пространстве имен `production`, то к нему можно обратиться, используя имя `mariadb-0-mariadb-service.production`.

Чтобы опробовать в деле наш экземпляр базы данных MariaDB, работающий в StatefulSet, мы можем перенаправить порт с помощью команды `kubectl port-forward sts/mariadb 3306:3306` и выполнить локальное подключение. Но мы сделаем кое-что более интересное: создадим клиент базы данных MariaDB, используя запущенный в кластере эфемерный под, и выполним подключение, задействуя присвоенное сервису имя хоста:

```
kubectl run my -it --rm --restart=Never --pod-running-timeout=3m \
  --image mariadb -- mariadb -h mariadb-0.mariadb-service -P 3306 -u root -p
```

Эта команда создаст в кластере под с клиентом базы данных MariaDB, настроенным на подключение к нашему основному поду из StatefulSet. Этот под является эфемерным и будет удален после закрытия интерактивного сеанса, благодаря чему такой подход удобен для выполнения разовых отладочных операций в пределах кластера. Когда этот под уже будет готов к использованию, введите в командной строке пароль базы данных, указанный в определении StatefulSet в переменной среды `MARIADB_ROOT_PASSWORD` (см. листинг 9.6 выше), после чего можете приступить к выполнению команд, позволяющих управлять базой данных. Завершив работу с базой, введите команду `exit` для выхода из сеанса.

REDIS

В качестве еще одного примера попробуем развернуть базу данных Redis. Это один из самых широко применяемых в Kubernetes видов рабочей нагрузки, который имеет широкий спектр возможных применений, в том числе кэширование и другие функции, требующие сохранения и извлечения данных в режиме реального времени. Предположим, что в данном случае нам нужно кэшировать данные, которые не слишком ценны. Мы хотим сделать возможным их сохранение на диск, чтобы кэш не приходилось заполнять заново в случае перезапуска, но в то же время можем обойтись без их резервного копирования. Далее я опишу способ настройки одноподовой рабочей нагрузки с базой данных Redis, который отлично работает на практике и подходит для таких приложений.

Сначала определим конфигурационный файл для настройки Redis, который можно будет смонтировать в контейнере в виде тома (листинг 9.7).

Листинг 9.7. Chapter09/9.2.1_StatefulSet_Redis/redis-configmap.yaml

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: redis-config
data:
  redis.conf: |
    bind 0.0.0.0
    port 6379
    protected-mode no
    appendonly yes
    dir /redis/data

```

Привязываемся ко всем сетевым интерфейсам, чтобы могли подключаться другие поды

Используемый порт

Отключаем защищенный режим, чтобы другие поды кластера могли подключаться без пароля

Активируем журнал добавления данных, чтобы данные сохранялись на диске

Определяем каталог для данных

Важным моментом здесь является то, что мы сохраняем состояние Redis в каталог `/redis/data`, чтобы его можно было загрузить заново в случае повторного создания пода, и теперь мы должны настроить том, который будет монтироваться в этот каталог.

В данном примере мы решили не настраивать аутентификацию для Redis, поэтому любой под кластера сможет подключаться к ней для чтения/записи. Если вы решите использовать данный пример в кластере на продуктиве, то обязательно подумайте о том, как вам лучше настроить свой кластер.

Теперь, наконец, создадим StatefulSet, который будет ссылаться на эту конфигурацию и монтировать каталог `/redis/data` в виде тома PersistentVolume (листинг 9.8).

Листинг 9.8. Chapter09/9.2.1_StatefulSet_Redis/redis-statefulset.yaml

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  selector:
    matchLabels:
      app: redis-sts
  serviceName: redis-service
  replicas: 1
  template:
    metadata:
      labels:
        app: redis-sts
    spec:
      terminationGracePeriodSeconds: 10
      containers:
        - name: redis-container
          image: redis:latest
          command: ["redis-server"]

```

Используем одну реплику, поскольку это StatefulSet с одной ролью


```

args: ["/redis/conf/redis.conf"]
volumeMounts:
- name: redis-configmap-volume
  mountPath: /redis/conf/
- name: redis-pvc
  mountPath: /redis/data
resources:
  requests:
    cpu: 1
    memory: 4Gi
volumes:
- name: redis-configmap-volume
  configMap:
    name: redis-config
volumeClaimTemplates:
- metadata:
    name: redis-pvc
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
spec:
  ports:
  - port: 6379
  clusterIP: None
  selector:
    app: redis-sts

```

Монтируем конфигурационный файл из листинга 9.7 в каталог внутри контейнера

Монтирование тома для данных Redis, определенное в разделе volumeClaimTemplates

Ссылаемся на объект ConfigMap, определенный в листинге 9.7

Это определение более простое по сравнению с набором StatefulSet для базы данных MariaDB, если не считать такие связанные с приложением нюансы, как использование другого порта и образа контейнера и монтирование тома ConfigMap в каталог `/redis/conf`.

Когда ресурсы, определенные в каталоге `Chapter09/9.2.1_StatefulSet_Redis`, будут созданы, вы сможете подключиться к базе данных Redis и убедиться в ее работоспособности. Для этого перенаправьте порт локальной машины и подключитесь к базе данных с помощью утилиты `redis-cli`:

```

$ kubectl port-forward pod/redis-0 6379:6379
Forwarding from 127.0.0.1:6379 -> 6379

$ docker run --net=host -it --rm redis redis-cli
27.0.0.1:6379> INFO
# Сервер
redis_version:7.2.1
127.0.0.1:6379> exit

```

Итак, мы рассмотрели два примера определения StatefulSet с одной репликой. Даже если используется только одна реплика, задействовать для такой нагрузки этот объект удобнее, чем развертывание, поскольку при этом Kubernetes автоматически создает заявку PersistentVolumeClaim.

Если вы удалите StatefulSet, это не приведет к удалению соответствующей заявки PersistentVolumeClaim. Если затем вы снова создадите этот StatefulSet, к нему будет снова привязана та же заявка PersistentVolumeClaim, и таким образом используемые им данные не будут утрачены. Эти данные *могут быть утрачены* при удалении самой заявки PersistentVolumeClaim, но лишь в зависимости от того, какой класс хранения вы будете использовать. Если вам нужно, чтобы ваши данные сохранялись и в этом случае (если это не что-то вроде кэша, который легко может быть восстановлен), то воспользуйтесь инструкциями из подраздела 9.1.3 и настройте класс хранения так, чтобы используемые облачные ресурсы сохранялись при удалении PersistentVolumeClaim.

Увеличив количество реплик в StatefulSet, мы получим дополнительные поды с собственными томами, но это совсем не означает, что эти поды смогут взаимодействовать. Так, в случае StatefulSet для Redis из представленного выше примера увеличение количества реплик приведет лишь к тому, что у нас будет несколько независимо работающих экземпляров базы данных. В следующем подразделе мы разберем, как выполняется настройка многоподовой версии StatefulSet, где каждый под имеет собственную конфигурацию и может взаимодействовать с другими подами.

9.2.2. Процесс развертывания StatefulSet с несколькими ролями

Все возможности StatefulSet проявляются лишь при использовании его многоподовой версии. Разрабатывая приложение, использующее такой набор, вы должны позаботиться о том, чтобы реплики пода, создаваемые этим объектом, знали о существовании друг друга и взаимодействовали в соответствии с дизайном вашего stateful-приложения. Использование многоподовой версии StatefulSet дает преимущество: каждый из ваших подов получает уникальный порядковый номер, идентифицирующий его в наборе. Эта уникальность и гарантированная упорядоченность подов позволяет назначать им разные роли и заново подключать их к тем же постоянным дискам после обновления этих подов и даже их повторного создания.

В качестве примера возьмем одноподовую версию StatefulSet для Redis из предыдущего подраздела и преобразуем ее в трехподовую конфигурацию, дополнительно использующую роли реплик. Redis выполняет репликацию по

схеме «ведущий — ведомый», что предполагает наличие основного пода (который в подразделе 9.2.1 был единственным) и нескольких дополнительных подов с ролью реплики (которую не стоит путать с «репликами» платформы Kubernetes, представляющими собой любые поды, используемые в StatefulSet или в разворачивании).

Взяв за основу пример из предыдущего подраздела, оставим исходную конфигурацию базы данных Redis в качестве основного пода и добавим для реплик Redis дополнительный конфигурационный файл со ссылкой на основной под. В листинге 9.9 показано, как выглядит объект ConfigMap, определяющий эти два конфигурационных файла.

Листинг 9.9. Chapter09/9.2.2_StatefulSet_Redis_Multi/redis-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: redis-role-config
data:
  primary.conf: |
    bind 0.0.0.0
    port 6379
    protected-mode no
    appendonly yes
    dir /redis/data
  replica.conf: |
    replicaof redis-0.redis-service 6379
    bind 0.0.0.0
    port 6379
    protected-mode no
    appendonly yes
    dir /redis/data
```

Первый файл в словаре конфигурации, с помощью которого можно настроить основную роль

Второй файл в словаре конфигурации, который служит для настройки экземпляров реплик

Настройка, позволяющая реплике Redis обращаться к основному поду по его имени

Объект ConfigMap используется здесь просто как более удобный способ определения двух конфигурационных файлов для наших двух ролей. С тем же успехом мы могли бы создать собственный контейнер на основе базового образа СУБД Redis, поместив в него эти два файла. Но эти два файла содержат все необходимые нам индивидуальные настройки, так что будет проще определить их в данном объекте и смонтировать его в контейнере.

Теперь изменим StatefulSet так, чтобы он использовал контейнер `init` (запускаемый на этапе инициализации пода) для настройки роли каждой реплики пода. В контейнере `init` будет запускаться скрипт, который будет проверять порядковый номер инициализируемого пода и копировать конфигурацию роли, соответствующей этому порядковому номеру. (Как вы помните, наборы StatefulSet имеют особенность: они присваивают каждому поду уникальный порядковый номер.

Мы можем использовать номер 0 для обозначения основной роли и присваивать остальным подам роль реплики.)

Этот прием подойдет при использовании самых разных stateful-нагрузок с несколькими ролями. В частности, в документации Kubernetes приводится отличный пример его использования для работы с базой данных MySQL (<https://kubernetes.io/docs/tasks/run-application/run-replicated-stateful-application/>) (листинг 9.10).

Листинг 9.10. Chapter09/9.2.2_StatefulSet_Redis_Multi/redis-statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  selector:
    matchLabels:
      app: redis-sts
  serviceName: redis-service
  replicas: 3
  template:
    metadata:
      labels:
        app: redis-sts
    spec:
      terminationGracePeriodSeconds: 10
      initContainers:
        - name: init-redis
          image: redis:latest
          command:
            - bash
            - "-c"
            - |
              set -ex
              # Генерация идентификатора сервера из порядкового номера пода.
              [[ `hostname` =~ -([0-9]+)$ ]] || exit 1
              ordinal=${BASH_REMATCH[1]}
              echo "ordinal ${ordinal}"
              # Скопируйте соответствующие файлы конфигурации из config-map в emptyDir.
              mkdir -p /redis/conf/
              if [[ $ordinal -eq 0 ]]; then
                cp /mnt/redis-configmap/primary.conf /redis/conf/redis.conf
              else
                cp /mnt/redis-configmap/replica.conf /redis/conf/redis.conf
              fi
              cat /redis/conf/redis.conf
      volumeMounts:
        - name: redis-config-volume
          mountPath: /redis/conf/
        - name: redis-configmap-volume
          mountPath: /mnt/redis-configmap
```

Три реплики для StatefulSet с несколькими ролями

Контейнер инициализации, который однократно запускается на этапе загрузки для копирования конфигурационного файла из тома ConfigMap в том emptyDir

Запускаем следующий далее скрипт

Том emptyDir, используемый вместе с основным контейнером

Том ConfigMap с двумя файлами из листинга 9.9

```

containers:
- name: redis-container
  image: redis:latest
  command: ["redis-server"]
  args: ["/redis/conf/redis.conf"]
  volumeMounts:
  - name: redis-config-volume
    mountPath: /redis/conf/
  - name: redis-pvc
    mountPath: /redis/data
  resources:
    requests:
      cpu: 1
      memory: 4Gi
volumes:
- name: redis-configmap-volume
  configMap:
    name: redis-role-config
- name: redis-config-volume
  emptyDir: {}
volumeClaimTemplates:
- metadata:
  name: redis-pvc
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
spec:
  ports:
  - port: 6379
  clusterIP: None
  selector:
    app: redis-sts

```

Основной контейнер
базы данных Redis

Том `emptyDir`, используемый
вместе с контейнером
инициализации

Том для данных базы данных
Redis, определяемый в разделе
`volumeClaimTemplates`

Ссылка на объект `ConfigMap`,
определенный в листинге 9.9

Определение
тома `emptyDir`

Некоторые моменты требуют разъяснения, поэтому рассмотрим содержимое данного файла чуть подробнее. Главное, чем он отличается от одноподовой версии `StatefulSet` для Redis, — наличие контейнера `init`. Этот контейнер запускается на этапе инициализации пода и монтирует два тома: уже известный нам том `ConfigMap` и том `redis-config-volume`:

```

volumeMounts:
- name: redis-config-volume
  mountPath: /redis/conf/
- name: redis-configmap-volume
  mountPath: /mnt/redis-configmap

```

Том `redis-config-volume` относится к типу `emptyDir`, который позволяет сделать данные доступными для нескольких контейнеров, но не обеспечивает их сохранение в случае перезапуска пода. Мы используем этот том исключительно в целях сохранения копии конфигурации, и том `emptyDir` идеально подходит для этого. Контейнер `init` запускает `bash`-скрипт, который определен внутри данного YAML-файла:

```
command:
- bash
- "-c"
- |
  set -ex
  # Генерация идентификатора сервера из порядкового номера пода
  [[ `hostname` =~ -([0-9]+)$ ]] || exit 1
  ordinal=${BASH_REMATCH[1]}
  # Скопируйте соответствующие файлы конфигурации из config-map в emptyDir
  mkdir -p /redis/conf/
  if [[ $ordinal -eq 0 ]]; then
    cp /mnt/redis-configmap/primary.conf /redis/conf/redis.conf
  else
    cp /mnt/redis-configmap/replica.conf /redis/conf/redis.conf
  fi
```

Этот скрипт копирует одну из двух конфигураций с тома `ConfigMap` (который монтируется в каталог `/mnt/redis-configmap`) в совместно используемый том `emptyDir` (который монтируется в каталог `/redis/conf`), в зависимости от порядкового номера пода. То есть в случае пода `redis-0` копируется файл `primary.conf`, а в случае остальных подов — файл `replica.conf`.

После этого основной контейнер монтирует тот же том `emptyDir` с именем `redis-config-volume` в каталог `/redis/conf`. Запускаясь, Redis использует конфигурацию, размещенную в файле `/redis/conf/redis.conf`.

Чтобы опробовать эту базу данных, вы можете подключиться к основному поду, используя уже описанные ранее способы с перенаправлением портов и запуском локального клиента или с созданием эфемерного пода. Кроме того, можно быстро записать небольшое количество данных, выполнив подключение напрямую с помощью команды `exec`:

```
$ kubectl exec -it redis-0 -- redis-cli
127.0.0.1:6379> SET capital:australia "Canberra"
OK
127.0.0.1:6379> exit
```

После этого подключитесь к реплике Redis и попробуйте считать записанные вами данные:

```
$ kubectl exec -it redis-1 -- redis-cli
Defaulted container "redis-container" out of: redis-container, init-redis (init)
127.0.0.1:6379> GET capital:australia
"Canberra"
```

Реплики Redis доступны только для чтения, поэтому вы не сможете записать в них данные напрямую:

```
127.0.0.1:6379> SET capital:usa "Washington"
(error) READONLY You can't write against a read only replica.
127.0.0.1:6379> exit
```

9.3. ПЕРЕМЕЩЕНИЕ/ВОССТАНОВЛЕНИЕ ДИСКОВ

Я знаю, о чем вы сейчас подумали. Ваши мысли звучат примерно так: «Могу ли я доверить системе Kubernetes свои данные? Очень уж просто здесь все; а что произойдет, если кластер Kubernetes выйдет из строя? Не пропадут ли мои данные в таком случае и смогу ли я их восстановить?»

Что ж, пришло время вселить в вас чуть больше уверенности. Для этого создадим в Kubernetes stateful-нагрузку, а затем попробуем удалить все связанные с ней объекты Kubernetes и снова создать ее с нуля, восстановив подключение к соответствующим облачным дисковым ресурсам.

Здесь важно иметь в виду следующий момент. Создаваемые в Kubernetes дисковые ресурсы обычно удаляются при удалении соответствующей заявки PersistentVolumeClaim, поскольку по умолчанию для них задается политика восстановления Delete. Удаление самого StatefulSet не ведет к удалению связанных с ним заявок PersistentVolumeClaim, вследствие чего их нужно удалять вручную. Это достаточно удобно: когда определенные данные становятся ненужными, вы удаляете соответствующие PersistentVolumeClaim (что, например, можно легко сделать, выполнив команду `kubectl delete` с параметром `--all`), что, в свою очередь, *ведет к удалению* соответствующих дисковых ресурсов.

Учитывая сказанное выше, вы должны позаботиться о том, чтобы при создании дисков для ценных данных использовался класс хранения с политикой восстановления Retain, а не Delete. В таком случае удаление объектов Kubernetes не приведет к удалению используемых ими облачных дисков, и вы сможете вручную восстановить соответствующую пару объектов PersistentVolumeClaim и PersistentVolume в том же или другом кластере (как это сделать, я покажу чуть позже).

Чтобы провести этот эксперимент, разверните Redis из подраздела 9.2.2, используя класс хранения по умолчанию или специально определенный класс с политикой восстановления Retain. Проверьте состояние созданных вами томов PersistentVolume с помощью команды `kubectl get pv` и при необходимости воспользуйтесь командой `kubectl edit pv $PV_NAME`, чтобы скорректировать содержимое поля `persistentVolumeReclaimPolicy`.

Должным образом настроив политику восстановления, мы можем сохранить некоторые данные, чтобы потом можно было убедиться в успешном восстановлении StatefulSet после его удаления. Для этого необходимо подключиться к основному

поду с помощью команды `exec` и запустить утилиту `redis-cli`. Следующая команда позволяет сделать сразу и то и другое:

```
kubectl exec -it redis-0 -- redis-cli
```

Теперь можно выполнить добавление данных. Если вы еще никогда не использовали Redis, то не переживайте — мы лишь добавим простейшие данные, чтобы потом убедиться в том, что можем восстановить их. Например, добавим несколько пар «ключ — значение» с информацией о столицах некоторых стран:

```
127.0.0.1:6379> SET capital:australia "Canberra"
OK
127.0.0.1:6379> SET capital:usa "Washington"
OK
```

Теперь удалим `StatefulSet` и создадим его заново. Затем снова подключимся к `redis-cli` с помощью команды `exec` и проверим, сохранились ли наши данные. Все это можно сделать так:

```
$ cd Chapter09/9.2.2_StatefulSet_Redis_Multi/
$ kubectl delete -f redis-statefulset.yaml
service "redis-service" deleted
statefulset.apps "redis" deleted

$ kubectl create -f redis-statefulset.yaml
service/redis-service created
statefulset.apps/redis created

$ kubectl exec -it redis-0 -- redis-cli
127.0.0.1:6379> GET capital:usa
"Washington"
```

Пример работает, и наши данные никуда не пропадают. Это объясняется тем, что при повторном создании `StatefulSet` он ссылается на тот же `PersistentVolumeClaim`, где хранятся наши данные для Redis, к которым он обращается при загрузке, и получается, будто Redis просто выключили и включили.

Что ж, с этим разобрались. Теперь сделаем более серьезный шаг и попробуем удалить и создать заново объекты `PVC` и `PV`. При этом операцию повторного создания можно выполнить в другом кластере, таким образом симитировав удаление целого кластера. Но проследите за тем, чтобы новый кластер находился в том же регионе облачной платформы, чтобы вы могли подключиться к тому же диску.

Прежде чем удалить эти объекты, сохраним их конфигурацию. В общем-то, можно и не сохранять, но эти объекты легко настроить заново, поэтому такой подход позволит сэкономить немного времени. Отобразите информацию об объектах `PVC`

и PV и сохраните их в файлы, как показано ниже (чтобы улучшить читабельность, я удалил часть выводимой информации):

```
$ kubectl get pvc,pv
NAME                                     STATUS   VOLUME
persistentvolumeclaim/redis-pvc-redis-0 Bound    pvc-64b52138
persistentvolumeclaim/redis-pvc-redis-1 Bound    pvc-4530141b
persistentvolumeclaim/redis-pvc-redis-2 Bound    pvc-5bbf6729

NAME                                     STATUS   CLAIM
persistentvolume/pvc-4530141b          Bound    default/redis-pvc-redis-1
persistentvolume/pvc-5bbf6729          Bound    default/redis-pvc-redis-2
persistentvolume/pvc-64b52138          Bound    default/redis-pvc-redis-0

$ kubectl get -o yaml persistentvolumeclaim/redis-pvc-redis-0 > pvc.yaml
$ PV_NAME=pvc-64b52138
$ kubectl get -o yaml persistentvolume/$PV_NAME > pv.yaml
```

Что ж, пришло время осуществить «массовое уничтожение»: удалим все объекты StatefulSet, PVC и PV в пределах текущего пространства имен:

```
kubectl delete statefulset,pvc,pv --all
```

ПРЕДУПРЕЖДЕНИЕ

Эту команду можно использовать только в тестовом кластере! Она удалит все объекты указанного типа в пределах текущего пространства имен, не ограничиваясь лишь объектами примера с базой данных Redis.

Надеюсь, вы не забыли выбрать для использования класс хранения с политикой восстановления **Retain**! Благодаря этому операция удаления не привела к удалению облачного дискового ресурса, и нам нужно просто вручную привязать к нему новый том PV, а к нему — новую заявку PVC.

У нас есть следующая информация (рис. 9.6):

- мы знаем (или можем выяснить) имя предоставленного облачным провайдером дискового ресурса;
- мы знаем имя заявки PVC, используемой набором StatefulSet (**redis-pvc-redis-0**).

Таким образом, мы должны создать заявку PVC с именем **redis-pvc-redis-0**, которая будет привязана к тому PV, который, в свою очередь, будет ссылаться на используемый нами диск. При этом в определении объекта PVC важно указать имя надлежащего объекта PV, а в определении объекта PV — имя надлежащего объекта PVC. В противном случае объект PVC может быть привязан не к тому объекту PV, а объект PV — не к тому объекту PVC.

К сожалению, мы не можем создать эти объекты непосредственно из сохраненной конфигурации, используя команды `kubectl create -f pv.yaml` и `kubectl create -f pvc.yaml`. Дело в том, что наряду с прочим в этой конфигурации было экспортировано и *состояние связывания*, использующее уникальные идентификаторы, которые не сохраняются при удалении и повторном создании объекта из конфигурации. Если мы попробуем создать эти объекты, не внося никаких изменений, то в результате получим объект PVC с состоянием `Lost` и объект PV с состоянием `Released`, а это нам совсем не нужно.

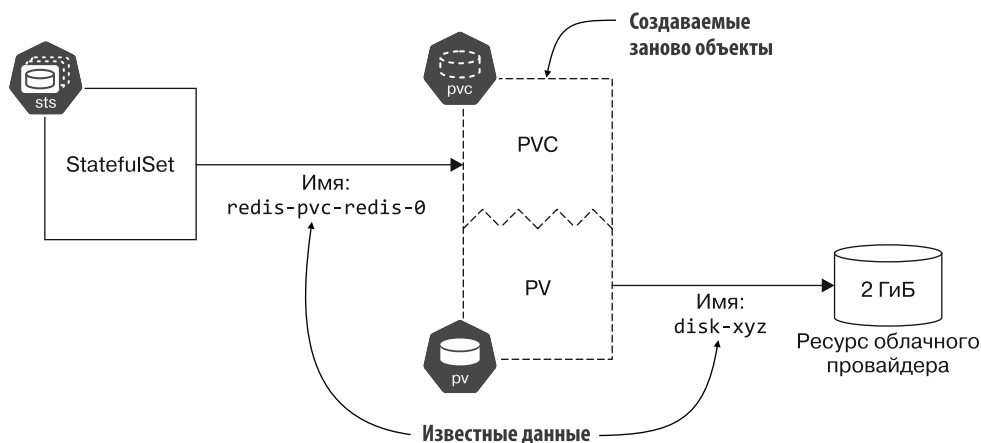


Рис. 9.6. Известная информация и создаваемые заново объекты

Исправить эту ситуацию можно, удалив из сохраненной конфигурации информацию о состоянии связывания и соответствующие идентификаторы `uid`.

- Внесите следующие два изменения в определение объекта PV (которое мы экспортировали в файл `pv.yaml`):
 - удалите поле `uid` в разделе `claimRef` (здесь определяется ссылка на объект PVC, но проблема в том, что у объекта PVC теперь другой идентификатор `uid`);
 - разместите пустую строку `""` в поле `storageClassName` (мы предоставляем ресурсы вручную, поэтому можем обойтись без указания класса хранения).
- Внесите следующие два изменения в определении объекта PVC (которое мы экспортировали в файл `pvc.yaml`):
 - удалите аннотацию `pv.kubernetes.io/bind-completed: "yes"` (она не дает выполнить повторное связывание объекта PVC);
 - разместите пустую строку `""` в поле `storageClassName` (это нужно сделать по той же причине, которая указана в предыдущем шаге).

Кроме того, при желании вы можете воссоздать эту конфигурацию с нуля; в таком случае важно проследить за тем, чтобы в поле `volumeName` объекта PVC было указано имя объекта PV, а в разделе `claimRef` объекта PV — имя и пространство имен объекта PVC и чтобы поле `storageClassName` обоих объектов содержало пустую строку (`""`).

Для наглядности разместим определения этих двух объектов рядом друг с другом. На рис. 9.7 показано, как выглядели экспортированные мной конфигурации после того, как я внес описанные выше изменения.

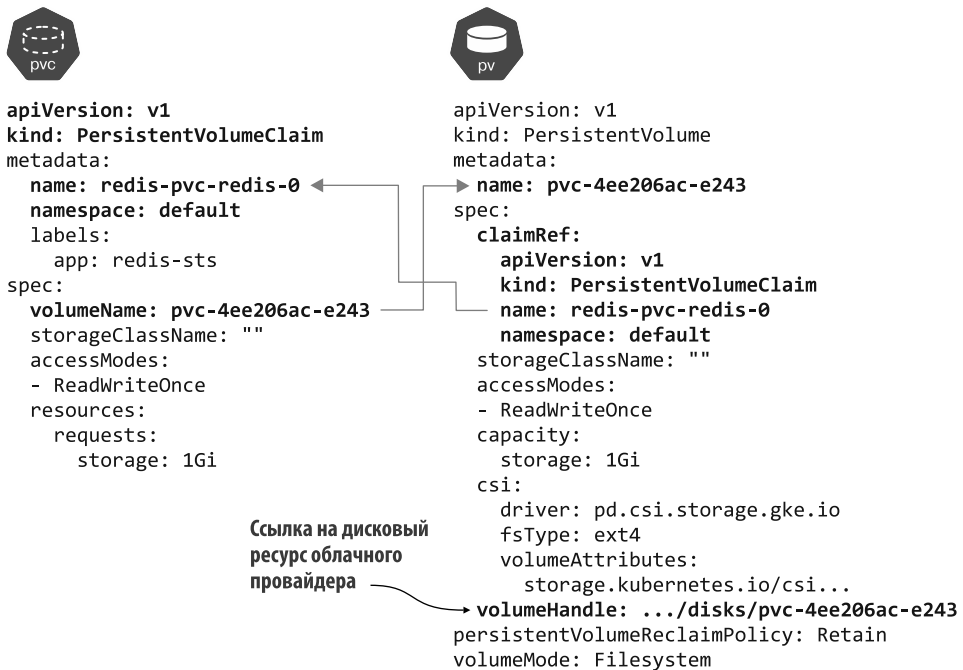


Рис. 9.7. Предварительно связанные объекты PVC и PV

Должным образом подготовив файлы конфигурации, создайте в кластере том и заявку, после чего проверьте их состояние с помощью команды `kubectl get pvc,pv`.

Если все было сделано правильно, то объект PV должен находиться в состоянии `Bound`, а объект PVC — в состоянии `Pending` (пока не будет создан объект `StatefulSet`). Если же оба объекта находятся в состоянии `Pending` или `Released`, то вернитесь к предыдущему шагу и проверьте, связаны ли они друг с другом и указана ли вся необходимая информация и не указана ли лишняя. Таким образом, приложив некоторые усилия, вы сможете заново привязать объекты PVC и PV к облачному дисковому ресурсу, при условии, что он по-прежнему существует (а это так, поскольку вы использовали класс хранения с политикой восстановления `Retain`).

284 Часть II. Переходим в продуктивную среду

Если все правильно, то информация о созданных объектах будет выглядеть примерно так (чтобы улучшить читабельность, я удалил некоторые столбцы):

```
$ kubectl create -f pv.yaml
persistentvolume/pvc-f0fea6ae-e229 created
```

```
$ kubectl get pv
NAME                                RECLAIM POLICY  STATUS    CLAIM
pvc-f0fea6ae-e229                  Retain          Available default/redis-pvc-redis-0
```

```
$ kubectl create -f pvc.yaml
persistentvolumeclaim/redis-pvc-redis-0 created
```

```
$ kubectl get pv,pvc
NAME                                RECLAIM POLICY  STATUS    CLAIM
pv/pvc-f0fea6ae-e229              Retain          Bound     default/redis-pvc-redis-0
```

```
NAME                                STATUS    VOLUME
persistentvolumeclaim/redis-pvc-redis-0  Pending  pvc-f0fea6ae-e229
```

Теперь, имея созданные вручную объекты PVC и PV, мы можем заново создать объект StatefulSet. Как показал предыдущий эксперимент с удалением и повторным созданием StatefulSet, если PVC с соответствующим именем есть, то этот объект заново привязывается к подам StatefulSet. При создании PVC StatefulSet присваивает им однозначно выбранные имена. И если он создается повторно, то обнаруживает созданные ранее PVC и ссылается на них, вместо того чтобы создавать новые. Поэтому в данном случае все должно работать практически так же, как и раньше, когда мы просто подключаем заново объекты с теми же именами.

Обратите внимание: изначально StatefulSet использует три PVC, благодаря чему становится возможным подключение трех дисков, но мы заново подключили вручную только один диск — тот, который подключается к основному поду. Используя этот источник, реплики Redis могут автоматически восстановить свои данные. Конечно, мы с тем же успехом могли бы переподключить вручную и все три диска.

```
$ kubectl create -f redis-statefulset.yaml
service/redis-service created
statefulset.apps/redis created
```

```
$ kubectl get pv,pvc,pods
NAME                                RECLAIM POLICY  STATUS    CLAIM
pv/pvc-f0fea6ae-e229              Retain          Bound     default/redis-pvc-redis-0
pv/pvc-9aabd1b8-ca4f              Retain          Bound     default/redis-pvc-redis-1
pv/pvc-db153655-88c2              Retain          Bound     default/redis-pvc-redis-2
```

```
NAME                                STATUS    VOLUME
persistentvolumeclaim/redis-pvc-redis-0  Bound  pvc-f0fea6ae-e229
persistentvolumeclaim/redis-pvc-redis-1  Bound  pvc-9aabd1b8-ca4f
persistentvolumeclaim/redis-pvc-redis-2  Bound  pvc-db153655-88c2
```

NAME	READY	STATUS	RESTARTS	AGE
pod/redis-0	1/1	Running	0	15m
pod/redis-1	1/1	Running	0	13m
pod/redis-2	1/1	Running	0	11m

Когда StatefulSet был создан, все наши объекты PV и PVC перешли в состояние Bound. Теперь подключимся к одной из реплик с помощью команды `exec` и посмотрим, содержит ли она данные, которые мы занесли в базу ранее:

```
$ kubectl exec -it redis-1 -- redis-cli
127.0.0.1:6379> GET capital:australia
"Canberra"
```

Если вам удалось успешно считать данные, которые вы занесли в Redis ранее, то примите мои поздравления! Вы сумели восстановить StatefulSet с нуля. Этот метод можно использовать не только для восстановления, но и для перемещения диска в StatefulSet, расположенный в другом кластере. Единственное различие — объекты создаются заново в другом кластере; кроме того, обычно нужно следить еще и за тем, чтобы диск и кластер находились в одном регионе облачной платформы.

Я надеюсь, что рассмотренный здесь пример помог вам обрести достаточную уверенность в том, что при использовании политики `Retain` ваши данные будут в полной сохранности. Как было показано, вы можете полностью удалить все объекты (даже весь кластер!), а затем воссоздать все связи с нуля. Это требует некоторых усилий, но, тем не менее, вполне осуществимо на практике. Чтобы немного уменьшить объем необходимой работы, рекомендуется экспортировать конфигурацию объектов PVC и PV в репозиторий для конфигураций, чтобы их можно было более быстро создавать в дальнейшем.

9.4. ИСПОЛЬЗОВАНИЕ УНИВЕРСАЛЬНЫХ ЭФЕМЕРНЫХ ТОМОВ В КАЧЕСТВЕ ВРЕМЕННОГО РАБОЧЕГО ПРОСТРАНСТВА

До сих пор мы говорили исключительно о том, как использовать `stateful-нагрузки` в сочетании с объектами `PersistentVolume` и `PersistentVolumesClaim`. Но что, если нам требуется просто большой диск для выполнения каких-либо временных вычислений — временное рабочее пространство для таких задач, как обработка данных? В начале этой главы я уже говорил о том, что в качестве такого временного пространства можно использовать том `emptyDir`, но данный подход имеет определенные недостатки. Для того чтобы можно было использовать этот том, мы должны заранее выделить на узле дисковое пространство, то есть вам придется заблаговременно планировать размер используемых узлами загрузочных дисков (что не всегда возможно в том случае, когда платформа не предоставляет доступ к узлам). Однако универсальные эфемерные тома позволяют выделять временное

рабочее пространство с помощью простого монтирования тома, как это делается в случае обычных постоянных томов.

Использование эфемерных томов — во многом более выгодный вариант по сравнению с томами `emptyDir` при работе с большими объемами временных данных. Размер загрузочного диска не имеет значения, поэтому вы можете динамически, без какого-либо предварительного планирования, выделить очень большой объем дискового пространства (например, на момент написания книги платформа Google Cloud позволяла выделить до 64 Тбайт). Причем даже этот, очень высокий лимит накладывается на размер отдельного тома, что позволяет обойти его с помощью подключения нескольких томов. Вы можете использовать разные классы хранения и особым образом настраивать атрибуты такого класса (например, выделив SSD-диск, более производительный, чем загрузочный диск самого узла). Пример использования эфемерного диска представлен в листинге 9.11.

Листинг 9.11. Chapter09/9.4_EphemeralVolume/ephemeralvolume_pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ephemeralvolume-pod
  labels:
    pod: timeserver-pod
spec:
  containers:
    - name: timeserver-container
      image: docker.io/wdenniss/timeserver:1
      volumeMounts:
        - mountPath: "/scratch"
          name: scratch-volume
      volumes:
        - name: scratch-volume
          ephemeral:
            volumeClaimTemplate:
              metadata:
                labels:
                  type: scratch-volume
              spec:
                accessModes: [ "ReadWriteOnce" ]
                storageClassName: "ephemeral"
                resources:
                  requests:
                    storage: 1Ti
```

Точка монтирования
временного тома

Определяем
эфемерный том
объемом 1 Тбайт

При использовании универсальных эфемерных томов вы должны задействовать класс хранения с политикой восстановления `Delete`. В противном случае эфемерное хранилище будет сохраняться, что не совсем правильно. Как определить такой класс хранения, показано в листинге 9.12.

Листинг 9.12. Chapter09/9.4_EphemeralVolume/ephemeral_storageclass.yaml

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ephemeral
parameters:
  type: pd-ssd
provisioner: pd.csi.storage.gke.io
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true

```

Определяем тип диска, исходя из требований к производительности эфемерного хранилища

Используем политику восстановления Delete, поскольку данный том будет служить эфемерным хранилищем

Сводя вышеизложенное воедино, попробуем создать такое хранилище, проверить его состояние и затем удалить. Ниже представлен пример того, как можно создать под с подключенным к нему диском объемом 1 ТиБ, а затем удалить его вместе со всеми ресурсами, которые он использует (эти команды должны выполняться из корневого каталога данного примера; чтобы улучшить читабельность, вывод команд я привожу в сокращенном виде):

```

$ kubectl create -f Chapter09/9.4_EphemeralVolume
storageclass.storage.k8s.io/ephemeral created
pod/ephemeralvolume-pod created

```

```
$ kubectl get pod,pvc,pv
```

NAME	READY	STATUS	RESTARTS	AGE
pod/ephemeralvolume-pod	1/1	Running	0	34s

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
pvc/scratch-volume	Bound	pvc-a5a2	1Ti	RwO	ephemeral

NAME	CAPACITY	RECLAIM POLICY	STATUS	CLAIM
pv/pvc-a5a2	1Ti	Delete	Bound	default/scratch-volume

```

$ kubectl exec -it ephemeralvolume-pod -- df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay          95G   4.6G   90G   5% /
/dev/sdb        1007G  28K 1007G   1% /scratch

```

Том объемом 1 ТиБ доступен для использования

```

$ kubectl delete pod ephemeralvolume-pod
pod "ephemeralvolume-pod" deleted

```

```

$ kubectl get pod,pvc,pv
No resources found

```

Как видите, в отличие от StatefulSet, при удалении которого требуется вручную удалять объекты PVC, при работе с эфемерными томами удаление пода ведет к автоматическому удалению связанных с ним объектов PVC. Кроме того, данный под можно обернуть в развертывание, предоставляющее каждой реплике собственный эфемерный том.

РЕЗЮМЕ

- Kubernetes позволяет легко разворачивать не только stateless-, но и stateful-нагрузки.
- Kubernetes дает возможность использовать различные типы томов, в том числе подключаемые напрямую постоянные дисковые ресурсы облачного провайдера.
- Объекты PersistentVolume и PersistentVolumeClaim, в сочетании с объектом StorageClass, составляют уровень абстракции, который используется платформой Kubernetes для представления динамически выделяемых дисковых ресурсов и обеспечивает переносимость рабочих нагрузок.
- StatefulSet — это высокоуровневый тип рабочей нагрузки, который специально предназначен для запуска stateful-нагрузок и предоставляет такие преимущества, как возможность определения индивидуальной роли для каждой реплики.
- Объекты PersistentVolume и PersistentVolumeClaim имеют сложный жизненный цикл: изначально создаются как заявка на ресурсы, а затем преобразуются в ссылку на конкретный логический объект.
- Используя объект StorageClass, вы можете получить возможность динамического выделения хранилища с нужными вам свойствами и, что самое главное, сохранения дисковых ресурсов облачного провайдера, если соответствующие объекты Kubernetes будут удалены.
- При использовании политики восстановления Retain вы сможете восстановить данные, даже если будут удалены все объекты в кластере.
- Универсальные эфемерные тома позволяют использовать монтируемые диски в качестве временного рабочего пространства.

10

Фоновая обработка

В этой главе

- ✓ Как в Kubernetes можно выполнять фоновые задачи.
- ✓ Объекты Job и CronJob платформы Kubernetes.
- ✓ Когда следует (и когда не следует) использовать объекты Job для своих фоновых рабочих нагрузок.
- ✓ Создание собственной очереди задач с помощью Redis.
- ✓ Реализация очереди фоновых задач в Kubernetes.

В предыдущих главах мы говорили о разработке сервисов, которые доступны на некотором IP-адресе, представляя собой либо внешний сервис с общедоступным адресом, либо внутренний сервис, имеющий локальный внутрикластерный адрес. Но что можно сказать о других видах выполняемых вами задач, которые не являются составной частью цепочки «запрос — ответ», таких как изменение размера множества изображений, отправка сообщений различным устройствам, задачи машинного обучения, обработка финансовых данных или рендеринг кадров видеоролика? Подобные задачи обычно выполняются как фоновые процессы, которые принимают входные данные и выдают результаты без привязки к синхронной обработке пользовательских запросов.

В Kubernetes выполнение фоновых задач можно реализовать с помощью объекта Deployment или с помощью объекта Job. Объект Deployment подходит для работы

с постоянно поддерживаемой очередью задач, которая, например, используется в большинстве веб-приложений для таких задач, как изменение размера изображений. Объект Job служит для выполнения разовых и периодических (с использованием объекта CronJob) задач и обработки пакетной рабочей нагрузки, когда необходимо выполнить определенный объем работ.

ТЕРМИНОЛОГИЯ: «ЗАДАЧА» ИЛИ «ЗАДАНИЕ»?

При работе с фоновыми вычислениями разработчики часто используют слова «задача» (task) и «задание» (job) как синонимы (например: «очередь заданий», «очередь задач», «фоновое задание», «фоновая задача»). У платформы Kubernetes есть объект Job, поэтому во избежание путаницы я всегда буду использовать слово «задание», когда речь будет идти об этом объекте, и слово «задача», когда будет иметься в виду обобщенная концепция фоновой обработки безотносительно способа ее реализации (например: «фоновая задача», «очередь задач»).

В этой главе мы рассмотрим оба способа реализации фоновой обработки: и с помощью развертывания, и с помощью задания. Прочитав ее, вы научитесь настраивать постоянно поддерживаемую очередь фоновых задач для своего веб-приложения и определять пакетные рабочие нагрузки с заданным конечным состоянием, в том числе периодические и разовые задачи, используя для этого только возможности платформы Kubernetes.

10.1. ОЧЕРЕДИ ФОНОВЫХ ЗАДАЧ

В состав большинства существующих в Интернете веб-приложений входит крупный компонент для обработки фоновых задач, обеспечивающий выполнение тех задач, которые не могут быть выполнены за короткий промежуток времени, в течение которого приложение предоставляет ответ на HTTP-запрос. Как показало исследование поведения пользователей, проведенное компанией Google, чем дольше длится загрузка веб-страницы, тем выше вероятность, что произойдет «отскок» посетителя (то есть он сразу же ее покинет), и при этом «вероятность «отскока» возрастает на 32 % при увеличении времени загрузки страницы от 1 до 3 секунд»¹. Поэтому обычно не стоит заставлять пользователя ждать, пока будет выполнена какая-либо ресурсоемкая задача; вместо этого все такие задачи следует помещать в фоновую очередь и уведомлять пользователя о ходе их выполнения. При этом

¹ Google/SOASTA Research, 2017. <https://www.thinkwithgoogle.com/marketing-strategies/app-and-mobile/page-load-time-statistics/>.

разработчики всех компонентов приложения, от фронтенда до бэкенда, должны уделять первостепенное значение скорости загрузки страницы.

Данная скорость зависит от большого количества факторов, многие из которых (как, например, размер изображений или используемый JavaScript-код) не могут быть скорректированы системой Kubernetes. Важным показателем при оценке эффективности работы разворачиваемых в Kubernetes рабочих нагрузок является «время до первого байта» (Time to First Byte, TTFB), которое часто называют «временем ответа сервера». Это промежуток времени, в течение которого веб-сервер успевает обработать запрос и клиент приступает к загрузке ответа. Чтобы обеспечить высокую среднюю скорость загрузки страницы, вы должны позаботиться о том, чтобы время ответа сервера (TTFB) не превышало одну секунду. В большинстве случаев это требует отказа от выполнения обработки данных в основном потоке процесса обработки запроса. Вам нужно преобразовать файл в ZIP-архив перед его отправкой пользователю или уменьшить размер изображения, загруженного пользователем на сервер? Постарайтесь сделать так, чтобы эти операции не были составной частью процесса обработки запроса.

Как следствие, общепринятым подходом стала непрерывная обработка очереди фоновых задач. Веб-приложение помещает в фоновую очередь задачи, которые не может выполнить в основном потоке, в том числе, в частности, обработку данных (рис. 10.1). При этом оно может отображать на экране индикатор загрузки или какой-то иной индикатор хода выполнения фоновой задачи, отправлять сообщение по электронной почте после получения результатов или просто сообщать о том, что результаты будут готовы позже, — вы можете использовать любой удобный для вас способ взаимодействия с пользователем. Но в этой главе мы подробно рассмотрим, как такую очередь фоновых задач можно реализовать в Kubernetes.

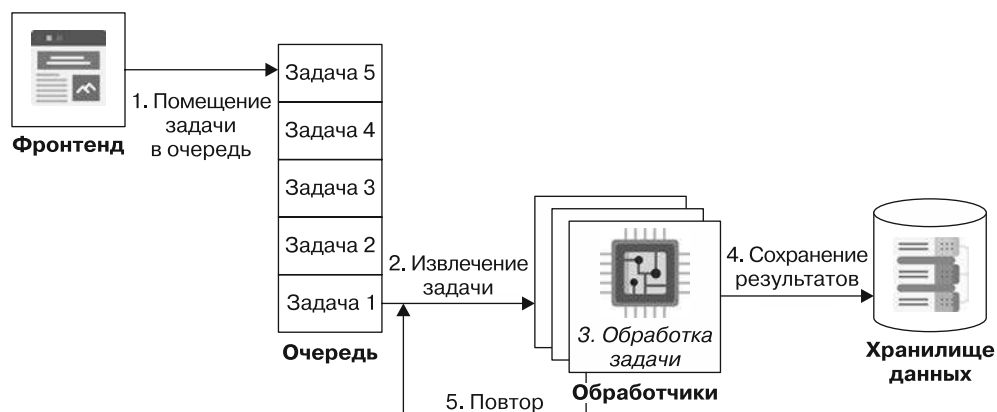


Рис. 10.1. Фронтендный веб-сервер с очередью фоновых задач

Как вы помните, в главе 3 мы говорили о том, что с помощью развертывания в Kubernetes можно управлять рабочими нагрузками, которые представляют собой набор непрерывно работающих подов. Это вполне подходит для очереди фоновых задач, поскольку в данном случае вы должны использовать набор непрерывно работающих подов в качестве обработчиков задач. При этом неважно, что в данном случае поды развертывания не предоставляются вовне в виде сервиса; главное, чтобы хотя бы один из ваших обработчиков работал непрерывно. Вы точно так же будете обновлять развертывание с помощью новых версий контейнера и масштабировать его вверх или вниз, как это делается при использовании развертывания для обслуживания принимаемых фронтендом запросов. Поэтому все, что мы узнали раньше, в равной мере применимо и к данному случаю, где развертывание служит для управления обработчиками фоновых задач.

10.1.1. Создание собственной очереди задач

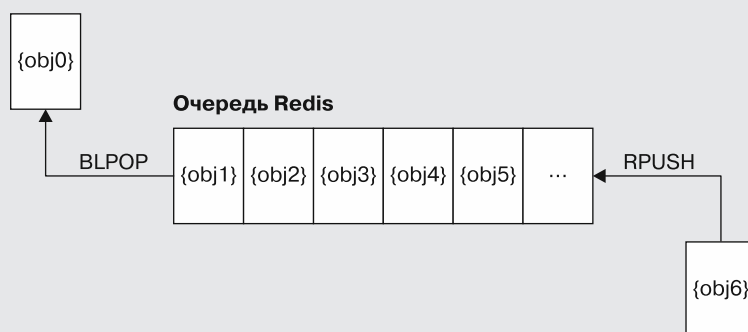
Рабочие поды, которые вы будете разворачивать в обрабатывающем задачи развертывании, должны просто принимать входные данные и выдавать некоторые результаты. Но где они будут брать эти данные? Чтобы обеспечить их предоставление, вы должны создать очередь, в которую могли бы добавлять задачи другие компоненты приложения. В ней будет храниться список ожидающих выполнения задач, которые будут последовательно выполнять поды с обработчиками. Существует несколько готовых реализаций такой фоновой очереди (часть которых мы рассмотрим в подразделе 10.1.4), но, чтобы лучше разобраться в том, как она работает, мы попробуем реализовать ее самостоятельно.

Сохранять данные нашей очереди мы будем с помощью того же варианта развертывания Redis, который мы создали в предыдущей главе. Благодаря встроенной поддержке очередей эта база прекрасно подходит для данной цели (и используется во многих готовых реализациях фоновой очереди). Наша система для обработки фоновых задач будет иметь очень простой дизайн, где веб-приложение помещает задачи в очередь Redis (что можно симитировать, добавляя задачи вручную), а поды с обработчиками, которыми управляет развертывание, извлекают задачи из очереди, выполняют их и ждут поступления следующей задачи.

В качестве простейшего конкретного примера возьмем такую ситуацию: допустим, наша задача сводится к тому, чтобы, используя в качестве входных данных целочисленное значение n , вычислить значение числа π с помощью формулы для ряда Лейбница, используя n итераций (чем больше итераций используется в этой формуле, тем более точный результат она выдает). На практике фоновые задачи могут представлять собой любые необходимые вам виды вычислений и принимать в качестве входных данных URL или извлекаемые из словаря пары «ключ — значение». Общий принцип действия при этом остается таким же.

ОЧЕРЕДИ В REDIS

Redis позволяет задействовать несколько удобных встроенных структур данных, одной из которых является используемая здесь очередь. Чтобы обеспечить организацию этой очереди по типичному для фоновых очередей принципу FIFO (First In, First Out — первым пришел, первым ушел), при котором элементы обрабатываются в порядке их поступления, мы воспользуемся двумя командами: `RPush`, которая добавляет элемент в конец очереди, и `BLPop`, которая извлекает элемент из начала очереди и выполняет блокировку при отсутствии элементов.



Если мы представим, что очередь протянулась слева направо и слева находится ее первый элемент, а справа — последний, то станет понятно, почему в названии этих команд есть буквы `L` и `R`, обозначающие соответственно левую (`left`) и правую (`right`) стороны. (Команда `RPush` «заталкивает» (`push`) крайний правый элемент, а команда `BLPop` — «вытаскивает» (`pop`) крайний левый элемент.) Дополнительная буква `B` в названии команды `BLPop` говорит о том, что это блокирующая версия команды `LPop`, которая ждет добавления нового элемента, если очередь является пустой, вместо того чтобы сразу же вернуть пустой указатель. Мы могли бы просто задействовать команду `LPop` в собственном цикле повторных попыток, однако использование блокировки позволяет нам обойтись без более ресурсоемкого режима «активного ожидания» и немного облегчить свою работу, возложив выполнение повторных попыток на Redis.

Создание контейнера с обработчиком

Прежде чем переходить к определению развертывания, следует создать контейнер с обработчиком. Я снова воспользуюсь языком Python, поскольку все, что нам нужно для реализации очереди задач, можно сделать с помощью всего нескольких

строк Python-кода. Все необходимые для данного контейнера файлы можно найти в репозитории с примерами кода из этой книги, в каталоге `Chapter10/pi_worker`. Там вы найдете три Python-файла, содержимое которых представлено в следующих листингах.

В листинге 10.1 представлена функция для выполнения задачи, которая непосредственно выполняет необходимые расчеты. Никак не учитывая факт существования очереди, эта функция просто обрабатывает данные необходимым нам образом. На практике вы будете использовать вычисления, которые потребуются в вашем случае (например, преобразование в ZIP-архив или изменение размера изображения).

Листинг 10.1. `Chapter10/pi_worker/pi.py`

```
from decimal import *

# Рассчитайте число пи, используя ряд бесконечности Грегори – Лейбница
def leibniz_pi(iterations):

    precision = 20
    getcontext().prec = 20
    piDiv4 = Decimal(1)
    odd = Decimal(3)

    for i in range(0, iterations):
        piDiv4 = piDiv4 - 1/odd
        odd = odd + 2
        piDiv4 = piDiv4 + 1/odd
        odd = odd + 2

    return piDiv4 * 4
```

В листинге 10.2 представлена реализация обработчика, который извлекает из начала очереди объект задачи, содержащий параметры выполняемой задачи, и выполняет эту задачу путем вызова функции `leibniz_pi`. При создании собственной реализации данного обработчика вам нужно будет позаботиться о том, чтобы помещаемый в очередь объект задачи содержал параметры функции, необходимые для выполнения задачи, например, параметры создаваемого ZIP-архива или обрабатываемого изображения. При этом будет полезно отделить логику обработки очереди от функции для выполнения задачи, чтобы эту функцию можно было повторно использовать в другой среде.

Элементы из нашей очереди на основе Redis извлекаются с помощью команды `Redis BLPOP`, которая либо извлекает из очереди первый элемент, либо блокирует вызов, если очередь является пустой, ожидая появления нового элемента. Чтобы данную функцию можно было использовать в продуктивной среде, потребуется добавить такие операции, как обработка сигналов в случае остановки пода (которой будет посвящен подраздел 10.1.2), а пока хватит и этого.

Листинг 10.2. Chapter10/pi_worker/pi_worker.py

```

import os
import redis
from pi import *

redis_host = os.environ.get('REDIS_HOST')
assert redis_host != None
r = redis.Redis(host=redis_host,
                 port='6379',
                 decode_responses=True)

print("starting")
while True:
    task = r.blpop('queue:task')
    iterations = int(task[1])
    print("got task: " + str(iterations))
    pi = leibniz_pi(iterations)
    print(pi)

```

Извлекаем имя хоста Redis из переменной среды (определенной в развертывании, которое будет представлено в листинге 10.5)

Подключаемся к сервису Redis

Извлекаем следующую задачу (и производим блокировку при отсутствии задач в очереди)

Выполняем задачу

Наконец, в листинге 10.3 представлен скрипт для добавления задач в очередь. На практике вы будете помещать задачи в очередь (выполняя команду RPUSH с параметрами задачи) по мере необходимости, например, в ответ на такие действия пользователя, как загрузка им изображения на сервер. В данном случае мы можем заполнить свою очередь задач неким набором случайных чисел. Представленный ниже код создает десять задач, используя в качестве входного значения каждой из них случайно выбранное целое число (в диапазоне от 1 до 10 миллионов).

Листинг 10.3. Chapter10/pi_worker/add_tasks.py

```

import os
import redis
import random

redis_host = os.environ.get('REDIS_HOST')
assert redis_host != None
r = redis.Redis(host=redis_host,
                 port='6379',
                 decode_responses=True)

random.seed()
for i in range(0, 10):
    rand = random.randint(10,100)
    iterations = rand * 100000
    r.rpush('queue:task', iterations)
    print("added task: " + str(iterations))

print("queue depth", str(r.llen('queue:task')))
print ("done")

```

Выполняем в цикле десять итераций для добавления десяти задач

Генерируем случайно выбранное число в качестве параметра задачи

Помещаем задачу в очередь

Метод `rpush` (который отображается на команду `RPUSH` (<https://redis.io/commands/rpush>)) добавляет указанное значение (в данном случае целочисленное) в список,

указанный с помощью ключа (в данном случае это ключ "queue:task"). Если вы думали, что использование Redis — нечто более сложное, то можете быть уверены: это все, что требуется для реализации очереди в данном случае. Не нужны никакие предварительные настройки или схемы.

Процесс упаковки этих трех Python-скриптов в контейнер не представляет ничего сложного. Как показано в листинге 10.4, мы можем воспользоваться официальным базовым образом языка Python, дополнив его необходимой нам зависимостью — самим клиентом Redis (если вы уже подзабыли, как создаются такие контейнеры, то еще раз вернитесь к главе 2). В качестве точки входа контейнера по умолчанию мы будем использовать запуск обработчика с помощью команды `python3 pi_worker.py`.

Листинг 10.4. Chapter10/pi_worker/Dockerfile

```
FROM python:3
RUN pip install redis
COPY . /app
WORKDIR /app
CMD python3 pi_worker.py
```

Добавляем необходимую зависимость — клиент Redis

Запускаем обработчик

Теперь, имея контейнер с обработчиком задач на Python, мы можем перейти к самому интересному: к процессу развертывания контейнера в Kubernetes.

Процесс развертывания в Kubernetes

На рис. 10.2 показано, как в данном случае выглядит архитектура объектов Kubernetes: у нас есть StatefulSet, который запускает Redis, и развертывание, которое запускает поды с обработчиками. Свою роль играет и веб-приложение, которое помещает задачи в очередь, но в данном случае мы делаем это вручную.

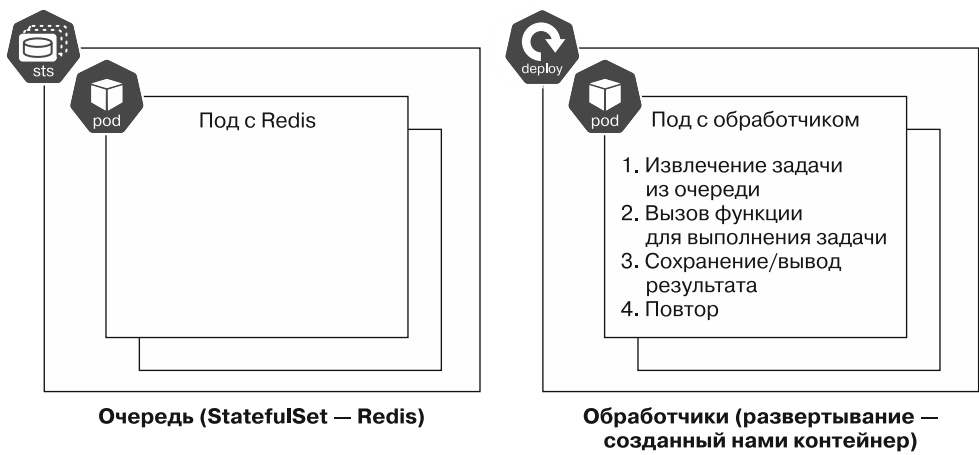


Рис. 10.2. Архитектура объектов Kubernetes при использовании очереди фоновых задач

Для развертывания подов с обработчиками в данном случае используется уже знакомый нам объект Deployment (который мы подробно рассмотрели в главе 3). При этом мы указываем расположение хоста с Redis, определяя переменную среды с именем хоста внутреннего сервиса (как мы делали это в подразделе 7.1.3) (листинг 10.5).

Листинг 10.5. Chapter10/10.1.1_TaskQueue/deploy_worker.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pi-worker
spec:
  replicas: 2
  selector:
    matchLabels:
      pod: pi
  template:
    metadata:
      labels:
        pod: pi
    spec:
      containers:
        - name: pi-container
          image: docker.io/wdenniss/pi_worker:1
          env:
            - name: REDIS_HOST
              value: redis-0.redis-service
            - name: PYTHONUNBUFFERED
              value: "1"
```

Образ контейнера с обработчиком

Имя хоста сервиса Kubernetes с основным подом базы данных Redis

Переменная среды, которая обеспечивает немедленный вывод сообщений, отображаемых с помощью функции print языка Python

Обратите внимание: данное развертывание выглядит совершенно так же, как любое из тех развертываний, которые мы уже использовали раньше для публикации веб-сервисов. Единственное различие состоит в том, что поды, развертываемые в данном случае, выполняют роль обработчика задач.

Наши поды с обработчиками требуют наличия экземпляра базы данных Redis, поэтому сначала мы развернем ее. Для этого можно использовать примеры из главы 9; подойдут и примеры из каталогов 9.2.1_StatefulSet_Redis и 9.2.2_StatefulSet_Redis_Multi. Перейдите в один из них и выполните следующие команды:

```
$ kubectl create -f Chapter09/9.2.2_StatefulSet_Redis_Multi
configmap/redis-config created
service/redis-service created
statefulset.apps/redis created
```

```
$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
redis-0   1/1     Running   0           20s
redis-1   1/1     Running   0           13s
redis-2   0/1     Init:0/1   0           7s
```

Теперь можно создать развертывание с обработчиками:

```
kubectl create -f Chapter10/10.1.1_TaskQueue/deploy_worker.yaml
```

Наконец, убедитесь в том, что все работает должным образом. Вы должны увидеть, что все пять подов перешли в состояние выполнения (Running):

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
pi-worker-55477bdf7b-7rmhp         1/1     Running   0           2m5s
pi-worker-55477bdf7b-ltcsd         1/1     Running   0           2m5s
redis-0                             1/1     Running   0           3m41s
redis-1                             1/1     Running   0           3m34s
redis-2                             1/1     Running   0           3m28s
```

НАБЛЮДЕНИЕ ЗА ХОДОМ ОБНОВЛЕНИЯ

Используемый здесь способ вызова команды `get` (с помощью команды вида `kubectl get pods`) позволяет получить лишь однократный вывод информации о состоянии объектов. Как вы помните, в главе 3 мы говорили о двух удобных способах отслеживания процесса обновления. Первый, встроенный в Kubernetes способ отслеживания изменений, сводится к тому, чтобы вызвать команду `kubectl get` с флагом `-w` (с помощью команды вида `kubectl get pods -w`). Второй и, на мой взгляд, более удобный способ — использовать команду `watch` операционной системы Linux. Так, например, команда `watch -d kubectl get pods` позволяет обновлять информацию о состоянии каждые 2 секунды, выделяя изменения. При желании вы можете задать и любой другой интервал обновления. Чтобы как можно больше упростить используемый в книге синтаксис, я не задействую флаг `-w` или команду `watch` в приводимых здесь примерах, однако имейте в виду, что вы всегда можете ими воспользоваться.

Теперь, когда наше приложение развернуто, мы можем посмотреть, что оно делает, проверив содержимое журналов. Kubernetes не позволяет организовать потоковый вывод записей журнала сразу из нескольких подов (например, из двух подов, выполняющих роль обработчика), но если мы укажем соответствующее развертывание, Kubernetes случайно выберет один из подов и начнет выводить его записи журнала:

```
$ kubectl logs -f deployment/pi-worker
Found 2 pods, using pod/pi-worker-55477bdf7b-7rmhp
starting
```

Кроме того, вы можете просмотреть записи журнала всех подов развертывания, не используя потоковый вывод. Для этого нужно сослаться на метку метаданных, указанную в шаблоне PodSpec, которая в данном случае выглядит как `pod=pi`:

```
$ kubectl logs --selector pod=pi
starting
starting
```

Каким бы способом просмотра записей журнала вы ни воспользовались, вы должны заметить, что наши поды не выводят никаких сообщений, кроме начального сообщения `starting`. Это объясняется тем, что они ждут, пока в очередь будет добавлена хотя бы одна задача. Что ж, добавим в очередь несколько задач, чтобы подам было над чем поработать.

Добавление задачи в очередь

Обычно операцию добавления задачи в фоновую очередь выполняет какой-то другой процесс или веб-приложение. Для этого достаточно выполнить вызов метода `redis.rpush('queue:task', object)`, передав в качестве параметра представляющий задачу объект.

В данном случае мы можем добавить в очередь несколько задач, запустив скрипт `add_tasks.py` (см. листинг 10.3), который мы вместе с прочим кодом упаковали в свой контейнер. Для этого нужно выполнить для контейнера следующую команду в одном из подов:

```
$ kubectl exec -it deploy/pi-worker -- python3 add_tasks.py
added task: 9500000
added task: 3800000
added task: 1900000
added task: 3600000
added task: 1200000
added task: 8600000
added task: 7800000
added task: 7100000
added task: 1400000
added task: 5600000
queue depth 8
done
```

Обратите внимание: когда мы передаем ссылку `deploy/pi-worker`, команда `exec` случайно выбирает один из подов, соответствующих этой ссылке (поэтому позаботьтесь о том, чтобы ни один из них не находился в состоянии `Terminating`). Вдобавок вы можете явно указать, в каком поде нужно выполнить скрипт, используя команду вида `kubectl exec -it $POD_NAME -- python3 add_tasks.py`.

Наблюдение за ходом выполнения задач

Добавив задачи в очередь, мы можем отобразить записи журнала одного из наших подов с обработчиками, чтобы посмотреть, как идет их выполнение:

```
$ kubectl logs -f deployment/pi-worker
Found 2 pods, using pod/pi-worker-54dd47b44c-bjccg
starting
got task: 9500000
3.1415927062213693620
got task: 8600000
3.1415927117293246813
got task: 7100000
3.1415927240123234505
```

В данном случае обработчик последовательно извлекает и выполняет добавленные нами задачи (которые сводятся к вычислению числа пи с помощью формулы для бесконечного ряда Грегори — Лейбница, с использованием n итераций).

10.1.2. Обработка сигналов в подах с обработчиками

Следует заметить, что в версии обработчика, которую мы рассмотрели выше, не реализована обработка сигналов SIGTERM, и потому она не позволит корректно завершить работы, если потребуется выполнить замену пода. Но поскольку поды иногда требуется перезапускать, например, в целях изменения конфигурации развертывания или используемых узлов, они должны уметь обрабатывать такие сигналы.

Дополнительно реализуем в нашем Python-коде обработчик сигналов SIGTERM, который будет давать обработчику задач указание завершить работу после выполнения текущей задачи. Кроме того, мы зададим определенное время ожидания в вызове для извлечения задачи из очереди, чтобы обработчик задач мог чаще проверять содержимое переменной, выступающей в роли выключателя. Если вы используете в своем проекте какой-то другой язык, то поищите информацию о том, как обработка сигналов SIGTERM реализуется в этом языке. В данном случае обеспечить завершение работы обработчика при получении сигнала SIGTERM можно следующим образом (листинг 10.6).

Листинг 10.6. Chapter10/pi_worker2/pi_worker.py

```
import os
import signal
import redis
from pi import *

redis_host = os.environ.get('REDIS_HOST')
assert redis_host != None
r = redis.Redis(host=redis_host,
                 port='6379',
                 decode_responses=True)

running = True

def signal_handler(signum, frame):
    print("got signal")
    running = False

signal.signal(signal.SIGTERM, signal_handler)

print("starting")
while running:
    task = r.blpop('queue:task', 5)
    if task != None:
        iterations = int(task[1])
        print("got task: " + str(iterations))
        pi = leibniz_pi(iterations)
        print(pi)
```

Вместо бесконечного цикла будет цикл, который закончится после присвоения этой переменной значения False в обработчике сигналов

Регистрируем обработчик сигналов, который будет присваивать значение False переменной running при получении сигнала SIGTERM

Извлекаем следующую задачу, однако на этот раз ждем только пять секунд, если очередь пуста (чтобы можно было проверить содержимое переменной running)

Теперь развернем новую версию обработчика в обновленной версии разворачивания, указав новый образ контейнера и задав в поле `terminationGracePeriodSeconds` двухминутный период времени для корректного завершения работы после получения сигнала SIGTERM (листинг 10.7).

Листинг 10.7. Chapter10/10.1.2_TaskQueue2/deploy_worker.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pi-worker
spec:
  replicas: 2
  selector:
    matchLabels:
      pod: pi
  template:
    metadata:
      labels:
        pod: pi
    spec:
      containers:
        - name: pi-container
          image: docker.io/wdenniss/pi_worker:2
          env:
            - name: REDIS_HOST
              value: redis-0.redis-service
            - name: PYTHONUNBUFFERED
              value: "1"
      resources:
        requests:
          cpu: 250m
          memory: 250Mi
      terminationGracePeriodSeconds: 120
```

Новая версия приложения, в которой реализована обработка сигналов SIGTERM

Добавлены запросы ресурсов, чтобы можно было использовать НРА

После получения сигнала SIGTERM контейнеру будет предоставляться 120 секунд для корректного завершения работы

Использование обработчика сигналов в сочетании с периодом корректного завершения работы означает, что, получив сигнал SIGTERM, данный под будет прекращать извлечение новых задач и завершать выполнение текущей задачи в течение 120 секунд. Если в вашем случае для этого требуется другой промежуток времени, то скорректируйте значение в поле `terminationGracePeriodSeconds` в большую или меньшую сторону.

Мы не рассмотрели здесь еще несколько других важных моментов. Например, если обработчик даст сбой во время выполнения задачи, то она будет потеряна, поскольку уже будет извлечена из очереди, но еще не будет выполнена. Кроме того, мы обрисовали здесь лишь самые минимальные возможности наблюдения за ходом выполнения задач. Поэтому не следует считать данный пример полноценной реализацией системы с очередью; он призван лишь показать принцип действия такой системы. Вы можете доработать данный пример, самостоятельно реализовав такие аспекты, как восстановление работоспособности в случае сбоя, или вместо этого использовать одну из свободно распространяемых реализаций очереди фоновых задач, которая сделает все это за вас. Выбор за вами.

10.1.3. Масштабирование подов с обработчиками

Масштабирование подов с обработчиками фоновых задач выполняется так же, как в случае любого другого развертывания (см. описание в главе 6). При этом можно задавать количество реплик вручную или с помощью горизонтального автоскейлера подов (HPA). В нашем примере рабочая нагрузка активно использует ресурсы процессора, поэтому мы можем использовать HPA в сочетании с показателем потребления ресурсов процессора (листинг 10.8).

Листинг 10.8. Chapter10/10.1.3_HPA/worker_hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: pi-worker-autoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: pi-worker
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 20
```

Ссылка на развертывание
из листинга 10.7

Данный автоскейлер автоматически масштабирует количество подов, используемых нашим развертыванием, в диапазоне от двух до десяти, стремясь к тому, чтобы поды в среднем использовали 20 % от запрашиваемого ими количества ресурсов процессора. Теперь создадим его с помощью следующей команды:

```
kubectl create -f Chapter10/10.1.3_HPA
```

Теперь еще раз выполним добавление задач в очередь, как мы уже делали в подразделе 10.1.1, и посмотрим, как наш HPA будет работать после этого. В команде `kubectl get` можно указать несколько типов ресурсов, поэтому мы можем проверить состояние всех используемых компонентов с помощью команды `kubectl get pods,hpa` (обычно я ставлю в начале еще и команду `watch` операционной системы Linux, чтобы отслеживать все дальнейшие изменения состояния):

```
$ kubectl exec -it deploy/pi-worker -- python3 add_tasks.py
$ kubectl get pods,hpa
```

NAME	READY	STATUS	RESTARTS	AGE
pod/pi-worker-54dd47b44c-22x9b	1/1	Running	0	2m42s
pod/pi-worker-54dd47b44c-9wppc	1/1	Running	0	2m27s
pod/pi-worker-54dd47b44c-bjccg	1/1	Running	0	13m

pod/pi-worker-54dd47b44c-f79hx	1/1	Running	0	2m42s
pod/pi-worker-54dd47b44c-fptj9	1/1	Running	0	2m27s
pod/pi-worker-54dd47b44c-hgbqd	1/1	Running	0	2m27s
pod/pi-worker-54dd47b44c-lj2bk	1/1	Running	0	2m27s
pod/pi-worker-54dd47b44c-wc267	1/1	Running	0	2m10s
pod/pi-worker-54dd47b44c-wk4dg	1/1	Running	0	2m10s
pod/pi-worker-54dd47b44c-x2s4m	1/1	Running	0	13m
pod/redis-0	1/1	Running	0	56m
pod/redis-1	1/1	Running	0	56m
pod/redis-2	1/1	Running	0	56m

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
pi-worker-autoscaler	Deployment/pi-worker	66%/20%	2	10	10

10.1.4. Свободно распространяемые реализации очереди задач

До сих пор мы использовали собственную реализацию очереди задач, поскольку это самый простой способ понять принцип действия таких очередей. Однако на практике обычно не требуется создавать очередь задач с нуля, так как вместо этого можно использовать одно из готовых решений.

Если вы работаете на Python, то популярным решением является библиотека RQ (<https://python-rq.org/>), которая позволяет помещать в очередь вызовы функций с заданными параметрами. Эти вызовы даже не нужно оборачивать в объекты, соответствующие требуемому протоколу.

Для Ruby популярным решением является созданная группой энтузиастов библиотека Resque (<https://github.com/resque/resque>), в которой для представления задач используются классы языка Ruby, реализующие метод `perform`. Это решение хорошо сочетается с фреймворком Active Job платформы Ruby on Rails, который позволяет использовать библиотеку Resque (или другую реализацию очереди) как формирующий очередь бэкэнд.

Прежде чем приступить к созданию собственной реализации очереди, подумайте о том, можете ли вы воспользоваться одним из таких готовых решений. Возможно, ни одно из них не подходит или вам нужно задействовать собственную реализацию по какой-то иной причине. Тем не менее я надеюсь, что теперь вы справитесь с созданием простейшей версии очереди.

10.2. ОБЪЕКТЫ JOB

Объект Job (Задание) служит в Kubernetes для представления конечного объема работы. Для выполнения пакетных заданий и фоновой обработки можно использовать как задание, так и развертывание. Однако задание рассчитано на выполнение конечного объема работы и, в принципе, может использоваться без реализации очереди с помощью такой структуры, как Redis. А развертывание

рассчитано на использование непрерывно работающей фоновой очереди, работу которой требуется координировать с помощью некой структуры (как мы делали в разделе 10.1). Кроме того, задание можно использовать для выполнения разовых или периодических задач наподобие операций технического обслуживания, для представления которых нецелесообразно использовать развертывание (поскольку оно перезапускает под в случае его остановки).

Возможно, вас удивляет то, зачем в Kubernetes была предусмотрена отдельная конструкция для однократного выполнения рабочей нагрузки, если для этого, по идее, можно использовать просто отдельный под. Вы действительно можете сделать так, чтобы отдельный под выполнял определенную задачу и после этого прекращал свою работу, но в Kubernetes нет контроллера, позволяющего гарантировать выполнение задачи в таком случае. Это может привести к проблемам, если, например, под будет вытеснен из-за выполнения некой операции технического обслуживания, так и не успев выполнить задачу. Задание предоставляет поду удобную надстройку, которая гарантирует выполнение задачи, повторно распределяя под в случае его сбоя или вытеснения, при необходимости отслеживая повторное выполнение и параллельную обработку.

Задание фактически представляет собой еще один высокоуровневый контроллер рабочих нагрузок, используемый в Kubernetes для управления подами наряду с развертыванием и StatefulSet. Эти три контроллера, в принципе, делают одно и то же: создают поды, обеспечивающие запуск реального кода. Но каждый из них использует собственную логику распределения подов и управления их работой. Развертывание служит для создания набора непрерывно работающих подов. Набор с сохранением состояния (StatefulSet) — для создания подов, которые имеют уникальный порядковый номер и позволяют подключать диски с помощью шаблона постоянного тома. Задание используется для создания подов, которые должны запускаться до тех пор, пока не будут выполнены, — если потребуется, то многократно.

10.2.1. Выполнение разовых задач с помощью объектов Job

Объекты Job хорошо подходят для выполнения разовых задач. Допустим, вам нужно выполнить очистку кэша или какую-то иную задачу технического обслуживания, которая, по сути, сводится просто к вызову команды внутри вашего контейнера. Вместо того чтобы выполнять команду `kubectl exec` в одном из запущенных подов, вы можете создать задание, которое запустит эту задачу как отдельный процесс с выделенными ему ресурсами, проследит за тем, чтобы он был успешно завершен (или сообщит о неудаче в случае сбоя), и позволит при необходимости выполнить его повторно.

На самом деле команду `exec` следует применять исключительно для отладки запущенных подов. Если использовать ее для выполнения задач технического

обслуживания, то выполняемая вами задача отнимает часть ресурсов у пода, а это плохо сказывается на производительности и может привести к тому, что у пода не хватит ресурсов для выполнения возросшего объема работы. Но если вы будете помещать свои задачи в задание, то каждая задача будет получать собственный под со специально выделенными ему ресурсами.

ИСПОЛЬЗОВАНИЕ МЕТОДОЛОГИИ «КОНФИГУРАЦИЯ КАК КОД» ПРИ ВЫПОЛНЕНИИ ЗАДАЧ ТЕХНИЧЕСКОГО ОБСЛУЖИВАНИЯ

В этой книге я постоянно упоминаю о том, насколько важно представлять все в виде конфигурации. Представив рутинные задачи технического обслуживания в виде заданий, а не в виде списка команд, выполняемых в командной оболочке с помощью копирования через буфер обмена, вы тем самым создадите воспроизводимую конфигурацию. Использование методологии GitOps, при которой все изменения продуктивного уровня проводятся через систему управления версиями Git (о чем мы подробно поговорим в следующей главе), позволяет пропускать задачи технического обслуживания через стандартный процесс ревью кода перед их применением в продуктивной среде.

В предыдущем разделе мы добавили задачи в очередь, выполняя команду в контейнере, а именно выполняя в одном из запущенных подов команду `python3 add_tasks.py` с помощью команды `kubectl exec`. Теперь вместо этого определим данный процесс как задание с отдельным подом. Мы можем использовать представленное ниже задание, которое делает возможным выполнение команды `python3 add_tasks.py` в контейнере с именем `pi` (листинг 10.9).

Листинг 10.9. Chapter10/10.2.1_Job/job_addwork.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: addwork
spec:
  backoffLimit: 2
  template:
    spec:
      containers:
      - name: pi-container
        image: docker.io/wdenniss/pi_worker:2
        command: ["python3", "add_tasks.py"]
        env:
        - name: REDIS_HOST
          value: redis-0.redis-service
        - name: PYTHONUNBUFFERED
          value: "1"
      restartPolicy: Never
```

Используем тот же образ контейнера, который выступает в качестве обработчика...

...но при этом даем указание выполнить другую команду, которая добавляет задачи

Вы можете заметить здесь уже знакомый нам паттерн: в поле `spec` размещается поле `template`, внутри которого размещается еще одно поле `spec`. Дело в том, что задание содержит шаблон `PodSpec`, как это делают развертывание и набор `StatefulSet` (структура этого объекта показана на рис. 10.3). В данном шаблоне можно определить любые параметры пода, в том числе, например, требования ресурсов и переменные среды; исключение составляют лишь несколько недопустимых комбинаций параметров, которые просто не имеют смысла в контексте заданий.

Заголовок	<pre>apiVersion: batch/v1 kind: Job</pre>	
	<pre>metadata: name: addwork spec:</pre>	
Определение объекта Job	<pre> backoffLimit: 2 template:</pre>	
	<pre> spec: containers: - name: pi-container image: docker.io/wdenniss/pi_worker:2 command: ["python3", "add_tasks.py"] env: - name: REDIS_HOST value: redis-0.redis-service - name: PYTHONUNBUFFERED value: "1" restartPolicy: Never</pre>	

Рис. 10.3. Структура объекта `Job`

В шаблоне `PodSpec` своего задания мы определяем те же переменные среды, которые мы определяли ранее в шаблоне `PodSpec` своего развертывания. Это приятное следствие единообразия структуры объектов, используемых в Kubernetes: вы можете задействовать одну и ту же спецификацию пода вне зависимости от того, внутри какого объекта она применяется. Что касается изменений, то мы добавили здесь поля `restartPolicy` и `backoffLimit`.

Поле `restartPolicy` представляет собой свойство встроенного в задание шаблона `PodSpec`, которое определяет, будет ли размещенная на узле утилита `kubelet` перезапускать контейнеры, завершившие свою работу с ошибкой. В случае задания этому полю можно присвоить значение `OnFailure`, чтобы контейнер перезапускался при сбое, или значение `Never`, чтобы сбой игнорировался. Вариант `Always`, благодаря которому под перезапускается при успешном завершении работы, больше подходит для развертывания и в случае задания не имеет смысла.

Поле `backoffLimit` составляет часть определения задания и задает количество его перезапусков, вне зависимости от того, что будет этому причиной: сбой в работе

самого задания или сбой в работе узла. Например, если задание два раза само даст сбой, а потом будет вытеснено вследствие выполнения на узле операции технического обслуживания, это будет засчитано как три перезапуска. Некоторые разработчики предпочитают использовать в ходе разработки политику перезапуска `Never`, поскольку это упрощает отладку: вы сразу видите все поды, которые дали сбой, и можете исследовать выдаваемые ими записи журналов.

Создайте задание, как любой другой объект Kubernetes, и затем проверьте, к каким результатам это привело:

```
$ kubectl create -f Chapter10/10.2.1_Job/job_addwork.yaml
job.batch/addwork created
$ kubectl get job,pods
```

NAME	COMPLETIONS	DURATION	AGE
job.batch/addwork	1/1	3s	9s

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-99q5k	0/1	Completed	0	9s
pod/pi-worker-6f6dfdb548-7krpm	1/1	Running	0	7m2s
pod/pi-worker-6f6dfdb548-pzxq2	1/1	Running	0	7m2s
pod/redis-0	1/1	Running	0	8m3s
pod/redis-1	1/1	Running	0	7m30s
pod/redis-2	1/1	Running	0	6m25s

После того как задание успешно выполнено, запускаются поды с обработчиками, которые начинают обрабатывать только что добавленные задачи. Если до запуска задания вы развернули НРА, как было описано ранее, то вскоре в вашей системе будут созданы дополнительные контейнеры. В моем случае это произошло так:

```
$ kubectl get pods,hpa
```

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-99q5k	0/1	Completed	0	58s
pod/pi-worker-6f6dfdb548-7krpm	1/1	Running	0	7m51s
pod/pi-worker-6f6dfdb548-h6pld	0/1	ContainerCreating	0	8s
pod/pi-worker-6f6dfdb548-pzxq2	1/1	Running	0	7m51s
pod/pi-worker-6f6dfdb548-qpgxp	1/1	Running	0	8s
pod/redis-0	1/1	Running	0	8m52s
pod/redis-1	1/1	Running	0	8m19s
pod/redis-2	1/1	Running	0	7m14s

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
pi-worker-autoscaler	Deployment/pi-worker	100%/20%	2	10	2

У задания есть важная особенность: независимо от того, как оно будет выполнено, успешно или нет, вы не сможете распределить его заново, дав ему то же имя (чтобы выполнить заново ту же работу), если сначала его не удалите. Это объясняется тем, что даже после выполнения своей работы объект `Job` продолжает присутствовать в системе Kubernetes. Его можно удалить, как любой другой объект, созданный на основе конфигурации:

```
kubectl delete -f Chapter10/10.2.1_Job/job_addwork.yaml
```

Напомню еще раз: объект `Job` предназначен для выполнения определенной задачи или работы. В нашем случае эта работа сводилась к выполнению простой команды, но в других ситуациях она может представлять собой очень объемную и сложную вычислительную задачу. Если вам нужно выполнить разовый фоновый процесс, то просто поместите его в контейнер, определите для него задание и запланируйте его исполнение. После того как оно перейдет в состояние `Completed` (закончив работу и выдав код успешного завершения), ваша работа будет выполнена.

У объекта `Job` есть еще два параметра, которые мы не задействовали, пока разбирались с нашей разовой задачей: `completions` и `parallelism`. С помощью этих параметров можно сделать возможной пакетную обработку множества задач, используя один объект `Job`, о чем мы поговорим подробнее в разделе 10.3. Но прежде поговорим о том, как распределять задания через регулярные интервалы времени.

10.2.2. Выполнение запланированных задач с помощью объектов `CronJob`

В предыдущем подразделе мы взяли команду, которая до этого выполнялась в кластере вручную, и обернули ее в объект `Kubernetes`, специально предназначенный для выполнения задач. Теперь любой член команды разработчиков может выполнять эту задачу путем создания объекта `Job`, избавившись от необходимости вспоминать точное написание сложной команды `exes`.

Но что, если вам требуется выполнять задание многократно, с определенной периодичностью? Для такого случая в `Kubernetes` предусмотрен объект `CronJob`, который инкапсулирует объект `Job`, дополняя его параметром частоты исполнения. С помощью этого параметра вы можете сделать так, чтобы задание выполнялось через любой требуемый временной интервал (листинг 10.10). Объект `CronJob` часто используется для регулярного выполнения таких задач, как ежедневная очистка кэша.

Листинг 10.10. Chapter10/10.2.2_CronJob/cronjob_addwork.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: addwork
spec:
  schedule: "*/5 * * * *"
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          containers:
            - name: pi-container
              image: docker.io/wdenniss/pi_worker:2
```

← Такая же спецификация задания, как в листинге 10.9

← Расписание cron, используемое для запуска задания

```

command: ["python3", "add_tasks.py"]
env:
- name: REDIS_HOST
  value: redis-0.redis-service
- name: PYTHONUNBUFFERED
  value: "1"
restartPolicy: Never

```

Как можно заметить, мы просто скопировали в объект `CronJob` всю спецификацию задания (то есть словарь `spec`) из листинга 10.9, разместив ее в словаре `spec` с ключом `jobTemplate` (Шаблон задания), и добавили на уровне спецификации еще одно поле с именем `schedule` (расписание). При этом, как вы помните, задание имеет собственный шаблон для создания подов, который содержит еще одну спецификацию.

Таким образом, в объект `CronJob` встроен объект `Job` со встроенным в него подом. Структура этого объекта представлена на рис. 10.4.



Рис. 10.4. Структура объекта `CronJob`

Таким образом, теперь мы знаем, как устроен объект `CronJob`. Но что можно сказать о поле `schedule`, добавляемом в спецификацию этого объекта? Оно служит для определения частоты выполнения задания в хорошо известном формате cron

операционных систем Unix. Он представляет собой очень гибкий способ описания нужной периодичности. Используемое в листинге 10.10 выражение `*/5 * * * *` означает, что задание должно выполняться с периодичностью 5 минут. Кроме того, мы могли бы указать, что задание нужно выполнять с периодичностью 30 минут (`*/30 * * * *`), ежедневно в полночь (`0 0 * * *`), по понедельникам в 16:00 (`0 16 * * 1`) и т. д. Я рекомендую использовать визуальный редактор формата cron (при желании вы легко найдете его в Интернете). Благодаря ему вы сможете сразу убедиться в правильности используемого расписания в формате cron, вместо того чтобы, например, ждать целую неделю, чтобы убедиться в том, что ваше задание действительно выполняется с заданной вами еженедельной периодичностью.

СТРУКТУРА ОБЪЕКТОВ В KUBERNETES

Из-за череды спецификаций и шаблонов, встроенных в другие спецификации и шаблоны, иногда кажется, что этот процесс последовательного встраивания одних объектов в другие продолжается в Kubernetes до бесконечности. Здесь мы видим объект `CronJob`, спецификация которого содержит шаблон выполняемого по расписанию задания, которое тоже содержит шаблон пода с еще одной спецификацией. В этой череде одних и тех же элементов поначалу непросто разобраться, но у такого подхода есть большой плюс: вы можете использовать в шаблоне `jobTemplate` любое указанное в документации поле объекта `Job`, а в разделе `spec` — любое поле пода, поскольку каждый объект в Kubernetes содержит другие объекты.

При этом будет полезно запомнить некоторые из принятых в этой системе обозначений. Так, при встраивании пода в другой объект спецификацию этого пода принято называть шаблоном *PodSpec* (например, данный шаблон содержится в определении объекта `Deployment`). Поды, создаваемые контроллером такого высокоуровневого объекта, ничем не отличаются от подов, которые создаются напрямую с помощью отдельной спецификации. Единственное различие состоит в том, что поды, которые создаются контроллером (например, поды объекта `Job` или `Deployment`), остаются под его наблюдением после их создания (и он выполняет над ними такие операции, как перезапуск в случае сбоя).

Создайте новый объект `CronJob`:

```
$ kubectl create -f Chapter10/10.2.2_CronJob/cronjob_addwork.yaml
cronjob.batch/addwork created
```

```
$ kubectl get cronjob,job
```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
cronjob.batch/addwork	<code>*/5 * * * *</code>	False	0	<none>	58s

Выждав несколько минут (в данном случае задание создается через каждые 5 минут, то есть в моменты времени :00, :05 и т. д.), вы увидите информацию о задании и создаваемых им подах:

```
$ kubectl get cronjob,job,pods
```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
cronjob.batch/addwork	* / 5 * * * *	False	0	2m38s	3m11s

NAME	COMPLETIONS	DURATION	AGE
job.batch/addwork-27237815	1 / 1	107s	2m38s

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-27237815-b44ws	0 / 1	Completed	0	2m38s
pod/pi-worker-6f6dfdb548-5czkc	1 / 1	Running	5	14m
pod/pi-worker-6f6dfdb548-gfkccq	1 / 1	Running	0	7s
pod/pi-worker-6f6dfdb548-pl584	1 / 1	Running	0	7s
pod/pi-worker-6f6dfdb548-qpgxp	1 / 1	Running	5	25m
pod/redis-0	1 / 1	Running	0	14m
pod/redis-1	1 / 1	Running	0	33m
pod/redis-2	1 / 1	Running	0	32m

Объект CronJob будет создавать с заданной периодичностью новое задание, которое, в свою очередь, будет создавать новый под. Вы будете видеть в списке уже выполненные задания, поскольку они продолжают существовать, перейдя в состояние Completed. Вы можете указать, сколько именно старых заданий должна «помнить» система, определив в спецификации объекта CronJob (<https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/cron-job-v1/#CronJobSpec>) параметры `successfulJobsHistoryLimit` и `failedJobsHistoryLimit`.

ЧАСОВЫЕ ПОЯСА

Имейте в виду, что объект CronJob будет отсчитывать время с помощью того часового пояса, который используется в вашем кластере, а для многих облачных платформ, в том числе Google Kubernetes Engine (GKE), это часовой пояс UTC. Используемый часовой пояс задается контроллерным компонентом системы Kubernetes, который работает на управляющем слое. Многие управляемые облачные платформы не позволяют напрямую связываться с узлами управляющего слоя. В таком случае можно связаться с рабочими узлами, которые обычно используют тот же часовой пояс. Вот, например, как можно создать разовый под, который выполнит команду `date` операционной системы Linux и завершит свою работу после этого (полученный результат выделен полужирным шрифтом):

```
$ kubectl run date --restart=Never -it --rm --image ubuntu -- date +%Z
UTC
pod "date" deleted
```

10.3. ПАКЕТНОЕ ВЫПОЛНЕНИЕ ЗАДАЧ С ИСПОЛЬЗОВАНИЕМ ОБЪЕКТОВ JOB

А как быть, если вам нужно сделать так, чтобы некий набор задач выполнялся однократно или регулярно? В разделе 10.1 я уже говорил, что объект `Deployment` следует использовать в том случае, когда нужно реализовать *непрерывно работающую* очередь задач. Но если речь идет о выполнении ограниченного набора задач, то для этой цели больше подойдет объект `Job`.

Если вам нужно, чтобы очередь задач была представлена в виде динамической структуры данных, как это было сделано в разделе 10.1, но чтобы при этом обработчики прекращали работу после того, как очередь станет пустой, то это можно реализовать с помощью задания. Вдобавок при использовании развертывания требуется задействовать такую вспомогательную систему, как горизонтальный автоскейлер подов (`HorizontalPodAutoscaler`), которая масштабирует количество подов с обработчиками вверх или вниз в зависимости от текущей длины очереди. При использовании задания поды с обработчиками могут сами уведомлять контроллер задания о том, что выполнение задач завершено и их можно удалить, чтобы высвободить ресурсы.

Еще один возможный подход сводится к тому, чтобы реализовать с помощью задания статическую очередь задач, не используя при этом какую-либо базу данных. Допустим, вы знаете, что вам нужно обработать очередь, содержащую 100 задач. В таком случае вы можете запустить задание 100 раз. Здесь есть одна сложность: экземпляр пода каждого задания нужно как-то уведомлять о том, какую из 100 задач он должен выполнить; эта проблема решается с помощью использования индексированных заданий.

В этом разделе мы рассмотрим оба способа реализации пакетной обработки: и динамический, и статический.

10.3.1. Обработка динамической очереди с помощью объекта `Job`

Переработаем дизайн динамической очереди из раздела 10.1 так, чтобы вместо развертывания она использовала задание. Оба этих объекта позволяют создавать несколько подов с обработчиками и перезапускают их в случае сбоя, но развертывание не дает использовать нотацию успешного выполнения (то есть окончания работы пода с выдачей кода успешного завершения). Развертывание просто следит за тем, чтобы в любой момент времени было запущено указанное вами количество реплик. Но когда для управления подами используется задание, окончание работы пода с выдачей кода успешного завершения (такого как `exit 0`) указывает контроллеру задания на то, что под уже не нужно перезапускать, поскольку он успешно выполнил свою работу.

Эта особенность объектов Job, позволяющая отдельным обработчикам сообщать о завершении своей работы, делает их очень удобным инструментом. Если используемая вами среда Kubernetes может меняться динамически, например, за счет применения автомасштабирования (как сервис GKE в режиме «Автопилот»), то с помощью задания вы можете обеспечить выполнение всей необходимой работы по принципу «настроил и забыл»: после своего распределения объект Job автоматически выполнит необходимую работу и затем освободит выделенные ему ресурсы. Обратите внимание: вы не можете снова запустить задание после успешного завершения его работы, но можете удалить его и создать заново, тем самым запустив новую обработку очереди.

Чтобы наш контейнер с обработчиком задач мог корректно работать под управлением объекта Job, нужно добавить условие успешного завершения работы, которое будет срабатывать, когда очередь будет становиться пустой. Как может выглядеть переработанная версия обработчика, показано в листинге 10.11.

Листинг 10.11. Chapter10/pi_worker3/pi_worker.py

```
import os
import signal
import redis
from pi import *

redis_host = os.environ.get('REDIS_HOST')
assert redis_host != None
r = redis.Redis(host=redis_host,
                port='6379',
                decode_responses=True)

running = True

def signal_handler(signum, frame):
    print("got signal")
    running = False

signal.signal(signal.SIGTERM, signal_handler)

print("starting")
while running:
    task = r.blpop('queue:task', 5)
    if task != None:
        iterations = int(task[1])
        print("got task: " + str(iterations))
        pi = leibniz_pi(iterations)
        print(pi)
    else:
        if os.getenv('COMPLETE_WHEN_EMPTY', '0') != '0':
            print("no more work")
            running = False

exit(0)
```

Когда используется настройка COMPLETE_WHEN_EMPTY=1, система не ждет добавления новых задач, если очередь пуста

Код завершения 0 указывает системе Kubernetes на то, что задание успешно завершило свою работу

Теперь, когда наш контейнер с обработчиком уже должным образом настроен для работы в контексте задания, мы можем создать задание, которое будет его запускать. При этом надо заметить, что если в случае развертывания количество одновременно работающих подов задается с помощью поля `replicas`, то в случае задания для этой цели используется параметр `parallelism` (листинг 10.12).

Листинг 10.12. Chapter10/10.3.1_JobWorker/job_worker.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: jobworker
spec:
  backoffLimit: 2
  parallelism: 2
  template:
    metadata:
      labels:
        pod: pi
    spec:
      containers:
        - name: pi-container
          image: docker.io/wdenniss/pi_worker:3
          env:
            - name: REDIS_HOST
              value: redis-0.redis-service
            - name: PYTHONUNBUFFERED
              value: "1"
            - name: COMPLETE_WHEN_EMPTY
              value: "1"
      restartPolicy: OnFailure
```

Сколько одновременно работающих подов следует использовать для обработки данной очереди задач

Новая версия контейнера, дополненная логикой сигнализации об успешном выполнении

Переменная среды, которая активирует логику сигнализации об успешном выполнении в обработчике

Если мы посмотрим, какая спецификация пода с обработчиком встроена в задание из листинга 10.12 и в развертывание из листинга 10.7, то увидим, что в обоих случаях используется одинаковый шаблон `PodSpec` (поля внутри элемента `template`). Разница лишь в том, что в случае задания он дополнительно содержит переменную среды `COMPLETE_WHEN_EMPTY` и поле `restartPolicy`. Переопределить политику перезапуска нужно было потому, что используемая по умолчанию политика `Always` не подходит для заданий, которые должны прекращать свою работу после успешного выполнения задачи. Благодаря использованию политики перезапуска `OnFailure` поды с обработчиками будут перезапускаться только после их сбоя, что и требуется. Применительно к заданию, в принципе, можно обойтись без метки метаданных, однако она может оказаться полезной, если нам потребуется отобразить записи журналов сразу нескольких подов, как было описано ранее (с помощью команды вида `kubectl logs --selector pod=pi`).

ПОДГОТОВИТЕЛЬНЫЙ ПРОЦЕСС

Прежде чем запустить версию обработчика задач, реализованную в виде задания, удалите предыдущую версию обработчика, реализованную в виде развертывания, а также удалите объекты Job и CronJob с именем addwork, чтобы их можно было запустить заново:

```
$ cd Chapter10
$ kubectl delete -f 10.1.2_TaskQueue2
deployment.apps "pi-worker" deleted
$ kubectl delete -f 10.2.1_Job
job.batch "addwork" deleted
$ kubectl delete -f 10.2.2_CronJob
cronjob.batch "addwork" deleted
```

Наша очередь на основе Redis может содержать задания, поэтому ее также можно очистить, используя команду ltrim:

```
$ kubectl exec -it pod/redis-0 -- redis-cli ltrim queue:task 0 0
OK
```

Если вы предпочитаете интерактивный подход, то очистить очередь можно и с помощью утилиты redis-cli:

```
$ kubectl exec -it pod/redis-0 -- redis-cli
127.0.0.1:6379> LTRIM queue:task 0 0
OK
```

Теперь попробуем воспользоваться нашим новым обработчиком, реализованным на основе задания. Сначала добавим задачи в нашу очередь на основе Redis, используя для этого разовое задание, как мы уже делали ранее:

```
$ cd Chapter10
$ kubectl create -f 10.2.1_Job
job.batch/addwork created
```

```
$ kubectl get job,pod
```

NAME	COMPLETIONS	DURATION	AGE
job.batch/addwork	0/1	19s	19s

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-l9fgg	0/1	ContainerCreating	0	19s
pod/redis-0	1/1	Running	0	19h
pod/redis-1	1/1	Running	0	19h
pod/redis-2	1/1	Running	0	19h

СОВЕТ

Один и тот же объект Job нельзя создать дважды, даже если к моменту создания второго экземпляра первый уже успешно завершит свою работу. Чтобы запустить задание повторно, сначала нужно удалить первый экземпляр.

После того как задание `addwork` выполнит свою работу, можно будет запустить наше новое задание для обработки очереди. В отличие от предыдущих примеров, в данном случае важно соблюдать последовательность запуска, поскольку при отсутствии задач в очереди поды с обработчиками будут сразу же прекращать функционирование. Поэтому сначала убедитесь в том, что задание `addwork` уже выполнило свою работу. Проверить его состояние можно так:

```
$ kubectl get job,pod
```

NAME	COMPLETIONS	DURATION	AGE
job.batch/addwork	1/1	22s	36s

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-19fgg	0/1	Completed	0	37s
pod/redis-0	1/1	Running	0	19h
pod/redis-1	1/1	Running	0	19h
pod/redis-2	1/1	Running	0	19h

Если вы видите, что под задания `addwork` перешел в состояние `Completed`, то можно запланировать исполнение задания для обработки очереди:

```
$ kubectl create -f 10.3.1_JobWorker
job.batch/jobworker created
```



```
$ kubectl get job,pod
```

NAME	COMPLETIONS	DURATION	AGE
job.batch/addwork	1/1	22s	3m45s
job.batch/jobworker	0/1 of 2	2m16s	2m16s

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-19fgg	0/1	Completed	0	3m45s
pod/jobworker-swb6k	1/1	Running	0	2m16s
pod/jobworker-tn6cd	1/1	Running	0	2m16s
pod/redis-0	1/1	Running	0	19h
pod/redis-1	1/1	Running	0	19h
pod/redis-2	1/1	Running	0	19h

После этого поды с обработчиками начнут обрабатывать очередь, и когда она станет пустой, каждый из них закончит выполнять свою текущую задачу и свою работу, выдав код успешного завершения. Если вы хотите отслеживать глубину очереди, чтобы знать, когда примерно закончится обработка, то можете выполнить команду `Redis llen`, которая выводит текущую длину очереди:

```
$ kubectl exec -it pod/redis-0 -- redis-cli llen queue:task
(integer) 5
```

После того как длина очереди станет равной нулю, все поды перейдут в состояние `Completed`. Но это произойдет не сразу, а лишь после того, как будут выполнены все выполняемые ими задачи:

```
$ kubectl get job,pod
```

NAME	COMPLETIONS	DURATION	AGE
job.batch/addwork	1/1	22s	3m45s
job.batch/jobworker	0/1 of 2	2m16s	2m16s

NAME	READY	STATUS	RESTARTS	AGE
pod/addwork-l9fgg	0/1	Completed	0	10m09s
pod/jobworker-swb6k	1/1	Completed	0	8m40s
pod/jobworker-tn6cd	1/1	Completed	0	8m40s
pod/redis-0	1/1	Running	0	19h
pod/redis-1	1/1	Running	0	19h
pod/redis-2	1/1	Running	0	19h

Помните о том, что повторно запустить любое задание можно только после того, как оно будет удалено и создано заново, даже если это задание уже было выполнено и все его поды завершили работу. Например, чтобы еще раз запустить рассмотренный выше пример, необходимо удалить и создать заново оба задания (и для добавления задач, и для обработки очереди):

```
$ kubectl delete -f 10.2.1_Job
job.batch "addwork" deleted
$ kubectl delete -f 10.3.1_JobWorker
job.batch "jobworker" deleted
$ kubectl create -f 10.2.1_Job
job.batch/addwork created
$ kubectl create -f 10.3.1_JobWorker
job.batch/jobworker created
```

10.3.2. Обработка статической очереди с помощью задания

Вместо того чтобы использовать динамическую очередь, такую как Redis, для хранения списка задач, как мы делали в предыдущем подразделе, можно использовать несколько способов запуска заданий с помощью статической очереди. При использовании статической очереди ее длина известна заранее и определяется как часть конфигурации задания, и для каждой задачи создается отдельный под. Вы могли бы заставлять обработчики задач работать до тех пор, пока очередь не станет пустой, но вместо этого заранее указываете, сколько раз создавать экземпляр пода с обработчиком.

Такой подход очень выгоден: он не требует, чтобы контейнер знал, как извлекаются задачи из динамической очереди (если контейнеры уже существуют, то потребуются приложить дополнительные усилия, чтобы наделить их этой функциональностью). Но есть у этого подхода и недостаток: вам нужно выполнять дополнительные настройки в Kubernetes. Фактически данный подход переносит часть необходимых настроек из контейнера с обработчиком в объекты Kubernetes.

Обратите внимание: если из-за имеющихся требований вы не можете изменить контейнер с обработчиком, это еще не значит, что вы должны использовать статическую очередь. В таком случае можно разместить в поде несколько контейнеров, один из которых будет извлекать задачи из очереди и передавать параметры другим контейнерам.

Как же определить статическую очередь задач в конфигурации Kubernetes? Это можно сделать разными способами, три из которых я опишу ниже.

Реализация статической очереди с помощью индексов

Наиболее интересный вариант, на мой взгляд, — реализация статической очереди с помощью индексируемых заданий. Этот подход удобно использовать в том случае, когда заранее известно, сколько задач необходимо выполнить, и их список можно легко проиндексировать. Хороший пример такой ситуации — рендеринг анимированного видеоряда. Вам заранее известно количество кадров (длина очереди), и вы легко можете передавать каждому экземпляру обработчика номер обрабатываемого кадра (то есть индекс в очереди).

Kubernetes запускает задание указанное вами количество раз (в параметре `completions`), создавая для каждой задачи отдельный под. При каждом следующем запуске задания ему передается последовательно возрастающий индекс (через переменную среды `$JOB_COMPLETION_INDEX`). Такой подход очень удобен, когда выполняемая работа является индексируемой по самой своей природе, как в случае отображения кадров анимированного видеоряда. Вы легко можете дать системе Kubernetes указание запустить задание 30 000 раз, чтобы она обработала 30 000 кадров, присваивая каждому поду номер кадра. Еще один напрашивающийся здесь прием сводится к тому, чтобы вместе с индексом предоставлять каждому заданию полный список задач, используя некую структуру данных (например, массив задач, представленный в виде YAML-кода, или обычный текст, содержащий описание каждой задачи в отдельной строке). Благодаря такому подходу каждое задание может извлекать нужную задачу из списка, используя полученный индекс.

В листинге 10.13 представлен пример конфигурации индексируемого задания, которое просто выводит на экран номер кадра. При желании вы можете доработать данный пример, заменив эту простейшую задачу реальной логикой рендеринга.

Листинг 10.13. Chapter10/10.3.2_IndexedJob/indexed_job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: echo
spec:
  completions: 5
  parallelism: 3
  completionMode: Indexed
  template:
```

Сколько раз нужно запустить задание (верхняя граница индекса)

Сколько параллельно работающих подов с обработчиком необходимо использовать

Используем данный объект в индексируемом режиме, передавая индекс в виде переменной среды `JOB_COMPLETION_INDEX`

```

metadata:
  labels:
    pod: framerender
spec:
  restartPolicy: Never
  containers:
  - name: 'worker'
    image: 'docker.io/library/busybox'
    command: ["echo", "render frame: ${JOB_COMPLETION_INDEX}"]

```

Команда для вывода
на экран текущего индекса

Запустите данное задание и проследите за его работой:

```

$ kubectl create -f 10.3.2_IndexedJob
job.batch/echo created
$ kubectl get job,pods

```

NAME	COMPLETIONS	DURATION	AGE
job.batch/echo	5/5	20s	58s

NAME	READY	STATUS	RESTARTS	AGE
pod/echo-0-r8v52	0/1	Completed	0	58s
pod/echo-1-nxwsm	0/1	Completed	0	58s
pod/echo-2-49kz2	0/1	Completed	0	58s
pod/echo-3-9lvt2	0/1	Completed	0	51s
pod/echo-4-2pstq	0/1	Completed	0	45s

Теперь отобразите записи журнала, используя следующую команду:

```

$ kubectl logs --selector pod=framerender
render frame: 0
render frame: 1
render frame: 2
render frame: 3
render frame: 4

```

Вместо того чтобы использовать в приложении переменную среды напрямую, можно задействовать контейнер инициализации, который будет получать индекс и выполнять подготовительные операции, необходимые для основного контейнера, — например, создавать выполняемый этим контейнером скрипт.

Реализация статической очереди с помощью очереди сообщений

Обойтись без изменения контейнера можно и по-другому: создавать очередь сообщений, из которой каждый под будет извлекать свою задачу. Контейнеры могут получать необходимые параметры через переменные среды, определенные в конфигурации Kubernetes, поэтому вы можете определить задание так, чтобы контейнер не знал о существовании очереди. Такая очередь по-прежнему является статической (ведь вы заранее указываете количество задач и запускаете отдельный под с обработчиком для каждой задачи), но в то же время требует использования некой структуры данных (такой как очередь сообщений). В документации по Kubernetes можно найти прекрасный пример использования этого подхода, когда очередь сообщений реализуется с помощью брокера сообщений RabbitMQ (<https://kubernetes.io/docs/tasks/job/coarse-parallel-processing-work-queue/>).

Реализация статической очереди с помощью скриптов

Еще один подход сводится к тому, чтобы просто создавать отдельное задание для каждой задачи с помощью скрипта. Иными словами, если вам нужно выполнить 100 задач, то вы можете написать скрипт, который обойдет в цикле все определения задач и создаст 100 индивидуальных заданий, предоставив каждому из них соответствующие входные данные. Мне эта громоздкая реализация нравится меньше всего, поскольку ей очень неудобно управлять. Например, допустим, что после создания очереди вам потребуется отменить выполнение всех задач. Вместо того чтобы просто удалить одно задание, как при использовании предыдущих подходов, вы должны будете удалить 100 заданий, а для этого вам потребуется еще один скрипт и т. д. Но если этот подход вас все же заинтересовал, то хороший пример его применения можно найти в документации по Kubernetes (<https://kubernetes.io/docs/tasks/job/parallel-processing-expansion/>).

10.4. ПРОВЕРКА ЖИЗНЕСПОСОБНОСТИ ДЛЯ ФОНОВЫХ ЗАДАЧ

Как и контейнеры для обработки HTTP-трафика, контейнеры для выполнения задач (которые вы можете определить как объекты Deployment или Job) должны подвергаться проверкам жизнеспособности. Такая проверка предоставляет системе Kubernetes информацию, необходимую для перезапуска контейнера в том случае, если он перестает работать должным образом (например, если приостанавливается какой-либо процесс или не удастся подключить внешнюю зависимость). Утилита kubelet автоматически перезапускает контейнеры *в случае выхода их из строя* (конечно, при условии, что в шаблоне PodSpec не определено поле `restartPolicy` со значением `Never`), но она не сможет выявить зависание процесса или какое-либо иное отклонение от нормального поведения, если вы не определите проверку жизнеспособности.

В главе 4 мы говорили о проверках готовности и жизнеспособности в контексте обслуживания HTTP-нагрузок. Если задачи фоновые, то мы можем обойтись без проверок готовности и сосредоточиться только на проверках жизнеспособности, поскольку у фоновых задач нет сервиса, к которому их нужно было бы подключать или от которого отключать в зависимости от их готовности. Как и в случае сервисных рабочих нагрузок, проверки жизнеспособности можно использовать для выявления зависших контейнеров фоновых задач, чтобы система Kubernetes могла их перезапустить.

У фоновых задач нет конечной точки HTTP или TCP, которую можно было бы использовать для проверки жизнеспособности, поэтому в данном случае такую проверку можно проводить только с помощью консольных команд. Вы можете дать контейнеру указание выполнять определенную команду, и если он выполняет

ее успешно, то будет считаться работоспособным. Но какую команду следует использовать для такой проверки? Один из возможных подходов сводится к тому, чтобы заставить задачи периодически записывать в файл текущую временную метку и проверять жизнеспособность с помощью скрипта, проверяющего, насколько давно была записана последняя метка.

Определим такую проверку жизнеспособности для нашего контейнера с обработчиком задач. Прежде всего, мы должны написать функцию, которая будет записывать в файл текущее значение времени (в виде временной метки Unix). Как может выглядеть такая функция, показано в листинге 10.14.

Листинг 10.14. Chapter10/pi_worker4/liveness.py

```
import os
import time

def update_liveness():
    timestamp = int(time.time())
    with open("logs/lastrun.date", "w") as myfile:
        myfile.write(f"{timestamp}")
```

Текущее время в виде временной метки Unix

Записываем временную метку в файл logs/lastrun.date

Теперь с помощью функции `update_liveness()` мы можем проверять, является ли «живым» обработчик, в различных точках цикла его работы. Очевидно, что, прежде всего, вызов этой функции следует поместить в основной цикл. Если задача выполняется очень долго, то можно добавить еще несколько вызовов в других местах. В листинге 10.15 показано, как добавить вызов этой функции в файле `pi_worker.py` (полную версию файла см. в каталоге с примерами кода для данной книги).

Листинг 10.15. Chapter10/pi_worker4/pi_worker.py

```
from liveness import *

# ...

while running:
    update_liveness()
    task = r.blpop('queue:task', 5)

# ...
```

Обновляем временную метку, чтобы задача считалась «живой» в течение основного цикла работы

Теперь нам нужно поместить в контейнер `bash`-скрипт, на который будет ссылаться команда для проверки жизнеспособности, чтобы проверять, насколько «свежа» временная метка. Пример реализации такого скрипта представлен в листинге 10.16. Этот скрипт принимает два параметра: имя считываемого файла (переменная `$1`) и промежуток времени в секундах, при превышении которого процесс уже не будет считаться «живым» (переменная `$2`). Скрипт сравнивает считанное из файла значение с текущим значением времени и возвращает код успешного завершения (`0`), если временная метка является достаточно «свежей»,

и код ошибки (ненулевой код завершения) в противном случае. Использовать этот скрипт можно так: `./check_liveness.sh logs/lastrun.date 300`. Данный вызов возвратит успешный результат, если метка, записанная в файл `lastrun.date`, находится в пределах 300 секунд (5 минут) от текущего времени. Полную версию скрипта, которая также содержит код для проверки входных данных, см. в каталоге с примерами кода для этой книги.

Листинг 10.16. Chapter10/pi_worker4/check_liveness.sh

```
#!/bin/bash

# ...

if ! rundate=$(cat $1); then
    echo >&2 "Failed: unable to read logfile"
    exit 2
fi

curdate=$(date +%s')
time_difference=$((curdate-rundate))

if [ $time_difference -gt $2 ]
then
    echo >&2 "Liveness failing, timestamp too old."
    exit 1
fi

exit 0
```

Производим чтение файла с временной меткой (имя которого передается во входном параметре \$1) и завершаем работу с кодом ошибки, если указанный файл не существует

Получаем текущее время в виде временной метки

Сравниваем две временные метки

Возвращаем код ошибки, если «возраст» временной метки процесса превышает пороговое значение (значение параметра \$2)

Возвращаем код успешного завершения

Теперь, когда наш обработчик задач сохраняет временную метку и у нас есть проверяющий ее `bash`-скрипт, нам остается только добавить проверку жизнеспособности в определении нашей рабочей нагрузки. Для этого используется поле `livenessProbe`. Оно является частью шаблона `PodSpec`, поэтому его можно добавить в обеих версиях обработчика задач: и в версии на основе развертывания, и в версии на основе задания. В листинге 10.17 показано, как добавить проверку жизнеспособности в версии на основе развертывания, которая была представлена выше, в листинге 10.7.

Листинг 10.17. Chapter10/10.4_TaskLiveness/deploy_worker.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pi-worker
spec:
  replicas: 2
  selector:
    matchLabels:
      pod: pi
```

```

template:
  metadata:
    labels:
      pod: pi
  spec:
    containers:
      - name: pi-container
        image: docker.io/wdenniss/pi_worker:4
        env:
          - name: REDIS_HOST
            value: redis-0.redis-service
          - name: PYTHONUNBUFFERED
            value: "1"
        livenessProbe:
          initialDelaySeconds: 60
          periodSeconds: 30
          exec:
            command: ["/check_liveness.sh", "logs/lastrun.date", "300"]
            successThreshold: 1
            timeoutSeconds: 1
        terminationGracePeriodSeconds: 120

```

Добавленная нами проверка жизнеспособности

Новая версия контейнера с логикой проверки жизнеспособности

Таким образом, теперь у нас есть процесс, позволяющий выявлять зависшие процессы обработчика задач. Если обработчик не успеет записать в файл новую временную метку за время, не превышающее максимально допустимый «возраст» метки (который в данном случае составляет 300 секунд), то команда для проверки жизнеспособности возвратит код ошибки и под будет перезапущен.

Проследите за тем, чтобы ваш обработчик успевал обновить временную метку, укладываясь в отведенное для этого время. Если определенная задача выполняется очень долго, то либо увеличьте максимально допустимый «возраст» метки, либо обновляйте файл с меткой в нескольких точках процесса обработки задачи, а не только в основном цикле, как мы делали в листинге 10.15. Кроме того, важно проследить за тем, чтобы метка записывалась только в том случае, если обработчик ведет себя нормально. Например, вряд ли стоит вызывать функцию `update_liveness()` внутри обработчика исключений.

ПРИМЕЧАНИЕ

В данном примере пороговое значение, при превышении которого процесс уже не считается «живым», не зависит от периодичности проверки жизнеспособности (которая указывается в поле `periodSeconds`). Чтобы увеличить это пороговое значение, нужно увеличить число, передаваемое проверке жизнеспособности в качестве третьего параметра, то есть число 300 в команде `./check_liveness.sh logs/lastrun.date 300`.

После того как вы настроите такую проверку жизнеспособности для своих фоновых задач, Kubernetes будет обладать информацией, которая позволит ему поддерживать ваш код в рабочем состоянии без вашего вмешательства.

РЕЗЮМЕ

- В Kubernetes можно использовать несколько способов обработки очереди фоновых задач и выполнения пакетных заданий.
- Развертывание можно использовать для реализации непрерывно работающей очереди задач, при этом можно управлять очередью с помощью такой структуры данных, как Redis.
- Фоновая обработка, выполняемая многими сайтами для отложенного выполнения ресурсоемких запросов, обычно запускается как объект Deployment.
- Выполнять задачи в Kubernetes можно с помощью объекта Job, который специально предназначен для этой цели.
- Задания можно использовать для выполнения разовых задач, таких как выполняемые вручную операции технического обслуживания.
- С помощью объекта CronJob можно сделать так, чтобы задания выполнялись по расписанию, например, запустить ежедневную операцию очистки.
- Задание может обработать очередь задач и затем самостоятельно прекратить свою работу, что очень удобно в случае выполнения разовых или периодических пакетных заданий.
- В отличие от фоновой очереди на основе развертывания, задание дает возможность разместить задачи в статической очереди, что позволяет представлять очередь, не используя такую структуру данных, как Redis.
- В поды для обработки фоновых задач тоже нужно добавлять проверки жизнеспособности, которые в данном случае выявляют зависшие процессы и могут быть настроены с помощью консольных команд.

11

Методология GitOps: конфигурация как код

В этой главе

- ✓ Использование пространств имен и конфигурационных файлов для репликации среды.
- ✓ Преимущества обращения с конфигурацией рабочей нагрузки Kubernetes как с исходным кодом.
- ✓ Управление процессами с помощью запросов на принятие изменений системы управления версиями Git.
- ✓ Управление секретами без сохранения их в виде простого текста в системе управления версиями.

Как вы могли заметить, на протяжении всех предыдущих глав мы постоянно создавали конфигурационные файлы в формате YAML, несмотря на то что с большинством объектов Kubernetes можно взаимодействовать и без написания конфигурационных файлов, используя такие императивные команды утилиты `kubectl`, как `kubectl run`, которые, как многие считают, проще запомнить. Почему же я постоянно использую в этой книге декларативный подход, основанный на конфигурациях? Одна из причин такова: это позволяет после переноса приложения в продуктивную среду обращаться с конфигурацией как с кодом, точно так же используя управление версиями и ревью кода.

Еще одной причиной является то, что такой подход позволяет легко запустить несколько сред с одинаковой конфигурацией. Допустим, вам нужно, чтобы пред-продуктивная и продуктивная среды практически не отличались друг от друга,

чтобы можно было эффективно проводить тестирование. Если ваши рабочие нагрузки будут определены в конфигурационных файлах, то вы сможете легко реплицировать среду. При этом вам не нужно будет беспокоиться о возможных конфликтах имен, поскольку Kubernetes позволяет создавать для разных сред отдельные пространства имен.

11.1. РАБОТА В ПРОДУКТИВНОЙ И ПРЕДПРОДУКТИВНОЙ СРЕДАХ С ИСПОЛЬЗОВАНИЕМ ПРОСТРАНСТВ ИМЕН

По мере подготовки приложения к переходу на продуктивную среду обычно требуется создать предпродуктивную среду, в которой можно было бы проверять изменения, которые вносятся до того, как продуктивная версия приложения будет обновлена. В Kubernetes вы можете легко это реализовать, используя пространства имен.

Как и предполагает их название, эти пространства обеспечивают уникальность имен в пределах отдельного логического пространства. То есть вы можете определить пространство имен `production` для продуктивной среды и пространство имен `staging` — для предпродуктивной и запустить в обеих средах развертывание `foo-deployment` и сервис `foo-service` (рис. 11.1). Благодаря такому подходу можно не вносить лишние изменения в версию конфигурации, используемую в другой среде (такие как использование объектов `Deployment` и `Service` с другими именами, например `foo-staging-deployment` и `foo-staging-service`). Кроме того, он

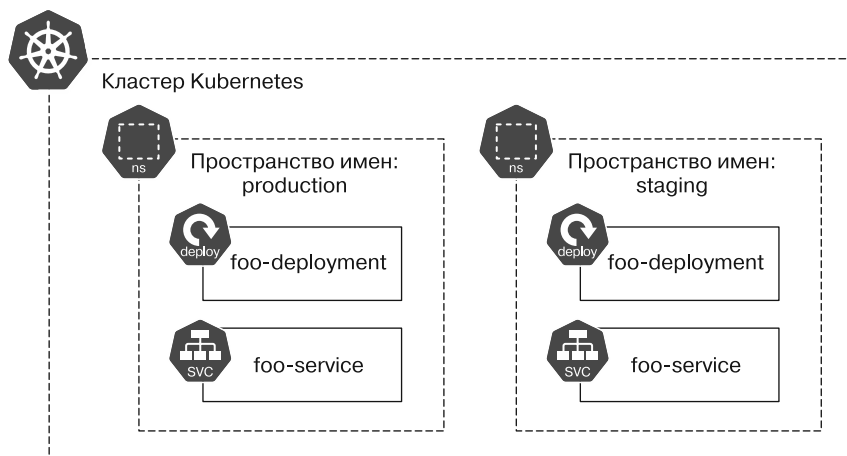


Рис. 11.1. Кластер Kubernetes с двумя пространствами имен. Обратите внимание на то, что в обоих пространствах имен используются объекты `Deployment` и `Service` с одинаковыми именами

в какой-то мере защищает вас от внесения случайных изменений, поскольку по умолчанию действие команд утилиты `kubectl` распространяется только на то пространство имен, которое активно в данный момент.

Различия в конфигурации между продуктивной и предпродуктивной средами, как правило, сводятся к использованию разной степени масштабирования (количества реплик) и учетных данных для доступа к внешним сервисам, которые не развертываются как часть пространства имен.

СОВЕТ

Чтобы упростить работу с очень сложными рабочими нагрузками или большим количеством по-разному настроенных сред, вы можете воспользоваться таким шаблонизатором, как `Kustomize` или `Helm`.

Создать пространство имен `staging` для размещения еще одного экземпляра приложения можно с помощью данной команды:

```
kubectl create namespace staging
```

Для взаимодействия с этим пространством имен можно либо запускать каждую команду утилиты `kubectl` с флагом `--namespace staging` (в сокращенном виде `-n staging`), либо изменить контекст утилиты `kubectl` так, чтобы все ее команды выполнялись в этом пространстве. Я настоятельно рекомендую вам воспользоваться вторым вариантом, поскольку в первом случае можно легко забыть добавить флаг `-n` и выполнить команду не в том пространстве. Лучше всегда переключайте контекст. Для этого следует отобразить список доступных пространств имен, используя команду `kubectl get namespace`, и переключить контекст на нужное вам пространство:

```
kubectl config set-context --current --namespace=staging
```

В списке доступных пространств имен можно заметить несколько встроенных пространств имен платформы Kubernetes. Пространство `kube-system`, в котором располагаются системные поды, следует использовать лишь в том случае, если вы точно знаете, что делаете. Пространство `default`, которое предоставляется по умолчанию, уже можно использовать, но я все же рекомендую вместо этого создавать отдельное пространство имен для каждой среды, с которой вы взаимодействуете. Чтобы вам не приходилось каждый раз писать громоздкую команду `kubectl` для переключения контекста, я настоятельно рекомендую вам настроить свою командную оболочку с помощью утилиты, позволяющей упростить процесс переключения. Я использую утилиты `kubens` (<https://github.com/ahmetb/kubectx>) и `kubectx`. С помощью `kubens` можно отобразить список пространств имен, используя команду `kubens`, и задать контекст, используя следующую команду:

```
kubens staging
```

Вторая упомянутая утилита, `kubectx`, позволяет быстро переключаться между кластерами. Эти два скрипта представляют собой просто сокращенный способ

записи более длинных команд `kubectl config set-context`, поэтому после настройки контекста вы можете снова вернуться к использованию обычных команд утилиты `kubectl`.

11.1.1. Процесс развертывания в созданном нами пространстве имен

После того как пространство имен будет создано, вы можете легко развернуть свое приложение из конфигурации. Именно поэтому в данной книге я постоянно использую конфигурационные файлы.

Вместо того чтобы заново выполнять множество императивных команд для создания тех же самых рабочих нагрузок, вы можете просто выполнить следующую команду в каталоге, в котором размещена конфигурация:

```
kubectl apply -f .
```

А если вам потребуется выкатить новую версию конфигурации или развернуть ее в еще одном пространстве имен, то для этого нужно будет просто выполнить эту команду еще раз в каждой используемой среде.

На самом деле процесс создания отдельных сред с помощью пространств имен в Kubernetes настолько прост, что вы можете найти определенное преимущество в использовании множества разных сред, если раньше вы имели дело с платформами, которые позволяли задействовать только одну общедоступную предпродуктивную среду. В Kubernetes вы можете создать отдельные пространства имен для команды разработчиков, предпродуктивной среды, интеграционного тестирования и т. д. При этом обычно вам не нужно платить за создание дополнительных пространств имен (однако следует иметь в виду, что продублированные вами поды будут потреблять больше вычислительных ресурсов).

11.1.2. Синхронизация изменений в пределах кластера

Что делать в том случае, если определенные изменения будут внесены императивно, без добавления в конфигурацию? А если вы масштабировали рабочую нагрузку с помощью команды `kubectl scale`, изменили образ, используя команду `kubectl set-image`, или создали объект `Deployment`, запустив команду `kubectl run`? Иногда такое случается, и я не собираюсь вас осуждать.

Kubernetes позволяет просмотреть и экспортировать конфигурацию, используя параметр `--output` (в сокращенном виде `-o`) в любом запросе `get`. Например, получить последнюю версию конфигурации развертывания в формате YAML можно так:

<code>kubectl get deploy timeserver -o yaml</code>	←	Просмотр конфигурации развертывания в формате YAML		Вывод YAML-конфигурации развертывания в файл
<code>kubectl get deploy timeserver -o yaml > timeserver-exported.yaml</code>			←	

Проблема в том, что Kubernetes добавляет много таких дополнительных полей, как сообщения о состоянии, которые фактически не нужны, если конфигурация размещается на диске. Раньше утилита `kubect1` позволяла легко удалять такие поля, используя флаг `--export`, но, к сожалению, в ее последних версиях он уже не доступен. Поэтому вам придется самостоятельно решать, какие строки следует удалить, а какие — оставить. Чтобы в какой-то мере облегчить себе решение этой задачи, вы можете сравнивать получаемые таким образом YAML-файлы с файлами из данной книги.

Если, как обычно бывает, вы собираетесь использовать полученную конфигурацию в нескольких местах, то вам определенно стоит удалить поле `metadata → namespace`. Тогда вы сможете выполнить развертывание конфигурации в текущем пространстве имен (а если оставить это поле, то изменения, которые вы вносите, будут применяться к объекту, размещенному в указанном пространстве). Это поле опасно тем, что вы можете случайно применить к продуктивной среде конфигурацию, размещенную в каталоге предпродуктивной среды. В разделе 11.3 мы еще подробно обсудим несколько способов, позволяющих выполнить безопасное развертывание в нескольких пространствах имен, но все эти способы требуют, чтобы вы *не указывали* пространство имен в объектах ресурсов напрямую.

Кроме того, для порядка можно удалить ряд полей из раздела `metadata`: поля `uid`, `resourceVersion`, `generation`, `creationTimestamp` и весь раздел `status`. Эти поля не препятствуют использованию конфигурации в других пространствах имен или кластерах, но не имеют смысла за пределами исходного контекста, и поэтому во избежание путаницы будет лучше не пропускать их через систему управления версиями.

11.2. КАК РЕАЛИЗУЕТСЯ МЕТОДОЛОГИЯ «КОНФИГУРАЦИЯ КАК КОД» В KUBERNETES

При возникновении ошибки в программном коде вы можете отобразить историю версий и посмотреть, когда производилось изменение кода (используя команды `git log` и `git blame`), а затем при необходимости откатиться к предыдущему состоянию, лишенному ошибок. Обращаясь с конфигурацией как с кодом (сохраняя «снимки» конфигурации в системе управления версиями), вы сможете применять тот же подход к своим системам продуктивного уровня.

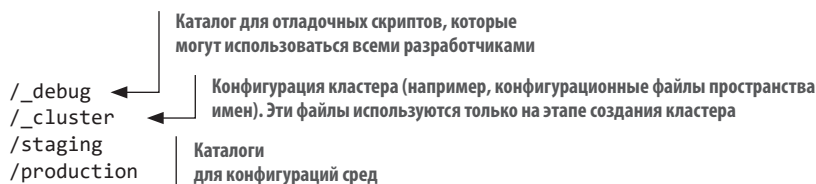
Если вы используете определенный процесс ревью кода, то такому же процессу вы можете подвергать и конфигурацию Kubernetes, ведь работоспособность запущенной вами системы зависит от конфигурации не меньше, чем от качества кода. Подвергая ревью кода репозитории с конфигурациями, вы сможете перехватывать имеющиеся в них ошибки до их развертывания. Например, если при редактировании конфигурации вы случайно удалите все реплики развертывания, то ваш коллега сможет перехватить эту ошибку на этапе ревью кода.

Такого подхода придерживаются все крупные интернет-компании. Например, Google выполняет развертывание большинства своих сервисов из того же репозитория кода, который компания использует на этапе разработки (<https://research.google/pubs/pub45424/>), размещая конфигурации рядом с исходным кодом. При этом код и конфигурация сервисов подвергаются совершенно одинаковому процессу ревью кода, лишь с тем отличием, что у кода и конфигурации могут быть разные списки владельцев (разработчиков, имеющих право выполнять слияние изменений).

Однако размещать конфигурацию в том же репозитории, что и код, как это делает компания Google, совсем не обязательно. Вы можете опираться на свои предпочтения (или на то, к чему в итоге придут ваши разработчики в ходе бесконечных споров). Описываемая здесь модель с сохранением конфигурации Kubernetes в Git хорошо подходит в моем случае, а вам следует адаптировать ее под свои методы разработки.

Я использую один Git-репозиторий для размещения конфигураций всех объектов Kubernetes, развертываемых в одном кластере. В репозитории я размещаю по одному каталогу для каждого пространства имен Kubernetes, а в каталогах — YAML-файлы объектов, расположенных в соответствующем пространстве имен (рис. 11.2). Возможен еще один подход: использовать для каждого пространства имен отдельную ветку, что позволяет получить некоторые преимущества, например, можно выполнять слияние изменений при их переходе от предпродуктивной версии к продуктивной. Но поскольку некоторые изменения обычно *не требуется* подвергать слиянию, то такой подход может привести к путанице (например, вы можете случайно внести в продуктивную версию изменения, которые требуются только в предпродуктивной).

Структура каталогов может выглядеть так:



Каждый каталог в данном репозитории отображается на пространство имен Kubernetes. Такая однозначность отображения хороша тем, что вы можете совершенно спокойно использовать команду `kubectl apply -f .` для развертывания в активном пространстве имен всех изменений, внесенных в каталог. Чтобы клонировать среду, достаточно скопировать весь соответствующий каталог и развернуть его в другом пространстве имен.

Частой практикой — особенно при развертывании не очень крупных рабочих нагрузок — является совместное использование кластера пространствами имен разных сред. Благодаря этому можно снизить как непосредственные расходы, так и операционные издержки по поддержке кластеров. Вдобавок при таком подходе

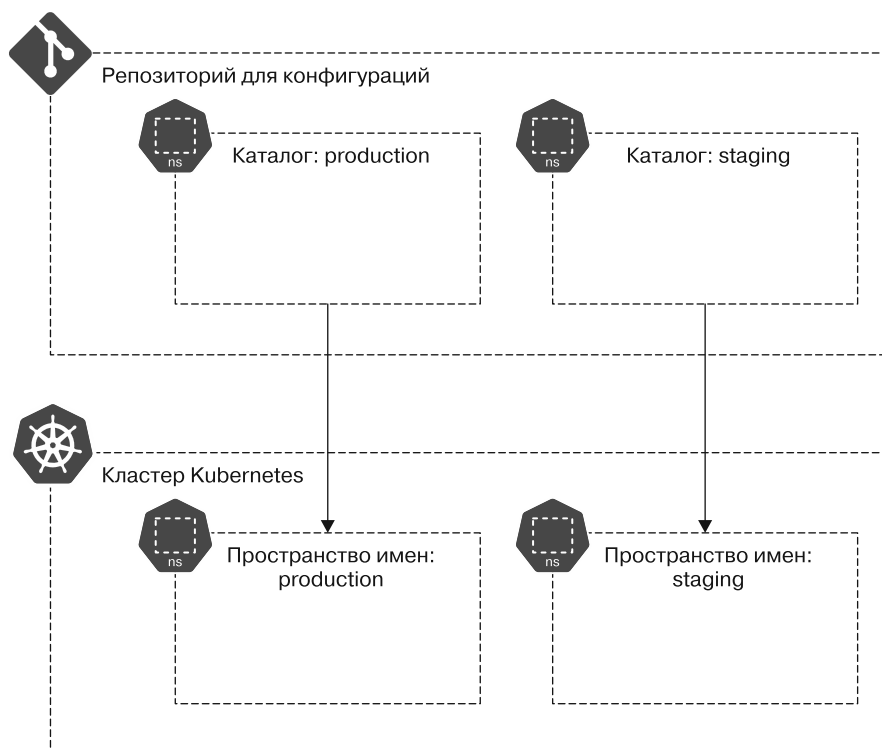


Рис. 11.2. Структура каталогов Git-репозитория и ее связь с пространствами имен Kubernetes

рабочие нагрузки могут совместно использовать общий пул вычислительных ресурсов. По мере того как ваши рабочие нагрузки будут увеличиваться, вероятно, будет лучше выделить для каждой среды собственный кластер, чтобы обеспечить дополнительную степень контроля доступа и изоляции ресурсов (рис. 11.3). Хорошая новость: для размещенных в репозитории конфигураций совершенно неважно, где именно будут располагаться эти пространства имен; они с тем же успехом могут находиться и в разных кластерах.

После того как вы настроите репозиторий для конфигураций, дальнейший процесс разработки будет выглядеть так.

1. Внесение изменений в конфигурацию для той или иной среды.
2. Фиксация этих изменений.
3. Обновление текущего состояния: текущий контекст переключается на соответствующее пространство имен с помощью утилиты `kubect1`, команда `kubect1 apply -f` . выполняется для соответствующего каталога.

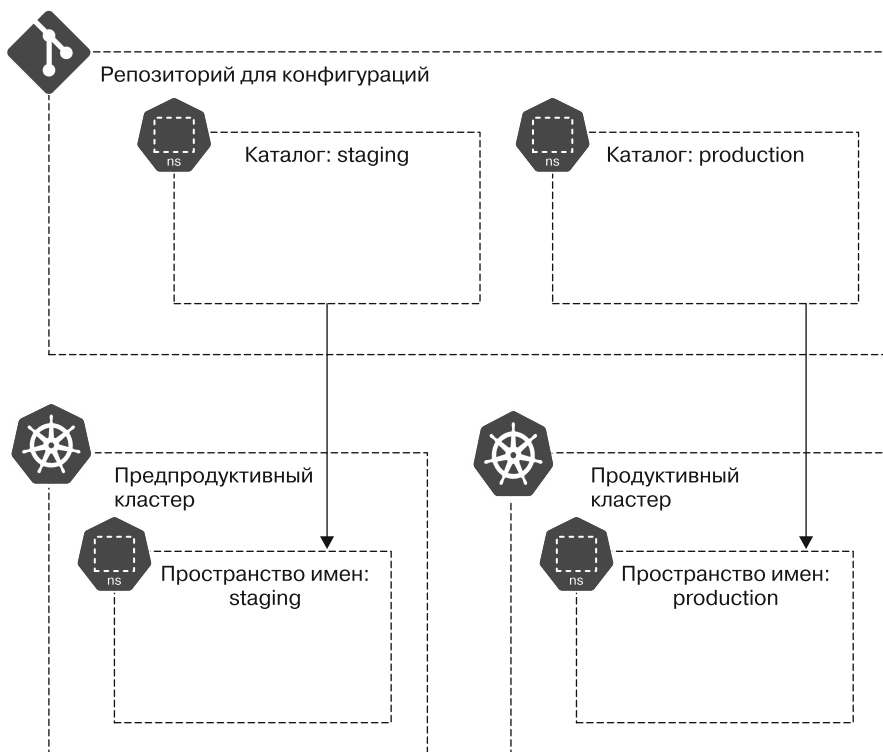


Рис. 11.3. Использование репозитория для конфигураций с развертыванием сред в разных кластерах

Благодаря такому подходу вы уже можете применять методологию «конфигурация как код», но это еще не все. Так, если вы используете этот подход, то возникает риск, что вы случайно развернете конфигурацию из какого-либо каталога не в то пространство имен. В следующих разделах мы поговорим о том, как обеспечить безопасное развертывание, исключив вероятность возникновения этой проблемы, и как сделать еще один шаг вперед и реализовать расширенный способ развертывания в стиле GitOps, при котором выполнение команды `git push` в репозитории для конфигураций будет запускать автоматический процесс развертывания.

11.3. БЕЗОПАСНОЕ РАЗВЕРТЫВАНИЕ

После настройки репозитория для конфигураций по методологии «конфигурация как код» встает вопрос о том, как лучше развертывать изменения из данного репозитория. Конечно, вы можете просто выбирать в репозитории нужный каталог и выполнять для него команду `kubectl apply -f .`, как мы делали ранее, но это чревато определенными рисками. При таком подходе вы можете случайно

развернуть конфигурацию не в то пространство имен. Мы используем одни и те же имена объектов в разных пространствах имен, поэтому такая ошибка может иметь очень серьезные последствия. Кроме того, ничто не мешает вам изменить текущее состояние кластера напрямую, не фиксируя данное изменение конфигурации в репозитории.

Чтобы исключить вероятность развертывания конфигурации не в то пространство имен, я рекомендую создать своего рода «защитное ограждение». Вместо того чтобы просто выполнять команду `kubectl apply -f .`, как мы делали ранее, оберните ее в скрипт, который будет проверять, правильно ли выбрано пространство имен. Если каждому каталогу будет присвоено такое же имя, как у соответствующего пространства имен, то проверка будет простой: если у текущего пространства имен такое же имя, как у текущего каталога, то разрешить развертывание; если нет — запретить. В листинге 11.1 показано, как может выглядеть скрипт, который сравнивает имя текущего каталога с именем текущего пространства имен и завершает работу, выдавая код ошибки в случае их несовпадения.

Листинг 11.1. Chapter11/gitops/gitops_check.sh

```
#!/bin/bash

CURRENT_DIR=`echo "${PWD##*/}"`
CURRENT_NAMESPACE=`kubectl config view --minify
-o=jsonpath='{.contexts[0].context.namespace}'`

if [ "$CURRENT_DIR" != "$CURRENT_NAMESPACE" ]; then
    >&2 echo "Wrong namespace (currently $CURRENT_NAMESPACE but" \
        "$CURRENT_DIR expected)"
    exit 1
fi

exit 0
```

Получаем имя каталога, которое является последним элементом текущего пути

Извлекаем имя текущего пространства имен

В противном случае завершаем работу, получив код успешного завершения

Завершаем работу, получив код ошибки, если скрипт был запущен в каталоге, имя которого не совпадает с именем пространства имен

Теперь вы можете добавить вызов этой проверки в скрипт, выполняющий развертывание, и запускать его вместо непосредственного выполнения команды `kubectl apply -f .`, чтобы гарантировать правильное сопоставление каталога с пространством имен (листинг 11.2).

Листинг 11.2. Chapter11/gitops/production/rollout.sh

```
#!/bin/sh

if [ $(../gitops_check.sh; echo $? ) != 0 ]; then exit 1; fi

kubectl apply -f .
```

Проверяем, совпадает ли имя каталога с именем пространства имен

Выполняем команду kubectl apply

Если вы хотите увидеть, как в данном случае выглядит вся структура репозитория в стиле GitOps вместе со всеми скриптами, то посмотрите каталог `Chapter11/gitops` внутри каталога с примерами кода для данной книги.

Конечно, это не единственный возможный подход. Еще один способ сводится к тому, чтобы задать нужное пространство имен в скрипте, выполняющем развертывание, используя команду `set namespace`. Однако имейте в виду, что в таком случае неудачное выполнение этой команды приведет к прекращению выполнения всего скрипта.

Все эти скрипты можно будет использовать только тогда, когда ни в одном из конфигурационных файлов не будет напрямую определено поле `metadata → namespace` (метаданные → пространство имен). Если в каком-то из файлов будет таким образом задано пространство имен, то он будет игнорировать текущий контекст, и скрипт не сможет этому помешать.

Правильное применение методологии GitOps подразумевает, что вам необходимо позаботиться и о том, чтобы развертывалась всегда именно та конфигурация, которая находится в нужном репозитории. Самый лучший способ добиться этого — вообще исключить человека из цепочки необходимых действий, настроив конвейер развертывания или GitOps-оператор.

11.3.1. Конвейеры развертывания

Конвейер развертывания — это просто набор функций, который запускается с помощью триггера репозитория для кода, например: «когда в репозиторий конфигураций отправлены изменения, выполнить развертывание конфигурации в кластер Kubernetes» (рис. 11.4). Использование конвейеров позволяет гарантировать, что развертываться всегда будет именно та конфигурация, которая была зафиксирована вами. Если оператору потребуется внести дополнительные изменения после развертывания (например, исправить ошибку), то он сможет сделать это таким же образом: отправить скорректированный код в репозиторий и тем самым еще раз запустить конвейер развертывания.

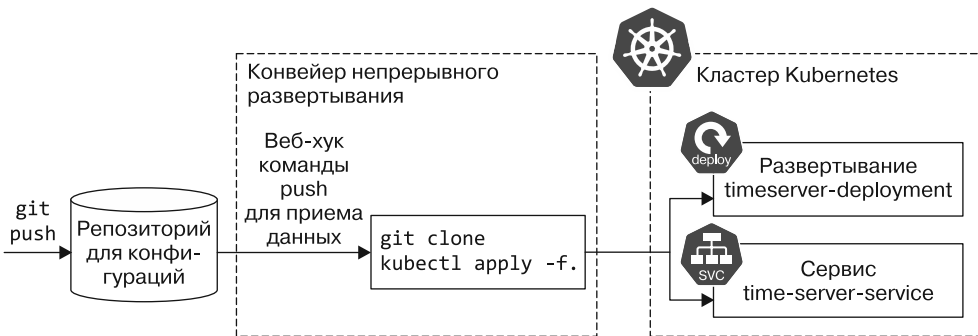


Рис. 11.4. Конвейер непрерывного развертывания в Kubernetes

Настроив такой конвейер, вы сможете выполнять развертывание в продуктивной среде, объединяя код в своем Git-репозитории (что и составляет суть методологии GitOps (Git-driven operations)). Главное при этом никогда не вносить изменения в кластере напрямую; все изменения должны проходить через репозиторий для конфигураций и конвейер непрерывного развертывания.

11.3.2. Непрерывное развертывание с помощью сервиса Cloud Build

Реализовать конвейер развертывания на практике можно с помощью множества разных имеющихся на рынке продуктов. Пользователи платформы Google Kubernetes Engine (GKE), в частности, могут использовать сервис Cloud Build. Вы можете настроить триггер так, чтобы при поступлении нового кода в репозиторий для конфигураций он выполнял команду `kubectl apply -f ..`

Выполнить такую настройку можно следующим образом.

1. Настройте разрешения системы управления идентификацией и доступом (Identity and Access Management, IAM) (<https://cloud.google.com/build/docs/securing-builds/configure-access-for-cloud-build-service-account>) для своего аккаунта сервиса Cloud Build так, чтобы у вас было разрешение на выполнение действий со своим кластером GKE.
2. Создайте новый триггер (настроив его так, чтобы он срабатывал при отправке изменений кода в репозиторий конфигураций).

Поместите в свой репозиторий конфигурационный файл сервиса Cloud Build (листинг 11.3) и добавьте ссылку на него в триггере.

Листинг 11.3. Chapter11/11.3.2_CloudBuild/cloudbuild-deploy.yaml

```
steps:
- name: 'gcr.io/cloud-builders/kubectl'
  id: Deploy
  args:
  - 'apply'
  - '-f'
  - '$FOLDER'
  env:
  - 'CLOUDSDK_COMPUTE_REGION=us-west1'
  - 'CLOUDSDK_CONTAINER_CLUSTER=my-cluster'
```

Я не буду погружаться в тему непрерывного развертывания очень глубоко. Если вы решили использовать сервис Cloud Build, то хорошим подспорьем для вас станет статья *GitOps-Style Continuous Delivery with Cloud Build* («Непрерывное развертывание в стиле GitOps с помощью Cloud Build») (<https://cloud.google.com/kubernetes-engine/docs/tutorials/gitops-cloud-build>), в которой эта тема освещается более подробно, а также приводится полный разбор процесса непрерывной интеграции и доставки (CI/CD, Continuous Integration, Continuous Delivery).

НЕПРЕРЫВНАЯ СИНХРОНИЗАЦИЯ

Описанный здесь подход можно сделать еще более эффективным, если использовать GitOps-оператор — запущенный в кластере управляющий контур, который непрерывно синхронизирует то, что запущено в кластере, с содержимым репозитория для конфигураций. В итоге вы получаете такой же управляемый событиями конвейер, какой мы рассмотрели выше; вдобавок он может выполнять дополнительную синхронизацию в случае каких-либо отклонений, а не срабатывает исключительно при помещении кода в репозиторий. Примером такого GitOps-оператора может служить инструмент Flux (<https://fluxcd.io/>).

11.4. СЕКРЕТЫ

Вы можете размещать в Git-репозитории свои конфигурации Kubernetes, но такой способ хранения не подходит для секретных данных, таких как пароли баз данных и ключи API. Если такие данные будут встроены непосредственно в код или указаны в качестве значения переменных среды, то их сможет получить любой пользователь, имеющий доступ к вашему исходному коду. Конечно, вы можете исправить эту ситуацию, сделав так, чтобы доступ к этим данным имели только те пользователи, которые имеют доступ к продуктивной версии вашей системы, или принять какие-то другие меры. Но в данной главе мы говорим о методологии GitOps, поэтому лучше сосредоточимся на том, как отделить секретные данные от репозитория для кода и конфигураций.

На самом деле в Kubernetes предусмотрен специальный объект для хранения секретных данных, который получил очень удачное название `Secret`. С помощью этих объектов вы можете передавать в рабочую нагрузку такую информацию, как учетные данные и ключи, обеспечивая их изоляцию от конфигурации рабочей нагрузки.

11.4.1. Строковые секреты (пароли)

Если до сих пор вы указывали такие секретные данные, как пароли, в обычных переменных среды, определенных в конфигурации рабочей нагрузки, то пришла пора перенести эти данные в объект `Secret`. Допустим, у нас есть секретный ключ со значением `secret_value` (на практике это может быть ключ, предоставленный вам облачным провайдером). Мы можем инкапсулировать это значение `secret_value` в объект `Secret` (листинг 11.4).

Листинг 11.4. Chapter11/11.4.1_StringSecrets/secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: secrets-production
```



```

type: Opaque
stringData:
  SECRET_KEY: secret_value
  ANOTHER_KEY: another_secret_value

```

Секреты можно предоставлять подам в виде смонтированного в контейнере файла или переменной среды. В виде файла обычно передаются те данные, которые приложения используют как конфигурационный файл (например, закрытые SSL-ключи), а в виде переменной среды — такие элементы, как пароли баз данных. Секрет, который мы определили в листинге 11.4, представляет собой простую строку, поэтому мы можем воспользоваться вторым способом и определить в развертывании переменную среды со ссылкой на наш секрет (листинг 11.5) (пример передачи секрета в виде файла см. в подразделе 11.4.3).

Листинг 11.5. Chapter11/11.4.1_StringSecrets/deploy.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
          env:
            - name: AVATAR_ENDPOINT
              value: http://robohash-internal
            - name: SECRET_KEY
              valueFrom:
                secretKeyRef:
                  name: secrets-production
                  key: SECRET_KEY

```

Переменная среды,
содержащая обычный текст

Переменная среды,
содержащая ссылку
на объект Secret

Чтобы убедиться в том, что мы все сделали правильно, создайте объекты Secret и Deployment из каталога Chapter11/11.4.1_StringSecrets и выполните следующую команду:

```

$ kubectl exec deploy/timeserver -- env
SECRET_KEY=secret_value

```

Она должна вывести на экран значение вашего секретного ключа. Теперь приложение получает это значение через переменную среды SECRET_KEY.

11.4.2. Секреты в формате Base64

В документации по Kubernetes и других источниках можно найти примеры использования секретных данных, представленных в кодировке base64 (с использованием поля `data` вместо поля `stringData`). Такой способ представления данных нужен не для их защиты (base64 — способ кодирования, а не шифрования), а для представления данных, которые нельзя легко определить в YAML-коде.

Обычно я не перевожу секретные данные в формат base64 по умолчанию, поскольку, на мой взгляд, это лишь затрудняет понимание кода. Но если вы имеете дело со строкой, которую нельзя легко определить в YAML-коде (как, например, в случае, когда секретные данные представляют собой целый файл), то будет лучше представить эти данные в формате base64. В листинге 11.6 показано, как определить объект Secret для секретного ключа `SECRET_KEY` из листинга 11.4, представив его значение в формате base64.

Листинг 11.6. Chapter11/11.4.2_Base64Secrets/secret-base64.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: secrets-production
type: Opaque
data:
  SECRET_KEY: c2VjcmV0X3ZhbHVlCg==
```

В любой Unix-подобной системе выполнить кодирование в этот формат и декодирование из него можно так:

```
$ echo "secret_value" | base64
c2VjcmV0X3ZhbHVlCg==

$ echo "c2VjcmV0X3ZhbHVlCg==" | base64 -d
secret_value
```

Если одни ваши значения требуют перевода в формат base64, а другие — нет, то вы можете добавить в один конфигурационный файл и поле `data`, и поле `stringData`. Кроме того, можно сохранить в одном объекте Secret несколько секретных значений (разместив их в отдельных строках). В листинге 11.7 представлен пример того, как определить в одном объекте Secret сразу три секретных значения, два из которых представляют собой обычный текст, а третье — значение в формате base64.

Листинг 11.7. Chapter11/11.4.2_Base64Secrets/secrets-multiple.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: secrets-production
```

```

type: Opaque
stringData:
  SECRET_KEY: secret_value
  ANOTHER_KEY: another_value
data:
  ENCODED_KEY:
    VGhpYBzdHJpbmcKbWlnaHQgYmUgaGFyZCB0byByZXByZXNlbnQgaW4gWUFNTCDwn5iFCg==

```

Если получать секретные данные от сервера вы будете с помощью консольных команд, то эти данные будут предоставляться вам в формате base64, и их нужно будет декодировать, чтобы просмотреть соответствующее неформатированное значение (но код вашего приложения будет получать их в уже декодированном виде).

Я предпочитаю создавать один объект Secret в каждом из своих пространств имен, определяя в нем сразу несколько секретных значений. Однако при этом я размещаю секреты и конфигурации в разных репозиториях. В подразделе 11.4.4 мы рассмотрим несколько способов размещения секретов, которые позволяют изолировать их от основного репозитория для конфигураций, не отказываясь от методологии GitOps.

11.4.3. Файловые секреты

Иногда секреты удобнее предоставлять приложению в виде файлов, а не строк, содержащихся в переменных среды. Kubernetes позволяет реализовать и такой подход. Процесс создания секрета при этом выглядит так же, но я все же приведу еще один пример, используя в качестве секрета многострочный текстовый файл, поскольку способ представления таких данных в YAML-коде имеет некоторые нюансы.

Допустим, нам нужно сохранить закрытый ключ. В качестве примера ниже представлен ключ, который я сгенерировал с помощью команды `openssl genrsa 256 -out example.key` (обычно лучше использовать ключи длиной 2048 или еще больше разрядов, но для краткости я использовал только 256 разрядов) (листинг 11.8).

Листинг 11.8. Chapter11/11.4.3_FileSecrets/example.key

```

-----BEGIN RSA PRIVATE KEY-----
MIGsAgEAAiEA4TneQFg/UMsVGrAvsm1wkonC/5jX+ykJAMeNffn1PQkCAwEAAQIh
ANgcs+MgClkXFQAP0SSvmJRmnRze3+zgUbN+u+rrYNR1AhEA+K0ghKRgK1zVn0xw
qltGtWIRA0fb8LCVNf6FAdD+bJGwHycCED6Yzf01sONZBQiAWAf6Am8CEQDIEXI8
fVSNHmp108UNZcNLAhEA3hHFV5jZppEHHHLy4F9Dnw==
-----END RSA PRIVATE KEY-----

```

Содержимое этого файла можно представить в YAML-коде (листинг 11.9). Крайне важной деталью является символ вертикальной черты, который позволяет сохранить разрывы строк в данных ключа.

Листинг 11.9. Chapter11/11.4.3_FileSecrets/secret_file.yaml

```

apiVersion: v1
kind: Secret
metadata:
  name: secret-files
type: Opaque
stringData:
  example.key: |
    -----BEGIN RSA PRIVATE KEY-----
    MIGsAgEAAiEA4TneQFg/UMsVGrAvsm1wkonC/5jX+ykJAMenffn1PQkCAwEAAQIh
    ANgcs+MgClkXFQAP0SSvmJRmnRze3+zgUbN+u+rrYNR1AhEA+K0ghKRgK1zVn0xw
    qltgTwIRA0fb8LCVNf6FAD+bJGwHycCED6Yzf01s0NZBQiAWAf6Am8CEQDIEXI8
    fVSNHmp108UNZcNLAhEA3hHFV5jZppEHHHLy4F9Dnw==
    -----END RSA PRIVATE KEY-----

```

Если вы уже немного устали от YAML-разметки, то вместо этого можно представить файловые данные в формате base64, используя команду `cat example.key | base64`, и определить их, как показано в листинге 11.10 (чтобы улучшить читабельность, я удалил часть данных). Обратите внимание: строковое значение в формате base64 необходимо целиком разместить в одной строке (не используя разрывы строк!).

Листинг 11.10. Chapter11/11.4.3_FileSecrets/secret_file_base64.yaml

```

apiVersion: v1
kind: Secret
metadata:
  name: secret-files
type: Opaque
data:
  example.key: LS0tLS1CRUdJTiBSU0EgUFJJVk...U0EgUFJJVkFURSBLRVktLS0tLQo=

```

Создавать конфигурационные файлы для секретов вручную достаточно утомительно, поэтому лучше автоматизировать процесс, используя утилиту `kubectl`. Команда, приведенная ниже, позволяет получить функционально такой же результат (чтобы улучшить читабельность, я удалил часть строкового значения в формате base64):

```

$ cd Chapter11/11.4.3_FileSecrets
$ kubectl create secret generic secret-files \
  --from-file=example.key=./example.key --dry-run=client -o yaml

```

```

apiVersion: v1
data:
  example.key: LS0tLS1CRUdJTiBSU0EgUFJJVk...U0EgUFJJVkFURSBLRVktLS0tLQo=
kind: Secret
metadata:
  creationTimestamp: null
  name: secret-files

```

Выражение `--dry-run=client -o yaml` здесь означает следующее: вам нужно, чтобы объект `Secret` не создавался на сервере, а был предоставлен вам в виде YAML-файла

(чтобы его можно было поместить в конфигурационный файл и впоследствии применить к серверу с помощью команды `kubectl apply -f secret.yaml`). Если вы опустите параметр `--dry-run`, то Secret будет создан непосредственно в кластере (то есть вы создадите этот объект Kubernetes в императивном стиле). На самом деле с помощью императивных команд утилиты `kubectl` можно реализовать все примеры, которые были приведены в этом подразделе. Но использование декларативного, основанного на конфигурациях подхода к эксплуатации кластера позволяет получить преимущества в долгосрочной перспективе, в том числе такие, которые были упомянуты ранее в этой главе.

Создав объект Secret, вы можете смонтировать все содержащиеся в нем файлы в виде каталога в своем контейнере. В листинге 11.11 показано, как смонтировать созданный нами секрет `secret-files` в каталог `/etc/config`. Каждый ключ секрета монтируется как отдельный файл; но в данном случае у нас только один ключ: `example.key`.

Листинг 11.11. Chapter11/11.4.3_FileSecrets/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
    spec:
      containers:
        - name: timeserver-container
          image: docker.io/wdenniss/timeserver:5
          volumeMounts:
            - name: secret-volume
              mountPath: "/etc/config"
              readOnly: true
          volumes:
            - name: secret-volume
              secret:
                secretName: secret-files
```

Путь монтирования
файловых секретов

Ссылка
на объект
Secret

Чтобы убедиться в том, что вы все сделали правильно, создайте объекты Secret и Deployment и отобразите содержимое каталога с помощью команды `exec`. Вы должны увидеть, что каталог содержит файл `example.key`:

```
$ kubectl exec deploy/timeserver -- ls /etc/config
example.key
```

Просмотреть содержимое самого файла можно, заменив команду `ls` командой `cat /etc/config/example.key`. Теперь вы можете сослаться на этот файл в своем коде как на любой другой файл, имеющийся в вашей системе.

11.4.4. Секреты и методология GitOps

Использование секретов требует, чтобы они были не только созданы, но и должным образом хранились. Если вы будете хранить их в одном репозитории с конфигурациями, то с тем же успехом сможете обойтись и без них, используя вместо этого обычные переменные среды. Здесь нет идеального варианта, подходящего на все случаи жизни, но я опишу несколько возможных подходов, в порядке возрастания их сложности.

Использование отдельного репозитория

Самый простой подход сводится к тому, чтобы использовать для секретов отдельный репозиторий для конфигураций, сделав его доступным более ограниченному количеству пользователей по сравнению с остальными вашими репозиториями. Вы по-прежнему будете иметь все преимущества методологии «конфигурация как код» (такие как ревью кода, возможность отката и т. д.), но аудитория будет ограничена. Если вы используете репозиторий с детализированным контролем доступа, то секреты можно разместить в нем же, в отдельном каталоге, доступ к которому находится под контролем.

Будет логичным использовать для секретов тот же репозиторий облачного провайдера, в котором вы храните продуктивные ресурсы, с тем же контролем доступа, который вы используете для продуктивной среды. Такая модель не приводит к снижению уровня защиты, поскольку злоумышленник, получая доступ к продуктивной среде, получает доступ и ко всем секретным данным.

Использование утилиты Sealed Secrets

Интересный подход предлагает утилита Sealed Secrets (<https://github.com/bitnami-labs/sealed-secrets>): вы шифруете свои секреты перед их сохранением в Git (чтобы никто не мог их прочитать), после чего контроллер декодирует их в кластере с помощью закрытого ключа. Такой подход рождает вопрос о том, где следует хранить закрытый ключ, но позволяет хранить зашифрованные секреты в основном репозитории для конфигураций, получая все преимущества методологии «конфигурация как код» (ревью кода, возможность отката и т. д.).

Использование сервиса

Еще один вариант сводится к тому, чтобы запустить отдельный сервис, который будет выполнять внедрение секретов в вашем кластере. Популярная реализация этого подхода — свободно распространяемая система Vault от компании HashiCorp (<https://www.vaultproject.io/>).

РЕЗЮМЕ

- С помощью пространств имен можно отделять друг от друга разные среды (например, предпродуктивную и продуктивную) и разные приложения.
- Обращение с конфигурацией как с кодом позволяет вам легко продублировать любую среду и поддерживать несколько сред.
- Придерживайтесь методологии «конфигурация как код», сохраняя конфигурации Kubernetes в системе контроля версий подобно программному коду (и подвергая их процессу ревью кода, если вы его используете).
- Вместо того чтобы напрямую изменять текущее состояние кластера, сначала вносите изменения в конфигурацию, а затем фиксируйте новую версию конфигурации в системе контроля версий и применяйте ее к кластеру.
- Конвейер развертывания можно использовать для автоматического развертывания изменений после того, как они будут зафиксированы и отправлены в репозиторий конфигурации.
- Используя такой GitOps-оператор, как Flux, можно обеспечить непрерывную синхронизацию с репозиторием для конфигураций.
- Хранить такую секретную информацию, как ключи баз данных, в виде отдельных объектов Kubernetes можно с помощью объектов Secret. Используйте для этих конфигураций способ хранения, позволяющий обеспечить контроль доступа.

12

Безопасность в Kubernetes

В этой главе

- ✓ Установка в кластере необходимых обновлений и патчей.
- ✓ Управление прерываниями.
- ✓ Использование объектов DaemonSet для развертывания на каждом узле узловых агентов.
- ✓ Запуск контейнеров от имени обычного пользователя (не root).
- ✓ Использование контроллеров доступа для проверки и изменения объектов Kubernetes.
- ✓ Применение стандартов безопасности пода.
- ✓ Управление доступом к пространствам имен с помощью ролевой модели (RBAC).

До сих пор мы говорили в основном о развертывании различных видов программного обеспечения в кластерах Kubernetes. Однако эта последняя глава будет посвящена важным вопросам, связанным с обеспечением безопасности. Безопасность — обширная тема в контексте практически любой системы, и Kubernetes не является исключением. Если вы развертываете код в кластер Kubernetes, которым управляет другая команда специалистов, то вас можно поздравить: вам не придется решать некоторые вопросы, связанные с безопасностью. Но для разработчиков, которые сами занимаются обслуживанием, и для операторов кластеров защита и обновление кластера — одна из ключевых сфер деятельности.

В этой главе мы рассмотрим такие темы, как обновление кластера, управление прерываниями, развертывание узловых агентов и создание контейнеров от имени обычного пользователя. Кроме того, мы подробно разберем, как создать для команды разработчиков специальное пространство имен и предоставить доступ к нему другим командам. Такой подход часто применяется в компаниях, где разные команды могут использовать кластеры совместно.

12.1. ВЫПОЛНЕНИЕ ОБНОВЛЕНИЙ

Развернутая в Kubernetes система имеет большую поверхность атаки: помимо ядра Linux и самой программы Kubernetes, которые работают на управляющем слое и на рабочих узлах. Она также содержит ваши контейнеры и все их зависимости, в том числе базовый образ. Это означает, что вам нужно обновлять и защищать от возможных атак большое количество программных компонентов.

12.1.1. Обновление кластера и узлов

При эксплуатации системы, развернутой в Kubernetes, важно регулярно выполнять обновление кластера и узлов. Это позволяет устранять известные уязвимости платформы и операционной системы, работающей на узлах.

В отличие от большинства других действий, которые мы обсуждали в предыдущих главах, процесс обновления кластера и узлов на самом деле не поддерживается API системы Kubernetes. Он осуществляется на уровне облачной платформы, поэтому вам нужно изучить документацию вашего облачного провайдера. К счастью, если вы используете управляемую платформу, то обеспечить регулярное обновление сможете достаточно легко. Если же вы решите выбрать ручную установку Kubernetes на виртуальные машины (что я не рекомендую делать!), то заниматься выполнением обновлений придется вам, поскольку вы сами будете выступать в роли провайдера платформы.

ОБНОВЛЕНИЕ С ПОМОЩЬЮ СЕРВИСА GOOGLE KUBERNETES ENGINE

При работе с сервисом GKE обеспечить регулярное обновление достаточно просто. Вам нужно подписаться на один из трех каналов обновлений: Stable, Regular и Rapid. Патчи безопасности выкладываются на каждом из этих каналов одинаково часто; разница лишь в скорости предоставления всех остальных новшеств системы Kubernetes и сервиса GKE.

После того как вы подпишетесь на канал обновлений, кластер и узлы будут автоматически обновляться до последней версии. Использовать более старую статическую версию не рекомендуется, поскольку в таком случае вам придется отслеживать обновления самостоятельно.

12.1.2. Обновление контейнеров

Обновление кластера Kubernetes — только часть работы, которую вам придется выполнять. Уязвимости защиты часто находят и в компонентах базовых образов, например, в базовом образе Ubuntu. А поскольку ваше контейнеризированное приложение устанавливается поверх базовых образов, то «получает в наследство» все имеющиеся в них уязвимости.

Выход в том, чтобы регулярно пересобирать и обновлять свои контейнеры, особенно при выявлении каких-либо уязвимостей в используемых базовых образах. Многие разработчики и компании используют сканеры уязвимостей (которые также называют *CVE-сканерами*, поскольку они используют список уязвимостей CVE (Common Vulnerabilities and Exposures, известные уязвимости)), чтобы проверять свои контейнеры на предмет наличия в них выявленных уязвимостей и определять, какие из них следует пересобрать и обновить в первую очередь.

При обновлении контейнера нужно указать базовый образ со всеми последними исправлениями. Обычно для этого достаточно указать минорный номер версии используемого базового образа, не указывая номер патча. Того же результата можно добиться, используя тег `latest`, но в таком случае вы можете получить ненужные вам изменения функциональности.

Возьмем в качестве примера базовый образ языка Python (https://hub.docker.com/_/python). В случае любой версии этого языка (например, 3.10.2) вам предлагается несколько вариантов: `3.10.2-bullseye`, `3.10-bullseye`, `3-bullseye` и просто `bullseye` (слово `bullseye` здесь обозначает используемую версию Debian), и кроме того, вы можете использовать тег последней версии `latest`. Если номера версий образов соответствуют принципам семантического версионирования, то я рекомендую выбрать вариант вида *мажорный_номер.минорный_номер*, то есть в данном случае вариант `3.10-bullseye`. Благодаря этому вы сможете автоматически получать патчи, выпускаемые для версии 3.10, не внося какие-либо революционные изменения. Но есть и минус: вам придется проследить за тем, когда прекратится поддержка версии 3.10, чтобы после этого перейти на более новую. Если же вы укажете только мажорный номер версии (то есть в данном случае выберете вариант `3-bullseye`), это позволит вам получать более длительную поддержку, но немного возрастет вероятность того, что какие-либо изменения приведут к некорректной работе кода. Теоретически использование мажорного номера версии не должно представлять опасности, поскольку вносимые при этом изменения должны быть обратно совместимыми. Но мой опыт показывает, что безопаснее применять минорный номер. Использовать тег последней версии `latest` не рекомендуется: хоть это и хороший вариант с точки зрения безопасности, его использование может привести к некорректной работе кода из-за внесения изменений, разрушающих обратную совместимость (breaking changes).

Какой бы способ настройки файла `Dockerfile` вы ни использовали, ключевые принципы сводятся к тому, чтобы регулярно пересобирать контейнеры, ссылаться

на обновленные базовые образы, регулярно обновлять рабочие нагрузки и использовать CVE-сканеры для выявления устаревших контейнеров. Еще больше повысить уровень безопасности вы сможете, если будете стремиться к тому, чтобы контейнеры приложений были как можно более легковесными и содержали только те компоненты, которые требуются для запуска приложения и его зависимостей. Типичный базовый образ такого дистрибутива, как Ubuntu, содержит такие компоненты, как менеджер пакетов и пакеты прикладных программ, которые, конечно, могут упростить вашу работу, но в то же время увеличивают поверхность атаки. Чем меньше в вашем контейнере будет кода, получаемого из других источников, тем реже потребуются выполнять обновление, необходимое для устранения выявленных в этом коде уязвимостей, и тем меньше ошибок потенциально может появиться.

Хороший пример использования этого принципа — файл `Dockerfile` из подраздела 2.1.8, посвященного многоэтапной сборке. Напомним, что там мы использовали один контейнер для сборки кода, а второй — для его запуска. Чтобы уменьшить потенциальную поверхность атаки, старайтесь выбирать для второго этапа сборки как можно более легковесный базовый образ со средой выполнения. Компания Google запустила проект с открытым исходным кодом `distroless` (<https://github.com/GoogleContainerTools/distroless>), который призван помочь разработчикам в создании сверхлегковесных контейнеров со средой выполнения. В листинге 12.1 показано, как выполнить сборку контейнера со средой выполнения Java, используя на втором этапе сборки `distroless`-образ.

Листинг 12.1. <https://github.com/GoogleContainerTools/distroless/tree/main/examples/java/Dockerfile>

```
FROM openjdk:11-jdk-slim-bullseye AS build-env
COPY . /app/examples
WORKDIR /app
RUN javac examples/*.java
RUN jar cfe main.jar examples.HelloJava examples/*.class
```

← Обычный образ набора инструментов OpenJDK, используемый для сборки кода

```
FROM gcr.io/distroless/java11-debian11
COPY --from=build-env /app /app
WORKDIR /app
CMD ["main.jar"]
```

← Distroless-образ со средой выполнения Java, используемый для запуска кода

12.1.3. Управление прерываниями

Применительно ко всем этим процедурам обновления у вас может возникнуть вопрос: что в ходе их выполнения будет происходить с запущенными рабочими нагрузками? При выполнении обновлений поды неизбежно будут удаляться и создаваться заново. Очевидно, это может сильно дестабилизировать работу нагрузок, запущенных в данных подах. К счастью, в Kubernetes можно снизить негативное влияние таких прерываний работы и потенциально полностью его исключить, если использовать несколько способов.

Проверки готовности

Прежде всего, если вы еще не настроили проверки готовности, как было показано в главе 4, то сделайте это наконец, поскольку это совершенно необходимо. Kubernetes полагается на сообщения о готовности, получаемые от контейнера. И если он не будет предоставлять такие данные, то будет считаться готовым к работе уже в момент запуска процесса, хотя приложение еще обычно не завершило инициализацию и на самом деле *не готово* к обслуживанию трафика. И чем больше подов вы будете перезапускать, например, при проведении обновлений, тем больше запросов будет завершаться ошибкой из-за направления их неготовым подам, если вы не реализуете корректные проверки готовности.

Обработка сигналов и корректное прекращение работы

Если с помощью проверок готовности можно определить, в какой момент приложение будет готово к запуску, то механизм корректного прекращения работы в Kubernetes помогает определить, в какой момент приложение будет готово прекратить свою работу. В случае задания, управляющего долго выполняющимся процессом, очевидно, лучше не выполнять внезапное принудительное завершение работы этого процесса. И даже когда речь идет о веб-приложении с короткоживущими запросами, резкое завершение работы может вызвать негативный эффект, приводя к неудачному завершению запросов.

Чтобы исключить вероятность возникновения этих проблем, важно реализовать в коде приложения логику, запускающую процесс завершения работы после получения сигнала SIGTERM, и определить достаточный период корректного завершения работы (с помощью поля `terminationGracePeriodSeconds`). Веб-приложения при поступлении сигнала SIGTERM должны завершать работу сервера после выполнения всех текущих запросов, а пакетные задания в идеале должны успевать выполнить всю текущую работу, не приступая к новым задачам.

В некоторых случаях прерывание работы задания, управляющего долго выполняющейся задачей, ведет к потере прогресса в выполнении этой задачи. В таких случаях можно задать очень большой период корректного завершения работы, чтобы после получения сигнала SIGTERM приложение продолжало свою работу, пока не завершит текущую задачу. Управляемые платформы часто накладывают лимит на продолжительность периода корректного завершения работы в случае ее системных прерываний.

В подразделе 10.1.2 мы рассмотрели несколько примеров обработки сигналов SIGTERM и настройки поля `terminationGracePeriodSeconds` в контексте объектов Job. Те же принципы действуют и в случае других типов рабочей нагрузки.

Скользящее обновление

При обновлении контейнеров, используемых в объектах Deployment или StatefulSet (например, для обновления базового образа), способ обновления определяется стратегией обновления. Чтобы свести к минимуму влияние прерываний работы при обновлении рабочих нагрузок, рекомендуется использовать рассмотренную в главе 4 скользющую стратегию обновления, последовательно обновляя целые группы подов, сохраняя при этом доступность приложения. При работе с объектом Deployment не забудьте настроить его параметр `maxSurge`, который временно увеличивает количество реплик пода на время проведения обновления, чтобы этот процесс не приводил к снижению доступности.

Квота недоступности подов

Обновление узлов выполняется немного *не так*, как обновление развертываний. Процесс выглядит следующим образом. Сначала узел блокируется, чтобы предотвратить развертывание на нем новых подов, а затем опустошается путем удаления подов с данного узла и повторного создания их на другом узле. По умолчанию Kubernetes сначала удаляет с узла сразу все поды, а потом (если они находятся под управлением такого ресурса рабочей нагрузки, как развертывание) распределяет их заново, создавая их в другом месте. Обратите внимание, что все происходит именно в такой последовательности, то есть повторное создание подов в другом месте *не выполняется* до их удаления. Работа нескольких реплик одного развертывания на одном и том же узле может привести к тому, что приложение будет недоступно в тот момент, когда все они будут одновременно вытеснены (рис. 12.1).

Чтобы опустошение узла, содержащего несколько подов одного развертывания, не приводило к снижению доступности (из-за резкого снижения количества работающих реплик), в Kubernetes предусмотрен ресурс с названием «Квота недоступности подов» (Pod Disruption Budget, PDB). С помощью этого объекта вы можете указать, каким должны быть количество или процент недоступных подов, чтобы ваша рабочая нагрузка сохраняла работоспособность (листинг 12.2).

Листинг 12.2. Chapter12/12.1_PDB/pdb.yaml

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: timeserver-pdb
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      pod: timeserver-pod
```

Объявляем, каким может быть
максимальное количество недоступных
подов при прерывании работы

Выбираем поды
по их метке

Без объекта PDB

все поды удаляются
одновременно, даже если
относятся к одному и тому же
развертыванию

Через некоторое время
поды заново создаются
на другом узле

Простой до того момента,
когда поды будут
распределены

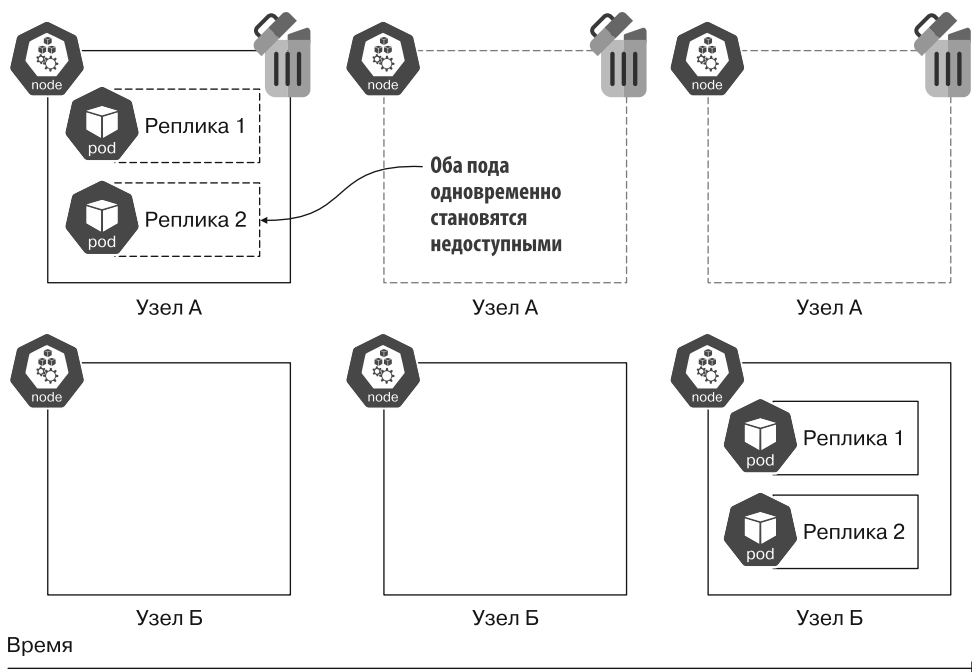


Рис. 12.1. Удаление узлов без использования квоты недоступности подов. Все размещенные на узле поды одновременно становятся недоступными

Если вы развернете объект PDB в своем кластере, то в случае прерывания работы у вас никогда не будет больше одного недоступного пода (рис. 12.2).

Альтернативный способ сводится к тому, чтобы задать минимальное количество реплик с помощью поля `minAvailable`. Я предпочитаю использовать поле `maxUnavailable`, поскольку оно лучше сочетается с масштабированием. Если вы решите использовать поле `minAvailable`, то в целях сохранения требуемого минимального уровня доступности придется масштабировать его значение в соответствии с изменением количества реплик.

ПРИМЕЧАНИЕ

Использование объекта PDB позволяет исключить произвольное вытеснение подов в ходе таких процессов, как обновление узлов, но не может защитить вас от прерываний, вызванных такими внезапными сбоями, как отказ узла.

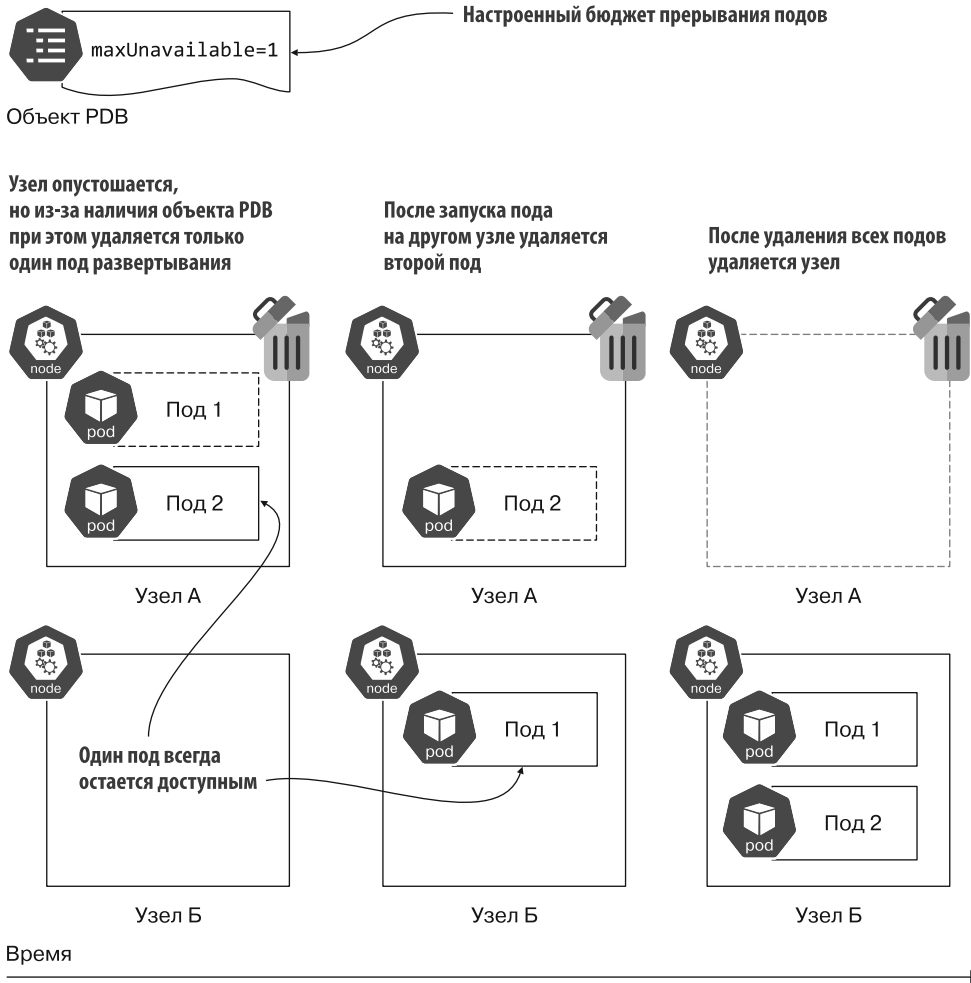


Рис. 12.2. При наличии объекта PDB Kubernetes будет ждать, пока у развертывания появится требуемое количество доступных подов, прежде чем удалять оставшиеся, тем самым снижая влияние прерывания работы

Управление прерываниями с помощью объекта PDB немного напоминает то, как скользящая стратегия обновления не допускает одновременное удаление слишком большого количества подов. Вы можете сделать так, чтобы ваше приложение оставалось доступным и в ходе иницилируемых вами обновлений, и при прерываниях работы, иницилируемых обновлениями кластера. Для этого вам потребуется настроить и скользящую стратегию обновления, и квоту недоступности подов.

12.2. РАЗВЕРТЫВАНИЕ УЗЛОВЫХ АГЕНТОВ С ПОМОЩЬЮ ОБЪЕКТОВ DAEMONSET

В этой книге мы уже рассмотрели несколько высокоуровневых конструкций, которые инкапсулируют поды, имеющие то или иное конкретное назначение, в том числе объект `Deployment` для приложений, объект `StatefulSet` для баз данных и объект `CronJob` для периодических задач. Объект `DaemonSet` — это еще один вид рабочей нагрузки, который позволяет запустить под на всех узлах.

В каких случаях требуется использовать этот объект? Он практически всегда выполняет в кластере служебную функцию, такую как журналирование, мониторинг или обеспечение безопасности. Как разработчик приложений, вы будете использовать его не слишком часто. Вы можете опубликовать сервисы на внутрикластерном IP-адресе, делая их доступными для любого пода, расположенного в вашем кластере, поэтому вам не нужно запускать сервисы на всех узлах просто для того, чтобы сделать их доступными в пределах кластера. А если требуется подключаться к сервису на локальной машине, то вы можете создать виртуальный адрес, используя сервис типа `NodePort`. Объекты `DaemonSet` обычно используются для выполнения определенных операций на уровне узлов, например, для отслеживания уровня загрузки или производительности, поэтому их можно отнести исключительно к сфере системного администрирования.

С помощью объектов `DaemonSet` обычно развертывают свои программы поставщики решений для журналирования, мониторинга и обеспечения безопасности. Эти программы выполняют следующие операции:

- чтение записей журналов на узле с их последующей загрузкой на центральный сервер журналирования;
- запрашивание у агента `kubelet` показателей производительности (таких как количество запущенных подов, время их начальной загрузки и т. д.);
- обеспечение безопасности путем отслеживания поведения контейнеров и хостов.

Все это примеры подов, которые должны находиться на каждом узле для сбора данных, необходимых для корректной работы продукта в целом.

Обычно в кластере работает несколько объектов `DaemonSet`, запущенных в пространстве имен `kube-system`, которые реализуют такую функциональность, как журналирование, мониторинг и внутрикластерный DNS-сервис. В качестве примера ниже представлен неполный список объектов `DaemonSet`, используемых в кластере GKE:

```
$ kubectl get daemonset -n kube-system
NAMESPACE   NAME
kube-system  filestore-node
kube-system  fluentbit-gke
kube-system  gke-metadata-server
```



```

kube-system    gke-metrics-agent
kube-system    kube-proxy
kube-system    metadata-proxy-v0.1
kube-system    netd
kube-system    node-local-dns
kube-system    pdcsi-node

```

Как правило, разработчики приложений не создают эти объекты сами, а используют уже готовые объекты DaemonSet, предоставленные поставщиками программных решений. Однако в качестве примера в листинге 12.3 показано, как определить простейший объект DaemonSet, который считывает записи журналов, созданные на узле, в стандартный поток вывода (stdout).

Листинг 12.3. Chapter12/12.2_DaemonSet/logreader.yaml

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: logreader
spec:
  selector:
    matchLabels:
      pod: logreader-pod
  template:
    metadata:
      labels:
        ds: logreaderpod
    spec:
      containers:
        - image: ubuntu
          command:
            - bash
            - "-c"
            - |
              tail -f /var/log/containers/*_kube-system_*.log
          name: logreader-container
      resources:
        requests:
          cpu: 50m
          memory: 100Mi
          ephemeral-storage: 100Mi
      volumeMounts:
        - name: logpath
          mountPath: /var/log
          readOnly: true
      volumes:
        - hostPath:
            path: /var/log
            name: logpath

```

Считываем и выводим записи журналов, созданные на узле контейнерами пространства имен kube-system

Объектам DaemonSet обычно требуется небольшое количество ресурсов

Монтируем том logpath в каталог /var/log

Определяем том logpath, используя каталог /var/log на хосте

Создать этот объект DaemonSet можно с помощью такой команды:

```

$ kubectl create -f Chapter12/12.2_DaemonSet/logreader.yaml
daemonset.apps/logreader created

```

354 Часть II. Переходим в продуктивную среду

Теперь после запуска подов мы можем обеспечить потоковый вывод записей журналов:

```
$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
logreader-2nbt4     1/1     Running   0           4m14s

$ kubectl logs -f logreader-2nbt4 --tail 10
==> /var/log/containers/filestore-node_kube-system_gcp-filestore-1b5.log <==
lock is held by gk3-autopilot-cluster-2sc2_e4337a2e and has not yet expired
```

На практике вы можете сталкиваться с объектами DaemonSet при разворачивании программных решений, предназначенных для журналирования, мониторинга и обеспечения безопасности.

12.3. КОНТЕКСТ БЕЗОПАСНОСТИ ПОДА

Шаблон PodSpec имеет свойство `securityContext`, с помощью которого можно определить атрибуты пода и его контейнеров, связанные с безопасностью. Если ваш под должен выполнять некую административную функцию (например, потому, что является частью DaemonSet, выполняющего некую операцию на уровне узла), то именно в этом свойстве вы будете определять привилегии, которые необходимы для корректной работы пода. В качестве примера ниже представлено определение DaemonSet, запрашивающего привилегии для своих подов (листинг 12.4).

Листинг 12.4. Chapter12/12.3_PodSecurityContext/admin-ds.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: admin-workload
spec:
  selector:
    matchLabels:
      name: admin-app
  template:
    metadata:
      labels:
        name: admin-app
    spec:
      containers:
        - name: admin-container
          image: ubuntu
          command: ["sleep", "infinity"]
          securityContext:
            privileged: true
```

Под с такими привилегиями фактически получает права суперпользователя `root` и может, например, смонтировать в контейнер файловую систему узла, как показано ниже:

```
$ kubectl exec -it admin-workload-px6xg -- bash
root@admin-workload-px6xg:/# df
Filesystem      1K-blocks    Used   Available   Use%    Mounted on
overlay         98831908    4652848   94162676    5%      /
tmpfs           65536        0       65536       0%      /dev
/dev/sda1       98831908    4652848   94162676    5%      /etc/hosts
shm             65536        0       65536       0%      /dev/shm
root@admin-workload-px6xg:/# mkdir /tmp/host
root@admin-workload-px6xg:/# mount /dev/sda1 /tmp/host
root@admin-workload-px6xg:/# cd /tmp/host
root@admin-workload-px6xg:/tmp/host# ls
dev_image etc home lost+found var var_overlay vmlinuz_hd.vblock
root@admin-workload-px6xg:/tmp/host#
```

Если вы попытаетесь сделать то же самое с контейнером без привилегий, то операция монтирования закончится неудачей.

В большинстве случаев вам нужно будет просто запустить в Kubernetes разработанное вами обычное приложение, поэтому вы гораздо чаще будете использовать свойства `securityContext`, чтобы, наоборот, сделать некоторые функции *недоступными* для пода и тем самым сделать его использование более безопасным. В качестве примера ниже представлен шаблон PodSpec, который, в отличие от предыдущего примера, блокирует для пода доступ к привилегиям, вследствие чего тот запускается без прав суперпользователя и не имея возможности повысить привилегии (листинг 12.5).

Листинг 12.5. Chapter12/12.3_PodSecurityContext/pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu
  labels:
    pod: ubuntu-pod
spec:
  containers:
  - name: ubuntu-container
    image: ubuntu
    command: ["sleep", "infinity"]
    securityContext:
      runAsNonRoot: true
      runAsUser: 1001
      allowPrivilegeEscalation: false
      capabilities:
        drop:
        - ALL
```

По умолчанию любой под может свободно запрашивать любые необходимые ему возможности, в том числе права суперпользователя (если, конечно, это не запрещает ваша платформа Kubernetes, как это делают некоторые безузловые платформы). Как оператору кластера, вам лучше наложить запрет на подобные действия, поскольку это, по сути, означает, что любой, кто может подключиться к кластеру с помощью утилиты `kubectl`, может получить права суперпользователя. Существуют и другие принципы обеспечения безопасности кластера, один из которых сводится к тому, чтобы не запускать контейнеры от имени суперпользователя, что в представленном выше примере выполняется с помощью настройки `runAsNonRoot: true`.

В следующих разделах мы подробно поговорим об этих принципах безопасности, начиная с того, как создавать контейнеры, чтобы их не нужно было запускать от имени суперпользователя, и как вы, будучи администратором кластера, можете побудить пользователей применять эту и другие настройки, служащие для обеспечения безопасности.

12.4. ЗАПУСК КОНТЕЙНЕРОВ НЕ ОТ ИМЕНИ СУПЕРПОЛЬЗОВАТЕЛЯ

Одна из основных рекомендаций по обеспечению безопасности при разворачивании контейнеров сводится к тому, чтобы не запускать их от имени суперпользователя. Это объясняется тем, что контейнеры Linux, по сути, представляют собой обычные процессы, запускаемые на хосте с использованием таких механизмов изоляции, как контрольные группы Linux (`cgroups`) и пространства имен. Если ваш контейнер будет создан с расчетом на запуск от имени суперпользователя, как это делается по умолчанию, то будет запущен на узле от имени суперпользователя, но с некоторой изоляцией, не позволяющей ему использовать права суперпользователя. Проблема заключается в том, что если в применяемом механизме контейнеризации Linux есть уязвимость типа «побег из контейнера», то изолированный процесс контейнера все же сможет получить привилегии суперпользователя. То есть при запуске контейнера от имени суперпользователя «побег из контейнера» позволяет злоумышленнику получить доступ к узлу со всеми правами суперпользователя, что нельзя назвать хорошей ситуацией.

Docker по умолчанию запускает все процессы от имени суперпользователя, поэтому любая уязвимость типа «побег из контейнера» может быть опасной. Такие уязвимости встречаются достаточно редко, но полностью исключить вероятность их возникновения нельзя, поэтому в соответствии с принципом *эшелонированной защиты* будет лучше принять против них определенные меры защиты. Он означает следующее: хоть изоляция контейнера и обеспечивает защиту хоста при взломе приложения, в идеале вы должны иметь дополнительные уровни защиты на случай, если будет взломана и эта изоляция. Этот принцип требует запуска контейнеров от имени обычного пользователя, чтобы в том случае, если злоумышленнику удастся взломать контейнер и воспользоваться уязвимостью типа «побег из контейнера»

в механизме контейнеризации Linux, он все равно не смог получить на узле повышенный уровень привилегий. Чтобы добиться этого, ему придется найти еще одну уязвимость, преодолев три уровня защиты (уровень приложения, механизма контейнеризации Linux и пользовательских привилегий Linux).

ПРИМЕЧАНИЕ

Возможно, у вас возник вопрос: если рекомендация сводится к тому, чтобы не запускать процессы контейнеров от имени суперпользователя, то почему Docker по умолчанию создает контейнеры с расчетом на запуск от имени суперпользователя? Эта ситуация объясняется тем, что такой подход удобен для разработчиков. Запуск контейнера от имени суперпользователя позволяет задействовать привилегированные порты (с номерами от 1 до 1023, в том числе, например, HTTP-порт по умолчанию с номером 80) и не иметь никаких проблем с разрешениями на доступ к каталогам. Как вы увидите далее в этом разделе, создание и запуск контейнеров от имени обычного пользователя могут вести к возникновению ошибок, устранение которых требует определенных усилий. Но если вы будете придерживаться данного принципа с самого начала, то быстро научитесь легко устранять эти ошибки по мере их возникновения, а в результате вы добавите еще один уровень защиты в свою систему.

Сделать так, чтобы контейнеры запускались от имени обычного пользователя, в Kubernetes достаточно легко. Но, как мы вскоре увидим, проблема заключается в том, что не рассчитанные на это контейнеры могут при этом не запускаться. Запретить запуск подов от имени суперпользователя можно, добавив в их определение соответствующую аннотацию. Это первый необходимый шаг. Если вы настраиваете кластер Kubernetes для большой команды разработчиков или сами являетесь членом большой команды, которая использует уже настроенный большой кластер, то будет лучше воспользоваться контроллером допуска платформы Kubernetes, который автоматически добавит эту аннотацию во все поды (см. подраздел 12.5.1). Оба способа приводят к одинаковому результату, поэтому я приведу только пример добавления данной аннотации вручную. Ниже показано, как наложить запрет на запуск контейнеров от имени суперпользователя в определении развертывания (листинг 12.6).

Листинг 12.6. Chapter12/12.4_NonRootContainers/1_permission_error/deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: timeserver
spec:
  replicas: 1
  selector:
    matchLabels:
      pod: timeserver-pod
  template:
    metadata:
      labels:
        pod: timeserver-pod
```

```

спес:
  containers:
  - name: timeserver-container
    image: docker.io/wdenniss/timeserver:6
    securityContext:
      runAsNonRoot: true

```

Запрет на запуск этого контейнера
от имени суперпользователя

Однако это еще не все, ведь мы не указали, от имени какого пользователя должен запускаться контейнер. Если вы попытаетесь создать это разворачивание, то Kubernetes просто применит определенный в поле `securityContext` запрет и не позволит запустить контейнер от имени суперпользователя. Вот как (в сокращенном виде) выглядит результат такой попытки:

```

$ kubectl get pods
NAME                                READY   STATUS                                RESTARTS
timeserver-pod-fd574695c-5t92p    0/1     CreateContainerConfigError           0

$ kubectl describe pod timeserver-pod-fd574695c-5t92p
Name:          timeserver-pod-fd574695c-5t92p
Events:
  Type      Reason      Age           From          Message
  ----      -
Warning    Failed      10s (x3 over 23s)    kubelet       Error: container has
runAsNonRoot and image will run as root

```

Чтобы решить эту проблему, мы должны указать, от имени какого пользователя должны запускаться поды. Суперпользователю `root` всегда присваивается номер `0`, поэтому нужно указать пользователя с любым другим номером, например `1001`. Это можно сделать в файле `Dockerfile` с помощью команды `USER 1001` или в конфигурации Kubernetes с помощью настройки `runAsUser: 1001`. Если используются сразу оба способа, то приоритет отдается конфигурации Kubernetes, подобно тому, как параметр `command` в шаблоне `PodSpec` платформы Kubernetes имеет приоритет над используемой в файле `Dockerfile` командой `CMD`. Пример настройки пользователя в файле `Dockerfile` выглядит так:

```

FROM python:3
COPY . /app
WORKDIR /app
RUN mkdir logs
CMD python3 server.py
USER 1001

```

Чтобы настроить пользователя в шаблоне `PodSpec`, необходимо добавить дополнительное поле в разделе `securityContext` (листинг 12.7):

Листинг 12.7. Chapter12/12.4_NonRootContainers/1_permission_error/deploy-runas.yaml

```

# ...
securityContext:
  runAsNonRoot: true
  runAsUser: 1001

```

Оба способа позволяют получить одинаковый результат, но я рекомендую настраивать контейнеры на запуск от обычного пользователя в Kubernetes, поскольку это позволяет лучше отделить среду разработки от продуктивной. Если, настроив обычного пользователя в файле `Dockerfile`, вы произведете локальный запуск контейнера, не используя Kubernetes, и попытаетесь смонтировать том, то увидите, что это можно сделать только от имени суперпользователя: это известная проблема Docker (<https://github.com/moby/moby/issues/2259>), которую разработчики не могут решить уже более семи лет. Проблемы безопасности, связанные с запуском контейнеров от имени суперпользователя, актуальны только в продуктивной среде, так почему бы не перенести туда и все заботы о решении этого вопроса? К счастью, вы легко можете сделать так, чтобы при локальном запуске в Docker контейнер запускался от имени суперпользователя, а при запуске в продуктивной среде с помощью Kubernetes — от имени обычного пользователя. Первое позволит вам сделать условия своей работы максимально удобными, а второе — обеспечить эшелонированную защиту.

Чтобы контейнер запускался от имени обычного пользователя, достаточно добавить в конфигурации настройку `runAsUser: 1001`. Если контейнер может работать в таком режиме, то на этом ваша работа будет закончена. Общедоступные и широко используемые контейнеры, как правило, рассчитаны на запуск от имени обычного пользователя, но применительно к контейнерам, которые вы создаете сами, это не всегда так.

Так, в нашем случае контейнер не рассчитан на запуск от имени обычного пользователя и требует корректировки. Запуск от имени обычного пользователя имеет следующие два основных различия: вы не можете прослушивать привилегированные порты (с номерами от 1 до 1023) и по умолчанию не можете выполнять запись в записываемый уровень контейнера (то есть записывать какие-либо файлы!). Это представляет проблему для шестой версии демонстрационного приложения Timeserver (`Chapter12/timeserver6/server.py`), которое прослушивает порт 80 и записывает файл журнала в каталог `/app/logs`.

12.4.1. Изменение контейнеров в целях их запуска от имени обычного пользователя

Если вы развернете новую версию развертывания, в которой будет задан параметр `runAsUser`, как было показано в листинге 12.7 (см. выше), то увидите, что, несмотря на отсутствие ошибки `CreateContainerConfigError` (Ошибка конфигурации создания контейнера) на этапе развертывания, ваш контейнер все равно выходит из строя после получения прав обычного пользователя, что вызвано, вероятно, этим изменением проблемы с правами доступа. Но, прежде чем приступить к отладке проблем, связанных с правами обычного пользователя, убедитесь в том, что ваш контейнер нормально работает с правами суперпользователя; в противном случае проблема может иметь совсем другие причины.

Отладка проблем с разрешениями для контейнеров, запущенных от имени обычного пользователя, может осуществляться по-разному, поэтому посмотрим, как найти и исправить те две распространенные ошибки, которые были допущены в нашем примере. Ниже в сокращенном виде показано, какие записи журнала были выведены для вышедшего из строя контейнера в моем случае:

```
$ kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
timeserver-demo-774c7f5ff9-fq94k   0/1     CrashLoopBackOff    5 (47s ago) 4m4s

$ kubectl logs timeserver-demo-76ddf6d5c-7s9zc
Traceback (most recent call last):
  File "/app/server.py", line 23, in <module>
    startServer()
  File "/app/server.py", line 17, in startServer
    server = ThreadingHTTPServer(('',80), RequestHandler)
  File "/usr/local/lib/python3.9/socketserver.py", line 452, in __init__
    self.server_bind()
  File "/usr/local/lib/python3.9/http/server.py", line 138, in server_bind
    socketserver.TCPServer.server_bind(self)
  File "/usr/local/lib/python3.9/socketserver.py", line 466, in server_bind
    self.socket.bind(self.server_address)
PermissionError: [Errno 13] Permission denied
```

К счастью, проблему с портом можно легко решить в Kubernetes без каких-либо негативных последствий для конечных пользователей. Мы можем изменить используемый контейнером порт, сохранив для балансировщика нагрузки стандартный порт 80. Сначала зададим другой порт для контейнера (листинг 12.8).

Листинг 12.8. Chapter12/timeserver7/server.py

```
//...

def startServer():
    try:
        server = ThreadingHTTPServer(('',8080), RequestHandler)
        print("Listening on " + ":".join(map(str, server.server_address)))
        server.serve_forever()
    except KeyboardInterrupt:
        server.shutdown()

if __name__ == "__main__":
    startServer()
```

Наше приложение будет использовать другой порт, поэтому мы должны изменить и конфигурацию соответствующего сервиса Kubernetes, указав новый порт в поле `targetPort`. К счастью, нам не нужно изменять внешний порт сервиса, поскольку для связи с сетью сервис использует связующий компонент, предоставленный

платформой Kubernetes. Данный компонент не запускается от имени какого-либо пользователя, что позволяет использовать порты с номерами от 1 до 1023 (листинг 12.9).

Листинг 12.9. Chapter12/12.4_NonRootContainers/2_fixed/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: timeserver
spec:
  selector:
    pod: timeserver-pod
  ports:
  - port: 80
    targetPort: 8080
    protocol: TCP
  type: LoadBalancer
```

Указываем в качестве целевого порта новый порт контейнера

Решив проблему с портами и перезапустив приложение, вы столкнетесь с еще одной сложностью, когда приложение попытается выполнить запись в файл журнала на диске. Эта ошибка не мешает приложению запускаться и возникает после поступления запроса. В моем случае записи журнала выглядели так:

```
$ kubectl logs timeserver-demo-5fd5f6c7f9-cxzrb
10.22.0.129 - - [24/Mar/2022 02:10:43] "GET / HTTP/1.1" 200 -
Exception occurred during processing of request from ('10.22.0.129' , 41702)
Traceback (most recent call last):
  File "/usr/local/lib/python3.10/socketserver.py", line 683, in
    process_request_thread
    self.finish_request(request, client_address)
  File "/usr/local/lib/python3.10/socketserver.py", line 360, in
    finish_request
    self.RequestHandlerClass(request, client_address, self)
  File "/usr/local/lib/python3.10/socketserver.py", line 747, in
    __init__
    self.handle()
  File "/usr/local/lib/python3.10/http/server.py", line 425, in
    handle
    self.handle_one_request()
  File "/usr/local/lib/python3.10/http/server.py", line 413, in
    handle_one_request
    method()
  File "/app/server.py", line 11, in do_GET
    with open("logs/log.txt", "a") as myfile:
PermissionError: [Errno 13] Permission denied: 'logs/log.txt'
```

Возникновение ошибки `Permission denied` при попытке записать файл из приложения, запущенного от имени обычного пользователя, — явный признак того, что

права доступа к каталогу не были настроены так, чтобы его могли использовать обычные пользователи.

Самый простой способ решить эту проблему — настроить групповые права доступа для используемого каталога. Я предпочитаю применять этот подход потому, что он позволяет использовать одну и ту же группу (например, 0) и для локального запуска с помощью Docker, и для развертывания в продуктивной среде с помощью Kubernetes. При этом в файл Dockerfile не вносятся какие-либо изменения, связанные со средой. Попробуем добавить в файл Dockerfile команду, предоставляющую группе 0 доступ с правом записи (листинг 12.10).

Листинг 12.10. Chapter12/timeserver7/Dockerfile

```
FROM python:3.12
ENV PYTHONUNBUFFERED 1
COPY . /app
WORKDIR /app
RUN mkdir logs
RUN chgrp -R 0 logs \      | Изменяем права доступа
    && chmod -R g+rwX logs | к каталогу с файлом журнала
CMD python3 server.py
```

Если перед развертыванием этого контейнера в Kubernetes вы хотите посмотреть, как он будет работать с правами обычного пользователя на локальной машине в Docker, то это можно сделать, задав пользователя во время выполнения с помощью команды `docker run --user 1001:0 $CONTAINER_NAME`.

На этом наша работа закончена — теперь новая 7-я версия контейнера будет успешно работать с правами обычного пользователя. Запустите конфигурацию из каталога Chapter12/12.4_NonRootContainers/2_fixed и убедитесь, что она работает. При желании вы можете отобразить все изменения, которые позволили контейнеру и конфигурации работать с правами обычного пользователя, сравнив с помощью команды `diff` старые и новые версии файлов:

```
cd Chapter12
diff -u timeserver6 timeserver7
diff -u 12.4_NonRootContainers/1_permission_error \
    12.4_NonRootContainers/2_fixed
```

12.5. КОНТРОЛЛЕРЫ ДОПУСКА

В предыдущем разделе мы добавили в определение пода настройку `runAsNonRoot: true`, чтобы он не запускался от имени суперпользователя, но сделали это вручную. Если нужно, чтобы такая настройка была у всех подов, то в идеале нужно настроить кластер так, чтобы он отвергал поды, у которых ее нет, или просто автоматически добавлял ее.

Здесь могут помочь *контроллеры допуска*. Это небольшие блоки кода, которые запускаются с помощью веб-хуков в момент создания объекта, например, с помощью команды `kubectl create ОБЪЕКТ` (рис. 12.3). Существуют два типа таких контроллеров: проверяющие и изменяющие. Проверяющие веб-хуки допуска либо одобряют, либо отвергают объект Kubernetes — например, они могут отвергать поды, не имеющие настройки `runAsNonRoot: true`. Изменяющие веб-хуки допуска могут изменять поступающий объект — например, добавлять настройку `runAsNonRoot: true`.

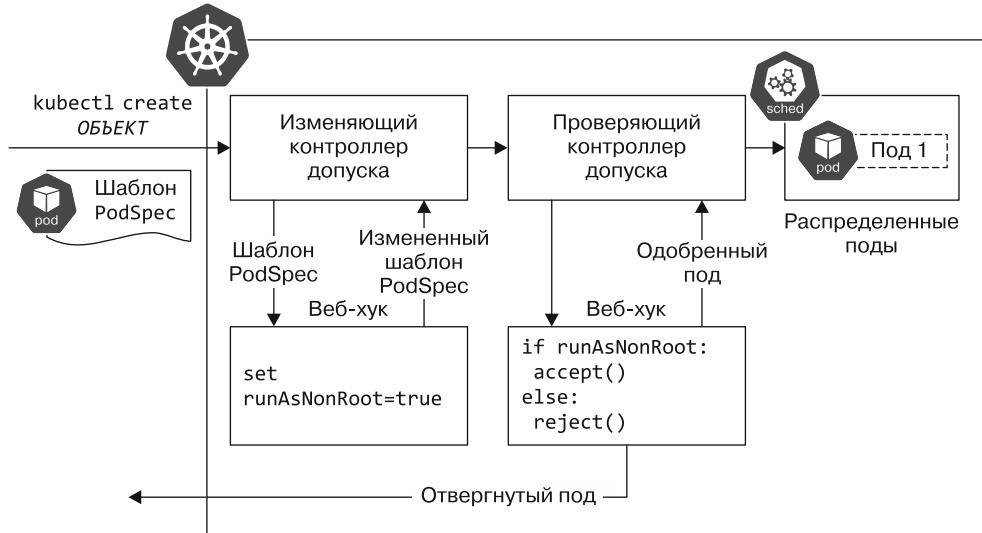


Рис. 12.3. Процесс допуска распределяемого пода

Чтобы реализовать нужное вам поведение, вы можете создать собственные контроллеры допуска, но в зависимости от того, какой результат хотите получить, вам это может и не понадобится. Вместо этого можно использовать встроенные контроллеры допуска платформы Kubernetes или сторонние контроллеры допуска, распространяемые как коммерческое программное обеспечение или ПО с открытым исходным кодом.

12.5.1. Допуск безопасности пода

Создание контроллера допуска — непростая задача. Вы должны настроить сертификаты, создать приложение, которое можно будет настроить в качестве веб-хука, соответствующего запросам/ответам API платформы Kubernetes, и организовать процесс разработки так, чтобы этот код обновлялся по мере каждого изменения платформы Kubernetes, что с ней происходит довольно часто. К счастью,

в большинстве случаев можно обойтись без создания собственных контроллеров допуска, используя вместо этого контроллеры допуска, предлагаемые платформой, или сторонние контроллеры допуска.

Используя встроенные контроллеры допуска платформы Kubernetes, вы можете обеспечить соблюдение таких политик безопасности, как обязательное использование настройки `runAsNonRoot: true`. До версии 1.25 для этой цели служил объект `PodSecurityPolicy`, который, однако, так и не преодолел этап бета-версии и был удален. Начиная с версии 1.25 для реализации политик безопасности рекомендуется использовать механизм допуска безопасности пода (`Pod Security Admission, PSA`) в виде контроллера допуска. Вы даже можете вручную разворачивать этот механизм в тех кластерах, где используется старая версия Kubernetes или где он не был активирован оператором платформы.

Стандарты безопасности пода

Стандарты безопасности пода (<https://kubernetes.io/docs/concepts/security/pod-security-standards/>) предусматривают три уровня политик безопасности, которые применяются на уровне пространства имен.

- *Привилегированный (Privileged)* — поды обладают неограниченным административным доступом и могут получать доступ к узлам с правами суперпользователя.
- *Базовый (Baseline)* — поды не могут повышать свои привилегии для получения административного доступа.
- *Ограниченный (Restricted)* — обеспечивает применение рекомендуемых методов обеспечения безопасности (среди которых использование принципа эшелонированной защиты), дополняя базовый профиль такими дополнительными уровнями защиты, как запрет на запуск от имени суперпользователя.

В общем случае уровень `privileged` должен предоставляться только системным рабочим нагрузкам; уровень `baseline` представляет собой хороший баланс между безопасностью и совместимостью; а уровень `restricted` обеспечивает усиленную эшелонированную защиту за счет некоторого ухудшения совместимости, например, обеспечивает запуск всех контейнеров от имени обычного пользователя, как это делалось в разделе 12.4.

Создание пространства имен с заданным уровнем безопасности подов

Продолжая сквозной пример этой главы, попробуем реализовать самый безопасный профиль защиты, создав пространство имен с политикой `restricted`. Она сделает недопустимым запуск подов от имени суперпользователя и обеспечит применение других рекомендуемых методов защиты.

Для начала создадим новое пространство имен с этой политикой. Пусть оно называется `team1` (команда 1), поскольку предполагается, что его будет использовать для развертывания своего кода первая команда разработчиков (листинг 12.11).

Листинг 12.11. Chapter12/12.5_PodSecurityAdmission/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: team1
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.28
```

Используемые здесь две метки (`labels`) указывают, какую политику и какую ее версию следует использовать. Версию необходимо указывать потому, что определение политики может меняться по мере выявления новых угроз безопасности. Чтобы не привязываться к конкретному номеру версии, можно использовать ключевое слово `latest` (последняя), что позволит применять самую последнюю версию политики. Помните, что изменение политик с выходом новых версий Kubernetes может плохо сказываться на работе запущенных рабочих нагрузок, поэтому рекомендуется всегда указывать конкретный номер версии. В идеале, прежде чем обновлять метку `enforce-version` в продуктивной среде, необходимо сначала тестировать новые версии политики в предпродуктивной среде или кластере.

Создадим это пространство имен:

```
kubect1 create -f Chapter12/12.5_PodSecurityAdmission/namespace.yaml
kubect1 config set-context --current --namespace=team1
```

Если теперь мы попробуем развернуть под из главы 3, в определении которого не задан параметр `runAsNonRoot`, система его отклонит:

```
$ kubect1 create -f Chapter03/3.2.4_ThePodSpec/pod.yaml
Error from server (Forbidden): error when creating
"Chapter03/3.2.4_ThePodSpec/pod.yaml": admission webhook
"pod-security-webhook.kubernetes.io" denied the request: pods "timeserver"
is forbidden: violates PodSecurity "restricted:v1.28":
allowPrivilegeEscalation != false (container "timeserver-container" must set
securityContext.allowPrivilegeEscalation=false), unrestricted capabilities
(container "timeserver-container" must set
securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or
container "timeserver-container" must set securityContext.runAsNonRoot=true)
```

Если же мы добавим контекст безопасности, требуемый политикой PSA (листинг 12.12), то наш под будет одобрен. Кроме того, важно, чтобы используемый

контейнер был настроен на запуск с правами обычного пользователя, как было показано в предыдущем подразделе, чтобы он мог работать в заданных нами новых условиях.

Листинг 12.12. Chapter12/12.5_PodSecurityAdmission/nonroot_pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: timeserver-pod
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: timeserver-container
    image: docker.io/wdenniss/timeserver:7
    securityContext:
      runAsNonRoot: true
      allowPrivilegeEscalation: false
      runAsUser: 1001
      capabilities:
        drop:
          - ALL
```

Контекст
безопасности,
требуемый
профилем restricted

Теперь создание этого пода с правами обычного пользователя должно пройти успешно:

```
$ kubectl create -f Chapter12/12.5_PodSecurityAdmission/nonroot_pod.yaml
pod/timeserver-pod created
```

ОТЛАДКА ОТКАЗОВ В ДОПУСКЕ ПОДОВ ПРИ ИСПОЛЬЗОВАНИИ РАЗВЕРТЫВАНИЯ

В представленных в этом разделе двух примерах я использовал самостоятельные поды, а не объекты Deployment. Я поступил так потому, что при таком подходе проще выполнять отладку отказов в допуске подов. Однако убедившись в том, что самостоятельный под работает успешно, вы всегда можете встроить его определение в виде шаблона PodSpec в нужное вам развертывание.

Если вы создадите развертывание, которое нарушает ограничения безопасности, в консоли не появится никакого сообщения об ошибке, как это произошло в представленном мной примере при попытке создать под напрямую. К сожалению, это неотъемлемая особенность работы реализации объекта Deployment, используемой в Kubernetes. Создание данного объекта проходит успешно, поэтому в консоли не выводится никакое сообщение об ошибке. Но когда он

приступает к созданию своих подов, эта попытка заканчивается неудачей. Кроме того, развертывание в действительности не само управляет подами, а создает объект `ReplicaSet`, который позволяет управлять подами конкретного варианта развертывания. Поэтому вы не сможете обнаружить данную ошибку, даже если отобразите подробную информацию о развертывании, поскольку для этого нужно исследовать информацию об объекте `ReplicaSet`.

До сих пор я не упоминал об объекте `ReplicaSet`, так как он фактически является внутренней деталью реализации. Это, по сути, конструкция для представления рабочей нагрузки, обеспечивающая управление набором подов. Развертывание создает новый объект `ReplicaSet` для каждой новой версии. Поэтому в ходе скользящего обновления развертывание в действительности использует два объекта `ReplicaSet`: один для старой версии, второй — для новой. Благодаря постепенному их масштабированию и осуществляется такое обновление. В большинстве случаев данная деталь реализации неважна, поэтому я не останавливался на ней до сих пор, однако здесь мы имеем дело с одним из тех редких случаев, где она имеет значение, поскольку именно в информации об объекте `ReplicaSet` скрываются такие ошибки.

Для отладки таких ошибок можно использовать следующий не совсем простой способ. Обычно после создания развертывания оно создает и свои поды. Выполнив в этот момент команду `kubectl get pods`, вы можете увидеть, что у вас есть несколько подов. Не все они могут находиться в состоянии готовности (`Ready`), так как по ряду причин могут еще оставаться в состоянии ожидания (`Pending`) (а в некоторых случаях и застревать в этом состоянии навсегда, как упоминалось в подразделе 3.2.3). Тем не менее в нормальной ситуации эти поды как минимум существуют, каким бы ни было их состояние. Если команда `kubectl get pods` не выдает для вашего развертывания никакой информации о подах, это может объясняться тем, что они были отвергнуты на этапе допуска.

Созданием подов на самом деле занимается объект `ReplicaSet`, принадлежащий развертыванию. Поэтому для того чтобы вывести на экран сообщение об этой ошибке, потребуется отобразить подробную информацию о данном объекте, используя команду `kubectl describe replicaset` (в сокращенном виде `kubectl describe rs`). Ниже представлен пример того, как это можно сделать (чтобы улучшить читабельность, вывод команд я привел в сокращенном виде):

```
$ kubectl create -f Chapter03/3.2_DeployingToKubernetes/deploy.yaml
deployment.apps/timeserver created
$ kubectl get deploy
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
timeserver    0/3      0             0            12s
$ kubectl get pods
```

```
No resources found in myapp namespace.

$ kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
timeserver-5b4fc5bb4               3         0         0       31s

$ kubectl describe rs
Events:
  Type            Reason            Age           From              Message
  ----            -
Warning FailedCreate 36s            replicaset-controller
Error creating: admission webhook "pod-security-webhook.kubernetes.io"
denied the request: pods "timeserver-5b4fc5bb4-hvqcm" is forbidden:
violates PodSecurity "restricted:v1.28": allowPrivilegeEscalation != false
(container "timeserver-container" must set
securityContext.allowPrivilegeEscalation=false), unrestricted capabilities
(container "timeserver-container" must set
securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or
container "timeserver-container" must set
securityContext.runAsNonRoot=true)
```

Завершив работу с этим примером, вы можете удалить созданное нами пространство имен и соответствующие ресурсы:

```
$ kubectl delete ns team1
namespace "team1" deleted
```

12.5.2. Соблюдение баланса между безопасностью и совместимостью

В предыдущем подразделе мы рассмотрели пример использования ограниченного профиля безопасности подов (**restricted**), содержащего настройку, которая позволяет контейнеру запускаться от имени обычного пользователя. Надеюсь, теперь в случае необходимости вы сможете легко запустить контейнеры в безопасном режиме. Такой подход является рекомендуемым и может даже быть обязательным к применению в регулируемых отраслях. Тем не менее он усложняет процесс разработки и не всегда применим на практике. В конечном счете вопрос о том, какой профиль безопасности лучше подойдет в вашем случае, должны решать вы сами, ваши специалисты по безопасности и, возможно, регулирующие органы. При этом я не рекомендую помещать в пространство имен с ограниченным профилем **restricted** буквально каждую рабочую нагрузку. На мой взгляд, лучше использовать базовый профиль **baseline** для всех неадминистративных нагрузок, поскольку он помогает защитить кластер в случае взлома одного из контейнеров, не вызывая при этом несовместимость со среднестатистическим приложением.

Административные нагрузки, которым требуется привилегированный профиль `privileged`, лучше запускать в их собственных пространствах имен, отдельно от остальных рабочих нагрузок.

12.6. РОЛЕВОЕ УПРАВЛЕНИЕ ДОСТУПОМ

Допустим, вам нужно, чтобы поды запускались от имени обычного пользователя (см. раздел 12.4), и вы соответствующим образом настроили контроллер допуска безопасности подов (PSA) (см. раздел 12.5). Все это звучит хорошо, но лишь при том условии, что вы уверены в том, что пользователи кластера не могут случайно или намеренно изменить ваши настройки и удалить эти ограничения. Необходимо обеспечить действительно надежное применение требований контроллера допуска и создать многоуровневую систему пользовательских разрешений, в которой будут использоваться такие роли, как «оператор платформы» (может настраивать пространства имен и контроллеры) и «разработчик» (может выполнять развертывание в пространстве имен, но не может удалять контроллеры допуска). Для этого можно использовать механизм ролевого управления доступом (role-based access control, RBAC).

Механизм RBAC позволяет управлять тем, какие права доступа будут получать пользователи кластера. Распространенный подход сводится к тому, чтобы предоставить членам команды разработчиков доступ к определенному пространству имен, настроив для него все необходимые политики безопасности подов. При этом они могут свободно развертывать в этом пространстве имен любые рабочие нагрузки, которые удовлетворяют определенным вами требованиям безопасности. В то же время такой подход не мешает разработчикам придерживаться методологии DevOps, поскольку, несмотря на установленные вами ограничения, они сами осуществляют процесс развертывания.

Настройка ролевого управления доступом выполняется с помощью двух объектов Kubernetes, которые действуют на уровне пространства имен: `Role` и `RoleBinding`. Первый позволяет задать для пространства имен определенную роль, например, разработчика, а второй — присвоить ее пользователям кластера (например, конкретным разработчикам). Существуют и кластерные версии этих объектов, `ClusterRole` и `ClusterRoleBinding`, которые отличаются лишь тем, что предоставляют доступ на уровне кластера.

12.6.1. Роль на уровне пространства имен

В определении роли вы можете указать, к каким группам API, ресурсам в пределах этих групп и командам должен предоставляться доступ. Права доступа определяются аддитивно (без возможности переключиться на субтрактивный способ), то есть доступ предоставляется ко всему, что вы перечисляете. Нам

нужно создать роль, дающую разработчикам право на выполнение практически любых действий в их пространстве имен, *за исключением* изменения самого пространства и удаления аннотации, определяющей уровень безопасности подов. Поэтому потребуется достаточно большое определение роли, которое представлено в листинге 12.13.

Листинг 12.13. Chapter12/12.6_RBAC/role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer-access
  namespace: team1
rules:
  - apiGroups:
    - ""
    resources:
    - namespaces
    verbs: ["get"]
  - apiGroups:
    - ""
    resources:
    - events
    - pods
    - pods/log
    - pods/portforward
    - services
    - secrets
    - configmaps
    - persistentvolumeclaims
    verbs: ["*"]
  - apiGroups:
    - apps
    resources:
    - deployments
    verbs: ["*"]
  - apiGroups:
    - autoscaling
    resources:
    - horizontalpodautoscalers
    verbs: ["*"]
  - apiGroups:
    - batch
    resources:
    - jobs
    verbs: ["*"]
  - apiGroups:
    - networking.k8s.io
    resources:
    - ingress
    verbs: ["*"]
  - apiGroups:
    - policy
    resources:
    - poddisruptionbudgets
    verbs: ["*"]
```

Пустая строка обозначает базовую группу API

Разрешаем разработчикам просматривать ресурсы в пределах пространства имен, но не редактировать их

Предоставляем разработчикам полный доступ к базовым типам рабочих нагрузок

Группа apps содержит такие ресурсы, как объект Deployment

Группа autoscaling содержит такие ресурсы, как объект HorizontalPodAutoscaler

Группа batch содержит такие ресурсы, как объект Job

Доступ к группе networking.k8s.io нужен для того, чтобы разработчики могли настраивать объект Ingress

Доступ к группе policy нужен для того, чтобы можно было настраивать бюджет прерывания подов (PodDisruptionBudget)

Данная роль предоставляет доступ к пространству имен `team1` (команда 1) и позволяет пользователю изменять поды, сервисы, секреты и объекты ConfigMap в базовой группе API и все ресурсы в группах `apps`, `autoscaling`, `batch`, `networking.k8s.io` и `policy`. Этот набор разрешений дает разработчику возможность развернуть практически любой YAML-файл из данной книги, в том числе развертывания, объекты StatefulSet, сервисы, объекты Ingress, горизонтальные автоскейлеры подов и задания. Обратите внимание: мы не указали ресурс пространства имен в базовой

группе API (которая обозначена с помощью пустой строки ""), чтобы пользователь не мог изменить пространство имен.

Создав роль, мы можем присвоить ее разработчику, указав его в определении объекта RoleBinding в поле subjects (субъекты) (листинг 12.14).

Листинг 12.14. Chapter12/12.6_RBAC/rolebinding.yaml

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: developerA
  namespace: team1
roleRef:
  kind: Role
  name: developer-access
  apiGroup: rbac.authorization.k8s.io
subjects:
# Учетная запись пользователя Google Cloud
- kind: User
  name: example@gmail.com
```

Ссылаемся на роль из листинга 12.13

Указываем обозначение конкретного разработчика. В случае сервиса GKE это пользователь Google, который имеет доступ к проекту с IAM-ролью Kubernetes Engine Cluster Viewer

Обратите внимание: то, какие имена можно указывать для субъекта User, зависит от того, какую платформу Kubernetes вы используете и как настроили систему идентификации. Если речь о платформе Google Cloud, то в качестве имени пользователя можно указать адрес электронной почты любого пользователя Google. Механизм RBAC авторизует пользователя, чтобы тот мог выполнить действия, которые определены в его роли. Однако, помимо этого, пользователь должен пройти аутентификацию, чтобы получить доступ к кластеру. Если вы работаете на платформе Google Cloud, то можете присвоить ему такую роль, как Kubernetes Engine Cluster Viewer. Она содержит разрешение `container.clusters.get`, которое позволяет пользователю проходить аутентификацию в целях получения доступа к кластеру, не предоставляя ему каких-либо разрешений на выполнение действий в кластере (что дает вам возможность выполнить тонкую настройку этих разрешений с помощью механизма RBAC). Как именно все это следует делать, зависит от того, какую облачную платформу вы используете.

Как администратор кластера, создайте пространство имен, роль и привязку роли, как показано ниже:

```
$ cd Chapter12/12.6_RBAC/
$ kubectl create ns team1
namespace/team1 created
$ kubectl create -f role.yaml
role.rbac.authorization.k8s.io/developer-access created
$ kubectl create -f rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/developerA created
```

Теперь, когда в кластере уже развернуты роль и привязка роли, наши пользователи-разработчики смогут взять практически любой из приведенных в этой книге примеров и развернуть его в пространстве имен `team1`, но не смогут при этом что-либо изменить в других пространствах или изменить само `team1`. Давайте проведем эксперимент и укажем в качестве субъекта User какого-либо реального пользователя в определении объекта RoleBinding — например, тестовый аккаунт разработчика.

АУТЕНТИФИКАЦИЯ И АВТОРИЗАЦИЯ

Аутентификация — это механизм, с помощью которого пользователь предоставляет системе свои учетные данные. Способность проходить аутентификацию для доступа к кластеру означает, что пользователь может получить необходимые для этого учетные данные с помощью утилиты `kubectl`. Авторизация — это процесс предоставления пользователям прав доступа в кластере. В зависимости от того, какую систему управления идентификацией и доступом (IAM) использует ваша платформа, обычно можно сделать так, чтобы пользователи могли пройти аутентификацию для доступа к кластеру (то есть получить учетные данные для использования утилиты `kubectl`), но не получали при этом право на выполнение каких-либо действий (то есть не проходили авторизацию). После этого можно выполнить точно настроенный процесс авторизации, задействуя механизм ролевого управления доступом (RBAC), чтобы предоставить пользователям только необходимые им права доступа. Допустим, вы работаете с сервисом GKE. В этом случае пользователи смогут проходить аутентификацию, если вы присвоите им роль `Kubernetes Engine Cluster Viewer` (Просмотр кластеров на движке Kubernetes) в системе IAM (за пределами Kubernetes), после чего можно авторизовать их в целях предоставления доступа к конкретным ресурсам, используя механизм RBAC, как показано в данной главе. Опять же, в зависимости от того, на какой платформе Kubernetes вы работаете, во многих случаях (например, при взаимодействии с сервисом GKE) некоторые IAM-роли авторизуют пользователя в целях предоставления доступа к определенным ресурсам в дополнение к тому, что указано в правилах RBAC. Один из примеров такой роли в случае сервиса GKE — действующая в рамках всего проекта роль `Viewer`, которая позволяет пользователям просматривать большинство ресурсов кластера, не определяя соответствующие правила RBAC.

Чтобы убедиться в том, что механизм RBAC настроен правильно, пройдите аутентификацию для доступа к кластеру, используя указанный в поле `subjects` (субъекты) тестовый аккаунт разработчика, и попробуйте развернуть что-либо в пространстве имен по умолчанию (`default`). Эта попытка закончится неудачей,

поскольку механизм RBAC не предоставляет никаких разрешений в этом пространстве имен:

```
$ kubectl config set-context --current --namespace=default
$ kubectl create -f Chapter03/3.2_DeployingToKubernetes/deploy.yaml
Error from server (Forbidden): error when creating
"Chapter03/3.2_DeployingToKubernetes/deploy.yaml": deployments.apps is
forbidden: User "example@gmail.com" cannot create resource "deployments" in
API group "apps" in the namespace "default": requires one of
["container.deployments.create"] permission(s).
```

Но если мы переключим контекст на пространство имен `team1`, для доступа к которому был настроен наш тестовый аккаунт с помощью представленной выше роли, то без каких-либо проблем сможем создать развертывание:

```
$ kubectl config set-context --current --namespace=team1
Context "gke_project-name_us-west1_cluster-name" modified.
$ kubectl create -f Chapter03/3.2_DeployingToKubernetes/deploy.yaml
deployment.apps/timeserver created
```

Теперь использующий этот аккаунт разработчик сможет успешно развернуть в данном пространстве имен нужные ему рабочие нагрузки. Но если он попытается изменить само пространство, чтобы получить привилегированный уровень безопасности подов, то ему будет в этом отказано, поскольку в определении ресурса пространства имен отсутствует разрешение на такое изменение:

```
$ kubectl label --overwrite ns team1 pod-security.kubernetes.io/enforce=privileged
Error from server (Forbidden): namespaces "team1" is forbidden: User
"example@gmail.com" cannot patch resource "namespaces" in API group "" in
the namespace "team1": requires one of ["container.namespaces.update"]
permission(s).
```

12.6.2. Роль на уровне кластера

До сих пор мы выполняли настройку объектов `Role` и `RoleBinding`, чтобы предоставить разработчику доступ к конкретному пространству имен. Настройка такой роли делает возможным развертывание практически любой конфигурации из этой книги. В то же время некоторые действия останутся недоступными: пользователь не сможет создать класс приоритета (`PriorityClass`) (глава 6) или класс хранения (`StorageClass`) (глава 9) либо отобразить список доступных в кластере постоянных томов (`PersistentVolume`) (глава 9). Эти ресурсы относятся к уровню кластера, поэтому обеспечивать доступ к ним нельзя с помощью созданной нами роли, которая действует на уровне пространства имен. Вместо этого мы должны использовать отдельные объекты `ClusterRole` и `ClusterRoleBinding`.

В листинге 12.15 представлено определение объекта `ClusterRole`, выдающего пользователю дополнительные разрешения на создание объектов `StorageClass` и `PriorityClass`.

Листинг 12.15. Chapter12/12.6_RBAC/clusterrole.yaml

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: developer-cluster-access
rules:
- apiGroups:
  - scheduling.k8s.io
  resources:
  - priorityclasses
  verbs: ["*"]
- apiGroups:
  - storage.k8s.io
  resources:
  - storageclasses
  verbs: ["*"]
- apiGroups:
  - ""
  resources:
  - persistentvolumes
  - namespaces
  verbs: ["get", "list"]

```

Предоставляем разработчику доступ к изменению любых объектов PriorityClass в пределах кластера	Предоставляем разработчику доступ к изменению любых объектов StorageClass в пределах кластера	Предоставляем разработчику доступ только для чтения к информации о постоянных томах (PersistentVolume) и пространствах имен (Namespace)
---	--	--

КАК ОПРЕДЕЛИТЬ, КАКИЕ ПРАВА ДОСТУПА СЛЕДУЕТ ПРЕДОСТАВЛЯТЬ

Приведенное выше определение кластерной роли позволяет предоставить все необходимые права доступа для развертывания любой из конфигураций, указанных в книге. Но не исключена вероятность того, что в контексте ваших рабочих нагрузок разработчикам нужно будет предоставить дополнительные разрешения. Чтобы определить, к каким группам, ресурсам и командам нужно предоставить доступ, вы можете обратиться к документации по API. В случае отладки проблем с правами доступа — если, например, разработчик жалуется на отсутствие необходимых ему прав доступа — вы можете просто изучить выдаваемое системой сообщение об ошибке. Вот пример такого случая:

```

$ kubectl create -f Chapter06/6.3.2_PlaceholderPod/placeholder-priority.yaml
Error from server (Forbidden): error when creating "placeholder-priority.yaml":
priorityclasses.scheduling.k8s.io is forbidden: User "example@gmail.com"
cannot create resource "priorityclasses" in API group "scheduling.k8s.io"
at the cluster scope: RBAC: clusterrole.rbac.authorization.k8s.io
"developer-cluster-access" not found requires one of
["container.priorityClasses.create"] permission(s).

```

Изучив это сообщение об ошибке, можно понять, что не хватает доступа к группе `scheduling.k8s.io`, ресурсу `priorityClasses` и команде создания `create` при использовании механизма RBAC в кластерном контексте. То есть мы должны добавить правило с этими значениями в определении объекта `ClusterRole`.

В листинге 12.16 представлено определение объекта ClusterRoleBinding, который привязывает кластерную роль к нашему тестовому пользователю подобно тому, как выполнял привязку роли объект RoleBinding, который был представлен выше.

Листинг 12.16. Chapter12/12.6_RBAC/clusterrolebinding.yaml

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: developerA
  namespace: team1
roleRef:
  kind: ClusterRole
  name: developer-cluster-access
  apiGroup: rbac.authorization.k8s.io
subjects:
- kind: User
  name: example@gmail.com
```

Ссылаемся на объект ClusterRole из листинга 12.15

Указываем обозначение конкретного разработчика. В случае сервиса GKE это пользователь Google, который имеет доступ к проекту с IAM-ролью Kubernetes Engine Cluster Viewer

Когда определение дополнительной кластерной роли и соответствующая привязка будут выполнены, как описано выше, наш пользователь сможет выполнить все указанные в этой книге действия.

ФЕДЕРАЦИЯ УДОСТОВЕРЕНИЙ

Чтобы механизм ролевого управления доступом (RBAC) мог использовать учетные данные пользователей-разработчиков в качестве субъектов типа User и Group, ваш кластер должен быть способным аутентифицировать эти учетные данные. В случае сервиса GKE кластер обладает встроенной способностью распознавать пользователей Google в поле User, а также группы Google, если активирован параметр Google Groups for RBAC. В зависимости от того, какую платформу и корпоративного поставщика удостоверений вы используете, вы уже можете иметь аналогичные возможности доступа; в противном случае вам придется настроить их должным образом. Подробное рассмотрение процесса настройки выходит за рамки данной книги; замечу лишь одно: для того чтобы механизм RBAC мог ссылаться на учетные данные, предоставленные системой идентификации, скорее всего, потребуется настроить интеграцию OpenID Connect (OIDC).

Вдобавок следует сказать, что при использовании плагина для системы идентификации, обеспечивающего поддержку групп, вместо того чтобы указывать в определении RoleBinding в качестве субъекта (subject) множество отдельных пользователей (kind: User), вы можете указать одного субъекта типа Group (kind: Group).

12.6.3. Применение профиля безопасности подов

При создании пространства имен в данном подразделе мы не определили требуемый уровень безопасности подов. Если мы вернемся на шаг назад и настроим это пространство с помощью меток профилей безопасности подов из раздела 12.5, то ему будет присвоен ограниченный профиль безопасности подов (**restricted**), и применяемый нами механизм ролевого управления доступом не позволит нашему пользователю-разработчику снять это ограничение. Миссия выполнена!

12.6.4. Использование механизма RBAC для аккаунтов сервисов

В примерах этого подраздела мы использовали механизм RBAC с субъектом User (Пользователь), поскольку создавали пространство имен для разработчиков, то есть для имеющейся у нас разновидности пользователей-людей. Однако еще одним распространенным сценарием использования механизма ролевого управления доступом (RBAC) является предоставление прав доступа сервисам, то есть работающим в кластере программам.

Допустим, что одному из подов запущенного в кластере развертывания требуется доступ к API Kubernetes — например, для отслеживания состояния подов другого развертывания. Чтобы предоставить необходимые права доступа такому пользователю-программе, вы можете создать объект Kubernetes ServiceAccount и указать его вместо пользователя в качестве субъекта при определении привязки механизма RBAC.

В Интернете можно найти документацию с описанием процедуры настройки объектов ServiceAccount для пользователей-людей, которая позволяет скачивать сертификаты аккаунта сервиса для взаимодействия с Kubernetes. Такой подход позволяет успешно настроить среду для разработчиков, обойдясь без настройки федерации удостоверений, но я не рекомендую вам его использовать, поскольку он действует за пределами системы идентификации. Если, например, кто-то из пользователей прекратит использовать вашу среду и будет удален из системы идентификации, то скачанные им токены для объекта ServiceAccount будут по-прежнему действовать. Поэтому лучше должным образом настроить федерацию удостоверений и использовать для пользователей-людей только субъект User, чтобы в случае их удаления из системы идентификации отзывались и предоставленные им права доступа к Kubernetes. Обратите внимание: этот процесс интеграции не доставляет больших проблем при использовании таких управляемых платформ, как Google Cloud, но при работе с другими платформами может потребоваться выполнить достаточно сложную настройку.

Объекты ServiceAccount предназначены для тех ситуаций, когда, например, одному из запущенных в кластере подов требуется непосредственный доступ к API

Kubernetes. Допустим, что вам нужно создать под, который будет отслеживать состояние другого развертывания. В таком случае вы можете создать `ServiceAccount`, указать его в качестве субъекта в `RoleBinding` и присвоить этот `ServiceAccount` своему поду. После этого ваш под сможет использовать данный аккаунт для вызова API, в том числе с помощью утилиты `kubectl`.

12.7. ЧТО ЕЩЕ МОЖНО СДЕЛАТЬ

Механизм допуска безопасности подов позволяет управлять тем, какие права доступа будут иметь запущенные на узле поды, а механизм ролевого управления доступом (RBAC) — тем, какие ресурсы будут доступны пользователям в пределах кластера. Это может стать хорошей отправной точкой, если требуется обеспечить дополнительную изоляцию на уровне сети и контейнеров; тем не менее вы можете принять дополнительные меры безопасности. Рассмотрим их более подробно.

12.7.1. Сетевые политики

По умолчанию любой под может взаимодействовать с любым другим подом в пределах кластера. Это удобно, поскольку команды разработчиков, работающие в разных пространствах имен, могут использовать сервисы совместно. Но в то же время следует иметь в виду, что в случае взлома пода злоумышленник может получить доступ и к другим внутренним сервисам. Чтобы обеспечить контроль над входящим и исходящим трафиком пода при его взаимодействии с сетью и другими подами и, например, запретить ему взаимодействие с подами, расположенными в других пространствах имен, вы можете использовать сетевые политики (<https://kubernetes.io/docs/concepts/services-networking/network-policies/>).

Принцип действия этих политик сводится к следующему. Если к поду не будет применена ни одна сетевая политика (`NetworkPolicy`), то для него будут разрешены любые виды трафика (именно так и обстоит дело по умолчанию). Но если вы определите сетевую политику для входящего или исходящего трафика пода, то в выбранном направлении смогут проходить лишь те виды трафика, которые будут явно указаны в политике. Это означает, что при ограничении трафика, идущего в определенном направлении, необходимо составлять полный список допустимых видов трафика, исходя из того, что нужно подам.

Например, чтобы запретить взаимодействие подов, расположенных в разных пространствах имен, можно создать набор правил, который будет разрешать обмен трафиком с Интернетом и с другими подами в пределах одного пространства. В данном наборе правил не будут упомянуты поды, расположенные в других пространствах имен, поэтому обмен трафиком с ними будет запрещен, что и требуется в данном случае.

Сетевые политики разрешают лишь явно указанные в них виды трафика, поэтому вам нужно будет внимательно изучить, какие виды трафика требуются в вашем случае (это могут быть и особые требования вашей платформы), и, вероятно, пройти некий путь проб и ошибок, прежде чем вам удастся настроить их должным образом. Я опубликовал несколько статей, которые можно использовать в качестве хорошего вводного курса по этой теме; они доступны по адресу <https://wdenniss.com/networkpolicy>.

12.7.2. Изоляция контейнеров

Контейнеризация в определенной мере изолирует процесс от узла, а механизм допуска безопасности пода позволяет ограничить предоставленные контейнеру права доступа, но время от времени вы можете сталкиваться с уязвимостями типа «побег из контейнера», которые позволяют процессам получать доступ к узлу. Можно добавить дополнительный уровень изоляции между контейнером и хостом, таким образом обеспечив дополнительный уровень эшелонированной защиты в дополнение к той защите, которая возможна благодаря контейнеризации. Эта дополнительная изоляция обычно не активируется по умолчанию, поскольку немного снижает производительность. Но если, например, вам нужно запускать в кластере недоверенный код в рамках многопользовательской системы, позволяющей пользователям предоставлять собственные контейнеры, то вам определенно стоит добавить такой дополнительный уровень изоляции.

Обеспечить дополнительную изоляцию подов можно, настроив их на использование безопасной среды выполнения с помощью ресурса `RuntimeClass` (<https://kubernetes.io/docs/concepts/containers/runtime-class/>). В качестве безопасной среды выполнения многие разработчики используют свободно распространяемое решение от Google — `gVisor` (<https://gvisor.dev/>). Оно реализует API ядра Linux и перехватывает системные вызовы между контейнером и ядром операционной системы, таким образом позволяя ему работать в изолированной «песочнице».

12.7.3. Защита кластера

Я надеюсь, что в этой главе достаточно подробно изложил основные принципы безопасной разработки и развертывания приложений в Kubernetes — процессов, в ходе которых в кластерах используются разрешения ролевого управления доступом (RBAC) и правила ограниченного допуска, такие как запрет на запуск контейнеров от имени суперпользователя. Читателям, работающим в роли оператора кластера, следует ознакомиться с более обширной темой усиления защиты кластера и его операционной среды (сети, узлов, облачных ресурсов и т. д.), причем многие из этих рекомендаций зависят от того, какая платформа используется.

Я рекомендую поискать в Интернете последнюю версию руководства по усилению защиты Kubernetes (Kubernetes Hardening Guide). Многое зависит от вашей

операционной среды, поэтому для начала стоит ознакомиться с руководством по усилению защиты, которое предоставляет поставщик вашей платформы. Хороший пример такого руководства — руководство по усилению защиты кластера для пользователей сервиса GKE (<https://cloud.google.com/kubernetes-engine/docs/how-to/hardening-your-cluster>). Сфера безопасности подвержена частым изменениям, поэтому старайтесь следить за тем, какие передовые практики рекомендуют использовать авторитетные источники.

РЕЗЮМЕ

- Чтобы снизить риск использования злоумышленниками уязвимостей защиты, регулярно обновляйте свой кластер и его узлы.
- Базовые образы Docker имеют собственную поверхность атаки, поэтому необходимо отслеживать состояние развернутых контейнеров и регулярно их обновлять. В этом вам может помочь система непрерывной интеграции и доставки (CI/CD).
- Используя максимально маленький базовый образ, вы можете уменьшить поверхность атаки, что позволяет реже обновлять приложение в целях устранения уязвимостей защиты.
- Объекты DaemonSet дают возможность запустить под на каждом узле и широко применяются для запуска в кластере программного обеспечения для журналирования, мониторинга и обеспечения безопасности.
- Контекст безопасности пода используется для предоставления подам расширенных или ограниченных разрешений.
- Контроллеры допуска позволяют изменять объекты Kubernetes на этапе их создания и обеспечить соблюдение требований, в том числе тех, которые связаны с контекстом безопасности пода.
- Kubernetes обладает встроенным контроллером допуска PSA (Pod Security Admission, допуск безопасности пода), с помощью которого вы можете применить определенный профиль безопасности, например, базовый (Baseline), который обеспечивает защиту от большинства известных атак, или ограниченный (Restricted), который дает возможность использовать расширенные методы защиты подов.
- Механизм ролевого управления доступом (RBAC) — это основанная на ролях система разрешений, которая позволяет пользователю с ролью администратора кластера выполнить тонкую настройку прав доступа для разработчиков, например, позволить им использовать только выделенное для них пространство имен.

- По умолчанию каждый под может взаимодействовать со всеми другими подами в пределах кластера. Используя сетевые политики, вы можете контролировать взаимодействие подов с сетью.
- Чтобы обеспечить дополнительный уровень изоляции, особенно когда требуется запускать в кластере недоверенный код, можно настроить поды на использование такой среды выполнения, как gVisor, используя для этого ресурс RuntimeClass (Класс среды выполнения).
- Чтобы изучить все подробные и зависящие от платформы рекомендации по обеспечению безопасности, ознакомьтесь с руководством по усилению защиты Kubernetes, которое предоставляет поставщик вашей платформы.

Уильям Деннис

Kubernetes для разработчиков

Перевел с английского В. Сауль

Научный редактор А. Патрин

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питиримов</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературный редактор	<i>Н. Хлебина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>О. Андриевич, Н. Терех</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес:
194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 02.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 24.01.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 30,960. Тираж 700. Заказ 0000.